



01-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Internet qanday ishlaydi? DNS, hosting, domen

Mijoz-server arxitekturasini, IP-manzil, DNS, domen va hosting tushunchalari, veb-sayt internetda qanday "yashaydi", Front-end va Back-end farqi.

1 soat · nazariy

Bu darsda nimani o'rganasiz

- Internet nima ekanini va ma'lumot unda qanday "sayohat" qilishini;
- Mijoz (client) va server nima ekanini;
- IP-manzil, domen, DNS va hosting tushunchalarini;
- Brauzerga manzil yozganda nima sodir bo'lishini bosqichma-bosqich;
- Front-end va Back-end dasturchilar nima bilan shug'ullanishini.

Internet nima?

Internet — butun dunyodagi millionlab kompyuterlar, telefonlar va serverlarni bir-biriga bog'lagan **ulkan tarmoq**. Ular kabellar (dengiz tubidagi optik kabellar ham!), Wi-Fi, mobil aloqa minoralari va sun'iy yo'ldoshlar orqali ulangan.



O'xshatish: pochta xizmati

Internetni butun dunyo bo'ylab ishlaydigan pochta deb tasavvur qiling. Siz xat (ma'lumot) yuborasiz, unga manzil (IP-manzil) yoziladi, pochta bo'limlari (routerlar) uni bir-biriga uzatib, kerakli uyga yetkazadi. Farqi — internet "xati" bir soniyadan ham kamroq vaqtda yetib boradi.

Internet va Web bir narsami?

Yo'q! Bu ko'p uchraydigan xato.

- **Internet** — "yo'llar tizimi" (tarmoqning o'zi).

- **Web (World Wide Web, WWW)** — shu yo'llarda ishlaydigan xizmatlardan biri: bir-biriga havolalar bilan bog'langan veb-sahifalar. Uni 1989-yilda olim **Tim Berners-Li** ixtiro qilgan.

Internetda boshqa xizmatlar ham bor: elektron pochta, Telegram'dagi xabarlar, onlayn o'yinlar, video qo'ng'iroqlar.

Ma'lumot qanday yuboriladi? Paketlar

Siz katta rasm yuborganingizda u bir butun holda emas, **mayda bo'laklarga — paketlarga** bo'linib ketadi. Har bir paketga qabul qiluvchining manzili yoziladi. Paketlar turli yo'llar bilan ketishi mumkin, manzilda esa yana to'g'ri tartibda yig'iladi.



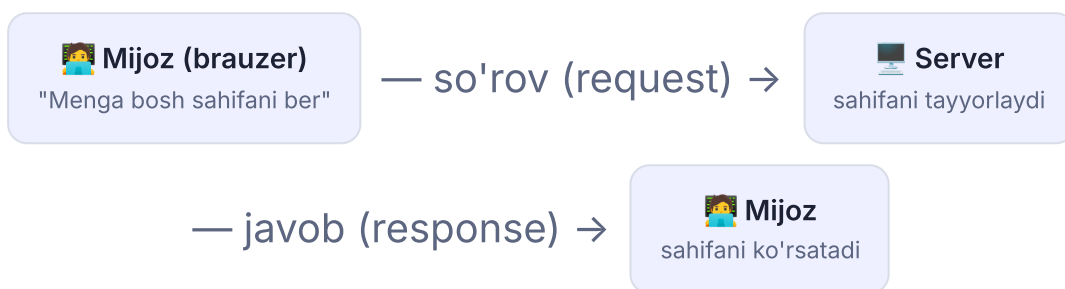
O'xshatish: mebelni ko'chirish

Katta shkafni bir mashinaga sig'dira olmasangiz, uni qismlarga ajratib, bir nechta mashinada jo'natasiz. Yangi uyda qismlar raqami bo'yicha qaytadan yig'iladi. Internet paketlari ham xuddi shunday.

Mijoz va server

Web **mijoz-server (client-server) arxitekturasida** ishlaydi:

- **Mijoz (client)** — ma'lumot *so'rovchi* tomon. Ko'pincha bu sizning brauzeringiz (Chrome, Edge) yoki telefondagi ilova.
- **Server** — so'rovni qabul qilib, *javob beruvchi* kuchli kompyuter. U kechayu kunduz yoqiq turadi va bir vaqtning o'zida minglab foydalanuvchiga xizmat qiladi.





O'xshatish: oshxona

Restoranda siz — **mijoz**, ofitsiantga buyurtma berasiz (**so'rov**). Oshxona — **server**: taomni tayyorlab, ofitsiant orqali qaytaradi (**javob**). Siz oshxonaga kirib ko'rmaysiz — faqat natijani olasiz. Veb-sayt ham xuddi shunday: siz serverni ko'rmaysiz, faqat u yuborgan sahifani ko'rasiz.

HTTP va HTTPS

Mijoz va server bir-biri bilan **HTTP** (HyperText Transfer Protocol) degan "til"da gaplashadi. **Protokol** — kelishilgan qoidalar to'plami.

- `http://` — ma'lumot ochiq holda uzatiladi. Yo'lda kimdir uni o'qishi mumkin.
- `https://` — **S** = Secure (xavfsiz). Ma'lumot **shifrlanadi**. Brauzerdagi  qulf belgisi shuni bildiradi.



Parol yoki karta raqamini faqat `https://` bilan boshlanadigan saytlarga kiriting!

Server javobida uch xonali **status kod** bo'ladi. Eng mashhurlari:

Kod	Ma'nosi	Qachon ko'rasiz
200	OK — hammasi joyida	Sahifa muvaffaqiyatli ochildi
301	Sahifa boshqa manzilga ko'chgan	Brauzer o'zi yangi manzilga o'tadi
404	Topilmadi	Mavjud bo'lmagan sahifaga kirdingiz
500	Serverda xato	Sayt dasturida muammo bor

IP-manzil

Internetga ulangan har bir qurilmaning o'z **IP-manzili** bor — xuddi uyning pochta manzili kabi. U nuqtalar bilan ajratilgan to'rtta sondan iborat: `142.250.74.46`. Har bir son 0 dan 255 gacha bo'ladi.

Lekin odamlar uchun bunday raqamlarni eslab qolish qiyin. Siz Google'ga kirish uchun `142.250.74.46` deb yozasizmi? Albatta yo'q — `google.com` deb yozasiz. Bu yerda **domen** va **DNS** yordamga keladi.

5 Brauzer shu IP-manzil bo'yicha serverga ulanadi.

Bu jarayon odatda **bir necha millisoniya** davom etadi.

Hosting

Hosting — saytingiz fayllari saqlanadigan va kechayu kunduz internetga ulangan turadigan **serverdagi joy**.



O'xshatish: uy va manzil

Domen — uyingizning manzili (ko'cha va raqam). **Hosting** — uyning o'zi, ichida mebellar (fayllar) turadi. **DNS** esa manzil bo'yicha uyni topib beruvchi xarita. Manzil bor-u, uy bo'lmasa — mehmon hech narsa topmaydi.

Hosting turlari

Tur	Tavsif	Kimga mos
Bepul statik hosting	Netlify, Vercel, GitHub Pages — HTML/CSS/JS fayllar uchun	O'quvchilar, portfolio (13-mavzuda ishlatamiz)
Shared (umumiy) hosting	Bitta serverda yuzlab sayt "qo'shni" bo'lib yashaydi	Kichik saytlar
VPS	Serverning sizga ajratilgan virtual qismi	O'rta loyihalar
Bulutli platformalar	Render, Railway, AWS — kerak bo'lsa avtomatik kengayadi	Django loyihalari (41-mavzu)

Brauzerga manzil yozganda nima bo'ladi?

Endi barcha tushunchalarni bitta zanjirga bog'laymiz. Siz `https://maktab.uz` deb yozib `Enter` bosdingiz:

1 **DNS** `maktab.uz` ni IP-manzilga aylantiradi.

- 2 Brauzer shu IP-manzildagi **serverga ulanadi** (HTTPS bo'lsa — xavfsiz, shifrlangan ulanish o'rnatiladi).
- 3 Brauzer serverga **HTTP so'rov** yuboradi: "Bosh sahifani ber".
- 4 **Server** (hosting) kerakli faylni topadi yoki tayyorlaydi va **javob** qaytaradi (status `200` + HTML kod).
- 5 Brauzer HTML'ni o'qiydi va unda ko'rsatilgan **CSS, rasmlar, JavaScript** fayllari uchun qo'shimcha so'rovlar yuboradi.
- 6 Brauzer hammasini birlashtirib, sahifani ekranga **chizadi**.

Front-end va Back-end

Har bir veb-sayt ikki qismdan iborat:

	 Front-end	 Back-end
Nima	Foydalanuvchi ko'radigan va bosadigan qism	Ko'rinmaydigan, serverda ishlaydigan qism
Qayerda ishlaydi	Foydalanuvchi brauzerida	Serverda
Texnologiyalar (bu kursda)	HTML, CSS, JavaScript	Python, Django, PostgreSQL
Misol	Tugma, menyu, rang, animatsiya	Login tekshirish, ma'lumotlarni bazaga saqlash



O'xshatish: restoran

Front-end — restoraning zali: chiroyli stollar, menyu, ofitsiant. **Back-end** — oshxona va ombor: oshpaz, retseptlar, mahsulotlar zaxirasi. **Ma'lumotlar bazasi** — ombor. **Full-stack dasturchi** — zalni ham, oshxonani ham yarata oladigan usta.

Masalan, siz Telegram'da xabar yozdingiz: matn kiritiladigan maydon va "Yuborish" tugmasi — **front-end**; xabarni qabul qilib, saqlab, do'stingizga yetkazish — **back-end**.

Amaliy mashg'ulot

- 1 Saytning IP-manzilini toping. Windows'da Win + R → cmd → Enter. Ochilgan oynaga yozing:

Terminal

```
nslookup google.com  
nslookup kun.uz
```

Natijadagi Address qatori — saytning IP-manzili. (macOS/Linux'da ham nslookup ishlaydi.)

- 2 Serverga "ping" yuboring — server javob beradimi va qancha vaqtda:

Terminal

```
ping google.com
```

time=25ms — paket serverga borib-kelishi uchun ketgan vaqt (millisoniya).

- 3 Brauzer so'rovlarini kuzating. Chrome'da istalgan saytni oching, F12 bosing → Network yorlig'i → sahifani F5 bilan yangilang. Har bir qator — brauzerning serverga yuborgan alohida so'rovi. Status ustunida 200 larni toping. Birinchi qatorni bosib, Remote Address (server IP-manzili)ni ko'ring.

- 4 404 xatoni ko'ring. Istalgan saytda mavjud bo'lmagan sahifani oching, masalan <https://github.com/bu-sahifa-yoq-12345>. Network panelida status 404 ekanini tekshiring.

- 5 Jadval to'ldiring. Daftaringizga 3 ta sevimli saytingiz uchun: domen nomi, zonasi (.uz / .com), IP-manzili, HTTPS bormi-yo'qmi va saytda nimalar front-end, nimalar back-end ekanini yozing.

Ko'p uchraydigan xato tushunchalar

- ✗ "Internet va Google bir narsa" → ✓ Google — internetdagi saytlardan (xizmatlardan) biri.
- ✗ "Domen sotib olinsa, sayt tayyor bo'ladi" → ✓ Domen — faqat manzil. Saytning fayllari uchun hosting ham kerak.
- ✗ "Brauzer = Internet" → ✓ Brauzer — internetdagi sahifalarni ochuvchi dastur (mijoz).
- ✗ "HTTPS saytlar hammasi ishonchli" → ✓ HTTPS faqat ma'lumot yo'lda shifrlanishini kafolatlaydi; firibgar sayt ham HTTPS bo'lishi mumkin. Domen nomini diqqat bilan tekshiring!

O'zingizni tekshiring

? DNS'ning asosiy vazifasi nima?

- A. Saytni chiroyli qilish
- B. Domen nomini IP-manzilga aylantirish
- C. Fayllarni saqlash
- D. Viruslardan himoyalash

? Saytning fayllari saqlanadigan, doim internetga ulangan server joyi nima deyiladi?

- A. Domen
- B. Brauzer
- C. Hosting
- D. Protokol

? Quyidagilardan qaysi biri front-end'ga tegishli?

- A. Saytdagi "Ro'yxatdan o'tish" tugmasining rangi
- B. Parolni ma'lumotlar bazasida tekshirish
- C. Buyurtmani serverda saqlash
- D. Server IP-manzili

? Server "404" status kodini qaytardi. Bu nimani bildiradi?

- A. Hammasi yaxshi
- B. Serverda dastur xatosi
- C. Sahifa boshqa manzilga ko'chgan
- D. So'ralgan sahifa topilmadi

? `https://blog.maktab.uz/maqolalar` manzilida subdomen qaysi?

- A. `https`
- B. `blog`
- C. `uz`
- D. `maqolalar`

? HTTPS'dagi "S" harfi nimani bildiradi?

- A. Speed — tezlik
- B. Server
- C. Secure — xavfsiz (shifrlangan)
- D. Simple — oddiy



Uyga vazifa

1. "Brauzerga manzil yozganda nima bo'ladi?" mavzusida **chizma (sxema)** chizing: mijoz, DNS, server, so'rov va javobni strekalar bilan ko'rsating.
2. Oila a'zolaringizga restoran o'xshatishi orqali front-end va back-end farqini tushuntirib bering.
3. O'zingiz yoqtirgan 5 ta mobil ilova uchun: ilovaning front-end qismi nima va back-end qismi nima qiladi — yozib chiqing.

✓ Yodda tuting

Internet

Dunyo kompyuterlarini bog'lovchi tarmoq. Web — undagi xizmatlardan biri.

Mijoz / Server

Mijoz so'raydi (brauzer), server javob beradi.

IP-manzil

Qurilmaning tarmoqdagi raqamli manzili: `142.250.74.46`.

Domen

Saytning eslab qolinadigan nomi: `maktab.uz`.

DNS

Domenni IP-manzilga aylantiruvchi "telefon kitobi".

Hosting

Sayt fayllari saqlanadigan, doim yoqiq server.

HTTP / HTTPS

Mijoz va server muloqot qoidalar; HTTPS — shifrlangan.

Front-end / Back-end

Ko'rinadigan qism / serverdagi ko'rinmas qism.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. DNS'ning asosiy vazifasi nima? — **B) Domen nomini IP-manzilga aylantirish**
DNS — "telefon kitobi": kun.uz kabi nomlarni kompyuter tushunadigan IP-manzilga aylantiradi.
2. Saytning fayllari saqlanadigan, doim internetga ulangan server joyi nima deyiladi? — **C) Hosting**
Hosting — "uy", domen — "manzil".
3. Quyidagilardan qaysi biri front-end'ga tegishli? — **A) Saytdagi "Ro'yxatdan o'tish" tugmasining rangi**
Foydalanuvchi ko'radigan hamma narsa — front-end. Qolganlari serverda bajariladi (back-end).
4. Server "404" status kodini qaytardi. Bu nimani bildiradi? — **D) So'ralgan sahifa topilmadi**
404 — "Not Found". 200 — muvaffaqiyat, 500 — server xatosi, 301 — ko'chirilgan.
5. <https://blog.maktab.uz/maqolalar> manzilida subdomen qaysi? — **B) blog**
blog — subdomen, maktab — domen nomi, uz — zona, /maqolalar — yo'l.
6. HTTPS'dagi "S" harfi nimani bildiradi? — **C) Secure — xavfsiz (shifrlangan)**
HTTPS ma'lumotni shifrlab uzatadi — yo'lda uni hech kim o'qiy olmaydi.



02-MAVZU · 42 TADAN · I BOB. HTML VA CSS

HTML hujjatining tuzilishi, asosiy teglar va atributlar

HTML nima, teg va element, hujjat skeleti (html, head, body), sarlavhalar, paragraflar, matn teglari va atributlar. Birinchi veb-sahifani yaratish.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- HTML nima va nima uchun kerakligini;
- Teg, element va atribut tushunchalarini;
- Har bir HTML sahifaning majburiy "skeleti"ni;
- Sarlavha, paragraf va matnni formatlovchi teglarni;
- VS Code'da birinchi veb-sahifangizni yaratib, brauzerda ochishni.

HTML nima?

HTML (HyperText Markup Language) — veb-sahifalarning **tuzilishini** yaratuvchi **belgilash tili**. U dasturlash tili emas: unda hisob-kitob yoki shartlar yo'q. HTML brauzerga faqat "bu — sarlavha, bu — paragraf, bu — rasm, bu — tugma" deb aytadi.



O'xshatish: uy qurilishi

Veb-sahifa — uy. **HTML** — uyning **g'ishtlari va devorlari** (tuzilma). **CSS** — bo'yoq, gilam, mebel (dizayn, 6-mavzudan). **JavaScript** — elektr va suv: chiroqni yoqish, eshik qo'ng'irog'i (harakat, 15-mavzudan). Devorsiz bo'yoq ham, elektr ham ma'nosiz.

Istalgan saytda sichqonchani o'ng tugmasini bosib "**Sahifa kodini ko'rish**" (**Ctrl** + **U**) ni tanlang — siz o'sha sahifaning HTML kodini ko'rasiz. Har bir sayt, hatto YouTube ham, HTML'dan boshlanadi.

Teg, element va atribut

Teg

Teg — burchak qavslar ichidagi kalit so'z. Ko'pchilik teglar **juft** bo'ladi: ochuvchi va yopuvchi.

HTML

```
<p>Bu paragraf matni.</p>
<!--
<p>    - ochuvchi teg
</p>    - yopuvchi teg (oldida / belgisi bor)
-->
```

Element

Element = ochuvchi teg + mazmun + yopuvchi teg. Yuqoridagi butun qator — bitta paragraf elementi.

Bo'sh (yolg'iz) teglar

Ba'zi teglarning mazmuni bo'lmaydi va ular yopilmaydi:

- `<br>` — yangi qatorga o'tish;
- `<hr>` — gorizontal chiziq;
- `<img>` — rasm (4-mavzu);
- `<input>` — kiritish maydoni (3-mavzu);
- `<meta>`, `<link>` — sahifa haqida ma'lumot.

Atribut

Atribut — elementga qo'shimcha ma'lumot yoki sozlama. U doim **ochuvchi tegda**, `nom="qiymat"` ko'rinishida yoziladi.

HTML

```
<a href="https://kun.uz" title="Yangiliklar sayti">Kun.uz ga o'tish</a>
<!--
a      - teg nomi (havo'la)
href   - atribut nomi: qayerga olib boradi
"https://kun.uz" - atribut qiymati (qo'shtirnoq ichida)
title  - yana bir atribut: sichqoncha ustiga borganda chiqadigan yozuv
-->
```

Hamma teglarda ishlatiladigan (global) atributlar:

Atribut	Vazifasi	Misol
id	Elementning yagona nomi (sahifada takrorlanmaydi)	<code><h1 id="sarlavha"></code>
class	Guruh nomi (CSS uchun, bir nechta element bir xil bo'lishi mumkin)	<code><p class="muhim"></code>
title	Sichqoncha ustiga kelganda chiqadigan izoh	<code><abbr title="..."></code>
style	Elementga to'g'ridan-to'g'ri CSS	<code><p style="color: red"></code>
lang	Matn tili	<code><html lang="uz"></code>

HTML hujjatining skeleti

Har bir HTML sahifa quyidagi majburiy tuzilmadan boshlanadi. Uni yod olish shart emas — VS Code'da bo'sh `.html` faylga `!` belgisini yozib `Enter` (yoki `Tab`) bossangiz, o'zi yaratiladi. Lekin **har bir qatorning ma'nosini** bilishingiz kerak:

```

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mening birinchi sahifam</title>
</head>
<body>
  <h1>Salom, dunyo!</h1>
  <p>Bu mening birinchi veb-sahifam.</p>
</body>
</html>

```

Qator	Ma'nosi
<code><!DOCTYPE html></code>	"Bu zamonaviy HTML5 hujjat" — brauzerga ko'rsatma. Doim birinchi qatorda.
<code><html lang="uz"></code>	Butun sahifaning "qutisi" (ildiz element). <code>lang="uz"</code> — sahifa tili o'zbekcha.
<code><head></code>	"Bosh qism" — sahifa haqidagi ko'rinmaydigan ma'lumotlar.
<code><meta charset="UTF-8"></code>	Belgilar kodlash: o'zbekcha <code>o'</code> , <code>g'</code> , kirill harflari va emoji 😊 to'g'ri ko'rinishi uchun.
<code><meta name="viewport" ...></code>	Telefonda sahifa to'g'ri o'lchamda ko'rinishi uchun (10-mavzuda muhim).
<code><title></code>	Brauzer yorlig'idagi (tab) nom va Google natijalaridagi sarlavha.
<code><body></code>	"Tana" — foydalanuvchi ko'radigan hamma narsa shu yerda.



O'xshatish: odam

`<head>` — odamning "pasporti": ismi, tili, yoshi (ko'rinmaydi, lekin muhim). `<body>` — odamning o'zi: ko'rinishi, kiyimi, harakatlari.

Ichma-ich joylashish (nesting)

Elementlar bir-birining ichida joylashadi — xuddi matryoshka qo'g'irchoqlari kabi. Asosiy qoida: **oxirgi ochilgan teg birinchi yopiladi**.

HTML

```
<!-- ✓ To'g'ri -->
<p>Bu <strong>juda muhim</strong> gap.</p>

<!-- ✗ Noto'g'ri: teglar "kesishib" qolgan -->
<p>Bu <strong>juda muhim</p> gap.</strong>
```



Kodni chiroyli o'qilishi uchun ichki elementlarni **2 ta bo'sh joy** bilan suring (indentatsiya). VS Code'da **Shift + Alt + F** kodni avtomatik tartiblaydi.

Sarlavhalar va paragraflar

HTML'da 6 darajali sarlavha bor: `<h1>` (eng muhim, eng katta) dan `<h6>` (eng kichik) gacha.

HTML

```
<h1>Maktabimiz yangiliklari</h1>      <!-- sahifaning asosiy sarlavhasi
(bitta) -->
<h2>Sport</h2>                        <!-- bo'lim -->
<h3>Futbol bo'yicha musobaqa</h3>    <!-- kichik bo'lim -->
<p>8-"A" sinf jamoasi shahar chempionatida g'olib bo'ldi.</p>
<h3>Shaxmat to'garagi</h3>
<p>Har seshanba kuni soat 15:00 da.</p>
<h2>Fanlar</h2>
<p>Informatika olimpiadasiga ro'yxatdan o'tish boshlandi.</p>
```

Qoidalar:

- Sahifada odatda **faqat bitta** `<h1>` bo'ladi — u sahifaning asosiy mavzusi.
- Darajalarni sakrab o'tmang: `h2` dan keyin `h4` emas, `h3`.
- Sarlavha tegini matnni **kattalashtirish uchun** ishlatmang — buning uchun CSS bor. Sarlavha — **ma'no** (kitob mundarijasi kabi).

<p> — paragraf. Brauzer paragraflar orasiga avtomatik bo'sh joy qo'yadi.



Diqqat: HTML bo'sh joylarni "yutadi"

Kodda nechta bo'sh joy yoki `Enter` qo'ysangiz ham, brauzer ularni **bitta bo'sh joy** qilib ko'rsatadi. Yangi qatorga o'tish uchun `<br>`, yangi abzats uchun yangi `<p>` ishlatish.

Matnni formatlash teglari

HTML

```
<p>
  <strong>Qalin va muhim</strong> matn,
  <b>shunchaki qalin</b> matn.<br>
  <em>Urg'u berilgan (kursiv)</em> va <i>shunchaki kursiv</i>.<br>
  <mark>Marker bilan belgilangan</mark> matn.<br>
  <u>Tagiga chizilgan</u> va <del>o'chirilgan</del> matn.<br>
  <small>Mayda yozuv</small> va oddiy matn.<br>
  Suv formulasi: H<sub>2</sub>O, maydon: 5 m<sup>2</sup><br>
  <code>console.log()</code> — dastur kodi.<br>
  <abbr title="HyperText Markup Language">HTML</abbr> — sichqonchani ustiga
  olib boring.
</p>
```

```
<blockquote>
  Bilim — eng katta boylik.
</blockquote>
```

```
<hr>
```

```
<pre>
Bu teg ichida      bo'sh joylar
    va qatorlar      SAQLANADI.
</pre>
```

Teg

Vazifasi

`<strong>` /

`<b>`

Qalin. `strong` — "bu muhim" degan **ma'no** ham beradi (ko'zi ojizlar uchun dasturlar ovozi o'zgartiradi).

<code></code> / <code><i></code>	Kursiv. <code>em</code> — urg'u ma'nosida.
<code><mark></code>	Marker bilan belgilash (sariq fon)
<code><sub></code> / <code><sup></code>	Pastki / yuqori indeks: H ₂ O, x ²
<code>
</code>	Yangi qatorga o'tish (yolg'iz teg)
<code><hr></code>	Mavzuni ajratuvchi chiziq (yolg'iz teg)
<code><blockquote></code>	Iqtibos (boshqa manbadan keltirilgan gap)
<code><pre></code>	Matnni yozilganidek (bo'sh joylari bilan) ko'rsatish

Izohlar (comments)

Izoh — brauzer ko'rsatmaydigan, faqat dasturchi uchun yozilgan eslatma:

HTML

```
<!-- Bu izoh. Sahifada ko'rinmaydi. -->
<!-- TODO: bu yerga rasm qo'shish kerak -->
```

Maxsus belgilar (entities)

Ba'zi belgilar HTML'da "band" — masalan `<` tegni boshlaydi. Ularni matnda ko'rsatish uchun maxsus kodlar ishlatiladi:

Yozilishi	Natija	Ma'nosi
<code>&lt;</code>	<	kichik belgisi
<code>&gt;</code>	>	katta belgisi
<code>&amp;</code>	&	"va" belgisi
<code>&copy;</code>	©	mualliflik huquqi
<code>&nbsp;</code>	(bo'sh joy)	uzilmaydigan bo'sh joy

Amaliy mashg'ulot: "Men haqimda" sahifasi

- 1 Ish stolida `men-haqimda` nomli papka yarating.
- 2 VS Code'ni oching: **File** → **Open Folder** → shu papkani tanlang.
- 3 Yangi fayl yarating: `index.html` (nomi aynan shunday — `index` veb-saytning bosh sahifasi degani).
- 4 Faylga `!` yozib `Enter` bosing — skelet paydo bo'ladi. `lang="en"` ni `lang="uz"` ga o'zgartiring.
- 5 `<title>` ga ismingizni yozing.
- 6 `<body>` ichiga quyidagi kodni (o'z ma'lumotlaringiz bilan) **qo'lda** yozing:

HTML

```
<h1>Sardor Aliyev</h1>
<p><em>8-"B" sinf o'quvchisi, kelajakdagi dasturchi</em></p>
<hr>

<h2>Men haqimda</h2>
<p>Men <strong>Samarqand</strong> shahrida yashayman. 14 yoshdaman.
Bo'sh vaqtimda kitob o'qiyman va futbol o'ynayman.</p>

<h2>Nima uchun dasturlashni o'rganyapman?</h2>
<p>Men o'z <mark>mobil o'yinimni</mark> yaratmoqchiman.<br>
Buning uchun avval web-dasturlash asoslarini o'rganyapman.</p>

<h2>Sevimli iqtibosim</h2>
<blockquote>Har bir dasturchi bir paytlar yangi boshlovchi bo'lgan.
</blockquote>

<hr>
<p><small>&copy; 2026 Sardor Aliyev</small></p>
```

- 7 Faylni saqlang (`Ctrl` + `S`). Papkadagi `index.html` ustiga ikki marta bosing — sahifa brauzerda ochiladi.
- 8 Qulayroq usul: VS Code'ga **Live Server** kengaytmasini o'rnatish (chap paneldagi Extensions belgisi → "Live Server" → Install). So'ng faylda o'ng tugma → **Open with Live Server**. Endi har saqlaganingizda brauzer **o'zi yangilanadi**.
- 9 `Ctrl` + `U` bilan o'z sahifangizning kodini brauzerda ko'ring.

Ko'p uchraydigan xatolar

- ❌ Yopuvchi tegni unutish: `<p>Matn` → keyingi butun matn ham paragraf ichida qolib ketishi mumkin.
- ❌ Yopuvchi tegda `/` ni unutish: `<h1>Sarlavha<h1>` — ikkinchisi yana *ochuvchi* teg bo'lib qoladi.
- ❌ Matnni `<head>` ichiga yozish — u ko'rinmaydi. Ko'rinadigan narsa faqat `<body>` da.
- ❌ Faylni `.html` emas, `.txt` kengaytmasi bilan saqlash (`index.html.txt`).
- ❌ Atribut qiymatini qo'shtirnoqsiz yoki "aqlli" qo'shtirnoq (`" "`) bilan yozish — doim oddiy `"` ishlatish.

O'zingizni tekshiring

? Foydalanuvchiga ko'rinadigan barcha mazmun qaysi teg ichida yoziladi?

- A. `<head>`
- B. `<title>`
- C. `<body>`
- D. `<meta>`

? `<a href="https://kun.uz">` da `href` nima?

- A. Teg
- B. Atribut
- C. Element
- D. Izoh

? Qaysi teg yolg'iz (yopuvchi tegi yo'q)?

- A. `<p>`
- B. `<h1>`
- C. `<strong>`
- D. `<br>`

? `<meta charset="UTF-8">` qatori nima uchun kerak?

- A. O'zbekcha harflar, kirill va emoji to'g'ri ko'rinishi uchun
- B. Sahifani tezlashtirish uchun
- C. Sahifaga rang berish uchun
- D. Telefonda ko'rinishi uchun

? Kodda 5 ta bo'sh joy yozsangiz, brauzer nechta ko'rsatadi?

- A. 5 ta
- B. Hech qancha
- C. 1 ta
- D. 10 ta

? Qaysi kod to'g'ri yozilgan?

- A. `<p><em>Salom</p></em>`
- B. `<p><em>Salom</em></p>`
- C. `<p><em>Salom<p></em>`
- D. `<em><p>Salom</em></p>`

Uyga vazifa

1. "Men haqimda" sahifangizga "Sevimli fanlarim" (`h2`) bo'limini qo'shing va har bir fan haqida bitta paragraf yozing.
2. `retsept.html` nomli sahifa yarating: sevimli taomingiz retsepti — sarlavha, kirish paragrafi, "Masalliqlar" va "Tayyorlash" bo'limlari (hozircha paragraflar bilan; ro'yxatlarni keyingi darsda o'rganamiz).
3. Sahifangizda `<sup>` va `<sub>` ishlatib, ikkita matematika va kimyo formulasini yozing.

Yodda tuting

- HTML — sahifaning tuzilishi (skeleti). Dasturlash tili emas, belgilash tili.
- Element = ochuvchi teg + mazmun + yopuvchi teg. Atributlar ochuvchi tegda yoziladi:
`nom="qiymat"`.
- Skelet: `<!DOCTYPE html>` → `<html>` → `<head>` (ko'rinmas) + `<body>` (ko'rinadi).
- `h1` — `h6` — sarlavhalar (ma'no bo'yicha), `p` — paragraf, `strong` / `em` — muhim/urg'u.
- Teglar to'g'ri tartibda yopiladi: oxirgi ochilgan — birinchi yopiladi.

✓ Test javoblari

1. Foydalanuvchiga ko'rinadigan barcha mazmun qaysi teg ichida yoziladi? — **C) <body>**
<head> — ko'rinmaydigan ma'lumotlar, <body> — ko'rinadigan mazmun.
2. da href nima? — **B) Atribut**
a — teg, href — atribut, "https://kun.uz" — atribut qiymati.
3. Qaysi teg yolg'iz (yopuvchi tegi yo'q)? — **D)
**

, <hr>, , <input>, <meta> — yolg'iz teglar.
4. <meta charset="UTF-8"> qatori nima uchun kerak? — **A) O'zbekcha harflar, kirill va emoji to'g'ri ko'rinishi uchun**
UTF-8 — dunyodagi deyarli barcha harf va belgilarni qamrab oluvchi kodlash.
5. Kodda 5 ta bo'sh joy yozsangiz, brauzer nechta ko'rsatadi? — **C) 1 ta**
HTML ketma-ket bo'sh joy va qator o'tishlarni bitta bo'sh joyga aylantiradi (<pre> bundan mustasno).
6. Qaysi kod to'g'ri yozilgan? — **B) <p>Salom</p>**
Oxirgi ochilgan (em) birinchi yopiladi.



03-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Jadval, ro'yxat va forma elementlari

Tartibli va tartibsiz ro'yxatlar, jadvallar (thead, tbody, colspan, rowspan), forma va uning elementlari: input turlari, label, select, textarea, tugmalar.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Tartibli va tartibsiz ro'yxatlar yaratishni;
- Dars jadvali kabi jadvallarni, kataklarni birlashtirishni;
- Foydalanuvchidan ma'lumot oluvchi formalarni: matn, parol, email, tanlash, belgilash maydonlari va tugmalarni;
- Forma ma'lumoti qayerga va qanday yuborilishini.

Ro'yxatlar

Tartibsiz ro'yxat — `<ul>`

ul = *unordered list*. Tartib muhim bo'lmaganda: xarid ro'yxati, ko'nikmalar, menyu. Har bir band — `<li>` (*list item*).

HTML

```
<h3>Bozorlik ro'yxati</h3>
<ul>
  <li>Non</li>
  <li>Sut</li>
  <li>Olma</li>
</ul>
```


Tartibli ro'yxat — `<ol>`

`ol` = *ordered list*. Tartib muhim bo'lganda: retsept qadamlari, reyting, ko'rsatmalar.

```
HTML

<h3>Choy damlash</h3>
<ol>
  <li>Suvni qaynating</li>
  <li>Choynakni issiq suv bilan chayib oling</li>
  <li>Choy soling va qaynoq suv quyding</li>
  <li>5 daqiqa damlang</li>
</ol>

<h3>Olimpiada g'oliblari (Rim raqamlari bilan)</h3>
<ol type="I">
  <li>Malika</li>
  <li>Jasur</li>
  <li>Dilnoza</li>
</ol>

<h3>3-dan boshlanuvchi, teskari ro'yxat</h3>
<ol start="3" reversed>
  <li>Uchinchi</li>
  <li>Ikkinchi</li>
  <li>Birinchi</li>
</ol>
```

Atribut (<code>ol</code> uchun)	Natija
<code>type="1"</code>	1, 2, 3 (standart)
<code>type="a"</code> / <code>type="A"</code>	a, b, c / A, B, C
<code>type="i"</code> / <code>type="I"</code>	i, ii, iii / I, II, III
<code>start="5"</code>	5 dan boshlaydi
<code>reversed</code>	Teskari tartibda sanaydi

Ichma-ich ro'yxat

Ro'yxat ichida ro'yxat — `<li>` ichiga yangi `ul` / `ol` yoziladi:

HTML

```
<ul>
  <li>Aniq fanlar
    <ul>
      <li>Matematika</li>
      <li>Informatika</li>
    </ul>
  </li>
  <li>Tabiiy fanlar
    <ul>
      <li>Fizika</li>
      <li>Kimyo</li>
    </ul>
  </li>
</ul>
```

Ta'riflar ro'yxati — `<dl>`

Lug'at yoki "atama — izoh" juftliklari uchun: `dt` — atama, `dd` — izoh.

HTML

```
<dl>
  <dt>HTML</dt>
  <dd>Sahifa tuzilishini yaratuvchi til</dd>
  <dt>CSS</dt>
  <dd>Sahifa dizaynini beruvchi til</dd>
</dl>
```

Jadvallar

Jadval — **qator va ustunlardan** iborat ma'lumot: dars jadvali, baholar, narxlar. Asosiy teglar:

Teg	Ma'nosi
<code><table></code>	Butun jadval
<code><tr></code>	table row — qator
<code><td></code>	table data — oddiy katak
<code><th></code>	table header — sarlavha katagi (qalin, markazda)
<code><caption></code>	Jadval nomi
<code><thead></code> , <code><tbody></code> , <code><tfoot></code>	Jadvalning bosh, asosiy va pastki qismlari



Eslab qolish uchun

Jadval **qatorma-qator** yoziladi: avval `<tr>` (qator) ochasiz, uning ichiga chapdan o'ngga kataklarni (`<td>`) yozasiz. Ustun degan alohida teg yo'q — ustunlar har qatordagi kataklar tartibidan hosil bo'ladi.

```

<table border="1">
  <caption>8-"A" sinf baholari</caption>
  <thead>
    <tr>
      <th>0' quvchi</th>
      <th>Matematika</th>
      <th>Informatika</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Ali</td>
      <td>5</td>
      <td>5</td>
    </tr>
    <tr>
      <td>Zarina</td>
      <td>4</td>
      <td>5</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th>0' rtacha</th>
      <td>4.5</td>
      <td>5</td>
    </tr>
  </tfoot>
</table>

```



`border="1"` — eski usul, faqat hozircha chegaralarni ko'rish uchun. 7-mavzudan boshlab chegaralarni CSS bilan chiroyli qilamiz.

Kataklarni birlashtirish: `colspan` va `rowspan`

- `colspan="2"` — katak **gorizontal** bo'yicha 2 ta ustunni egallaydi.
- `rowspan="2"` — katak **vertikal** bo'yicha 2 ta qatorni egallaydi.

```

<table border="1">
  <caption>Dars jadvali</caption>
  <tr>
    <th>Vaqt</th><th>Dushanba</th><th>Seshanba</th>
  </tr>
  <tr>
    <td>08:30</td><td>Matematika</td><td rowspan="2">Informatika<br>(juft dars)
  </td>
  </tr>
  <tr>
    <td>09:20</td><td>Ona tili</td>
    <!-- Seshanba katagi yo'q: tepadagi rowspan uni egallab turibdi -->
  </tr>
  <tr>
    <td>10:10</td><td colspan="2">🍎 Katta tanaffus</td>
  </tr>
</table>

```



Birlashtirilgan katak egallagan joyga keyingi qatorlarda katak **yozi olmaydi**, aks holda jadval "siljib" ketadi. Har bir qatordagi kataklar soni (birlashtirishlarni hisobga olganda) teng bo'lishi kerak.



Jadvalni faqat **jadval ko'rinishidagi ma'lumot** uchun ishlating. Sahifa maketini (menyu, ustunlar) jadval bilan qurish — eskirgan usul; buning uchun Flexbox va Grid bor (8-mavzu).

Formalar — foydalanuvchidan ma'lumot olish

Ro'yxatdan o'tish, qidiruv, izoh qoldirish, test topshirish — barchasi **forma** orqali. Forma — frontend va backend o'rtasidagi ko'priq: foydalanuvchi ma'lumot kiritadi, forma uni serverga yuboradi.

HTML

```
<form action="/royxatdan-otish" method="POST">
  <label for="ism">Ismingiz:</label>
  <input type="text" id="ism" name="ism" placeholder="Masalan: Aziz" required>

  <button type="submit">Yuborish</button>
</form>
```

Qism	Vazifasi
<code><form></code>	Barcha maydonlarni o'rab turuvchi "konvert"
<code>action</code>	Ma'lumot qaysi manzilga (serverdagi qaysi sahifaga) yuboriladi
<code>method</code>	<code>GET</code> — ma'lumot manzil satriga qo'shiladi (qidiruv uchun); <code>POST</code> — yashirin yuboriladi (parollar, ro'yxatdan o'tish)
<code><label></code>	Maydon yozuvi. <code>for</code> atributi input'ning <code>id</code> siga teng bo'lsa, yozuv bosilganda maydon faollashadi
<code>name</code>	Eng muhim atribut! Serverga ma'lumot <code>name=qiymat</code> ko'rinishida boradi. <code>name</code> siz maydon yuborilmaydi
<code>placeholder</code>	Maydon bo'sh turganda ko'rindigan kulrang maslahat
<code>required</code>	Majburiy maydon — to'ldirilmasa forma yuborilmaydi

Input turlari

Bitta `<input>` tegi `type` atributiga qarab butunlay boshqacha ko'rinadi:

HTML

```
<form>
  <p>Matn: <input type="text" name="ism"></p>
  <p>Parol: <input type="password" name="parol"></p>
  <p>Email: <input type="email" name="email"></p>
  <p>Son (1-10): <input type="number" name="son" min="1" max="10"></p>
  <p>Telefon: <input type="tel" name="tel" placeholder="+998 90 123 45 67"></p>
  <p>Sana: <input type="date" name="sana"></p>
  <p>Vaqt: <input type="time" name="vaqt"></p>
  <p>Rang: <input type="color" name="rang" value="#4f46e5"></p>
  <p>Diapazon: <input type="range" name="daraja" min="0" max="100"></p>
  <p>Fayl: <input type="file" name="rasm" accept="image/*"></p>

  <p>Jinsingiz:
    <label><input type="radio" name="jins" value="o'g'il"> O'g'il bola</label>
    <label><input type="radio" name="jins" value="qiz"> Qiz bola</label>
  </p>

  <p>Qiziqishlar:
    <label><input type="checkbox" name="qiziqish" value="sport"> Sport</label>
    <label><input type="checkbox" name="qiziqish" value="musiq" checked>
Musiq</label>
    <label><input type="checkbox" name="qiziqish" value="kitob"> Kitob</label>
  </p>
</form>
```

type	Qachon ishlatiladi
text	Oddiy matn: ism, shahar
password	Parol — belgilar nuqta bo'lib ko'rinadi
email	Email — @ borligini brauzer o'zi tekshiradi
number	Faqat son; min, max, step
radio	Bir nechta variantdan faqat bittasi . Bir guruh radiolarda name bir xil bo'lishi shart!
checkbox	Bir nechtasini belgilash mumkin; checked — oldindan belgilangan

date, time, color,
range, file

Sana, vaqt, rang, slayder, fayl tanlash

submit, reset

Yuborish va tozalash tugmalari (eski usul; hozir <button>)

Tanlash ro'yxati va katta matn maydoni

HTML

```
<form>
  <label for="viloyat">Viloyat:</label>
  <select id="viloyat" name="viloyat">
    <option value="">- Tanlang -</option>
    <option value="toshkent">Toshkent</option>
    <option value="samarqand" selected>Samarqand</option>
    <option value="buxoro">Buxoro</option>
    <option value="xorazm">Xorazm</option>
  </select>

  <br><br>
  <label for="fikr">Fikringiz:</label><br>
  <textarea id="fikr" name="fikr" rows="5" cols="40" placeholder="Bu yerga
yozing..."></textarea>

  <br>
  <button type="submit">Yuborish</button>
  <button type="reset">Tozalash</button>
</form>
```

- `<select>` — ochiladigan ro'yxat; har bir variant `<option>`. Serverga `value` qiymati yuboriladi, foydalanuvchi esa teg ichidagi matnni ko'radi.
- `<textarea>` — ko'p qatorli matn (izoh, xat). `rows` — balandlik (qatorlarda), `cols` — kenglik.
- `<button type="submit">` — formani yuboradi; `type="reset"` — barcha maydonlarni tozalaydi.

Brauzer tekshiruvi (validatsiya)

Brauzer ba'zi tekshiruvlarni o'zi bajaradi va xato bo'lsa formani yubormaydi:

- `required` — bo'sh bo'lmasin;
- `minlength="8"`, `maxlength="20"` — matn uzunligi;

- `min`, `max` — son yoki sana chegarasi;
- `type="email"` — email formati.



Bu tekshiruvlar faqat foydalanuvchiga qulaylik uchun. Ayyor foydalanuvchi ularni chetlab o'tishi mumkin, shuning uchun ma'lumotlar **serverda ham** tekshiriladi (37-mavzu, Django Forms).

Amaliy mashg'ulot: "To'garakka yozilish" sahifasi

- 1 `togarak.html` faylini yarating va HTML skeletini yozing.
- 2 Sahifa tepasiga `<h1>` sarlavha va to'garaklar haqida qisqa paragraf yozing.
- 3 To'garaklar jadvalini yarating: ustunlar — "To'garak", "Kunlar", "Vaqt", "Joylar soni". Kamida 4 qator. Bitta to'garakni ikki kunda o'tishini `rowspan` bilan ko'rsating.
- 4 "Nima uchun yozilish kerak?" — `<ul>` ro'yxati (3 ta sabab).
- 5 "Qanday yozilish mumkin?" — `<ol>` (qadamlar).
- 6 Ro'yxatdan o'tish formasini yozing:

```

<h2>Ro'yxatdan o'tish</h2>
<form action="#" method="POST">
  <p>
    <label for="fio">To'liq ism *</label><br>
    <input type="text" id="fio" name="fio" required minlength="3">
  </p>
  <p>
    <label for="sinf">Sinf *</label><br>
    <select id="sinf" name="sinf" required>
      <option value="">- Tanlang -</option>
      <option>7</option><option>8</option><option>9</option>
    </select>
  </p>
  <p>
    <label for="tel">Ota-ona telefoni *</label><br>
    <input type="tel" id="tel" name="tel" placeholder="+998 __ __ __ __"
required>
  </p>
  <p>To'garak *<br>
    <label><input type="radio" name="togarak" value="robototexnika"
required> Robototexnika</label><br>
    <label><input type="radio" name="togarak" value="dasturlash">
Dasturlash</label><br>
    <label><input type="radio" name="togarak" value="shaxmat">
Shaxmat</label>
  </p>
  <p>Qaysi kunlar qulay?<br>
    <label><input type="checkbox" name="kun" value="du"> Dushanba</label>
    <label><input type="checkbox" name="kun" value="chor">
Chorshanba</label>
    <label><input type="checkbox" name="kun" value="ju"> Juma</label>
  </p>
  <p>
    <label for="izoh">Qo'shimcha izoh</label><br>
    <textarea id="izoh" name="izoh" rows="3" cols="40"></textarea>
  </p>
  <p>
    <label><input type="checkbox" name="rozilik" required> Qoidalar bilan
tanishdim</label>
  </p>
  <button type="submit">Yozilish</button>

```

```
<button type="reset">Tozalash</button>
</form>
```

- 7 Sahifani brauzerda oching. Formani bo'sh holda yuborib ko'ring — brauzer qaysi maydonlarni ko'rsatadi? Telefon raqamisiz yuborib ko'ring.
- 8 **Tajriba:** `method="POST"` ni `method="GET"` ga o'zgartiring va formani to'ldirib yuboring. Manzil satriga qarang — barcha ma'lumotlar `?fio=...&sinf=8&...` ko'rinishida ko'rinadi. Shuning uchun parollar uchun `GET` ishlatilmaydi!

Ko'p uchraydigan xatolar

- ✗ `<li>` ni `ul` / `ol` siz yozish — ro'yxat bandi doim ro'yxat ichida bo'ladi.
- ✗ Radio tugmalarga turli `name` berish — hammasini birdaniga belgilash mumkin bo'lib qoladi.
- ✗ `<label for="...">` dagi qiymat input'ning `id` siga mos kelmasligi.
- ✗ Input'da `name` yo'qligi — ma'lumot serverga bormaydi.
- ✗ Jadvalda `<td>` ni `<tr>` siz to'g'ridan-to'g'ri `<table>` ichiga yozish.

O'zingizni tekshiring

? Retsept qadamlari uchun qaysi ro'yxat mos?

- A. `<ul>`
- B. `<ol>`
- C. `<dl>`
- D. `<table>`

? Jadvalda yangi qator qaysi teg bilan boshlanadi?

- A. `<td>`
- B. `<th>`
- C. `<tr>`
- D. `<row>`

? Katakni ikki ustun kengligida birlashtirish uchun qaysi atribut kerak?

- A. `colspan="2"`
- B. `rowspan="2"`
- C. `span="2"`
- D. `width="2"`

? "Jinsingiz" savoliga faqat bitta javob tanlash uchun qaysi input turi kerak?

- A. `checkbox`
- B. `text`
- C. `select multiple`
- D. `radio`

? Forma ma'lumoti serverga qaysi atribut nomi bilan boradi?

- A. `id`
- B. `name`
- C. `placeholder`
- D. `class`

? Parol yuboradigan forma uchun qaysi `method` to'g'ri?

- A. `GET`
- B. `SEND`
- C. `POST`
- D. `PUT`



Uyga vazifa

- O'z sinfingizning to'liq **haftalik dars jadvalini** HTML jadval sifatida yarating (`caption` , `thead` , tanaffuslar uchun `colspan`).
- 2-mavzudagi retsept sahifangizni yangilang: masallqlarni `ul` , tayyorlash bosqichlarini `ol` bilan yozing.
- "Kutubxonaga a'zo bo'lish" formasini yarating: ism, tug'ilgan sana, email, sevimli janr (`select`), qaysi tillarda o'qiydi (`checkbox`), izoh (`textarea`).



Yodda tuting

- `ul` — tartibsiz, `ol` — tartibli ro'yxat; bandlar `li` .
- Jadval: `table` → `tr` (qator) → `td/th` (katak); `colspan` — gorizontal, `rowspan` — vertikal birlashtirish.
- Forma: `action` (qayerga), `method` (GET/POST), har bir maydonda `name` va `label` .
- `radio` — bitta tanlov (bir xil name), `checkbox` — bir nechta, `select` — ro'yxatdan, `textarea` — ko'p qatorli matn.

✓ Test javoblari

1. Retsept qadamlari uchun qaysi ro'yxat mos? — **B) **

Qadamlar tartibi muhim — shuning uchun tartibli ro'yxat (ol).

2. Jadvalda yangi qator qaysi teg bilan boshlanadi? — **C) <tr>**

tr = table row. Kataklar (td/th) uning ichida.

3. Katakni ikki ustun kengligida birlashtirish uchun qaysi atribut kerak? — **A) colspan="2"**

col = ustun (gorizontal), row = qator (vertikal).

4. "Jinsingiz" savoliga faqat bitta javob tanlash uchun qaysi input turi kerak? — **D) radio**

Radio — bir guruhdan bitta tanlov. Guruhdagi barcha radiolarning namei bir xil bo'lishi kerak.

5. Forma ma'lumoti serverga qaysi atribut nomi bilan boradi? — **B) name**

Server name=qiymat juftliklarini oladi. id esa label va CSS uchun.

6. Parol yuboradigan forma uchun qaysi method to'g'ri? — **C) POST**

GET'da ma'lumot manzil satrida ochiq ko'rinadi va brauzer tarixida saqlanadi.



04-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Havolalar va navigatsiya. Rasmlar va media elementlari

Giperhavolalar (a, href, target), absolyut va nisbiy yo'llar, sahifa ichidagi havolalar, navigatsiya menyusi; rasmlar (img, alt), audio, video va iframe.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Havolalar orqali sahifalarni bir-biriga bog'lashni;
- Absolyut va nisbiy yo'llar farqini (bu juda ko'p xatoning sababi!);
- Sahifa ichidagi bo'limlarga o'tish, email va telefon havolalarini;
- Bir nechta sahifali sayt va navigatsiya menyusini yaratishni;
- Rasm, audio, video va YouTube videosini sahifaga joylashni.

Havola — webning "H" harfi

HTML'dagi "HyperText" — havolalar orqali bir-biriga bog'langan matn degani. Havolalar bo'lmaganda web — bir-biridan ajralgan alohida hujjatlar bo'lardi. Havola `<a>` (*anchor* — langar) tegi bilan yaratiladi:

HTML

```
<a href="https://uz.wikipedia.org">O'zbekcha Vikipediya</a>
```

- `href` (*hypertext reference*) — havola **qayerga** olib borishi.
- Teglar orasidagi matn — foydalanuvchi bosadigan yozuv.

Yangi oynada ochish

HTML

```
<a href="https://github.com" target="_blank" rel="noopener">GitHub (yangi yorliqda ochiladi)</a>
```

`target="_blank"` — havolani yangi yorliqda (tab) ochadi. Tashqi saytlarga havola berganda qulay: foydalanuvchi sizning saytingizni yo'qotib qo'ymaydi. `rel="noopener"` — xavfsizlik uchun qo'shiladi (ochilgan sayt sizning sahifangizni boshqara olmasligi uchun).

Absolyut va nisbiy yo'llar

Bu darsning eng muhim qismi. Havola, rasm, CSS fayl — barchasi qandaydir **yo'l** (manzil) bilan ko'rsatiladi.

Absolyut yo'l

To'liq internet manzili, `https://` bilan boshlanadi. **Boshqa saytlarga** havola uchun ishlatiladi.

HTML

```
<a href="https://www.w3schools.com/html/">W3Schools HTML darsliklari</a>
```

Nisbiy yo'l

Joriy fayldan **nisbatan** ko'rsatilgan yo'l. **O'z saytingiz ichidagi** sahifa va fayllar uchun ishlatiladi. Quyidagi papka tuzilmasini olaylik:

Matn

```
mening-saytim/
├── index.html          ← bosh sahifa
├── haqida.html
├── rasmlar/
│   ├── logo.png
│   └── men.jpg
└── sahifalar/
    ├── loyihalar.html
    └── aloqa.html
```

Qaysi fayldan	Qaysi faylga	Nisbiy yo'l	Tushuntirish
index.html	haqida.html	haqida.html	Bir papkada — shunchak nomi
index.html	rasmlar/logo.png	rasmlar/logo.png	Ichki papkaga kirish: papka/fayl
index.html	sahifalar/aloqa.html	sahifalar/aloqa.html	Ichki papkaga kirish
sahifalar/aloqa.html	index.html	../index.html	../ — bir papka yuqoriga chiqish
sahifalar/aloqa.html	rasmlar/men.jpg	../rasmlar/men.jpg	Yuqoriga chiqib, keyin boshqa papkaga kirish
sahifalar/aloqa.html	sahifalar/loyihalar.html	loyihalar.html	Bir papkada



O'xshatish: maktab binosi

Siz 8-"A" sinfidasiz (`sahifalar/`). Do'stingizga "qo'shni sinfga o't" desangiz — bu **nisbiy** yo'l (`loyihalar.html`). "Koridorga chiq, keyin kutubxonaga kir" — `../kutubxona/` . **Absolyut** yo'l esa maktabning to'liq pochta manzilini aytish: "Samarqand shahri, 5-maktab, 2-qavat, 8-A sinf".



Eng ko'p uchraydigan xato

`C:\Users\Ali\Desktop\sayt\rasm.jpg` kabi kompyuteringizdagi to'liq yo'lni yozmang! Saytni hostingga joylaganingizda yoki boshqa kompyuterda ochganda bunday yo'l **ishlamaydi**. Har doim nisbiy yo'l ishlatang. Shuningdek, fayl nomlarida bo'sh joy, katta harf va o'zbekcha belgilarni (`o'`) ishlatmang: `Mening Rasmim.JPG` ❌ → `mening-rasmim.jpg` ✅.

Boshqa havola turlari

HTML

```
<!-- Sahifa ichidagi bo'limga o'tish: href="#id" -->
<a href="#aloqa">Aloqa bo'limiga o'tish ↓</a>

<p style="height: 400px">... uzun matn ...</p>

<h2 id="aloqa">Aloqa</h2>

<!-- Email yozish (pochta dasturi ochiladi) -->
<a href="mailto:maktab5@mail.uz">Bizga xat yozing</a><br>

<!-- Telefon qilish (telefonda qo'ng'iroq oynasi ochiladi) -->
<a href="tel:+998901234567">+998 90 123 45 67</a><br>

<!-- Faylni yuklab olish -->
<a href="dars-jadvali.pdf" download>Dars jadvalini yuklab olish</a><br>

<!-- Sahifa tepasiga qaytish -->
<a href="#">↑ Yuqoriga</a>
```

Navigatsiya menyusi

Ko'p sahifali saytda har bir sahifaning tepasida bir xil menyu bo'ladi. U `<nav>` tegi ichida havolalar ro'yxati sifatida yoziladi:

HTML

```
<nav>
  <ul>
    <li><a href="index.html">Bosh sahifa</a></li>
    <li><a href="haqida.html">Men haqimda</a></li>
    <li><a href="sahifalar/loyihalar.html">Loyihalar</a></li>
    <li><a href="sahifalar/aloqa.html">Aloqa</a></li>
  </ul>
</nav>
```

Hozircha bu oddiy ro'yxatga o'xshaydi — 8-mavzuda Flexbox bilan uni chiroyli gorizontal menyuga aylantiramiz.

Rasmlar

HTML

```

```

Atribut	Vazifasi
<code>src</code>	(<i>source</i>) Rasm fayli yo'li — nisbiy yoki absolyut
<code>alt</code>	(<i>alternative</i>) Rasmning matnli tavsifi . Majburiy!
<code>width</code> , <code>height</code>	Kenglik va balandlik (piksellarda)
<code>title</code>	Sichqoncha ustiga borganda chiqadigan yozuv
<code>loading="lazy"</code>	Rasm faqat unga yaqinlashganda yuklanadi — sahifa tezroq ochiladi

alt nima uchun shunchalik muhim?

- Rasm yuklanmasa (internet sekin, fayl topilmadi), uning o'rnida `alt` matni ko'rinadi.
- **Ko'zi ojiz** foydalanuvchilarning ekran o'quvchi dasturlari aynan shu matnni ovoz chiqarib o'qiydi.
- **Google** rasmda nima borligini `alt` orqali tushunadi (SEO, 11-mavzu).

Yaxshi `alt` rasmda **nima tasvirlanganini** aytadi: `alt="rasm"` ❌ → `alt="Registon maydoni kechqurun chiroqlarda"` ✅.



Rasm o'lchami

Faqat `width` ni bersangiz, brauzer balandlikni proporsiyani saqlagan holda o'zi hisoblaydi. Ikkalasini noto'g'ri bersangiz, rasm cho'zilib, buzilib ko'rinadi.

Rasm formatlari

Format	Qachon
<code>.jpg</code> / <code>.jpeg</code>	Fotosuratlar
<code>.png</code>	Shaffof fonli rasmlar, skrinshotlar
<code>.svg</code>	Logotip va ikonkalar — har qanday o'lchamda tiniq
<code>.webp</code>	Zamonaviy format: JPG/PNG dan 25–35% yengil
<code>.gif</code>	Oddiy animatsiyalar

Rasm + izoh: `figure`

HTML

```
<figure>
  
  <figcaption>1-rasm. Maktabimiz hovlisi bahorda</figcaption>
</figure>
```

Rasmni havola qilish

HTML

```
<a href="index.html">
  
</a>
```

Audio va video

HTML

```
<h3>Audio</h3>
<audio controls>
  <source src="https://interactive-examples.mdn.mozilla.net/media/cc0-audio/t-
rex-roar.mp3" type="audio/mpeg">
  Brauzeringiz audioni qo'llab-quvvatlamaydi.
</audio>

<h3>Video</h3>
<video controls width="400" poster="https://picsum.photos/seed/video/400/225">
  <source src="https://interactive-examples.mdn.mozilla.net/media/cc0-
videos/flower.mp4" type="video/mp4">
  Brauzeringiz videoni qo'llab-quvvatlamaydi.
</video>
```

Atribut	Vazifasi
controls	Boshqaruv tugmalarini (play, pauza, ovoz) ko'rsatish
autoplay	Avtomatik boshlash (brauzerlar odatda faqat muted bilan ruxsat beradi)
muted	Ovozsiz
loop	Takroriy ijro
poster	Video boshlanguncha ko'rinadigan rasm

Teglar orasidagi matn — eski brauzerlar uchun zaxira xabar.

YouTube video va xarita joylash: `iframe`

`<iframe>` — sahifa ichida boshqa saytni "oyna" sifatida ko'rsatadi. YouTube'da video ostidagi **Ulashish** → **Joylashtirish (Embed)** tugmasini bossangiz, tayyor kod beriladi:

HTML

```
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="YouTube video"
  allowfullscreen></iframe>

<!-- Google Xaritadan: "Ulashish" → "Xaritani joylashtirish" →
<iframe src="https://www.google.com/maps/embed?pb=..." width="400" height="300"
loading="lazy"></iframe>
```

Amaliy mashg'ulot: 3 sahifali shaxsiy sayt

- 1 Quyidagi papka tuzilmasini yarating:

Matn

```
vizitka/
├── index.html
├── hobbilar.html
├── aloqa.html
├── rasmlar/
│   └── (2-3 ta rasm: men.jpg, hobbi.jpg ...)
```

- 2 Uchala sahifaga ham **bir xil** navigatsiya menyusini qo'ying (nusxa olib qo'yish mumkin):

HTML

```
<nav>
  <a href="index.html">🏠 Bosh sahifa</a> |
  <a href="hobbilar.html">⚽ Hobbilarim</a> |
  <a href="aloqa.html">✉ Aloqa</a>
</nav>
<hr>
```

- 3 **index.html:** ismingiz (`h1`), rasmingiz (`img` + mazmunli `alt`), o'zingiz haqingizda 2 ta paragraf.
- 4 **hobbilar.html:** 3 ta hobbi — har biri (`h2` , `figure` + `figcaption`) bilan rasm va paragraf. Bitta hobbingiz haqida YouTube video joylashtiring.
- 5 **aloqa.html:** email (`mailto:`), telefon (`tel:`), ijtimoiy tarmoqlarga havolalar (`target="_blank"`) va 3-mavzudagidek kichik "Xabar yuborish" formasi.
- 6 Barcha havolalarni bosib tekshiring. Biror sahifa ochilmasa — yo'lni tekshiring (fayl nomi, katta-kichik harf, papka).
- 7 **Qo'shimcha:** uzun (`hobbilar.html`) sahifasining tepasiga "Tezkor o'tish" havolalarini qo'ying (`href="#futbol"`) va har bir bo'lim oxirida "↑ Yuqoriga" havolasini.



Bu sizning **1-loyihangiz** — "Shaxsiy vizitka sahifasi". Keyingi darslarda unga semantik teglar, so'ng CSS dizayn qo'shib, oxirida internetga joylaymiz.

Ko'p uchraydigan xatolar

- ❌ Rasm ko'rinmayapti → fayl nomida xato (`Men.JPG` va `men.jpg` serverda **har xil** fayl!), noto'g'ri papka yoki `../` unutilgan.
- ❌ `href="www.google.com"` — `https://` siz brauzer buni nisbiy yo'l deb o'ylaydi va sizning saytingizdan "www.google.com" faylini qidiradi.
- ❌ `alt` ni yozmaslik yoki `alt="rasm"` deb yozish.
- ❌ Juda katta (5–10 MB) rasmlarni to'g'ridan-to'g'ri qo'yish — sahifa sekin ochiladi. Rasmni oldin kichraytiring (masalan, squoosh.app saytida).

O'zingizni tekshiring

? Havolani yangi yorliqda ochish uchun qaysi atribut kerak?

- A. `href="_new"`
- B. `open="tab"`
- C. `target="_blank"`
- D. `rel="new"`

? `sahifalar/aloqa.html` faylidan `index.html` (bir papka yuqorida) ga to'g'ri havola qaysi?

- A. `index.html`
- B. `../index.html`
- C. `/sahifalar/index.html`
- D. `sahifalar/index.html`

? `alt` atributi nima uchun kerak?

- A. Rasm o'lchamini belgilash
- B. Rasmga havola qo'yish
- C. Rasmni tezroq yuklash
- D. Rasm ko'rinmaganda va ko'zi ojizlar uchun uning matnli tavsifi

? Sahifadagi `id="narxlar"` bo'limiga o'tuvchi havola qaysi?

- A. `<a href="#narxlar">`
- B. `<a href="narxlar">`
- C. `<a id="#narxlar">`
- D. `<a href=".narxlar">`

? Videoda play/pauza tugmalari ko'rinishi uchun qaysi atribut kerak?

- A. `autoplay`
- B. `loop`
- C. `controls`
- D. `buttons`

? Logotip uchun eng mos rasm formati qaysi?

- A. JPG
- B. SVG
- C. GIF
- D. BMP



Uyga vazifa

- Vizitka saytingizga to'rtinchi sahifa — `sahifalar/kitoblar.html` qo'shing (ichki papkada!). Undan bosh sahifaga va rasmlarga to'g'ri havolalar bering (`../`).
- Sevimli 3 ta kitob haqida: muqova rasmi (`figure`), qisqa tavsif va kitob haqidagi tashqi saytga havola.
- Maktabingiz joylashgan joyni Google Xaritadan `iframe` bilan aloqa sahifasiga joylang.

✓ Yodda tuting

- `<a href="...">` — havola; `target="_blank"` — yangi yorliqda.
- Tashqi saytlarga — **absolyut** (`https:// ...`), o'z fayllaringizga — **nisbiy** yo'l; `../` — bir papka yuqoriga.
- `#id` — sahifa ichidagi bo'limga, `mailto:` — email, `tel:` — telefon.
- `<img src alt>` — rasm; `alt` doim mazmunli bo'lsin.
- `<audio>`, `<video controls>`, `<iframe>` — media va boshqa saytlarni joylash.

✓ Test javoblari

1. Havolani yangi yorliqda ochish uchun qaysi atribut kerak? — **C) target="_blank"**
target="_blank" + xavfsizlik uchun rel="noopener".
2. sahifalar/aloqa.html faylidan index.html (bir papka yuqorida) ga to'g'ri havola qaysi? — **B) ../index.html**
../ — bir papka yuqoriga chiqish.
3. alt atributi nima uchun kerak? — **D) Rasm ko'rinmaganda va ko'zi ojizlar uchun uning matnli tavsifi**
Shuningdek, Google rasm mazmunini alt orqali tushunadi.
4. Sahifadagi id="narxlar" bo'limiga o'tuvchi havola qaysi? — **A) **
belgisi — sahifa ichidagi id'ga ishora.
5. Videoda play/pauza tugmalari ko'rinishi uchun qaysi atribut kerak? — **C) controls**
controls — boshqaruv paneli.
6. Logotip uchun eng mos rasm formati qaysi? — **B) SVG**
SVG — vektorli format, har qanday o'lchamda tiniq va juda yengil.



05-MAVZU · 42 TADAN · I BOB. HTML VA CSS

HTML5 semantik teglari va ularning roli

Semantik HTML nima, header, nav, main, section, article, aside, footer teglarining vazifalari, div va span farqi, SEO va foydalanish qulayligi (accessibility) uchun ahamiyati.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- "Semantik" so'zining ma'nosini va nima uchun muhimligini;
- Sahifaning asosiy qismlarini ifodalovchi HTML5 teglarini;
- `section` va `article`, `div` va `span` farqini;
- Semantik teglar Google (SEO) va ko'zi ojiz foydalanuvchilarga qanday yordam berishini;
- Vizitka saytingizni to'g'ri semantik tuzilmaga keltirishni.

"Semantik" nima degani?

Semantika — ma'no. **Semantik teg** — nomi uning **ichidagi mazmun nima ekanini** aytib turadigan teg.

- `<div>` — hech narsa demaydi: "bu shunchaki quti".
- `<nav>` — "bu yerda navigatsiya (menyu) bor".
- `<footer>` — "bu sahifaning pastki qismi".

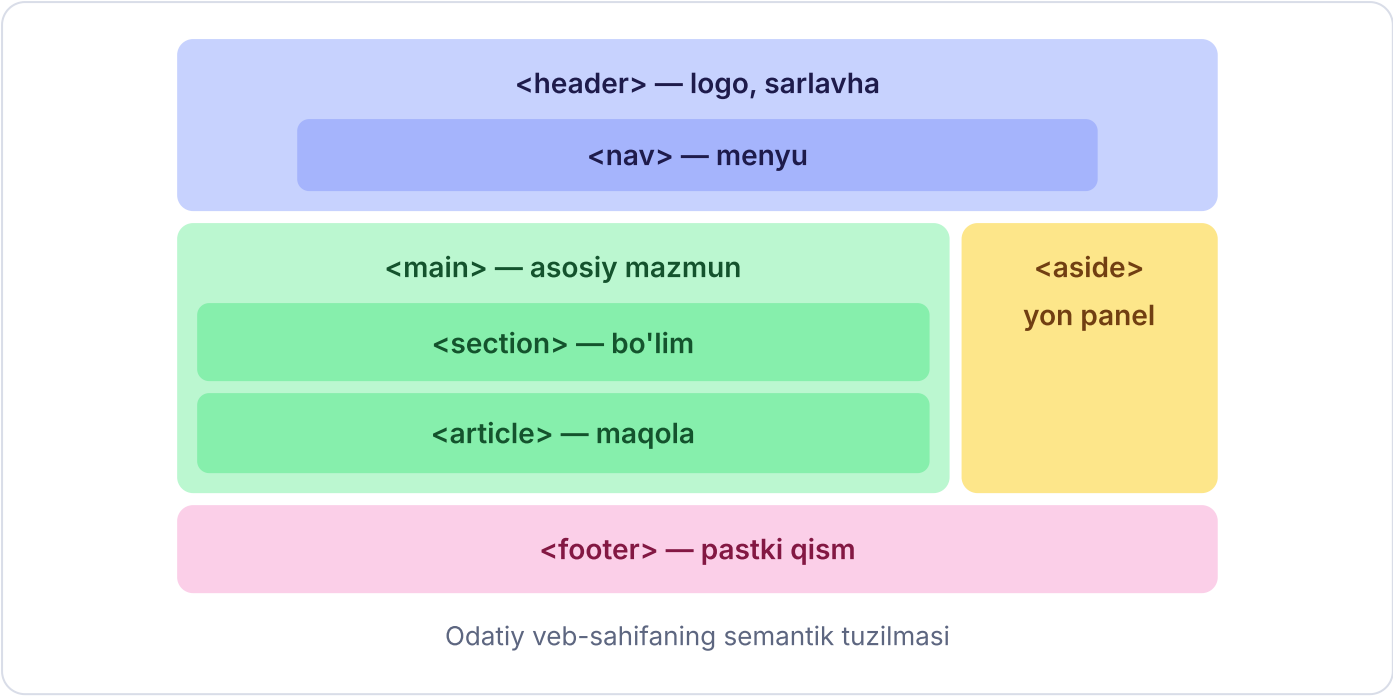


O'xshatish: ko'chib o'tish qutilari

Ko'chib o'tayotganda barcha qutilarga "quti" deb yozsangiz (`div`), yangi uyda nima qayerdaligini topish uchun hammasini ochishga to'g'ri keladi. Agar "Oshxona idishlari", "Kitoblar", "Kiyimlar" deb yozsangiz (`header`, `article`, `footer`) — hamma, hatto begona odam ham, darhol tushunadi. Semantik teglar — qutilarga yozilgan yorliqlar.

2014-yilgacha (HTML4) saytlar asosan `<div>` lardan qurilgan: `<div id="header">`, `<div class="menu">`. **HTML5** bu keng tarqalgan qismlar uchun maxsus teglarni qo'shdi.

Asosiy semantik teglar



Teg	Vazifasi	Ichida odatda nima bo'ladi
<code><header></code>	Sahifa yoki bo'limning kirish qismi	Logo, sayt nomi, menyu, qidiruv
<code><nav></code>	Asosiy navigatsiya havolalari	Sayt menyusi, mundarija
<code><main></code>	Sahifaning asosiy, noyob mazmuni . Sahifada faqat bitta	Maqolalar, mahsulotlar, asosiy matn
<code><section></code>	Mavzu bo'yicha guruhlangan bo'lim (odatda sarlavhasi bor)	"Biz haqimizda", "Xizmatlar", "Narxlar"
<code><article></code>	Mustaqil , alohida olib tarqatsa ham tushunarli mazmun	Yangilik, blog posti, mahsulot kartasi, izoh
<code><aside></code>	Asosiy mazmunga qo'shimcha , yondosh ma'lumot	Yon panel, "O'xshash maqolalar", reklama

<code><footer></code>	Sahifa yoki bo'limning pastki qismi	Mualliflik huquqi, aloqa, ijtimoiy tarmoqlar
<code><figure></code> / <code><figcaption></code>	Rasm, diagramma va uning izohi	Rasm + "1-rasm. ..."
<code><time></code>	Sana yoki vaqt	<code><time datetime="2026-09-01">1-sentabr</time></code>
<code><address></code>	Aloqa ma'lumotlari	Manzil, email, telefon

section va article — farqi nimada?

Oddiy savol bering: **"Bu qismni boshqa saytga alohida olib qo'ysam, o'zi tushunarli bo'ladimi?"**

- **Ha** → `<article>`. Masalan: yangilik maqolasi, mahsulot kartasi, foydalanuvchi izohi, forumdagi post.
- **Yo'q, u sahifaning bir qismi** → `<section>`. Masalan: "Bizning xizmatlar" bo'limi faqat shu sayt kontekstida ma'noli.

Ular bir-birining ichida bo'lishi mumkin: "Yangiliklar" `section` i ichida bir nechta `article` ; yoki uzun `article` ichida bir nechta `section` .

div va span — ma'nosiz konteynerlar

Semantik teglar bor ekan, `div` kerak emasmi? Kerak! Ular **faqat dizayn uchun** guruhlash kerak bo'lganda ishlatiladi:

- `<div>` — **blok** konteyner (yangi qatordan boshlanadi, butun kenglikni egallaydi).
- `<span>` — **qator ichidagi** konteyner (matnning bir bo'lagiga rang berish uchun).

HTML

```
<div class="karta">
  <p>Bugun havo <span style="color: orange; font-weight: bold">+28°C</span>,
  quyoshli.</p>
</div>
```



Qoida

Avval mos semantik teg bormi — o'ylang. Agar hech biri mos kelmasa, faqat CSS uchun guruhlash kerak bo'lsa — `div` / `span`.

Blok va qator ichidagi (inline) elementlar

	Blok elementlar	Qator ichidagi (inline) elementlar
Misollar	<code>div, p, h1-h6, ul, ol, li, table, form, header, section, article...</code>	<code>span, a, strong, em, img, input, label, code...</code>
Joylashuvi	Doim yangi qatordan boshlanadi	Matn bilan bir qatorda turadi
Kengligi	Butun mavjud kenglikni egallaydi	Faqat o'z mazmunicha

HTML

```
<div style="background: lightblue">Men blokman — butun kenglikni  
egallayman</div>  
<div style="background: lightgreen">Men ham blokman — yangi qatordan  
boshlandim</div>  
<span style="background: pink">Men inline'man</span>  
<span style="background: gold">Men ham — yoningizda turibman</span>
```

Semantika nima uchun muhim?

1. SEO — qidiruv tizimlari uchun

Google'ning roboti sahifani "ko'zi" bilan emas, kod orqali o'qiydi. `<main>` va `<article>` unga "asosiy mazmun shu yerda", `<nav>` va `<footer>` esa "bular takrorlanuvchi qismlar" deb aytadi. Natijada sahifangiz qidiruvda to'g'riroq va yuqoriroq chiqadi.

2. Accessibility (foydalanish qulayligi)

Ko'zi ojiz odamlar **ekran o'quvchi** dasturlar (screen reader) yordamida internetdan foydalanadi. Bu dasturlar semantik teglar orqali "Navigatsiyaga o'tish", "Asosiy mazmunga o'tish", "Keyingi maqola" kabi buyruqlarni beradi. `div` lardan qurilgan sahifada esa ular uchun hamma narsa bir xil "quti".

3. Kodni o'qish va jamoada ishlash

Kodingizni ochgan boshqa dasturchi (yoki bir oydan keyingi o'zingiz!) `<footer>` ni ko'rib, darhol nima ekanini tushunadi.

Misol: "div sho'rva"dan semantik kodga

❌ **Oldin** — hammasi `div`:

HTML

```
<div id="header">
  <div class="logo">Maktab №5</div>
  <div class="menu">
    <a href="#">Bosh sahifa</a> <a href="#">Yangiliklar</a>
  </div>
</div>
<div id="content">
  <div class="news">
    <div class="title">Bilimlar bellashuvi</div>
    <div class="text">8-sinflar o'rtasida...</div>
  </div>
</div>
<div id="footer">© 2026</div>
```

✅ **Keyin** — semantik:


```

<header>
  <h1>Maktab №5</h1>
  <nav>
    <a href="#">Bosh sahifa</a> | <a href="#">Yangiliklar</a>
  </nav>
</header>

<main>
  <section>
    <h2>So'nggi yangiliklar</h2>

    <article>
      <h3>Bilimlar bellashuvi</h3>
      <p><time datetime="2026-10-02">2-oktabr, 2026</time></p>
      <p>8-sinflar o'rtasida informatika fanidan bilimlar bellashuvi bo'lib
o'tdi.
      G'olib – 8-"B" sinf jamoasi!</p>
    </article>

    <article>
      <h3>Yangi kompyuter xonasi</h3>
      <p><time datetime="2026-09-15">15-sentabr, 2026</time></p>
      <p>Maktabimizda 20 ta kompyuterli zamonaviy sinf xonasi ochildi.</p>
    </article>
  </section>
</main>

<aside>
  <h2>E'lonlar</h2>
  <p>0ta-onalar majlisi – shanba, soat 10:00.</p>
</aside>

<footer>
  <address>Samarqand sh., Mustaqillik ko'chasi, 5-uy · <a
href="tel:+998662000000">+998 66 200 00 00</a></address>
  <p><small>&copy; 2026 Maktab №5</small></p>
</footer>

```

Brauzerda ikkala variant **bir xil** ko'rinadi — farq kod ichida. Lekin ikkinchisini Google ham, ekran o'quvchi ham, boshqa dasturchi ham to'g'ri tushunadi.

SEO uchun `head` qismi

Semantik `body` bilan birga `head` ichida ham to'g'ri ma'lumot bo'lishi kerak:

HTML

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Maktab №5 – Samarqand shahridagi umumta'lim maktabi</title>
  <meta name="description" content="Maktab №5 yangiliklari, dars jadvali,
to'garaklar va ustozlar haqida ma'lumot.">
</head>
```

- `<title>` — Google natijasidagi ko'k sarlavha. Har bir sahifada **har xil** va mazmunli bo'lsin.
- `<meta name="description">` — sarlavha ostidagi qisqa tavsif (150–160 belgi).

Amaliy mashg'ulot: vizitkani semantik qilish

- 1 4-mavzuda yaratgan `vizitka/index.html` faylini oching.
- 2 Navigatsiya menyusini `<header>` ichiga, menyuni esa `<nav>` ichiga o'rang. Ismingizni `<header>` ichidagi logotip sifatida qo'ying.
- 3 Sahifaning asosiy mazmunini `<main>` ichiga oling.
- 4 "Men haqimda", "Ko'nikmalarim", "Maqsadlarim" qismlarini alohida `<section>` larga ajrating, har birida `<h2>` sarlavha bo'lsin.
- 5 `hobbilar.html` da har bir hobbini `<article>` qiling (u alohida olib tarqatsa ham tushunarli).
- 6 Sahifa oxiriga `<footer>` qo'shing: mualliflik huquqi (`©`) va `<address>` ichida aloqa.
- 7 "Qiziqarli fakt" kabi qo'shimcha ma'lumotni `<aside>` ga joylang.
- 8 Har bir sahifaga mazmunli `<title>` va `<meta name="description">` yozing.
- 9 **Tekshirish:** validator.w3.org saytiga kodingizni joylab, "Check" bosing. Xatolar ro'yxati bo'sh bo'lguncha tuzating.

Ko'p uchraydigan xatolar

- ❌ Sahifada bir nechta `<main>` — faqat bitta bo'ladi.
- ❌ Har bir havola guruhini `<nav>` qilish — `nav` faqat **asosiy** navigatsiya uchun.
- ❌ Faqat dizayn uchun `<section>` ishlatish — sarlavhasi yo'q va mavzusi aniq bo'lmasa, bu `div`.
- ❌ `<header>` ni `<head>` bilan adashtirish: `head` — ko'rinmas meta-ma'lumot, `header` — ko'rinadigan sahifa tepasi.

O'zingizni tekshiring

? Sahifaning asosiy, noyob mazmuni qaysi teg ichida bo'ladi?

- A. `<section>`
- B. `<main>`
- C. `<body>`
- D. `<header>`

? Blogdagi alohida post uchun eng mos teg qaysi?

- A. `<article>`
- B. `<aside>`
- C. `<nav>`
- D. `<span>`

? "O'xshash maqolalar" yon paneli uchun qaysi teg mos?

- A. `<footer>`
- B. `<main>`
- C. `<section>`
- D. `<aside>`

? Qaysi element **inline** (qator ichida)?

- A. `<div>`
- B. `<p>`
- C. `<span>`
- D. `<section>`

? Semantik teglar nima uchun foydali EMAS?

- A. Google sahifani yaxshiroq tushunadi
- B. Sahifa avtomatik chiroyli bo'lib qoladi
- C. Ekran o'quvchi dasturlar qulayroq ishlaydi
- D. Kod o'qilishi osonlashadi



Uyga vazifa

1. Sevimli yangiliklar saytingizni oching, `F12` → Elements orqali kodini ko'ring. Qaysi semantik teglar ishlatilganini yozib chiqing.
2. "Sinfimiz gazetasi" sahifasini yarating: `header` (gazeta nomi + `nav`), `main` ichida 3 ta `article` (yangilik), `aside` (tug'ilgan kunlar), `footer`. Har bir yangilikda `<time>` ishlatib.

✓ Yodda tuting

- Semantik teg — nomi mazmunini aytadigan teg.
- `header`, `nav`, `main` (bitta), `section`, `article`, `aside`, `footer` — sahifaning asosiy qismlari.
- Mustaqil mazmun — `article`; sahifa bo'limi — `section`; faqat dizayn uchun — `div` / `span`.
- Semantika SEO, accessibility va kod o'qilishi uchun muhim; ko'rinishga ta'sir qilmaydi.

✓ Test javoblari

1. Sahifaning asosiy, noyob mazmuni qaysi teg ichida bo'ladi? — **B) <main>**
main — sahifada bitta, asosiy mazmun uchun.
2. Blogdagi alohida post uchun eng mos teg qaysi? — **A) <article>**
Post mustaqil mazmun — uni alohida olib tarqatsa ham tushunarli.
3. "O'xshash maqolalar" yon paneli uchun qaysi teg mos? — **D) <aside>**
aside — asosiy mazmunga qo'shimcha, yondosh ma'lumot.
4. Qaysi element inline (qator ichida)? — **C) **
span, a, strong, img — inline; qolganlari blok.
5. Semantik teglar nima uchun foydali EMAS? — **B) Sahifa avtomatik chiroyli bo'lib qoladi**
Semantik teglar ko'rinishni o'zgartirmaydi — dizayn CSS'ning ishi.



06-MAVZU · 42 TADAN · I BOB. HTML VA CSS

CSS ulash usullari, selektorlar va sintaksis. Matn va shriftlar

CSS nima, Inline, Internal va External usullar, qoida sintaksisi, teg/class/id va boshqa selektorlar, kaskad va ustuvorlik, ranglar, matn va shrift xossalari, Google Fonts.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- CSS nima va u HTML bilan qanday ishlashini;
- CSS'ni sahifaga ulashning 3 usulini va qaysi biri eng to'g'ri ekanini;
- CSS qoidasining tuzilishi va asosiy selektorlarni;
- Bir elementga bir nechta qoida to'g'ri kelsa, qaysi biri "g'olib" bo'lishini;
- Ranglar, matn va shriftlar bilan ishlashni.

CSS nima?

CSS (Cascading Style Sheets) — "kaskadli uslublar jadvali". HTML sahifaning **tuzilishini** yaratsa, CSS uning **ko'rinishini** belgilaydi: ranglar, shriftlar, o'lchamlar, joylashuv, animatsiyalar.



O'xshatish

HTML — odamning skeleti va tanasi. CSS — kiyimi, soch turmagi, rangi. Bir odam turli kiyimda butunlay boshqacha ko'rinadi — xuddi shunday, bitta HTML sahifa turli CSS bilan butunlay boshqacha ko'rinadi. Buni CSS Zen Garden saytida o'zingiz ko'ring: HTML bir xil, faqat CSS fayl almashadi!

CSS qoidasining tuzilishi

CSS

```
h1 {  
  color: blue;  
  font-size: 32px;  
}  
/*  
h1          – selektor: QAYSI elementlarga  
{ ... }    – deklaratsiyalar bloki  
color       – xossa (property): NIMANI o'zgartiramiz  
blue        – qiymat (value): QANDAY qilib  
color: blue; – bitta deklaratsiya. Oxirida nuqta-vergul (;) majburiy!  
*/
```



CSS'da izohlar `/* ... */` ko'rinishida yoziladi (HTML'dagi `<!-- -->` emas).

CSS'ni ulashning 3 usuli

1. Inline — teg ichida

HTML

```
<p style="color: red; font-size: 20px;">Bu paragraf qizil va katta.</p>
```

Faqat **bitta** elementga ta'sir qiladi. Tez sinab ko'rish uchun qulay, lakin 100 ta paragrafni qizil qilish uchun 100 marta yozish kerak — noqulay.

2. Internal — `<style>` tegi ichida

HTML

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>Internal CSS</title>
  <style>
    p {
      color: green;
    }
  </style>
</head>
<body>
  <p>Birinchii paragraf yashil.</p>
  <p>Ikkinchisi ham yashil!</p>
</body>
</html>
```

Butun **bitta sahifaga** ta'sir qiladi. Lekin saytda 10 ta sahifa bo'lsa, har biriga nusxalash kerak.

3. External — alohida `.css` fayl

index.html

```
<head>
  <meta charset="UTF-8">
  <title>External CSS</title>
  <link rel="stylesheet" href="style.css">
</head>
```

style.css

```
p {
  color: purple;
}
```

Eng to'g'ri usul. Bitta `style.css` fayli **barcha sahifalarga** ulanadi. Dizaynni o'zgartirish uchun bitta faylni tahrirlash kifoya. Brauzer faylni bir marta yuklab, xotirada saqlaydi — sayt tezroq ishlaydi.

Usul	Qayerda	Ta'sir doirasi	Qachon
Inline	<code>style="..."</code> atributi	Bitta element	Tez sinash, kamdan-kam
Internal	<code><style></code> tegi (<code>head</code> da)	Bitta sahifa	Bir sahifali kichik loyihalar
External	Alohida <code>.css</code> fayl + <code><link></code>	Butun sayt	Deyarli har doim

Selektorlar — "kimga?" savoliga javob

Asosiy selektorlar

```
HTML

<style>
  /* 1. Teg selektori — BARCHA <p> lar */
  p { color: #333; }

  /* 2. Class selektori — class="muhim" bo'lgan BARCHA elementlar (nuqta bilan) */
  .muhim { color: crimson; font-weight: bold; }

  /* 3. ID selektori — id="sarlavha" bo'lgan BITTA element (panjara bilan) */
  #sarlavha { color: navy; text-decoration: underline; }

  /* 4. Universal selektor — sahifadagi HAMMA narsa */
  * { font-family: Arial, sans-serif; }

  /* 5. Guruh selektori — bir nechtaga birdan (vergul bilan) */
  h2, h3 { color: teal; }
</style>

<h1 id="sarlavha">Maktab gazetasi</h1>
<h2>Yangiliklar</h2>
<p>Oddiy paragraf.</p>
<p class="muhim">Muhim e'lon: ertaga dars yo'q!</p>
<h3>Sport</h3>
<p>Yana oddiy paragraf, lekin <span class="muhim">bu so'z muhim</span>.</p>
```

Selektor	Yozilishi	Kimni tanlaydi
Teg	p	Barcha <p> elementlari
Class	.muhim	class="muhim" bo'lganlar (bir nechta bo'lishi mumkin)
ID	#sarlavha	id="sarlavha" bo'lgan yagona element
Universal	*	Hammasi

**class yoki id?**

Dizayn uchun deyarli doim **class** ishlatish. Bitta elementga bir nechta class berish mumkin:

`class="tugma katta qizil"`. `id` esa sahifada takrorlanmaydi va asosan havolalar (`#bo'lim`) va JavaScript uchun kerak.

Kombinatsiyalangan selektorlar

HTML

```
<style>
  /* Avlod: <nav> ichidagi BARCHA <a> lar (istalgan chuqurlikda) – bo'sh joy
bilan */
  nav a { color: white; background: #4f46e5; padding: 4px 10px; text-
decoration: none; }

  /* Bevosita farzand: faqat birinchi darajadagi <li> – ">" bilan */
  .menu > li { font-weight: bold; }

  /* Teg + class: faqat class="eslatma" bo'lgan <p> lar */
  p.eslatma { background: lightyellow; }

  /* Atribut bo'yicha */
  input[type="email"] { border: 2px solid green; }
</style>

<nav><a href="#">Bosh sahifa</a> <a href="#">Aloqa</a></nav>

<ul class="menu">
  <li>Birinchi daraja
    <ul><li>Ikkinchi daraja (qalin emas)</li></ul>
  </li>
</ul>

<p class="eslatma">Bu eslatma paragrafi</p>
<div class="eslatma">Bu div – sariq bo'lmaydi</div>
<input type="email" placeholder="email"> <input type="text" placeholder="matn">
```

Kaskad: qaysi qoida g'olib?

Bir elementga bir nechta qoida to'g'ri kelsa, brauzer quyidagi tartibda hal qiladi:

1. **Aniqlik (spetsifiklik)** — aniqroq selektor yutadi: **inline > id > class > teg**.
2. **Tartib** — aniqlik teng bo'lsa, **keyinroq yozilgan** qoida yutadi.

HTML

```
<style>
  p { color: blue; }           /* teg – eng kuchsiz */
  .matn { color: green; }      /* class – kuchliroq */
  #asosiy { color: orange; }   /* id – yanada kuchliroq */
  p { color: red; }           /* teg, keyinroq yozilgan – lekin class va id dan
baribir kuchsiz */
</style>

<p>Men qizilman (ikkala teg qoidasidan keyingisi yutdi)</p>
<p class="matn">Men yashilman (class tegdan kuchli)</p>
<p class="matn" id="asosiy">Men to'q sariqman (id class'dan kuchli)</p>
<p class="matn" id="asosiy" style="color: purple">Men binafshaman (inline eng
kuchli)</p>
```



O'xshatish: ko'rsatmalar

Direktor (**inline**) "Ali, sen bugun navbatchisan" desa, bu sinf rahbari (**id**) yoki sinf sardori (**class**) aytganidan ustun. Umumiy maktab qoidasi (**teg**) esa eng kuchsiz — maxsus ko'rsatma bo'lmagandagina amal qiladi.

Meros (inheritance)

Ba'zi xossalar (asosan matnga oid: `color`, `font-family`, `font-size`) ota elementdan ichidagi elementlarga **meros** bo'lib o'tadi. Shuning uchun shriftni bir marta `body` ga berish kifoya — butun sahifaga tarqaladi.

Ranglar

HTML

```
<style>
  .r1 { color: tomato; }           /* nom bilan (140+ ta tayyor nom) */
  .r2 { color: #4f46e5; }         /* HEX: #RRGGBB (qizil, yashil, ko'k)
*/
  .r3 { color: #f00; }             /* qisqa HEX = #fff0000 */
  .r4 { color: rgb(16, 185, 129); } /* RGB: har biri 0-255 */
  .r5 { color: rgba(0, 0, 0, 0.4); } /* RGBA: 4-son — shaffoflik 0..1 */
  .r6 { color: hsl(280, 70%, 50%); } /* HSL: rang tusi, to'yinganlik,
yorqinlik */
  .fon { background-color: #fef3c7; color: #92400e; padding: 8px; }
</style>
<p class="r1">tomato</p>
<p class="r2">#4f46e5</p>
<p class="r3">#f00</p>
<p class="r4">rgb(16, 185, 129)</p>
<p class="r5">rgba — yarim shaffof</p>
<p class="r6">hsl(280, 70%, 50%)</p>
<p class="fon">Fon rangi va matn rangi</p>
```

HEX kodda har bir juft — 16 lik sanoq sistemasidagi son (00 – ff): #ff0000 — to'liq qizil, #000000 — qora, #ffffff — oq. Ranglarni tanlash uchun VS Code'da rang ustiga sichqonchani olib boring yoki coolors.co saytidan tayyor palitralardan foydalaning.



Matn va fon o'rtasida **yetarli kontrast** bo'lsin: och kulrang fonda och sariq matnni hech kim o'qiy olmaydi.

Matn xossalari

HTML

```
<style>
  .demo p { border-bottom: 1px dashed #ccc; padding: 4px; }
  .a { text-align: center; } /* left | right | center |
justify */
  .b { text-decoration: underline; } /* none | underline | line-
through */
  .c { text-transform: uppercase; } /* uppercase | lowercase |
capitalize */
  .d { letter-spacing: 4px; } /* harflar orasidagi masofa */
  .e { word-spacing: 15px; } /* so'zlar orasidagi masofa */
  .f { line-height: 2; } /* qatorlar orasidagi masofa
(shrift o'lchamiga nisbatan) */
  .g { text-indent: 40px; } /* xatboshi */
  .h { text-shadow: 2px 2px 4px rgba(0,0,0,.4); } /* soya: o'ngga, pastga,
xiralik, rang */
  .i { text-decoration: none; } /* havola tagidagi chiziqni olib
tashlash */
</style>
<div class="demo">
  <p class="a">Markazga tekislangan matn</p>
  <p class="b">Tagi chizilgan matn</p>
  <p class="c">katta harflarga aylantirilgan matn</p>
  <p class="d">Keng harflar</p>
  <p class="e">So'zlar orasi keng</p>
  <p class="f">Qatorlar orasi ikki baravar. Bu matn ikki qatorga tushishi uchun
ataylab uzunroq yozildi, natijani ko'ring.</p>
  <p class="g">Xatboshi bilan boshlangan abzats. Kitoblardagi kabi birinchi
qator ichkariga surilgan.</p>
  <p class="h">Soyalı matn</p>
  <p><a class="i" href="#">Tagida chizig'i yo'q havola</a></p>
</div>
```

Shriftlar

CSS

```
body {
  font-family: "Segoe UI", Arial, sans-serif; /* birinchisi bo'lmasa —
keyingisi (zaxira) */
  font-size: 16px; /* shrift o'lchami */
  font-weight: 400; /* qalinlik: 100..900 (400 =
normal, 700 = bold) */
  font-style: normal; /* normal | italic */
  line-height: 1.6; /* o'qish uchun qulay qator
oralig'i */
}
h1 {
  font-family: Georgia, "Times New Roman", serif;
  font-weight: 700;
}
```

Shrift oilalari:

- `serif` — harflarida "oyoqchalari" bor (Times New Roman, Georgia). Kitob uslubi.
- `sans-serif` — oyoqchasiz, zamonaviy (Arial, Roboto). Ekranda o'qish uchun qulay.
- `monospace` — har bir harf bir xil kenglikda (Courier). Dastur kodi uchun.

Ro'yxat oxiriga doim umumiy oilani (`sans-serif`) yozing — agar foydalanuvchi kompyuterida birorta shrift bo'lmasa, brauzer shu oiladan biror shriftni tanlaydi.

Google Fonts — bepul chiroyli shriftlar

- 1 fonts.google.com saytiga kiring va shrift tanlang (masalan, **Montserrat**). O'zbekcha harflar uchun **Latin Extended**ni qo'llab-quvvatlaydiganini tanlang.
- 2 "Get font" → "Get embed code" bosing va `<link>` kodini nusxalang.
- 3 Uni HTML'ning `<head>` qismiga, **o'z CSS faylingizdan oldin** qo'ying:

HTML

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?
family=Montserrat:wght@400;700&display=swap" rel="stylesheet">
<link rel="stylesheet" href="style.css">
```

4

CSS'da ishlatilgan: `body { font-family: "Montserrat", sans-serif; }`

O'lcham birliklari (qisqacha)

- `px` — piksel, aniq o'lcham.
- `em` — ota element shrift o'lchamiga nisbatan (`2em` = 2 barobar).
- `rem` — sahifaning asosiy shrifti (odatda 16px) ga nisbatan: `1.5rem` = 24px. Shriftlar uchun tavsiya etiladi.
- `%` — ota elementga nisbatan foiz.

Batafsil 8-mavzuda.

Amaliy mashg'ulot: vizitka saytiga dizayn

1

`vizitka/` papkasida `css/style.css` faylini yarating.

2

Har bir HTML sahifaning `<head>` qismiga ulang: `<link rel="stylesheet" href="css/style.css">` (ichki papkadagi sahifalarda — `../css/style.css`!).

3

Google Fonts'dan ikkita shrift ulang: sarlavhalar uchun bittasi, matn uchun boshqasi.

4

`style.css` ga quyidagilarni yozing va o'zingizga moslang:

```
/* Butun sahifa uchun asosiy uslub */
body {
  font-family: "Segoe UI", Arial, sans-serif;
  font-size: 17px;
  line-height: 1.7;
  color: #1f2937;
  background-color: #f9fafb;
}

h1, h2, h3 {
  font-family: Georgia, serif;
  color: #4f46e5;
}

h1 {
  font-size: 2.5rem;
  text-align: center;
  letter-spacing: 1px;
}

h2 {
  font-size: 1.6rem;
  text-transform: uppercase;
  letter-spacing: 2px;
}

nav a {
  color: #4f46e5;
  font-weight: bold;
  text-decoration: none;
}

.ajratilgan {
  color: #b91c1c;
  font-weight: 700;
}

footer {
  text-align: center;
  font-size: 14px;
}
```

```
color: #6b7280;  
}
```

- 5 HTML'da 2–3 ta muhim so'zni `<span class="ajratilgan">` bilan o'rang.
- 6 Brauzerda `F12` → **Elements** → biror elementni tanlang. O'ng tomondagi **Styles** panelida unga qaysi CSS qoidalar ta'sir qilayotganini va qaysilari ustidan chizilganini (yutqazganini) ko'ring. Qiymatlarni shu yerning o'zida o'zgartirib tajriba qiling.

Ko'p uchraydigan xatolar

- ❌ Nuqta-vergulni unutish: `color: red font-size: 20px;` — ikkala xossa ham ishlamaydi.
- ❌ Class selektorida nuqtani unutish: `muhim { }` o'rniga `.muhim { }`.
- ❌ HTML'da `class=".muhim"` deb yozish — HTML'da nuqtasiz: `class="muhim"`.
- ❌ CSS fayl yo'li noto'g'ri — `F12` → Console'da "404 style.css" xatosini ko'rasiz.
- ❌ Xossa nomida xato: `colour`, `font-colour`, `text-color` — to'g'risi `color`.

O'zingizni tekshiring

? Butun saytga bitta dizayn berishning eng to'g'ri usuli qaysi?

- A. Inline CSS
- B. Internal CSS
- C. External CSS (alohida .css fayl)
- D. Har sahifaga `<style>`

? `class="karta"` bo'lgan elementlarni tanlovchi selektor qaysi?

- A. `#karta`
- B. `.karta`
- C. `karta`
- D. `*karta`

? `<p class="a" id="b">` uchun `p {color:red}`, `.a {color:green}`, `#b {color:blue}` yozilgan. Matn qanday rangda?

- A. Qizil
- B. Yashil
- C. Qora
- D. Ko'k

? `nav a` selektori nimani tanlaydi?

- A. `nav` ichidagi barcha havolalarni
- B. `nav` va barcha `a` larni
- C. Faqat birinchi havolani
- D. `a` ichidagi `nav` ni

? **Matnni markazga tekislovchi xossa qaysi?**

- A. `align: center`
- B. `font-align: center`
- C. `text-align: center`
- D. `center: true`

? `font-family: "Roboto", Arial, sans-serif;` — agar kompyuterda Roboto bo'lmasa?

- A. Matn ko'rinmaydi
- B. Arial ishlatiladi
- C. Xato chiqadi
- D. Times New Roman ishlatiladi



Uyga vazifa

1. CSS Diner o'yinida kamida 12-darajagacha o'ting (selektorlarni o'yin orqali mashq qilish).
2. Vizitka saytingiz uchun 4–5 rangdan iborat palitra tanlang (colors.co) va barcha sahifalarga qo'llang.
3. 5-mavzudagi "Sinfimiz gazetasi" sahifasiga gazeta uslubida dizayn bering: serif shrift, markazdagi katta sarlavha, ustunlar orasida chiziq.



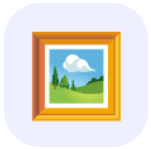
Yodda tuting

- Qoida: `selektor { xossa: qiymat; }`.
- External CSS (`<link rel="stylesheet" href="style.css">`) — asosiy usul.
- Selektorlar: `teg` , `.class` , `#id` , `*` , `a, b` (guruh), `a b` (ichidagi), `a > b` (bevosita farzand).
- Ustuvorlik: `inline > id > class > teg`; teng bo'lsa — keyingisi yutadi.

- Matn: `color`, `text-align`, `text-decoration`, `line-height`; shrift: `font-family`, `font-size`, `font-weight`.

✓ Test javoblari

1. Butun saytga bitta dizayn berishning eng to'g'ri usuli qaysi? — **C) External CSS (alohida .css fayl)**
Bitta fayl — barcha sahifalar. O'zgartirish oson va brauzer faylni keshlaydi.
2. class="karta" bo'lgan elementlarni tanlovchi selektor qaysi? — **B) .karta**
Nuqta — class, panjara — id.
3. <p class="a" id="b"> uchun p {color:red}, .a {color:green}, #b {color:blue} yozilgan. Matn qanday rangda? — **D) Ko'k**
ID selektori class va tegdan kuchliroq.
4. nav a selektori nimani tanlaydi? — **A) nav ichidagi barcha havolalarni**
Bo'sh joy — "ichidagi" (avlod) selektori. Vergul esa "va" (guruh) bo'lardi.
5. Matnni markazga tekislovchi xossa qaysi? — **C) text-align: center**
6. font-family: "Roboto", Arial, sans-serif; — agar kompyuterda Roboto bo'lmasa? — **B) Arial ishlatiladi**
Ro'yxat — zaxira shriftlar ketma-ketligi.



07-MAVZU · 42 TADAN · I BOB. HTML VA CSS

CSS background va border xossalari

Fon rangi, fon rasmi, takrorlanish, joylashuv va o'lcham (cover/contain), gradientlar; chegaralar: qalinlik, uslub, rang, border-radius, box-shadow va outline.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Elementlarga fon rangi, fon rasmi va gradient berishni;
- Fon rasmini to'g'ri joylashtirish va o'lchamlashni (`cover` , `contain`);
- Chegaralar (border) yaratish, ularning uslublari va yumaloq burchaklarni;
- Soyalar (`box-shadow`) bilan "kartochka" dizaynini yaratishni.

Fon rangi — `background-color`

HTML

```
<style>
  body { background-color: #f1f5f9; }
  .quti { background-color: #4f46e5; color: white; padding: 20px; }
  .shaffof { background-color: rgba(79, 70, 229, 0.2); padding: 20px; }
</style>
<div class="quti">To'q ko'k fonli quti</div>
<div class="shaffof">Yarim shaffof fonli quti (rgba)</div>
```



`padding` — matn va chegara orasidagi ichki bo'shliq. Usiz matn fon chetiga yopishib qoladi.
Batafsil — keyingi mavzuda.

Fon rasmi — background-image

HTML

```
<style>
  .banner {
    height: 250px;
    background-image: url("https://picsum.photos/seed/tog/1200/600");
    background-size: cover;          /* rasmni butun maydonga yoyish */
    background-position: center;     /* rasmning markazini ko'rsatish */
    background-repeat: no-repeat;    /* takrorlamaslik */
    color: white;
    display: flex; align-items: center; justify-content: center;
    font-size: 32px; font-weight: bold;
    text-shadow: 0 2px 8px rgba(0,0,0,.7);
  }
</style>
<div class="banner">Tog'larga sayohat</div>
```

Xossa	Qiymatlar	Vazifasi
background-image	url("rasm.jpg")	Fon rasmi (yo'l nisbiy: CSS fayldan hisoblanadi!)
background-repeat	repeat no-repeat repeat-x repeat-y	Rasm kichik bo'lsa takrorlanishi
background-position	center, top right, 50% 20%	Rasmning qaysi qismi ko'rinadi
background-size	cover contain 200px 100px 50%	Rasm o'lchami
background-attachment	scroll fixed	fixed — sahifa aylantirilganda fon joyida qoladi (parallax effekti)

cover va contain farqi

HTML

```
<style>
  .q { width: 300px; height: 150px; border: 3px solid #333; display: inline-
block; margin: 8px;
      background: #fde68a url("https://picsum.photos/seed/olma/400/400") no-
repeat center; }
  .cover { background-size: cover; }
  .contain { background-size: contain; }
</style>
<div class="q cover"></div>
<div class="q contain"></div>
<p><b>cover</b> (chapda): butun qutini to'ldiradi, ortiqcha qismi kesiladi.<br>
<b>contain</b> (o'ngda): rasm to'liq ko'rinadi, lekin bo'sh joy qolishi mumkin.
</p>
```



O'xshatish

cover — telefonga devor qog'ozi (oboy) qo'yish: ekran to'ladi, rasmning chetlari kesiladi.

contain — rasmni ramkaga qo'yish: rasm butunligicha ko'rinadi, lekin atrofida bo'sh joy qoladi.



Fon rasmi yo'li

`url()` ichidagi yo'l **CSS faylga nisbatan** hisoblanadi. Agar CSS `css/style.css` da, rasm `rasmlar/fon.jpg` da bo'lsa: `url("../rasmlar/fon.jpg")`.

Qisqa yozuv (shorthand)

CSS

```
/* Uzun yozuv */
.banner {
  background-color: #1e293b;
  background-image: url("fon.jpg");
  background-repeat: no-repeat;
  background-position: center;
  background-size: cover;
}
/* Qisqa yozuv — hammasi bitta qatorda (size "/" belgisidan keyin) */
.banner {
  background: #1e293b url("fon.jpg") no-repeat center / cover;
}
```

Gradientlar

Gradient — bir rangdan boshqasiga silliq o'tish. U ham fon rasmi sifatida ishlaydi.

HTML

```
<style>
  .g { height: 70px; margin: 8px 0; border-radius: 10px; color: white; font-
weight: bold; padding: 10px; }
  .g1 { background: linear-gradient(to right, #4f46e5, #06b6d4); }
  .g2 { background: linear-gradient(135deg, #f97316, #ec4899, #8b5cf6); }
  .g3 { background: radial-gradient(circle, #fde047, #f97316); color: #333; }
  .g4 { background: linear-gradient(rgba(0,0,0,.5), rgba(0,0,0,.5)),
url("https://picsum.photos/seed/shahar/800/200") center / cover; }
</style>
<div class="g g1">linear-gradient: chapdan o'ngga</div>
<div class="g g2">135° burchak ostida 3 ta rang</div>
<div class="g g3">radial-gradient: markazdan atrofga</div>
<div class="g g4">Rasm ustida qoraytiruvchi gradient — matn o'qilishi
oson</div>
```

Oxirgi usul juda ko'p ishlatiladi: rasm ustiga yarim shaffof qora qatlam qo'yilsa, oq matn har qanday rasmda ham yaxshi o'qiladi.

Chegaralar — border

Chegara uchta qismdan iborat: **qalinlik**, **uslub** va **rang**.

HTML

```
<style>
  div { padding: 10px; margin: 10px 0; }
  .b1 { border: 2px solid #4f46e5; }
  .b2 { border: 3px dashed tomato; }
  .b3 { border: 4px dotted green; }
  .b4 { border: 6px double #333; }
  .b5 { border: 6px groove #aaa; }
  .b6 { border-bottom: 3px solid orange; }           /* faqat pastki */
  .b7 { border-left: 6px solid #4f46e5; background: #eef2ff; } /* "eslatma"
uslubi */
  .b8 { border-width: 1px 4px 8px 12px; border-style: solid; border-color:
crimson; } /* tepa o'ng past chap */
</style>
<div class="b1">solid — yaxlit chiziq</div>
<div class="b2">dashed — shtrix</div>
<div class="b3">dotted — nuqtali</div>
<div class="b4">double — qo'sh chiziq</div>
<div class="b5">groove — o'yilgan</div>
<div class="b6">Faqat pastki chegara</div>
<div class="b7">Chap tomonda qalin chiziq — eslatma bloki</div>
<div class="b8">Har tomonda har xil qalinlik</div>
```



Soat strelkasi qoidasi

To'rt tomonli qiymatlar doim **tepa** → **o'ng** → **past** → **chap** tartibida yoziladi (soat strelkasi bo'yicha, 12 dan boshlab). Bu qoida `margin` va `padding` uchun ham amal qiladi.

Yumaloq burchaklar — border-radius

HTML

```
<style>
  .r { width: 120px; height: 120px; background: #4f46e5; color: white; display:
inline-grid; place-items: center; margin: 6px; font-size: 13px; }
  .r1 { border-radius: 10px; }
  .r2 { border-radius: 30px; }
  .r3 { border-radius: 50%; } /* doira! */
  .r4 { border-radius: 40px 0 40px 0; } /* barg shakli */
  .r5 { width: 160px; height: 50px; border-radius: 999px; } /* "tabletka"
tugma */
  img.avatar { width: 120px; height: 120px; border-radius: 50%; object-fit:
cover; border: 4px solid white; box-shadow: 0 0 0 3px #4f46e5; }
</style>
<div class="r r1">10px</div>
<div class="r r2">30px</div>
<div class="r r3">50% – doira</div>
<div class="r r4">barg</div>
<div class="r r5">tabletka</div>
<br>

```

`border-radius: 50%` + teng kenglik va balandlik = **doira**. Profil rasmlari aynan shunday qilinadi.

Soyalar — box-shadow

CSS

```
box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
/*
  |   |   |   L rang (yarim shaffof qora – eng tabiiy ko'rinadi)
  |   |   |   L xiralik (blur) – qancha katta, shuncha yumshoq
  |   |   |   L pastga siljish (y)
  |   |   |   L o'ngga siljish (x) */
```

```

<style>
  body { background: #f1f5f9; font-family: sans-serif; }
  .s { background: white; padding: 20px; margin: 18px; border-radius: 12px;
display: inline-block; width: 180px; }
  .s1 { box-shadow: 0 1px 3px rgba(0,0,0,.12); }
  .s2 { box-shadow: 0 8px 24px rgba(0,0,0,.12); }
  .s3 { box-shadow: 0 20px 40px rgba(79,70,229,.35); }
  .s4 { box-shadow: inset 0 2px 6px rgba(0,0,0,.2); } /* ichki soya */
</style>
<div class="s s1">Yengil soya</div>
<div class="s s2">O'rtacha soya</div>
<div class="s s3">Rangli kuchli soya</div>
<div class="s s4">Ichki soya (inset)</div>

```

`outline` — chegaraga o'xshaydi, lekin joy egallamaydi. Brauzer klaviatura bilan tanlangan elementni (`Tab`) shu bilan ko'rsatadi — uni `outline: none` bilan butunlay o'chirib qo'ymang, aks holda klaviaturadan foydalanuvchilar qayerdaligini bilmay qoladi.

Amaliy mashg'ulot: "O'quvchi kartochkasi"

Ko'plab zamonaviy saytlardagi "karta" (card) dizaynini o'rganilgan xossalar bilan yaratamiz.

1 `karta.html` va `karta.css` fayllarini yarating, CSS'ni HTML'ga ulang.

2 HTML tuzilmasini yozing:

karta.html

```
<body>
  <article class="karta">
    <div class="karta-fon"></div>
    
    <h2>Malika Karimova</h2>
    <p class="karta-sinf">8-"A" sinf · Frontend yo'nalishi</p>
    <p class="karta-matn">HTML va CSS bilan chiroyli sahifalar yaratishni
yaxshi ko'raman.
    Maqsadim – o'z maktabim uchun sayt yaratish.</p>
    <a class="karta-tugma" href="#">Portfolioni ko'rish</a>
  </article>
</body>
```

3

CSS'ni yozing:

```

<style>
body {
  background: linear-gradient(135deg, #e0e7ff, #fce7f3);
  font-family: "Segoe UI", Arial, sans-serif;
  min-height: 100vh;
  margin: 0;
  padding: 40px;
}
.karta {
  width: 320px;
  margin: 0 auto;
  background-color: white;
  border-radius: 20px;
  box-shadow: 0 15px 35px rgba(79, 70, 229, 0.2);
  text-align: center;
  padding-bottom: 28px;
  overflow: hidden; /* fon rasmi burchaklardan chiqib
ketmasin */
}
.karta-fon {
  height: 110px;
  background: linear-gradient(rgba(79,70,229,.6), rgba(236,72,153,.6)),
    url("https://picsum.photos/seed/kutubxona/640/220") center
/ cover;
}
.karta-avatar {
  width: 110px;
  height: 110px;
  border-radius: 50%;
  border: 5px solid white;
  margin-top: -55px; /* rasmni fonning ustiga "chiqarish" */
  object-fit: cover;
}
.karta h2 { margin: 8px 0 0; color: #1e1b4b; }
.karta-sinf { color: #6366f1; font-weight: 600; margin: 4px 0 12px; }
.karta-matn { color: #4b5563; padding: 0 24px; line-height: 1.6; }
.karta-tugma {
  display: inline-block;
  margin-top: 8px;
  padding: 10px 22px;
  background: linear-gradient(to right, #4f46e5, #ec4899);
  color: white;
}

```



```

text-decoration: none;
border-radius: 999px;
font-weight: 600;
box-shadow: 0 6px 15px rgba(236, 72, 153, 0.35);
}
</style>
<article class="karta">
  <div class="karta-fon"></div>
  
  <h2>Malika Karimova</h2>
  <p class="karta-sinf">8-"A" sinf · Frontend yo'nalishi</p>
  <p class="karta-matn">HTML va CSS bilan chiroyli sahifalar yaratishni
yaxshi ko'raman. Maqsadim – o'z maktabim uchun sayt yaratish.</p>
  <a class="karta-tugma" href="#">Portfolioni ko'rish</a>
</article>

```

4

O'z ma'lumotlaringiz va ranglaringiz bilan moslang. Soya, radius va gradient qiymatlarini o'zgartirib, natija qanday o'zgarishini kuzating.

Ko'p uchraydigan xatolar

- ✗ Fon rasmi ko'rinmayapti — `url()` dagi yo'l CSS faylga nisbatan emas, HTML'ga nisbatan yozilgan. Yoki elementning **balandligi 0** (ichida mazmun yo'q va `height` berilmagan).
- ✗ `border: 2px red;` — uslub (`solid`) unutilgan, chegara ko'rinmaydi.
- ✗ Doira chiqmayapti — `width` va `height` teng emas, natijada oval bo'ladi.
- ✗ Juda qora, qattiq soya (`box-shadow: 5px 5px 0 black`) — zamonaviy dizaynda yumshoq, shaffof soyalar ishlatiladi.

O'zingizni tekshiring

? Fon rasmi butun maydonni to'ldirishi (ortig'i kesilishi) uchun qaysi qiymat kerak?

- A. `background-size: contain`
- B. `background-size: cover`
- C. `background-repeat: repeat`
- D. `background-position: fill`

? `border: 3px ____ blue;` — bo'sh joyga nima yozilsa shtrixli chiziq bo'ladi?

- A. `solid`
- B. `dotted`
- C. `double`
- D. `dashed`

? Kvadrat rasmni doiraga aylantirish uchun nima kerak?

- A. `border-radius: 50%`
- B. `border: circle`
- C. `shape: round`
- D. `border-radius: 10px`

? `margin: 10px 20px 30px 40px;` da chap tomon necha piksel?

- A. 10px
- B. 20px
- C. 40px
- D. 30px

? `box-shadow: 0 10px 20px rgba(0,0,0,.2)` da `20px` nima?

- A. O'ngga siljish
- B. Xiralik (blur)
- C. Pastga siljish
- D. Soya rangi



Uyga vazifa

1. Vizitka saytingiz bosh sahifasiga gradient ustidagi fon rasmlari "banner" qo'shing (isingiz va qisqa shiorga).
2. Sinfdozlaringizdan 3 nafari uchun o'quvchi kartochkalarini yarating (har biri boshqa rang sxemasida).
3. cssgradient.io saytida o'z gradientingizni yarating va ishlating.



Yodda tuting

- Fon: `background-color`, `background-image: url()`, `-repeat`, `-position`, `-size: cover/contain`; qisqasi — `background`.
- Gradient: `linear-gradient(yo'nalish, rang1, rang2)`, `radial-gradient()`.
- Chegara: `border: qalinlik uslub rang`; tomonlar tartibi — tepa, o'ng, past, chap.
- `border-radius` — yumaloq burchak (50% = doira); `box-shadow` — soya.

✓ Test javoblari

1. Fon rasmi butun maydonni to'ldirishi (ortig'i kesilishi) uchun qaysi qiymat kerak? — **B) background-size: cover**
2. border: 3px ____ blue; — bo'sh joyga nima yozilsa shtrixli chiziq bo'ladi? — **D) dashed**
3. Kvadrat rasmni doiraga aylantirish uchun nima kerak? — **A) border-radius: 50%**
4. margin: 10px 20px 30px 40px; da chap tomon necha piksel? — **C) 40px**
Tartib: tepa (10) → o'ng (20) → past (30) → chap (40).
5. box-shadow: 0 10px 20px rgba(0,0,0,.2) da 20px nima? — **B) Xiralik (blur)**



08-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Box model va o'lcham birliklari. Flexbox va Grid

Har bir element — quti: content, padding, border, margin; box-sizing; px, %, em, rem, vw, vh; display; Flexbox bilan bir o'lchamli va CSS Grid bilan ikki o'lchamli sahifa maketlari.

3 soat · 1 nazariy + 2 amaliy

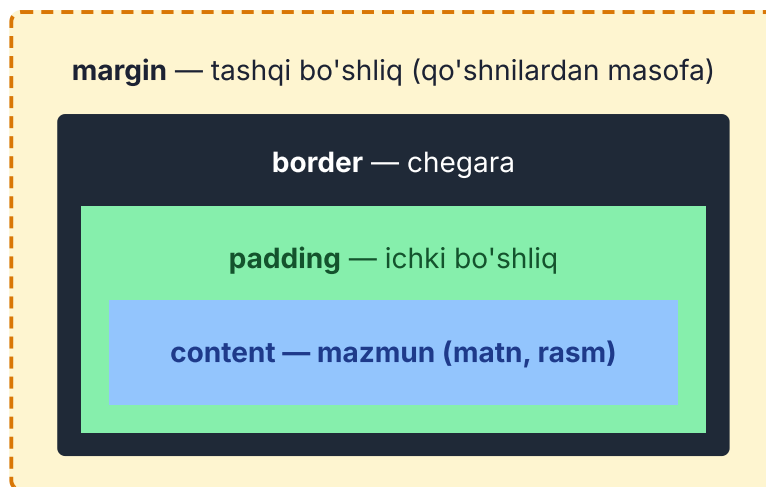
Bu darsda nimani o'rganasiz

- Brauzer har bir elementni qanday "quti" sifatida ko'rishini (box model);
- `margin`, `padding`, `border` va `box-sizing` ni;
- O'lcham birliklari: `px`, `%`, `em`, `rem`, `vw`, `vh`;
- **Flexbox** bilan menyu, kartalar qatori, markazlash;
- **CSS Grid** bilan butun sahifa maketi va galereya yaratishni.

Bu kursdagi eng muhim CSS mavzusi — 3 soat ajratilgan. Shoshilmang, har bir misolni ► tugmasi bilan o'zgartirib ko'ring.

Box model — har bir element quti

Brauzer sahifadagi **har bir** elementni (sarlavha, paragraf, rasm, tugma) to'rtburchak quti sifatida chizadi. Bu quti 4 qatlamdan iborat:



O'xshatish: rasmlı ramka

Devordagi rasmni tasavvur qiling. **Content** — rasmning o'zi. **Padding** — rasm atrofıdagi oq qog'oz hoshiya (paspartu). **Border** — yog'och ramka. **Margin** — devorda shu ramka bilan qo'shni ramka orasidagi bo'sh joy.

HTML

```
<style>
  .quti {
    width: 250px;
    padding: 20px;           /* ichki bo'shliq — fon rangi shu yerga ham
tarqaladi */
    border: 5px solid #4f46e5;
    margin: 30px;           /* tashqi bo'shliq — har doim shaffof */
    background: #e0e7ff;
  }
</style>
<div class="quti">Men qutiman. F12 bosib, meni tanlang va "Computed" panelidagi
box model chizmasiga qarang!</div>
<div class="quti">Men ikkinchi qutiman</div>
```

padding va margin yozish usullari

CSS

```
padding: 20px; /* 4 tomon ham 20px */
padding: 10px 20px; /* tepa-past 10px, chap-o'ng 20px */
padding: 10px 20px 30px; /* tepa 10, chap-o'ng 20, past 30 */
padding: 10px 20px 30px 40px; /* tepa, o'ng, past, chap (soat strełkasi bo'yicha) */
padding-top: 10px; /* faqat bitta tomon: -top, -right, -bottom, -left */

margin: 0 auto; /* tepa-past 0, chap-o'ng auto → blokni GORIZONTAL MARKAZLASH */
```



Blokni markazga qo'yish

Kengligi berilgan blokni sahifa o'rtasiga qo'yish uchun: `width: 600px; margin: 0 auto;`.
Brauzer chap va o'ng bo'shliqni teng taqsimlaydi.

box-sizing — eng muhim bitta qator

Yuqoridagi `.quti` ning kengligi aslida necha piksel? `width: 250px` deb yozdik, lekin brauzer standart holatda **padding va border'ni qo'shib** hisoblaydi:

`250 + 20 + 20 (padding) + 5 + 5 (border) = 300px` 😊

Bu hisob-kitobni juda chalkashtiradi. Yechim:

CSS

```
/* Har bir loyihaning CSS faylini shu qoida bilan boshlang */
* {
  box-sizing: border-box; /* endi width = padding + border + content. 250px — haqiqatan 250px */
}
```

HTML

```
<style>
  div { width: 250px; padding: 20px; border: 5px solid; margin: 10px 0; }
  .content-box { box-sizing: content-box; border-color: tomato; }
  .border-box { box-sizing: border-box; border-color: green; }
</style>
<div class="content-box">content-box: men 300px kenglikdaman</div>
<div class="border-box">border-box: men aynan 250px kenglikdaman</div>
```

display xossasi

Qiymat	Xatti-harakat
block	Yangi qatordan, butun kenglik; width / height ishlaydi (div, p, h1)
inline	Qatorda, faqat mazmuncha; width / height va tepa-past margin ishlamaydi (span, a)
inline-block	Qatorda turadi, lekin width / height ishlaydi (tugmalar uchun qulay)
none	Element butunlay yashiriladi va joy egallamaydi
flex / grid	Zamonaviy maket tizimlari (pastda)

O'lcham birliklari

Birlik	Nimaga nisbatan	Misol	Qachon ishlatish
px	Mutlaq (ekran pikseli)	border: 1px	Chegaralar, soyalar, mayda detallar
%	Ota element o'lchamiga	width: 50%	Kengliklar

em	Joriy elementning shrift o'lchamiga	padding: 0.5em 1em	Matn o'lchamiga mos padding (tugmalar)
rem	Sahifa asosiy shriftiga (odatda 16px)	font-size: 1.5rem = 24px	Shriftlar va bo'shliqlar
vw	Ekran kengligining 1%	width: 100vw	Ekran kengligida bloklar
vh	Ekran balandligining 1%	min-height: 100vh	To'liq ekranli bosh qism (hero)

HTML

```
<style>
  html { font-size: 16px; }
  .ota { width: 400px; background: #e0e7ff; padding: 10px; }
  .yarim { width: 50%; background: #a5b4fc; } /* 400 ning 50% =
200px */
  .rem { font-size: 1.5rem; } /* 16 × 1.5 = 24px */
  .em-ota { font-size: 20px; }
  .em { font-size: 2em; } /* 20 × 2 = 40px */
  .ekran { height: 20vh; background: #fbcfe8; margin-top: 10px; }
</style>
<div class="ota"><div class="yarim">50% kenglik</div></div>
<p class="rem">1.5rem = 24px</p>
<div class="em-ota">Ota 20px – <span class="em">2em</span></div>
<div class="ekran">Ekran balandligining 20%i. Oynani kichraytirib ko'ring.
</div>
```

Flexbox — elementlarni bir qatorga terish

Flexbox — elementlarni **bir yo'nalishda** (qator yoki ustun) joylashtirish tizimi. Navigatsiya menyusi, tugmalar qatori, kartalarni yonma-yon qo'yish, biror narsani markazga joylash — Flexbox'ning ishi.

Ikki asosiy tushuncha

- **Flex konteyner** — `display: flex` berilgan **ota** element.
- **Flex elementlar** — uning **bevosita farzandlari**. Ular avtomatik ravishda bir qatorga tiziladi.

```
<style>
  .konteyner {
    display: flex;                /* ← shu bitta qator hammasini o'zgartiradi */
    gap: 10px;                   /* elementlar orasidagi masofa */
    background: #f1f5f9;
    padding: 10px;
  }
  .element { background: #4f46e5; color: white; padding: 20px; border-radius:
8px; }
</style>
<div class="konteyner">
  <div class="element">1</div>
  <div class="element">2</div>
  <div class="element">3</div>
</div>
<p>display: flex ni o'chirib ko'ring – elementlar yana ustma-ust tushadi.</p>
```

Asosiy va kesishgan o'q



O'xshatish: avtobus bekati

Odamlar bekatda navbatga turdi. Navbat yo'nalishi — **asosiy o'q** (main axis): `justify-content` odamlarni shu chiziq bo'ylab taqsimlaydi (boshida to'planish, o'rtada, oralari teng...). Navbatga perpendikulyar yo'nalish — **kesishgan o'q** (cross axis): `align-items` odamlarni bo'yi bo'yicha tekislaydi (tepasi, o'rtasi, pastki qismi bo'yicha).

Konteyner xossalari

```
HTML

<style>
  .k { display: flex; background: #f1f5f9; margin: 8px 0; padding: 8px; height: 90px; border-radius: 8px; }
  .k div { background: #4f46e5; color: #fff; padding: 8px 14px; border-radius: 6px; }
  .k div:nth-child(2) { padding: 22px 14px; } /* ikkinchisi balandroq */

  .k1 { justify-content: flex-start; } /* boshida (standart) */
  .k2 { justify-content: center; } /* markazda */
  .k3 { justify-content: flex-end; } /* oxirida */
  .k4 { justify-content: space-between; } /* chetlarga yopishgan, oralari teng */
  .k5 { justify-content: space-around; } /* atrofida teng bo'shliq */
  .k6 { justify-content: space-evenly; } /* barcha bo'shliqlar teng */
  .k7 { align-items: center; } /* kesishgan o'q: vertikal markaz */

  .k8 { align-items: flex-end; } /* pastga tekislash */
  .k9 { justify-content: center; align-items: center; } /* ✨ to'liq markazlash */
</style>

<div class="k k1"><div>flex-start</div><div>2</div><div>3</div></div>
<div class="k k2"><div>center</div><div>2</div><div>3</div></div>
<div class="k k3"><div>flex-end</div><div>2</div><div>3</div></div>
<div class="k k4"><div>space-between</div><div>2</div><div>3</div></div>
<div class="k k5"><div>space-around</div><div>2</div><div>3</div></div>
<div class="k k6"><div>space-evenly</div><div>2</div><div>3</div></div>
<div class="k k7"><div>align-items: center</div><div>2</div><div>3</div></div>
<div class="k k8"><div>align-items: flex-end</div><div>2</div><div>3</div></div>
</div>
<div class="k k9"><div>to'liq markaz</div><div>2</div><div>3</div></div>
```

Xossa (konteynerga)	Qiymatlar	Vazifasi
flex-direction	row column row-reverse column-reverse	Asosiy o'q yo'nalishi (qator yoki ustun)

<code>justify-content</code>	<code>flex-start</code> , <code>center</code> , <code>flex-end</code> , <code>space-between</code> , <code>space-around</code> , <code>space-evenly</code>	Asosiy o'q bo'yicha taqsimlash
<code>align-items</code>	<code>stretch</code> (standart), <code>flex-start</code> , <code>center</code> , <code>flex-end</code>	Kesishgan o'q bo'yicha tekislash
<code>flex-wrap</code>	<code>nowrap</code> <code>wrap</code>	Sig'masa keyingi qatorga o'tsinmi
<code>gap</code>	<code>16px</code>	Elementlar orasidagi masofa



`flex-direction: column` qilinsa, o'qlar **almashadi**: endi `justify-content` vertikal, `align-items` esa gorizontal ishlaydi.

Element xossalari: `flex`

HTML

```
<style>
.k { display: flex; gap: 8px; background: #f1f5f9; padding: 8px; margin: 8px 0; }
.k div { background: #4f46e5; color: #fff; padding: 14px; border-radius: 6px; }
.yon { flex: 0 0 150px; background: #f97316 !important; } /* qat'iy 150px */
.asosiy { flex: 1; } /* qolgan BARCHA joyni oladi */
.ikki { flex: 2; } /* flex:1 dan 2 barobar ko'p joy */
</style>
<div class="k"><div class="yon">Yon panel 150px</div><div class="asosiy">Asosiy qism – flex: 1</div></div>
<div class="k"><div class="asosiy">flex: 1</div><div class="ikki">flex: 2 (ikki barobar keng)</div><div class="asosiy">flex: 1</div></div>
```

CSS Grid — panjara (to'r) maketi

Grid — elementlarni **qator va ustunlardan** iborat to'rga joylashtiradi (ikki o'lchamli). Butun sahifa maketi, rasm galereyasi, mahsulotlar katalogi uchun ideal.



O'xshatish: shaxmat taxtasi yoki Excel jadvali

Grid — bu katakli taxta. Siz avval nechta ustun va qator bo'lishini belgilaysiz, keyin elementlarni kataklarga joylashtirasiz. Ba'zi elementlar bir nechta katakni egallashi mumkin — Excel'dagi kataklarni birlashtirish kabi.

HTML

```
<style>
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr; /* 3 ta teng ustun */
  gap: 12px;
}
.grid div { background: #10b981; color: white; padding: 24px; border-radius:
8px; text-align: center; font-weight: bold; }
</style>
<div class="grid">
  <div>1</div><div>2</div><div>3</div>
  <div>4</div><div>5</div><div>6</div>
  <div>7</div>
</div>
```

fr birligi va repeat()

- `fr` (*fraction*) — **bo'sh joyning ulushi**. `1fr 2fr` — ikkinchi ustun birinchisidan 2 barobar keng.
- `repeat(4, 1fr)` = `1fr 1fr 1fr 1fr`.
- Aralash: `grid-template-columns: 250px 1fr;` — chapda qat'iy 250px yon panel, qolgani asosiy qism.

✨ Sehrli qator: avtomatik moslashuvchi galereya

HTML

```
<style>
  .galereya {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
    gap: 10px;
  }
  .galereya img { width: 100%; height: 120px; object-fit: cover; border-radius:
8px; }
</style>
<div class="galereya">
  
  
  
  
  
  
  
  
</div>
<p>"🎮 Yangi oynada" tugmasini bosib, oyna kengligini o'zgartiring – ustunlar
soni o'zi o'zgaradi!</p>
```

`repeat(auto-fill, minmax(150px, 1fr))` — "har bir ustun kamida 150px bo'lsin, nechta sig'sa shuncha ustun qil, ortgan joyini teng taqsimla". Telefonda 2 ta, planshetda 4 ta, kompyuterda 6 ta ustun — hech qanday qo'shimcha kodsiz.

Elementni bir nechta katakka cho'zish

HTML

```
<style>
  .g { display: grid; grid-template-columns: repeat(4, 1fr); gap: 10px; }
  .g div { background: #8b5cf6; color: #fff; padding: 20px; border-radius: 8px;
text-align: center; }
  .katta { grid-column: span 2; grid-row: span 2; background: #f97316
!important; }
  .keng { grid-column: 1 / 5; background: #0ea5e9 !important; } /* 1-
chiziqdan 5-chiziqqacha = 4 ustun */
</style>
<div class="g">
  <div class="katta">2x2 katta</div>
  <div>b</div><div>c</div><div>d</div><div>e</div>
  <div class="keng">Butun qator bo'ylab</div>
</div>
```

Nomlangan hududlar — sahifa maketi

HTML

```
<style>
  * { box-sizing: border-box; }
  body { margin: 0; font-family: sans-serif; }
  .sahifa {
    display: grid;
    grid-template-columns: 200px 1fr;
    grid-template-rows: 70px 1fr 50px;
    grid-template-areas:
      "header header"
      "aside main"
      "footer footer";
    min-height: 100vh;
    gap: 8px;
  }
  header { grid-area: header; background: #4f46e5; }
  aside { grid-area: aside; background: #a5b4fc; }
  main { grid-area: main; background: #e0e7ff; }
  footer { grid-area: footer; background: #312e81; }
  .sahifa > * { padding: 16px; color: #111; font-weight: bold; }
  header, footer { color: white !important; }
</style>
<div class="sahifa">
  <header>header</header>
  <aside>aside (yon panel)</aside>
  <main>main — asosiy mazmun</main>
  <footer>footer</footer>
</div>
```

`grid-template-areas` — maketni so'zlar bilan "chizish". Har bir qo'shtirnoq — bitta qator, har bir so'z — bitta ustun.

Flexbox yoki Grid?

	Flexbox	Grid
O'lcham	Bir o'lchamli: qator yoki ustun	Ikki o'lchamli: qator va ustun

Eng yaxshi joyi	Menyu, tugmalar, kartaning ichi, markazlash	Sahifa maketi, galereya, katalog
Qoida	Ular raqobatchi emas! Sahifa skeleti — Grid, uning ichidagi kichik qismlar — Flexbox.	

Amaliy mashg'ulot: portfolio sayti maketi (2-loyiha)

- 1 Yangi papka: `portfolio/`, ichida `index.html` va `style.css`.
- 2 HTML tuzilmasi (semantik!):

```

<body>
  <header class="header">
    <a class="logo" href="#">Ali<span>.dev</span></a>
    <nav class="menu">
      <a href="#haqida">Men haqimda</a>
      <a href="#loyihalar">Loyihalar</a>
      <a href="#aloqa">Aloqa</a>
    </nav>
  </header>

  <main>
    <section class="hero">
      <h1>Salom, men Ali – yosh veb-dasturchi</h1>
      <p>HTML, CSS va JavaScript o'rganyapman.</p>
      <a class="tugma" href="#loyihalar">Ishlarimni ko'rish</a>
    </section>

    <section id="loyihalar">
      <h2>Loyihalarim</h2>
      <div class="loyihalar">
        <article class="loyiha"><h3>Vizitka sayti</h3><p>HTML va CSS</p>
      </article>
        <article class="loyiha"><h3>Dars jadvali</h3><p>HTML
jadvallar</p></article>
        <article class="loyiha"><h3>O'quvchi kartasi</h3><p>CSS
dizayn</p></article>
        <article class="loyiha"><h3>Kalkulyator</h3><p>Tez kunda...</p>
      </article>
    </div>
  </section>
</main>

  <footer class="footer">© 2026 Ali</footer>
</body>

```

3

CSS — har bir qoidani o'qib, nima qilayotganini tushunib yozing:

```
<style>
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: "Segoe UI", Arial, sans-serif; color: #1f2937; line-height: 1.6; }

/* HEADER: logo chapda, menyu o'ngda — Flexbox */
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 40px;
  background: white;
  box-shadow: 0 2px 10px rgba(0,0,0,.06);
}
.logo { font-size: 1.5rem; font-weight: 800; color: #111; text-decoration: none; }
.logo span { color: #4f46e5; }
.menu { display: flex; gap: 24px; }
.menu a { color: #374151; text-decoration: none; font-weight: 500; }

/* HERO: to'liq markazlash — Flexbox ustun bo'yicha */
.hero {
  min-height: 60vh;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  text-align: center;
  gap: 16px;
  padding: 40px 20px;
  background: linear-gradient(135deg, #eef2ff, #fdf2f8);
}
.hero h1 { font-size: 2.4rem; max-width: 700px; }
.tugma { background: #4f46e5; color: white; padding: 12px 26px; border-radius: 999px; text-decoration: none; font-weight: 600; }

/* LOYIHALAR: avtomatik moslashuvchi Grid */
#loyihalar { max-width: 1100px; margin: 0 auto; padding: 60px 20px; }
#loyihalar h2 { text-align: center; margin-bottom: 30px; font-size: 2rem; }
.loyihalar {
  display: grid;

```

```

    grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
    gap: 20px;
}
.loyiha {
    padding: 24px;
    border-radius: 14px;
    background: white;
    border: 1px solid #e5e7eb;
    box-shadow: 0 6px 20px rgba(0,0,0,.05);
}
.loyiha h3 { color: #4f46e5; margin-bottom: 6px; }

.footer { text-align: center; padding: 24px; background: #111827; color:
#9ca3af; }
</style>
<header class="header">
    <a class="logo" href="#">Ali<span>.dev</span></a>
    <nav class="menu"><a href="#haqida">Men haqimda</a><a
href="#loyihalar">Loyihalar</a><a href="#aloqa">Aloqa</a></nav>
</header>
<main>
    <section class="hero">
        <h1>Salom, men Ali – yosh veb-dasturchi</h1>
        <p>HTML, CSS va JavaScript o'rganyapman.</p>
        <a class="tugma" href="#loyihalar">Ishlarimni ko'rish</a>
    </section>
    <section id="loyihalar">
        <h2>Loyihalarim</h2>
        <div class="loyihalar">
            <article class="loyiha"><h3>Vizitka sayti</h3><p>HTML va CSS</p>
        </article>
            <article class="loyiha"><h3>Dars jadvali</h3><p>HTML jadvallar</p>
        </article>
            <article class="loyiha"><h3>O'quvchi kartasi</h3><p>CSS dizayn</p>
        </article>
            <article class="loyiha"><h3>Kalkulyator</h3><p>Tez kunda...</p>
        </article>
        </div>
    </section>
</main>
<footer class="footer">© 2026 Ali</footer>

```

- 4 "Men haqimda" bo'limini qo'shing: chapda rasmingiz, o'ngda matn — **Flexbox** bilan (`display: flex; gap: 30px; align-items: center`).
- 5 "Ko'nikmalarim" bo'limini qo'shing: HTML, CSS, JavaScript, Python... nishonlari (badge) — `flex-wrap: wrap` bilan.
- 6 F12 → Elements'da `display: grid` yonidagi **grid** belgisini bosing — brauzer panjarani chizib ko'rsatadi. Flex uchun ham xuddi shunday.

Ko'p uchraydigan xatolar

- ❌ `display: flex` ni **farzandga** yozish — u **otaga** (konteynerga) yoziladi.
- ❌ `justify-content: center` vertikal markazlamayapti — vertikal uchun `align-items` kerak (va konteynerning balandligi bo'lishi kerak!).
- ❌ `box-sizing: border-box` ni unutish — elementlar kutilganidan kengroq chiqib, qatorga sig'may qoladi.
- ❌ `inline` elementga (`a`, `span`) `width` / `height` berish — ishlamaydi; `display: inline-block` yoki `block` kerak.
- ❌ Grid'da `grid-template-columns: 1fr, 1fr` — vergul qo'yilmaydi: `1fr 1fr`.

O'zingizni tekshiring

? Element chegarasi bilan mazmuni orasidagi bo'shliq nima deyiladi?

- A. margin
- B. border
- C. padding
- D. gap

? `width: 200px; padding: 10px; border: 5px solid;` + `box-sizing: border-box`. Elementning to'liq kengligi?

- A. 230px
- B. 200px
- C. 215px
- D. 210px

? Menyuda logotipni chapga, havolalarni o'ngga surish uchun konteynerga nima yoziladi?

- A. `display: flex; justify-content: space-between`
- B. `display: flex; align-items: center`
- C. `display: grid; gap: 10px`
- D. `text-align: justify`

? Root shrift 16px bo'lsa, `2rem` necha piksel?

- A. 2px
- B. 16px
- C. 18px
- D. 32px

? `grid-template-columns: 1fr 3fr;` — konteyner 800px bo'lsa (gap yo'q), birinchi ustun necha piksel?

- A. 400px
- B. 100px
- C. 200px
- D. 267px

? Butun sahifa maketi (header, yon panel, asosiy qism, footer) uchun eng qulay vosita?

- A. Faqat Flexbox
- B. CSS Grid (`grid-template-areas`)
- C. HTML jadval
- D. `float`

Uyga vazifa

- Flexbox Froggy (24 daraja) va Grid Garden (28 daraja) o'yinlarini to'liq o'ting.
- Portfolio saytingizni yakunlang: "Men haqimda" (Flex), "Loyihalar" (Grid), "Ko'nikmalar" (flex-wrap), "Aloqa" (forma) va footer.
- Rasm galereyasi sahifasi yarating: 12 ta rasm, ulardan 2 tasi `span 2` bilan katta.

Yodda tuting

- Box model: content + padding + border + margin. Har doim `* { box-sizing: border-box; }`.
- Birliklar: `px` (aniq), `%` (otaga), `rem` (shriftlar), `vw/vh` (ekranga).
- Flexbox (konteynerga `display: flex`): `justify-content` — asosiy o'q, `align-items` — kesishgan o'q, `gap`, `flex: 1`.

- Grid: `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))`, `span`, `grid-template-areas`.

✓ Test javoblari

1. Element chegarasi bilan mazmuni orasidagi bo'shliq nima deyiladi? — **C) padding**
padding — ichki, margin — tashqi bo'shliq.
2. width: 200px; padding: 10px; border: 5px solid; + box-sizing: border-box. Elementning to'liq kengligi? — **B) 200px**
border-box'da width padding va border'ni o'z ichiga oladi.
3. Menyuda logotipni chapga, havolalarni o'ngga surish uchun konteynerga nima yoziladi? — **A)**
display:flex; justify-content: space-between
4. Root shrift 16px bo'lsa, 2rem necha piksel? — **D) 32px**
5. grid-template-columns: 1fr 3fr; — konteyner 800px bo'lsa (gap yo'q), birinchi ustun necha piksel? — **C) 200px**
Jami 4 ulush: $800/4 = 200\text{px}$; ikkinchisi 600px.
6. Butun sahifa maketi (header, yon panel, asosiy qism, footer) uchun eng qulay vosita? — **B) CSS Grid**
(grid-template-areas)



09-MAVZU · 42 TADAN · I BOB. HTML VA CSS

CSS pseudo-class va pseudo-elementlar. CSS position

:hover, :focus, :active, :nth-child kabi holat selektorlari; ::before, ::after, ::first-letter pseudo-elementlari; static, relative, absolute, fixed, sticky pozitsiyalari va z-index.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Element **holatiga** qarab uslub berishni: sichqoncha ustida, bosilganda, tanlanganda;
- Ro'yxatning birinchi, oxirgi, juft-toq elementlarini tanlashni;
- HTML'siz "virtual" elementlar yaratishni (`::before` , `::after`);
- Elementlarni aniq joyga qo'yish: `relative` , `absolute` , `fixed` , `sticky` ;
- Ustma-ust tushgan elementlar tartibini `z-index` bilan boshqarishni.

Pseudo-class — element holati

Pseudo-class (soxta sinf) — elementning **holatini** yoki **o'rnini** bildiruvchi selektor. U **bitta ikki nuqta** bilan yoziladi: `a:hover` .



O'xshatish: svetofor

Svetofor bitta, lekin holati o'zgaradi: qizil, sariq, yashil. Tugma ham bitta, lekin holatlari bor: oddiy, sichqoncha ustida (`:hover`), bosilgan payt (`:active`), klaviatura bilan tanlangan (`:focus`). Har bir holatga alohida "rang" beramiz.

Foydalanuvchi harakati holatlari

HTML

```
<style>
  .tugma {
    background: #4f46e5; color: white; border: none;
    padding: 12px 24px; border-radius: 8px; font-size: 16px; cursor: pointer;
  }
  .tugma:hover { background: #4338ca; } /* sichqoncha ustida
*/
  .tugma:active { transform: scale(0.95); } /* bosilgan payt */
  .tugma:focus { outline: 3px solid #fbbf24; } /* Tab bilan
tanlanganda */

  input { padding: 8px; border: 2px solid #cbd5e1; border-radius: 6px; }
  input:focus { border-color: #4f46e5; outline: none; background: #eef2ff; }

  a:link { color: #2563eb; } /* hali bosilmagan havola */
  a:visited { color: #7c3aed; } /* ilgari bosilgan havola */
  a:hover { color: #dc2626; }

  button:disabled { background: #9ca3af; cursor: not-allowed; }
</style>
<button class="tugma">Ustiga olib boring va bosing</button>
<button class="tugma" disabled>O'chirilgan tugma</button>
<p><input placeholder="Meni bosing (focus)"></p>
<p><a href="#">Havola</a></p>
```



"LoVe HAte" qoidasi

Havola holatlarini shu tartibda yozing: `:link` → `:visited` → `:hover` → `:active` (L-V-H-A). Aks holda ba'zilar ishlamay qoladi.



`cursor: pointer` — sichqoncha ko'rsatkichini "qo'l"ga aylantiradi. Bosiladigan har qanday narsaga qo'ying, foydalanuvchi uni bosish mumkinligini tushunadi.

Tuzilma bo'yicha tanlash

HTML

```
<style>
  li { padding: 6px 10px; }
  li:first-child { font-weight: bold; color: #4f46e5; } /* birinchi */
  li:last-child { color: #dc2626; } /* oxirgi */
  li:nth-child(3) { background: #fef3c7; } /* uchinchi */

  tr:nth-child(even) { background: #f1f5f9; } /* juft qatorlar (2,
4, 6...) */
  tr:nth-child(odd) { background: white; } /* toq qatorlar */
  td, th { padding: 6px 14px; }

  p:not(.oddiy) { font-style: italic; } /* .oddiy dan
TASHQARI hammasi */
</style>
<ol>
  <li>Birinchi – ko'k va qalin</li>
  <li>Ikkinchi</li>
  <li>Uchinchi – sariq fon</li>
  <li>To'rtinchi</li>
  <li>Oxirgi – qizil</li>
</ol>
<table>
  <tr><th>Ism</th><th>Ball</th></tr>
  <tr><td>Ali</td><td>95</td></tr>
  <tr><td>Vali</td><td>87</td></tr>
  <tr><td>Gani</td><td>92</td></tr>
  <tr><td>Sobir</td><td>78</td></tr>
</table>
<p class="oddiy">Oddiy paragraf</p>
<p>Bu kursiv (chunki .oddiy emas)</p>
```

Selektor	Nimani tanlaydi
<code>:first-child</code> / <code>:last-child</code>	Otasining birinchi / oxirgi farzandi
<code>:nth-child(3)</code>	3-farzand
<code>:nth-child(even)</code> / <code>(odd)</code>	Juft / toq o'rindagilar ("zebra" jadval)

<code>:nth-child(3n)</code>	Har uchinchi: 3, 6, 9...
<code>:not(.x)</code>	<code>.x</code> sinfi bo'lmaganlar
<code>:checked</code>	Belgilangan checkbox/radio
<code>:valid</code> / <code>:invalid</code>	Forma maydoni to'g'ri / noto'g'ri to'ldirilgan

Pseudo-element — virtual qism

Pseudo-element — elementning **bir qismini** tanlaydi yoki HTML'da yo'q **yangi qism yaratadi**. U **ikki nuqta ikki marta** bilan yoziladi: `:: before`.

```

<style>
  /* ::before va ::after — element ichida, boshida/oxirida virtual "bola"
  yaratadi.
      content xossasi MAJBURIY (hatto bo'sh bo'lsa ham: content: "") */
  .majburiy::after { content: " *"; color: red; font-weight: bold; }
  .yangi::before { content: "🔥 YANGI "; color: #f97316; font-size: 12px; font-
  weight: bold; }
  .tel::before { content: "☎ "; }

  /* Bezak chiziq: sarlavha ostida qisqa rangli chiziq */
  h2.chiziqli::after {
    content: "";
    display: block;
    width: 60px; height: 4px;
    background: #4f46e5; border-radius: 2px; margin-top: 6px;
  }

  p.kitob::first-letter { font-size: 2.5em; color: #4f46e5; font-weight: bold;
  float: left; margin-right: 6px; line-height: 1; }
  p.kitob::first-line { font-variant: small-caps; }
  ::selection { background: #fde047; } /* sichqoncha bilan belgilangan
  matn */
  input::placeholder { color: #a78bfa; font-style: italic; }
</style>
<label class="majburiy">Ism</label><br>
<p class="yangi">Robototexnika to'garagi ochildi</p>
<p class="tel">+998 90 123 45 67</p>
<h2 class="chiziqli">Bo'lim sarlavhasi</h2>
<p class="kitob">Qadim zamonlarda bir dono podsho yashagan ekan. U o'z yurtida
ilm-fanni rivojlantirish uchun ko'p ishlar qilibdi. Matnni sichqoncha bilan
belgilab ko'ring.</p>
<input placeholder="Placeholder rangi o'zgargan">

```



`::before` / `::after` yordamida yaratilgan matn ekran o'quvchilarda har doim ham o'qilmaydi va uni nusxalab bo'lmaydi. Ularni faqat **bezak** uchun ishlatib; muhim ma'lumot HTML'da bo'lsin.

Position — elementni aniq joyga qo'yish

Odatda elementlar **oqim** bo'yicha — yuqoridan pastga, birin-ketin joylashadi. `position` xossasi bu tartibni o'zgartirishga imkon beradi. Joy `top`, `right`, `bottom`, `left` xossalari bilan ko'rsatiladi.

Qiymat	Xatti-harakat	Nimaga nisbatan siljiydi	Misol
<code>static</code>	Standart — oddiy oqimda	Siljimaydi	Hamma elementlar
<code>relative</code>	Oqimda qoladi, joyi saqlanadi	O'zining asl joyiga	Kichik siljitish; <code>absolute</code> uchun "langar"
<code>absolute</code>	Oqimdan chiqadi (joy egallamaydi)	Eng yaqin <code>position</code> li ajdodiga	Kartadagi "-20%" belgisi
<code>fixed</code>	Oqimdan chiqadi, scroll qilinganda qimirlamaydi	Ekranga (brauzer oynasiga)	"Yuqoriga" tugmasi, chat tugmasi
<code>sticky</code>	Oddiy turadi, scroll paytida chegaraga yetgach yopishib qoladi	Scroll konteyneriga	Yopishqoq menyu

relative + absolute — eng ko'p ishlatiladigan juftlik

HTML

```
<style>
.karta {
  position: relative;          /* ← "langar": ichidagi absolute elementlar
shunga nisbatan joylashadi */
  width: 260px; border-radius: 12px; overflow: hidden;
  box-shadow: 0 6px 20px rgba(0,0,0,.12); font-family: sans-serif;
}
.karta img { width: 100%; display: block; }
.chegirma {
  position: absolute;          /* ← oqimdan chiqdi */
  top: 12px;                   /* kartaning tepasidan 12px */
  left: 12px;                  /* chapidan 12px */
  background: #ef4444; color: white; font-weight: bold; padding: 4px 10px;
border-radius: 6px;
}
.sevimli {
  position: absolute; top: 12px; right: 12px;
  background: white; width: 36px; height: 36px; border-radius: 50%;
  display: grid; place-items: center; font-size: 18px;
}
.karta-matn { padding: 12px 16px; }
</style>
<div class="karta">
  
  <span class="chegirma">−20%</span>
  <span class="sevimli">♥</span>
  <div class="karta-matn"><b>Smartfon X</b><br>3 999 000 so'm</div>
</div>
```



O'xshatish: muzlatgichdagi magnit

`position: relative` berilgan karta — muzlatgich eshigi. `position: absolute` berilgan belgi — magnit: siz uni eshikning istalgan burchagiga yopishtirasiz (`top` , `left` ...). Eshik (karta) qayerga borsa, magnit ham u bilan birga ketadi. Agar ota elementda `position: relative` bo'lmasa, "magnit" butun sahifa burchagiga uchib ketadi!

fixed va sticky

HTML

```
<style>
  body { margin: 0; font-family: sans-serif; }
  .menyu { position: sticky; top: 0; background: #4f46e5; color: white;
padding: 14px; z-index: 10; }
  .yuqoriga {
    position: fixed; bottom: 20px; right: 20px;
    width: 50px; height: 50px; border-radius: 50%;
    background: #f97316; color: white; display: grid; place-items: center;
    font-size: 22px; text-decoration: none; box-shadow: 0 4px 12px
    rgba(0,0,0,.3);
  }
  .matn { padding: 20px; }
  .matn p { height: 120px; background: #f1f5f9; margin: 10px 0; padding: 10px;
}
</style>
<div class="matn"><b>Sahifani pastga aylantiring...</b></div>
<nav class="menyu">Men sticky menyuman – tepaga yetganda yopishib qolaman</nav>
<div class="matn">
  <p>1-paragraf</p><p>2-paragraf</p><p>3-paragraf</p><p>4-paragraf</p><p>5-
paragraf</p><p>6-paragraf</p>
</div>
<a class="yuqoriga" href="#">↑</a>
```

z-index — qatlamlar tartibi

Elementlar ustma-ust tushsa, qaysi biri yuqorida ko'rinadi? **z-index** — "qavat raqami": kattasi yuqorida. U faqat **position** berilgan (static bo'lmagan) elementlarda ishlaydi.

```

<style>
  .q { position: absolute; width: 120px; height: 120px; color: white; font-
weight: bold; padding: 8px; border-radius: 10px; }
  .a { background: #ef4444; top: 20px; left: 20px; z-index: 1; }
  .b { background: #22c55e; top: 60px; left: 70px; z-index: 3; }
  .c { background: #3b82f6; top: 100px; left: 120px; z-index: 2; }
  .sahna { position: relative; height: 260px; }
</style>
<div class="sahna">
  <div class="q a">z-index: 1</div>
  <div class="q b">z-index: 3 (eng yuqorida)</div>
  <div class="q c">z-index: 2</div>
</div>

```

Amaliy mashg'ulot: maktab sahifasi (3-loyiha)

- 1 maktab/ papkasida `index.html` va `style.css` yarating.
- 2 **Sticky header:** maktab logotipi va menyu, `position: sticky; top: 0; z-index: 100;`. Menyu havolalariga `:hover` holatini qo'shing.
- 3 **Hero bo'lim:** maktab rasmi fonda (7-mavzu), ustida sarlavha.
- 4 **"Yangiliklar":** 3–4 ta karta (Grid, 8-mavzu). Har bir kartaning burchagida `position: absolute` bilan sana belgisi (masalan, "15 OKT"). Kartaga `:hover` da soya kuchaysin.
- 5 **"Ustozlarimiz":** doira shaklidagi rasmlar va ism; `:hover` bo'lganda rasm atrofida rangli chegara paydo bo'lsin.
- 6 **Dars jadvali:** `tr:nth-child(even)` bilan "zebra" uslubi, `tr:hover` bilan qatorni ajratib ko'rsatish.
- 7 **"Yuqoriga" tugmasi:** `position: fixed` bilan o'ng pastki burchakda.
- 8 Sarlavhalar ostiga `::after` bilan qisqa rangli chiziq qo'shing.

```
.yangilik { position: relative; background: white; border-radius: 14px;
overflow: hidden;
            box-shadow: 0 4px 14px rgba(0,0,0,.08); transition: box-shadow .2s;
}
.yangilik:hover { box-shadow: 0 12px 30px rgba(0,0,0,.18); }
.yangilik .sana { position: absolute; top: 12px; left: 12px; background: white;
border-radius: 8px; padding: 4px 10px; font-weight: 800;
color: #4f46e5;
                text-align: center; line-height: 1.1; }
.yangilik img { width: 100%; height: 160px; object-fit: cover; display: block;
}
```

Ko'p uchraydigan xatolar

- ❌ `absolute` element butun sahifa burchagiga "uchib ketdi" — ota elementga `position: relative` berilmagan.
- ❌ `::before` ko'rinmayapti — `content` xossasi yozilmagan.
- ❌ `sticky` ishlamayapti — `top: 0` berilmagan yoki ota elementda `overflow: hidden` bor.
- ❌ `z-index: 9999` ham yordam bermayapti — elementda `position` yo'q (static).
- ❌ `a:hover` ni `a :hover` deb yozish (orada bo'sh joy) — bu butunlay boshqa selektor bo'lib qoladi.

O'zingizni tekshiring

? Sichqoncha element ustiga kelganda ishlaydigan pseudo-class qaysi?

- A. `:active`
- B. `:hover`
- C. `:focus`
- D. `:over`

? Jadvalning faqat juft qatorlarini tanlash uchun?

- A. `tr:even`
- B. `tr:nth(2)`
- C. `tr:first-child`
- D. `tr:nth-child(even)`

? `::after` ishlashi uchun qaysi xossa majburiy?

- A. `content`
- B. `display`
- C. `position`
- D. `color`

? Sahifa aylantirilganda ham ekranning bir joyida qimirlamay turadigan tugma uchun qaysi position?

- A. `relative`
- B. `absolute`
- C. `fixed`
- D. `static`

? Kartaning burchagidagi "-20%" belgisi kartaga nisbatan joylashishi uchun kartaga nima beriladi?

- A. `position: absolute`
- B. `position: relative`
- C. `position: fixed`
- D. `z-index: 1`

? `z-index: 5` va `z-index: 2` bo'lgan elementlar ustma-ust tushdi. Qaysi biri ko'rinadi?

- A. `z-index: 2`
- B. HTML'da birinchi yozilgani
- C. `z-index: 5`
- D. Ikkalasi ham ko'rinmaydi

Uyga vazifa

- Maktab sahifasini yakunlang va barcha interaktiv holatlarni (`:hover` , `:focus`) qo'shing.
- 5 xil tugma dizaynini yarating: har birida `:hover` , `:active` va `:disabled` holatlari bo'lsin.
- Faqat CSS bilan "tooltip" yasang: sichqoncha so'z ustiga borganda uning tepasida izoh chiqsin (`position: absolute` + `:hover`).

Yodda tuting

- Pseudo-class (`:`) — holat: `:hover` , `:focus` , `:active` , `:nth-child()` , `:first-child` , `:not()`.
- Pseudo-element (`::`) — virtual qism: `::before` , `::after` (`content` majburiy), `::first-letter` , `::placeholder`.
- `relative` — langar; `absolute` — langarga nisbatan; `fixed` — ekranga; `sticky` — yopishqoq.

- `z-index` — qatlamlar tartibi (faqat position berilganlarda).

✓ Test javoblari

1. Sichqoncha element ustiga kelganda ishlaydigan pseudo-class qaysi? — **B) :hover**
2. Jadvalning faqat juft qatorlarini tanlash uchun? — **D) tr:nth-child(even)**
3. ::after ishlashi uchun qaysi xossa majburiy? — **A) content**
4. Sahifa aylantirilganda ham ekranning bir joyida qimirlamay turadigan tugma uchun qaysi position? — **C) fixed**
5. Kartaning burchagidagi "-20%" belgisi kartaga nisbatan joylashishi uchun kartaga nima beriladi? — **B) position: relative**
Ota — relative (langar), belgi — absolute.
6. z-index: 5 va z-index: 2 bo'lgan elementlar ustma-ust tushdi. Qaysi biri ko'rinadi? — **C) z-index: 5**



10-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Moslashuvchan (responsiv) dizayn va media so'rovlar. Figma bilan ishlash

Responsiv dizayn nima, viewport, moslashuvchan rasmlar, media query sintaksisi, mobile-first yondashuv, breakpointlar, DevTools'da telefon rejimi; Figma'da maket bilan ishlash va uni kodga o'tkazish.

3 soat · 1 nazariy + 2 amaliy

Bu darsda nimani o'rganasiz

- Nima uchun saytlar telefon, planshet va kompyuterda turlicha ko'rinishi kerakligini;
- `viewport` meta-tegi va moslashuvchan rasmlarni;
- **Media query** — ekran o'lchamiga qarab turli CSS yozishni;
- **Mobile-first** yondashuvini va breakpointlarni;
- **Figma** dasturida maket bilan ishlash va uni HTML/CSS'ga aylantirishni;
- To'liq responsiv Landing Page yaratishni (4 va 5-loyihalar).

Responsiv dizayn nima?

Bugun internetdan foydalanuvchilarning **60% dan ortig'i** telefondan kiradi. Kompyuter uchun yasalgan 1200 piksellik sayt 390 piksellik telefon ekranida yo juda mayda ko'rinadi, yo yonga sig'may qoladi.

Responsiv (moslashuvchan) dizayn — sayt **har qanday ekran o'lchamiga** o'zi moslashishi.



O'xshatish: suv

Suv qaysi idishga quyilsa — stakan, choynak yoki chelak — o'sha shaklni oladi. Responsiv sayt ham suvga o'xshaydi: telefon, planshet yoki katta monitor — har biriga moslashadi. Bunga erishishning 3 asosiy vositasi bor: **moslashuvchan maket** (`%`, `fr`, `flex`, `grid`), **moslashuvchan rasmlar** va **media so'rovlar**.

Istalgan katta saytni (masalan, kun.uz) oching va brauzer oynasini asta-sekin toraytiring — menyu, ustunlar va rasmlar qanday o'zgarishini kuzating.

1-qadam: viewport meta-tegi

HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Bu qator bo'lmasa, telefon brauzeri sahifani "980 piksellik kompyuter ekrani" deb o'ylaydi va hammasini kichraytirib ko'rsatadi — media query'lar umuman ishlamaydi. `width=device-width` — "sahifa kengligi = qurilma ekran kengligi". Bu qatorni **har bir sahifaga** yozing (VS Code'dagi  shabloni uni avtomatik qo'shadi).

2-qadam: moslashuvchan o'lchamlar va rasmlar

CSS

```
/* ❌ Qattiq o'lcham — telefonda sig'maydi */
.konteyner { width: 1100px; }

/* ✅ Moslashuvchan: kengligi ekranga qarab, lekin 1100px dan oshmaydi */
.konteyner {
  width: 100%;
  max-width: 1100px;
  margin: 0 auto;
  padding: 0 16px;      /* telefonda matn ekran chetiga yopishmasin */
}

/* ✅ Rasmlar hech qachon konteynerdan chiqib ketmasin */
img {
  max-width: 100%;
  height: auto;         /* proporsiya saqlansin */
}

/* ✅ Shrift o'lchami ekranga qarab silliq o'zgaradi: kamida 1.8rem, ko'pi
bilan 3.5rem */
h1 { font-size: clamp(1.8rem, 5vw, 3.5rem); }
```

8-mavzudagi `repeat(auto-fill, minmax(220px, 1fr))` ham responsivlikning kuchli vositasi — ustunlar soni o'zi o'zgaradi.

3-qadam: media so'rovlar (media queries)

Media query — "agar ekran shunday bo'lsa, quyidagi CSS'ni qo'lla" degan **shart**.

CSS

```
@media (max-width: 768px) {  
  /* Bu qoidalar FAQAT ekran kengligi 768px yoki undan KICHIK bo'lganda  
  ishlaydi */  
  .menyu { flex-direction: column; }  
  h1 { font-size: 1.8rem; }  
}  
  
@media (min-width: 1024px) {  
  /* FAQAT ekran 1024px yoki undan KATTA bo'lganda */  
  .kartalar { grid-template-columns: repeat(4, 1fr); }  
}
```

- `max-width: 768px` — "**ko'pi bilan** 768px" (ya'ni 768 va undan kichik ekranlar).
- `min-width: 1024px` — "**kamida** 1024px" (1024 va undan katta).
- Shartlarni birlashtirish: `@media (min-width: 600px) and (max-width: 900px) { ... }`

HTML

```
<style>  
  body { font-family: sans-serif; background: #e0e7ff; transition: background  
.3s; }  
  .xabar::after { content: "💻 Katta ekran (900px dan katta)"; }  
  
  @media (max-width: 900px) {  
    body { background: #fef3c7; }  
    .xabar::after { content: "📱 O'rta ekran (600-900px)"; }  
  }  
  @media (max-width: 600px) {  
    body { background: #dcfce7; }  
    .xabar::after { content: "📱 Kichik ekran (600px gacha)"; }  
  }  
</style>  
<h2 class="xabar"></h2>  
<p>📺 Yangi oynada" bosing va brauzer oynasini toraytirib-kengaytirib ko'ring —  
fon rangi va yozuv o'zgaradi.</p>
```

Breakpointlar (to'xtash nuqtalari)

Breakpoint — dizayn o'zgaradigan ekran kengligi. Ko'p ishlatiladiganlari:

Qurilma	Taxminiy kenglik	Odatiy maket
 Telefon	360–480px	1 ustun, "burger" menyu
 Katta telefon / kichik planshet	481–768px	1–2 ustun
 Planshet	769–1024px	2–3 ustun
 Noutbuk / kompyuter	1025px+	3–4 ustun, to'liq menyu



Breakpointlarni qurilmalarga emas, **dizayningizga** qarab tanlang: oynani toraytiring va dizayn "buzila boshlagan" joyda breakpoint qo'ying.

Mobile-first yondashuvi

Ikki yo'l bor:

- **Desktop-first**: avval katta ekran uchun yozasiz, keyin `max-width` bilan kichraytirasiz.
- **Mobile-first** : avval **telefon** uchun oddiy CSS, keyin `min-width` bilan katta ekranlar uchun qo'shimchalar.

CSS

```
/* MOBILE-FIRST */

/* 1. Standart – telefon: hammasi bitta ustunda */
.kartalar { display: grid; grid-template-columns: 1fr; gap: 16px; }

/* 2. Planshet va undan katta */
@media (min-width: 640px) {
  .kartalar { grid-template-columns: repeat(2, 1fr); }
}

/* 3. Kompyuter */
@media (min-width: 1024px) {
  .kartalar { grid-template-columns: repeat(4, 1fr); }
}
```

Nima uchun mobile-first yaxshiroq? Telefon uchun dizayn odatda oddiyroq (bitta ustun) — undan murakkabga o'tish osonroq. Telefonlar kamroq CSS yuklaydi va eng ko'p foydalanuvchi aynan telefonda.

DevTools'da telefon rejimi

- 1 Sahifangizni Chrome'da oching, `F12` bosing.
- 2 Chap yuqoridagi telefon-planshet belgisini bosing (**Toggle device toolbar**, `Ctrl` + `Shift` + `M`).
- 3 Tepadan qurilmani tanlang: iPhone SE, Pixel 7, iPad... yoki "Responsive" rejimida chetini sudrab o'lchamni o'zgartiring.
- 4 Eng muhim tekshiruv: **gorizontal scroll** (yonga aylantirish) paydo bo'lmasligi kerak.

Responsiv navigatsiya

Kompyuterda menyu gorizontal, telefonda esa sig'magani uchun "☰ burger" tugmasi orqali ochiladi. JavaScript'siz, faqat CSS bilan:

```

<style>
  * { box-sizing: border-box; margin: 0; }
  body { font-family: sans-serif; }
  .header { display: flex; justify-content: space-between; align-items: center;
flex-wrap: wrap;
      padding: 14px 20px; background: #1e1b4b; color: white; }
  .logo { font-weight: 800; font-size: 1.3rem; }
  .menu { display: flex; gap: 20px; }
  .menu a { color: white; text-decoration: none; }
  #burger, .burger-tugma { display: none; }

  @media (max-width: 640px) {
    .burger-tugma { display: block; font-size: 1.6rem; cursor: pointer; }
    .menu { display: none; width: 100%; flex-direction: column; gap: 12px;
padding-top: 14px; }
    #burger:checked ~ .menu { display: flex; } /* checkbox belgilansa -
menu ochiladi */
  }
</style>
<header class="header">
  <div class="logo">Maktab №5</div>
  <input type="checkbox" id="burger">
  <label for="burger" class="burger-tugma">☰</label>
  <nav class="menu">
    <a href="#">Bosh sahifa</a><a href="#">Yangiliklar</a><a
href="#">Ustozlar</a><a href="#">Aloqa</a>
  </nav>
</header>
<p style="padding:20px">Yangi oynada ochib, oynani 640px dan toraytiring va ☰
ni bosing.</p>

```

Hiyla: yashirin `checkbox` va uning `label` i tugma vazifasini bajaradi. `:checked ~ .menu` — "checkbox belgilangan bo'lsa, undan keyingi `.menu` ni ko'rsat". 19-mavzuda buni JavaScript bilan ham qilamiz.

Figma — dizayndan kodga

Professional jamoalarda sayt avval **dizayner** tomonidan maxsus dasturda chiziladi, keyin **frontend dasturchi** uni kodga aylantiradi. Eng mashhur dastur — **Figma**. U brauzerda ishlaydi va o'quvchilar uchun

bepul tarifi bor.

Figma bilan tanishish

- 1
- figma.com da Gmail orqali ro'yxatdan o'ting.
- 2
- Figma Community** bo'limidan bepul "Landing page" yoki "Portfolio" shablonini toping va **Open in Figma** bosing (nusxa sizning akkauntingizga tushadi).
- 3
- Asosiy tushunchalar:
 - **Frame** — ekran yoki sahifa (masalan, "Desktop 1440", "iPhone 14"). Ichidagi elementlarning "konteyneri".
 - **Layers** (chap panel) — barcha elementlar daraxti. HTML tuzilmasiga juda o'xshaydi!
 - **Design** (o'ng panel) — tanlangan elementning o'lchami, rangi, shrifti, radiusi, soyasi.
 - **Auto layout** — Figma'dagi Flexbox! Elementlarni qator yoki ustunga teradi, `gap` va `padding` beradi.

Dizayndan qiymatlarni olish

Elementni tanlang va o'ng paneldan o'qing — yoki yuqoridagi **Dev Mode** (`Shift` + `D`) ni yoqing, Figma tayyor CSS ko'rsatadi:

Figma'da	CSS'da
Fill: #4F46E5	<code>background-color: #4f46e5;</code> (matn uchun — <code>color</code>)
Text: Inter, Bold, 48	<code>font-family: "Inter"; font-weight: 700; font-size: 48px;</code>
Corner radius: 12	<code>border-radius: 12px;</code>
Stroke: 1, #E5E7EB, Inside	<code>border: 1px solid #e5e7eb;</code>
Drop shadow: X 0, Y 4, Blur 12, #000 10%	<code>box-shadow: 0 4px 12px rgba(0,0,0,.1);</code>
Auto layout: horizontal, gap 24, padding 16 32	<code>display:flex; gap:24px; padding:16px 32px;</code>
Auto layout: vertical, center	<code>flex-direction:column; align-items:center;</code>



Elementlar orasidagi masofani o'lchash

Bitta elementni tanlab, `Alt` tugmasini bosib turing va sichqonchani boshqa element ustiga olib boring — Figma ular orasidagi masofani qizil raqam bilan ko'rsatadi.

Rasmlarni eksport qilish

Rasm yoki ikonkani tanlang → o'ng paneldagi **Export** → formatni tanlang (rasmlar uchun PNG/JPG, ikonkalar uchun SVG) → **Export**.

Maketni kodlash tartibi

- 1 Dizaynni "bloklarga" bo'ling: header, hero, xizmatlar, sharhlar, footer. Har biri — HTML'da `section`.
- 2 Avval faqat **HTML tuzilmasini** yozing (dizaynsiz).
- 3 Ranglar va shriftlarni bir joyga yig'ing (CSS boshida izoh sifatida yoki keyinchalik CSS o'zgaruvchilari bilan).
- 4 Bloklarni yuqoridan pastga ketma-ket uslublang. Avval katta maket (flex/grid), keyin detallar.
- 5 Oxirida telefon versiyasini (Figma'da mobil frame bo'lsa — unga qarab) media query bilan qo'shing.

Amaliy mashg'ulot: responsiv Landing Page (4-loyiha)

"IT Akademiya" uchun bir sahifali reklama sayti. Avval to'liq kodni o'qing, ► bilan sinab ko'ring, so'ng o'zingizning mavzungizda (kitob do'koni, sport zali, sayohat) qayta yarating.


```

<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>IT Akademiya – dasturlashni o'rganing</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: "Segoe UI", Arial, sans-serif; color: #1f2937; line-height: 1.6; }
  img { max-width: 100%; height: auto; display: block; }
  .konteyner { width: 100%; max-width: 1100px; margin: 0 auto; padding: 0 16px; }

  /* ----- MOBILE (standart) ----- */
  .header { position: sticky; top: 0; background: #fff; box-shadow: 0 2px 8px rgba(0,0,0,.06); z-index: 10; }
  .header .konteyner { display: flex; justify-content: space-between; align-items: center; padding-top: 14px; padding-bottom: 14px; }
  .logo { font-weight: 800; font-size: 1.3rem; color: #4f46e5; }
  .menyu { display: none; } /* telefonda yashirin */
  .header .tugma { padding: 8px 14px; font-size: .9rem; }

  .tugma { display: inline-block; background: #4f46e5; color: #fff; text-decoration: none; padding: 12px 24px; border-radius: 999px; font-weight: 600; }

  .hero { padding: 48px 0; background: linear-gradient(135deg, #eef2ff, #fdf2f8); }
  .hero .konteyner { display: flex; flex-direction: column; gap: 28px; text-align: center; align-items: center; }
  .hero h1 { font-size: clamp(1.9rem, 6vw, 3.2rem); line-height: 1.15; }
  .hero h1 span { color: #4f46e5; }
  .hero p { color: #4b5563; font-size: 1.1rem; }
  .hero img { border-radius: 20px; }

  .bolim { padding: 56px 0; }
  .bolim h2 { text-align: center; font-size: 1.9rem; margin-bottom: 28px; }
  .kurslar { display: grid; grid-template-columns: 1fr; gap: 18px; }
  .kurs { border: 1px solid #e5e7eb; border-radius: 16px; padding: 22px; box-shadow: 0 6px 18px rgba(0,0,0,.05); }
  .kurs .ikon { font-size: 2rem; }

```

```

.kurs h3 { margin: 6px 0; }
.narx { color: #4f46e5; font-weight: 800; margin-top: 8px; }

.raqamlar { background: #1e1b4b; color: #fff; }
.raqamlar .konteyner { display: grid; grid-template-columns: repeat(2, 1fr);
gap: 24px; text-align: center; }
.raqamlar b { display: block; font-size: 2rem; color: #a5b4fc; }

.footer { padding: 24px 0; text-align: center; color: #6b7280; font-size:
.9rem; }

/* ----- PLANSKET: 640px va undan katta ----- */
@media (min-width: 640px) {
.kurslar { grid-template-columns: repeat(2, 1fr); }
.raqamlar .konteyner { grid-template-columns: repeat(4, 1fr); }
}

/* ----- KOMPYUTER: 960px va undan katta ----- */
@media (min-width: 960px) {
.menyu { display: flex; gap: 28px; }
.menyu a { color: #374151; text-decoration: none; font-weight: 500; }
.menyu a:hover { color: #4f46e5; }
.hero { padding: 90px 0; }
.hero .konteyner { flex-direction: row; text-align: left; }
.hero .matn, .hero img { flex: 1; }
.kurslar { grid-template-columns: repeat(3, 1fr); }
}
</style>
</head>
<body>
<header class="header">
<div class="konteyner">
<div class="logo">IT Akademiya</div>
<nav class="menyu"><a href="#kurslar">Kurslar</a><a
href="#natijalar">Natijalar</a><a href="#aloqa">Aloqa</a></nav>
<a class="tugma" href="#aloqa">Yozilish</a>
</div>
</header>

<section class="hero">
<div class="konteyner">
<div class="matn">
<h1>Kelajak kasbini <span>bugun</span> o'rganing</h1>
<p>8-11-sinf o'quvchilari uchun dasturlash kurslari. Amaliy loyihalar

```

```

va tajribali mentorlar.</p>
    <br><a class="tugma" href="#kurslar">Kurslarni ko'rish</a>
</div>

</div>
</section>

<section class="bolim" id="kurslar">
    <div class="konteyner">
        <h2>Kurslarimiz</h2>
        <div class="kurslar">
            <article class="kurs"><div class="ikon">🌐</div><h3>Web Frontend</h3>
<p>HTML, CSS, JavaScript – 4 oy</p><p class="narx">450 000 so'm/oy</p>
</article>
            <article class="kurs"><div class="ikon">🐍</div><h3>Python Backend</h3>
<p>Python, Django, PostgreSQL – 5 oy</p><p class="narx">500 000 so'm/oy</p>
</article>
            <article class="kurs"><div class="ikon">🤖</div><h3>Telegram
botlar</h3><p>Python, Aiogram – 2 oy</p><p class="narx">400 000 so'm/oy</p>
</article>
        </div>
    </div>
</section>

<section class="bolim raqamlar" id="natijalar">
    <div class="konteyner">
        <div><b>1200+</b>bitiruvchi</div>
        <div><b>35</b>mentor</div>
        <div><b>87%</b>ishga joylashgan</div>
        <div><b>4.9★</b>reyting</div>
    </div>
</section>

<footer class="footer" id="aloqa">© 2026 IT Akademiya · +998 90 000 00
00</footer>
</body>
</html>

```

Topshiriq:

- 1 Kodni o'z mavzungizga moslang (ranglar, matnlar, rasmlar). Kamida 5 ta bo'lim: header, hero, xizmatlar/mahsulotlar, raqamlar yoki sharhlar, footer.

- 2 DevTools'da 3 ta o'lchamda tekshiring: iPhone SE (375px), iPad (768px), kompyuter (1280px). Gorizontaal scroll bo'lmasin!
- 3 Telefonda menyu yashiringan — yuqoridagi "burger" usulini qo'shing.
- 4 **5-loyiha:** Figma Community'dan oddiy bir sahifali shablonni tanlang va uni o'lchamlari, ranglari va shriftlarini Dev Mode'dan olib, imkon qadar aniq kodlang.

Ko'p uchraydigan xatolar

- ❌ `viewport` meta-tegini unutish — telefonda media query'lar ishlamaydi.
- ❌ `width: 1200px` kabi qattiq kengliklar — gorizontaal scroll paydo bo'ladi. `max-width` ishlatish.
- ❌ Media query'ni CSS faylning **boshiga** yozish — keyingi oddiy qoidalar uni "bosib" ketadi. Media query'lar asosiy qoidalaridan **keyin** yoziladi.
- ❌ `@media (max-width: 768)` — birlik (`px`) unutilgan.
- ❌ Faqat kompyuterda tekshirish. Haqiqiy telefonda ham oching (Live Server + bir Wi-Fi tarmog'ida telefondan kompyuter IP-manziliga kirish mumkin).

O'zingizni tekshiring

? `@media (max-width: 600px) { ... }` ichidagi qoidalar qachon ishlaydi?

- A. Ekran 600px dan katta bo'lganda
- B. Ekran 600px yoki undan kichik bo'lganda
- C. Faqat aynan 600px da
- D. Har doim

? **Mobile-first yondashuvida asosan qaysi shart ishlatiladi?**

- A. `min-width`
- B. `max-width`
- C. `orientation`
- D. `max-height`

? **Rasm konteynerdan chiqib ketmasligi uchun eng to'g'ri CSS?**

- A. `width: 1000px`
- B. `overflow: scroll`
- C. `width: 100vw`
- D. `max-width: 100%; height: auto;`

? **Figma'dagi "Auto layout" CSS'dagi qaysi texnologiyaga o'xshaydi?**

- A. `position: absolute`
- B. HTML jadval
- C. Flexbox
- D. Media query

? **`<meta name="viewport">` bo'lmasa telefonda nima bo'ladi?**

- A. Sayt umuman ochilmaydi
- B. Sahifa kompyuter kengligida, kichraytirilgan holda ko'rinadi
- C. Ranglar o'zgaradi
- D. Hech narsa o'zgarmaydi



Uyga vazifa

1. Portfolio va maktab saytlaringizni to'liq responsiv qiling (3 ta breakpoint).
2. Figma'da o'z portfoliingiz uchun oddiy maket chizing: 1 ta Desktop frame (1440px) va 1 ta Mobile frame (390px). Auto layout ishlating.
3. Telefoningizda 3 ta sevimli saytingizni oching va ularning mobil versiyasida nimalar o'zgarganini (menyu, ustunlar, shrift) yozib chiqing.



Yodda tuting

- Responsivlik = viewport meta + moslashuvchan o'lchamlar (`%` , `max-width` , `fr`) + `img { max-width: 100% }` + media query.
- `@media (min-width: 768px) { ... }` — shartli CSS; mobile-first'da `min-width` .
- DevTools → Device toolbar (`Ctrl` + `Shift` + `M`) bilan tekshiring; gorizontal scroll bo'lmasin.
- Figma: Frame, Layers, Auto layout (= Flexbox), Dev Mode'dan tayyor CSS qiymatlar, Export.

✓ Test javoblari

1. @media (max-width: 600px) { ... } ichidagi qoidalar qachon ishlaydi? — **B) Ekran 600px yoki undan kichik bo'lganda**
2. Mobile-first yondashuvida asosan qaysi shart ishlatiladi? — **A) min-width**
Avval telefon uchun yoziladi, keyin "kamida shuncha keng bo'lsa" deb katta ekranlar qo'shiladi.
3. Rasm konteynerdan chiqib ketmasligi uchun eng to'g'ri CSS? — **D) max-width: 100%; height: auto;**
4. Figma'dagi "Auto layout" CSS'dagi qaysi texnologiyaga o'xshaydi? — **C) Flexbox**
5. <meta name="viewport"> bo'lmasa telefonda nima bo'ladi? — **B) Sahifa kompyuter kengligida, kichraytirilgan holda ko'rinadi**



11-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Git va GitHub, hosting va SEO bo'yicha umumiy bilimlar

Versiyalarni boshqarish tizimi nima, Git'ni o'rnatish, init, add, commit, status, log; GitHub'da repozitoriya, push va clone; GitHub Pages; SEO asoslari: title, description, sarlavhalar, alt, tezlik.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Versiyalarni boshqarish nima va dasturchilarga nima uchun kerakligini;
- Git'ning asosiy buyruqlari: `init`, `status`, `add`, `commit`, `log`;
- GitHub'da repozitoriya yaratish va loyihani `push` qilishni;
- Saytni GitHub Pages orqali bepul internetga chiqarishni;
- Saytingiz Google'da yaxshi chiqishi uchun SEO asoslarini.

Versiyalarni boshqarish nima?

Tanish vaziyat: `referat.docx`, `referat_yangi.docx`, `referat_yangi_0XIRGI.docx`, `referat_yangi_0XIRGI_2.docx` ... Qaysi biri haqiqatan oxirgisi? Kecha nimani o'zgartirgan edingiz? Dasturchilar bu muammoni **versiyalarni boshqarish tizimi** bilan hal qiladi.

Git — eng mashhur versiyalarni boshqarish tizimi. U loyihangizning har bir saqlangan holatini ("surat"ini) eslab qoladi.



O'xshatish: o'yindagi saqlash nuqtalari

Kompyuter o'yinlarida qiyin bosqichdan oldin "save" qilasiz. Yutqazsangiz — o'sha nuqtaga qaytasiz. Git'dagi **commit** — xuddi shunday saqlash nuqtasi. Har bir commit'ga izoh yozasiz ("Menyu qo'shildi") va istalgan paytda istalgan nuqtaga qaytishingiz mumkin.

Git nima beradi?

- 📅 Loyihaning to'liq **tarixi**: kim, qachon, nimani o'zgartirgan.
- ⏮️ Xato qilsangiz — oldingi ishlaydigan holatga **qaytish**.
- 👥 **Jamoa** bo'lib bitta loyihada bir vaqtda ishlash (13-mavzu).
- ☁️ GitHub orqali kodni **internetda saqlash** — kompyuter buzilsa ham loyiha yo'qolmaydi.

Git va GitHub — farqi

	Git	GitHub
Nima	Dastur (kompyuteringizga o'rnatiladi)	Veb-sayt (github.com)
Vazifasi	Versiyalarni saqlaydi va boshqaradi	Git loyihalarini internetda saqlaydi, ulashadi
Internetsiz	Ishlaydi	Ishlamaydi
O'xshatish	Fotoapparat	Instagram (suratlarni saqlash va ulashish joyi)

Git'ni o'rnatish va sozlash

- 1 git-scm.com dan yuklab, o'rnatish (barcha sozlamalarni standart holda qoldiring).
- 2 VS Code'da terminalni oching: **Terminal** → **New Terminal** (`Ctrl` + ```).
- 3 Tekshiring va o'zingizni tanishtiring (bir marta qilinadi):

Terminal

```
git --version
git config --global user.name "Ismingiz Familiyangiz"
git config --global user.email "sizning@gmail.com"
```

Git'ning asosiy buyruqlari

Uchta hudud



O'xshatish: guruh surati

Ishchi papka — sinf xonasi, hamma o'z ishi bilan band. **git add** — suratga tushadiganlarni oldinga chaqirish. **git commit** — "chik!" — surat olindi va albomga (tarixga) qo'shildi.

Terminal

```
# 1. Loyiha papkasida Git'ni ishga tushirish (bir marta)
git init

# 2. Holatni ko'rish: qaysi fayllar yangi, qaysilari o'zgargan
git status

# 3. Fayllarni "suratga" tayyorlash
git add index.html      # bitta fayl
git add .               # barcha o'zgarishlar (nuqta = hammasi)

# 4. Saqlash nuqtasi (commit) yaratish — izoh bilan!
git commit -m "Bosh sahifa va menyu qo'shildi"

# 5. Tarixni ko'rish
git log --oneline
```

Natija

```
a3f9c2e Bosh sahifa va menyu qo'shildi
1b7d004 Loyiha boshlandi
```



Yaxshi commit izohi

Nima qilinganini aniq yozing: `"fix"`, `"o'zgarish"`, `"asdf"` ❌ → `"Aloqa sahifasiga forma qo'shildi"`, `"Telefonda menyu xatosi tuzatildi"` ✅. Kichik, mantiqiy qadamlar bilan tez-tez commit qiling.



VS Code'da chap paneldagi **Source Control** belgisi (shoxlangan chiziqlar) orqali ham xuddi shu ishlarni sichqoncha bilan qilish mumkin: "+" — add, matn yozib ✓ — commit.

GitHub'ga yuklash

- 1 github.com da ro'yxatdan o'ting (foydalanuvchi nomini jiddiy tanlang — u portfoliingiz manzili bo'ladi).
- 2 O'ng yuqoridagi + → **New repository**. Nom bering (masalan, `portfolio`), **Public** tanlang, "Add a README" belgisini **qo'ymang** → **Create repository**.
- 3 GitHub ko'rsatgan buyruqlarni terminalda bajaring:

Terminal

```
git branch -M main
git remote add origin https://github.com/USERNAME/portfolio.git
git push -u origin main
```

Birinchi marta brauzer orqali GitHub'ga kirish so'raladi — ruxsat bering.

- 4 GitHub sahifasini yangilang — fayllaringiz internetda! 🎉
- 5 Keyingi o'zgarishlardan so'ng uchta buyruq yetarli:

Terminal

```
git add .
git commit -m "Loyihalar bo'limi qo'shildi"
git push
```

Boshqa loyihani yuklab olish — `clone`

Terminal

```
git clone https://github.com/USERNAME/portfolio.git
# Maktab kompyuteridan uyga: o'zgarishlarni olish
git pull
```

 **GitHub'ga nimani yuklamaslik kerak**

Parollar, shaxsiy ma'lumotlar, juda katta fayllar (videolar). Ochiq (public) repozitoriyaning **butun dunyo** ko'radi.

Hosting: GitHub Pages

1-mavzuda hosting — sayt fayllari saqlanadigan server ekanini bildik. HTML/CSS/JS saytlar uchun **GitHub Pages** — bepul hosting:

- 1
- Repozitoriyada **Settings** → **Pages** bo'limiga kiring.
- 2
- Source:** "Deploy from a branch", **Branch:** `main`, papka `/ (root)` → **Save**.
- 3
- 1-2 daqiqadan so'ng saytingiz `https://USERNAME.github.io/portfolio/` manzilida ochiladi. Havolani do'stlaringizga yuboring!

Muhim: bosh sahifa fayli `index.html` deb nomlangan bo'lishi kerak. 13-mavzuda Netlify va Vercel'ni ham o'rganamiz.

SEO — Google'da yuqorida chiqish

SEO (Search Engine Optimization) — saytni qidiruv tizimlari uchun "tushunarli" qilish, natijada u qidiruvda yuqoriroq chiqadi. Google sayt "o'quvchi"si — robot. U dizaynni emas, **kodni** o'qiydi.

Asosiy SEO qoidalari (siz hozir qila oladiganlar)

Qoida	Qanday
-------	--------

Mazmunli <code><title></code>	Har bir sahifada har xil, 50–60 belgi: <code>Ali Valiyev – yosh veb-dasturchi portfoliosi</code>
<code>meta description</code>	Google natijasidagi tavsif, 150–160 belgi
Bitta <code><h1></code> va sarlavhalar tartibi	h1 → h2 → h3 (2-mavzu)
Semantik teglar	<code>header</code> , <code>main</code> , <code>article</code> (5-mavzu)
Rasmlarda <code>alt</code>	Mazmunli tavsif (4-mavzu)
Tushunarli manzillar	<code>/loyihalar.html</code> ✓, <code>/page1.html</code> ✗
Tezlik	Rasmlarni siqish, <code>loading="lazy"</code>
Mobilga moslik	Responsiv dizayn (10-mavzu) — Google mobil versiyani asosiy deb hisoblaydi
HTTPS	GitHub Pages, Netlify avtomatik beradi
Sifatli mazmun	Eng muhimi! Foydali, noyob, o'z so'zlaringiz bilan yozilgan matn

HTML

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ali Valiyev – yosh veb-dasturchi portfoliosi</title>
  <meta name="description" content="Samarqandlik 8-sinf o'quvchisi Ali Valiyevning HTML, CSS va JavaScript loyihalari.">

  <!-- Telegram/Facebook'da havola yuborilganda chiroyli "kartochka" uchun -->
  <meta property="og:title" content="Ali Valiyev – portfolio">
  <meta property="og:description" content="Mening veb-loyihalarim">
  <meta property="og:image"
content="https://USERNAME.github.io/portfolio/rasmlar/preview.jpg">
</head>
```

Tekshirish: Chrome → `F12` → **Lighthouse** yorlig'i → "SEO" va "Performance" belgilang → **Analyze page load**. 90+ ball — a'lo natija. Lighthouse har bir muammoni va uni qanday tuzatishni ko'rsatadi.

Amaliy mashg'ulot (6 va 7-loyihalar)

- 1 Git'ni o'rnatish va sozlash (`user.name`, `user.email`).
- 2 8-mavzudagi `portfolio/` papkasini VS Code'da oching, terminalda `git init`.
- 3 `git status` → `git add .` → `git commit -m "Portfolio: birinchi versiya"`.
- 4 GitHub'da `portfolio` repozitoriyasini yarating va `push` qiling.
- 5 Biror o'zgarish kiriting (masalan, yangi loyiha kartasi), yana `add` → `commit` → `push`. GitHub'da **commits** tarixini oching.
- 6 GitHub Pages'ni yoqing va saytingizni telefoningizdan oching.
- 7 Barcha sahifalarga `title` va `description` qo'shing, Lighthouse bilan SEO ballini tekshiring va 90+ ga yetkazing.
- 8 Repozitoriyaga `README.md` fayl qo'shing — loyiha haqida qisqacha:

README.md

```
# Mening portfoliom

8-sinf o'quvchisi Ali Valiyevning shaxsiy sayti.

**Sayt:** https://USERNAME.github.io/portfolio/

## Texnologiyalar
- HTML5 (semantik teglar)
- CSS3 (Flexbox, Grid, responsiv dizayn)
```

Ko'p uchraydigan xatolar

- ✗ `git commit` qilishdan oldin `git add` ni unutish — "nothing added to commit" xabari.
- ✗ `git init` ni noto'g'ri papkada (masalan, butun "Ish stoli"da) bajarish. Doim loyiha papkasi ichida.
- ✗ GitHub Pages'da sahifa ochilmayapti — bosh fayl `index.html` emas (masalan, `Index.html` yoki `bosh.html`).
- ✗ Rasmlar GitHub Pages'da ko'rinmayapti — fayl nomida katta-kichik harf farqi (`Logo.PNG` ≠ `logo.png`).

O'zingizni tekshiring

? O'zgarishlarni "saqlash nuqtasi" sifatida tarixga yozuvchi buyruq?

- A. `git add`
- B. `git push`
- C. `git commit`
- D. `git init`

? Git va GitHub farqi nimada?

- A. Git — kompyuterdagi dastur, GitHub — loyihalarni internetda saqlovchi sayt
- B. Ular bir xil narsa
- C. GitHub — dastur, Git — sayt
- D. Git faqat internetda ishlaydi

? Lokal commit'larni GitHub'ga yuborish buyrug'i?

- A. `git pull`
- B. `git clone`
- C. `git status`
- D. `git push`

? Google natijasida sahifa ostida ko'rinadigan qisqa tavsif qaysi teg bilan beriladi?

- A. `<title>`
- B. `<meta name="description">`
- C. `<h1>`
- D. `<p class="seo">`

? SEO uchun foydali EMAS narsa qaysi?

- A. Rasmlarda alt matni
- B. Saytning telefonga moslashishi
- C. Sahifada 5 ta h1 sarlavha
- D. Tez yuklanish



Uyga vazifa

1. Barcha oldingi loyihalaringizni (vizitka, maktab sahifasi, landing) alohida repozitoriyalarga joylang va GitHub Pages'ni yoqing.
2. Har bir loyihada kamida 5 ta mazmunli commit bo'lsin.
3. GitHub profilingizni bezang: rasm, bio, va `USERNAME/USERNAME` nomli maxsus repozitoriyada profil README yozing.



Yodda tuting

- Git — versiyalar tizimi (dastur), GitHub — kodni internetda saqlash platformasi.
- Ish sikli: o'zgartirish → `git add .` → `git commit -m "izoh"` → `git push`.
- `git status` — holat, `git log --oneline` — tarix, `git clone` / `git pull` — yuklab olish.
- GitHub Pages: Settings → Pages → main → sayt `USERNAME.github.io/repo` da.
- SEO: title, description, bitta h1, semantika, alt, tezlik, mobilga moslik, sifatli mazmun.

✓ Test javoblari

1. O'zgarishlarni "saqlash nuqtasi" sifatida tarixga yozuvchi buyruq? — **C) git commit**
2. Git va GitHub farqi nimada? — **A) Git — kompyuterdagi dastur, GitHub — loyihalarni internetda saqlovchi sayt**
3. Lokal commit'larni GitHub'ga yuborish buyrug'i? — **D) git push**
4. Google natijasida sahifa ostida ko'rinadigan qisqa tavsif qaysi teg bilan beriladi? — **B) <meta name="description">**
5. SEO uchun foydali EMAS narsa qaysi? — **C) Sahifada 5 ta h1 sarlavha**



12-MAVZU · 42 TADAN · I BOB. HTML VA CSS

CSS animatsiyalar va transformatsiyalar

transform (translate, scale, rotate, skew), transition bilan silliq o'tishlar, @keyframes va animation xossalari; interaktiv tugmalar, yuklanish belgisi, kartalar effektlari.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- Elementlarni siljitish, kattalashtirish, burish (`transform`);
- Holat o'zgarganda silliq o'tish yaratish (`transition`);
- O'zi harakatlanadigan animatsiyalar (`@keyframes` + `animation`);
- Zamonaviy saytlardagi effektlar: "tirik" tugmalar, yuklanish belgisi, paydo bo'luvchi kartalar.

Transform — shaklni o'zgartirish

`transform` elementni **boshqa elementlarga ta'sir qilmasdan** siljitadi, o'lchamini o'zgartiradi yoki buradi.

```

<style>
  .qator { display: flex; gap: 30px; flex-wrap: wrap; padding: 30px; font-
family: sans-serif; }
  .q { width: 90px; height: 90px; background: #4f46e5; color: #fff; display:
grid; place-items: center;
    border-radius: 10px; font-size: 12px; text-align: center; }
  .t1 { transform: translateX(30px); } /* o'ngga 30px */
  .t2 { transform: translateY(-20px); } /* tepaga 20px */
  .t3 { transform: scale(1.3); } /* 1.3 barobar katta */
  .t4 { transform: scale(0.7); } /* kichraytirish */
  .t5 { transform: rotate(45deg); } /* 45 gradus burish */
  .t6 { transform: skew(15deg); } /* qiyshaytirish */
  .t7 { transform: rotate(-10deg) scale(1.1); } /* bir nechta birga */
</style>
<div class="qator">
  <div class="q">0ddiy</div>
  <div class="q t1">translateX</div>
  <div class="q t2">translateY</div>
  <div class="q t3">scale 1.3</div>
  <div class="q t4">scale 0.7</div>
  <div class="q t5">rotate 45°</div>
  <div class="q t6">skew</div>
  <div class="q t7">rotate + scale</div>
</div>

```

Funksiya	Vazifasi	Misol
<code>translate(x, y)</code>	Siljitish	<code>translate(10px, -5px)</code>
<code>scale(n)</code>	Kattalashtirish (1 = asl o'lcham)	<code>scale(1.1)</code> — 10% katta
<code>rotate(deg)</code>	Burish (gradus)	<code>rotate(180deg)</code> — teskari
<code>skew(deg)</code>	Qiyshaytirish	<code>skew(10deg)</code>



`transform-origin` — burish yoki kattalashtirish qaysi nuqtadan bo'lishi: `center` (standart), `top left`, `bottom`. Masalan, eshik ochilishi kabi effekt uchun `transform-origin: left`.

Transition — silliq o'tish

9-mavzuda **:hover** holatini o'rgandik — lekin rang **keskin** o'zgarardi. **transition** o'zgarishni **silliq** qiladi.



O'xshatish: chiroq

Oddiy chiroq bir zumda yonadi (transition'siz). Kinoteatrdagi chiroqlar esa asta-sekin so'nadi va yonadi (transition bilan). Siz faqat **boshlang'ich** va **oxirgi** holatni berasiz, oradagi barcha "kadrlar"ni brauzer o'zi chizadi.

HTML

```
<style>
    .tugma {
        background: #4f46e5; color: white; border: none; padding: 14px 28px;
        border-radius: 10px; font-size: 16px; cursor: pointer; margin: 10px;
    }
    .keskin:hover { background: #db2777; transform: scale(1.1); }

    .silliq { transition: background-color 0.3s ease, transform 0.3s ease; }
    .silliq:hover { background: #db2777; transform: scale(1.1); }
</style>

<button class="tugma keskin">Keskin (transition yo'q)</button>
<button class="tugma silliq">Silliq (transition bor)</button>
```

CSS

Tezlik egri chiziqlari

HTML

```
<style>
  .yolak { background: #f1f5f9; border-radius: 8px; margin: 8px 0; padding:
4px; font-family: sans-serif; font-size: 13px; }
  .shar { width: 120px; padding: 6px; background: #4f46e5; color: #fff; border-
radius: 20px; text-align: center;
    transition: transform 1.5s; }
  .sahna:hover .shar { transform: translateX(300px); }
  .linear .shar { transition-timing-function: linear; }
  .ease .shar { transition-timing-function: ease; }
  .ease-in .shar { transition-timing-function: ease-in; }
  .ease-out .shar { transition-timing-function: ease-out; }
  .ease-in-out .shar { transition-timing-function: ease-in-out; }
</style>
<div class="sahna">
  <p><b>Bu qutiga sichqonchani olib keling:</b></p>
  <div class="yolak linear"><div class="shar">linear</div></div>
  <div class="yolak ease"><div class="shar">ease</div></div>
  <div class="yolak ease-in"><div class="shar">ease-in</div></div>
  <div class="yolak ease-out"><div class="shar">ease-out</div></div>
  <div class="yolak ease-in-out"><div class="shar">ease-in-out</div></div>
</div>
```

- `linear` — bir tekis tezlikda (robotdek).
- `ease` — sekin boshlanib, tezlashib, sekin tugaydi (standart, tabiiy).
- `ease-in` — sekin boshlanadi; `ease-out` — sekin tugaydi.



`transition` ni **oddiy holatga** yozing (`.tugma`), `:hover` ga emas. Aks holda effekt faqat bir tomonga silliq bo'ladi — sichqoncha ketganda keskin qaytadi.

Animation — o'zi harakatlanuvchi effektlar

`transition` faqat holat o'zgarganda (hover, click) ishlaydi va faqat A dan B ga. **Animatsiya** esa o'zi boshlanadi, ko'p bosqichli bo'lishi va cheksiz takrorlanishi mumkin.

1-qadam: @keyframes — "ssenariy" yozish

CSS

```
@keyframes sakrash {  
  0%   { transform: translateY(0); }      /* boshlanish */  
  50%  { transform: translateY(-40px); }  /* o'rtasida — tepada */  
  100% { transform: translateY(0); }      /* oxiri — yana pastda */  
}  
/* Faqat ikki bosqich bo'lsa: from { ... } to { ... } */
```

2-qadam: elementga qo'llash

HTML

```
<style>
  @keyframes sakrash {
    0%, 100% { transform: translateY(0); }
    50%      { transform: translateY(-40px); }
  }
  @keyframes aylanish { to { transform: rotate(360deg); } }
  @keyframes yurak {
    0%, 100% { transform: scale(1); }
    15%      { transform: scale(1.25); }
    30%      { transform: scale(1); }
    45%      { transform: scale(1.15); }
  }
  @keyframes rang {
    0%   { background: #4f46e5; }
    33%  { background: #db2777; }
    66%  { background: #f59e0b; }
    100% { background: #4f46e5; }
  }
  .sahna { display: flex; gap: 40px; align-items: flex-end; padding: 60px 30px 20px; font-size: 48px; }
  .koptok { animation: sakrash 1s ease-in-out infinite; }
  .gilDIRAK { animation: aylanish 2s linear infinite; }
  .yurak { animation: yurak 1.2s infinite; }
  .rangli { width: 80px; height: 80px; border-radius: 16px; animation: rang 4s infinite; }
</style>
<div class="sahna">
  <div class="koptok">⚽</div>
  <div class="gilDIRAK">⚙️</div>
  <div class="yurak">❤️</div>
  <div class="rangli"></div>
</div>
```

CSS

```
animation: sakrash 1s ease-in-out 0s infinite alternate forwards;  
/* nomi | davomiyligi | tezlik | kechikish | takror soni | yo'nalish | oxirgi  
holat */
```

Xossa	Qiymatlar
animation-name	@keyframes nomi
animation-duration	2s , 500ms
animation-iteration-count	1 , 3 , infinite (cheksiz)
animation-direction	normal , reverse , alternate (borib-kelish)
animation-delay	0.5s — kechikib boshlash
animation-fill-mode	forwards — tugagach oxirgi holatda qolish

Amaliy misollar

1. Yuklanish belgisi (spinner)

HTML

```
<style>
.spinner {
  width: 50px; height: 50px;
  border: 5px solid #e0e7ff;
  border-top-color: #4f46e5;      /* faqat tepa qismi rangli */
  border-radius: 50%;
  animation: aylan 0.8s linear infinite;
  margin: 30px auto;
}
@keyframes aylan { to { transform: rotate(360deg); } }
</style>
<div class="spinner"></div>
<p style="text-align:center;font-family:sans-serif">Yuklanmoqda... </p>
```

2. Paydo bo'luvchi kartalar (ketma-ket)

HTML

```
<style>
  body { font-family: sans-serif; background: #f1f5f9; }
  @keyframes paydo {
    from { opacity: 0; transform: translateY(30px); }
    to   { opacity: 1; transform: translateY(0); }
  }
  .kartalar { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px;
padding: 20px; }
  .karta {
    background: white; padding: 24px; border-radius: 14px; box-shadow: 0 6px
18px rgba(0,0,0,.08);
    animation: paydo 0.6s ease both;
    transition: transform .25s, box-shadow .25s;
  }
  .karta:nth-child(2) { animation-delay: 0.15s; }
  .karta:nth-child(3) { animation-delay: 0.3s; }
  .karta:hover { transform: translateY(-8px); box-shadow: 0 16px 30px
rgba(79,70,229,.25); }
</style>
<div class="kartalar">
  <div class="karta">🍪 <b>HTML/CSS</b><br>Sahifa yaratish</div>
  <div class="karta">⚡ <b>JavaScript</b><br>Interaktivlik</div>
  <div class="karta">🐍 <b>Python</b><br>Backend</div>
</div>
<p style="text-align:center">"Ў Asl holiga" ni bosib, animatsiyani qayta
ko'ring</p>
```

3. Porlovchi tugma

HTML

```
<style>
  .porla {
    background: linear-gradient(90deg, #4f46e5, #db2777, #4f46e5);
    background-size: 200% 100%;
    color: white; border: none; padding: 16px 34px; font-size: 18px;
    border-radius: 999px; cursor: pointer; font-weight: bold;
    animation: oqim 3s linear infinite;
    transition: transform .2s;
  }
  .porla:hover { transform: scale(1.06); }
  .porla:active { transform: scale(.96); }
  @keyframes oqim { to { background-position: -200% 0; } }
</style>
<div style="padding:40px;text-align:center"><button class="porla">Kursga
yozilish 🚀</button></div>
```

Unumdorlik va foydalanuvchilar haqida qayg'urish

- Animatsiya uchun eng yaxshi xossalar — `transform` va `opacity`: ular tez ishlaydi. `width`, `top`, `margin` ni animatsiya qilish sahifani sekinlashtiradi.
- Animatsiyani **me'yorida** ishlatish: hamma narsa sakrab tursa, ko'z charchaydi.
- Ba'zi odamlarda harakatdan bosh aylanadi. Ular uchun: `@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }`

Amaliy mashg'ulot

- 1 Portfolio saytingizdagi barcha tugma va havolalarga `transition` bilan silliq `:hover` effektlari qo'shing.
- 2 Loyiha kartalari hover'da 8px tepaga ko'tarilsin va soyasi kuchaysin.
- 3 Hero bo'limidagi sarlavha va tugma sahifa ochilganda pastdan yuqoriga "paydo" bo'lsin (`@keyframes paydo` + kechikish).

- 4 Profil rasmingizga sekin "nafas olish" effekti: `scale(1)` → `scale(1.03)` → `scale(1)`, 4 soniya, cheksiz.
- 5 **Ijodiy topshiriq:** faqat CSS bilan "Quyosh tizimi" yarating: markazda quyosh 🌞, atrofida 2 ta sayyora turli tezlikda aylanadi. (Maslahat: sayyorani o'zidan katta shaffof doira ichiga qo'ying va doirani aylantiring.)

Ko'p uchraydigan xatolar

- ✗ Bir elementga ikkita `transform` yozish — ikkinchisi birinчисini o'chiradi. Bittada yozing:
`transform: rotate(10deg) scale(1.2);`
- ✗ `transition: all` ni hamma joyda ishlatish — kutilmagan xossalar ham animatsiya bo'ladi; aniq xossani ko'rsating.
- ✗ `@keyframes` nomi va `animation` dagi nom mos emas (masalan, `aylanish` va `aylanis`).
- ✗ `display: none` ni animatsiya qilishga urinish — u animatsiya bo'lmaydi; `opacity` va `visibility` ishlatish.

O'zingizni tekshiring

? Elementni 1.2 barobar kattalashtiruvchi kod?

- A. `transform: size(1.2)`
- B. `zoom: 120px`
- C. `transform: scale(1.2)`
- D. `transform: grow(20%)`

? **Hover effekti silliq bo'lishi uchun `transition` qayerga yoziladi?**

- A. Elementning oddiy holatiga
- B. Faqat `:hover` ga
- C. `@keyframes` ichiga
- D. `body` ga

? **Animatsiyani cheksiz takrorlash uchun qaysi qiymat?**

- A. `repeat`
- B. `loop`
- C. `forever`
- D. `infinite`

? **`transition` va `animation` ning asosiy farqi?**

- A. Farqi yo'q
- B. `transition` holat o'zgarganda ishlaydi; `animation` o'zi boshlanadi va ko'p bosqichli bo'lishi mumkin
- C. `animation` faqat ranglar uchun
- D. `transition` faqat telefonda ishlaydi

? Qaysi xossalarni animatsiya qilish eng tez (unumdor)?

- A. `width` va `height`
- B. `transform` va `opacity`
- C. `margin` va `padding`
- D. `top` va `left`



Uyga vazifa

1. "Quyosh tizimi" ijodiy topshirig'ini yakunlang.
2. 3 xil yuklanish belgisi (spinner) yarating: aylanuvchi doira, sakrovchi uchta nuqta, to'lib boruvchi chiziq.
3. Kartani 180° aylantirib, orqa tomonini ko'rsatuvchi "flip card" effektini yarating (`rotateY` , `backface-visibility` xossalari internetdan o'rganing).



Yodda tuting

- `transform`: `translate` , `scale` , `rotate` , `skew` — boshqa elementlarga ta'sir qilmaydi.
- `transition`: xossa vaqt tezlik; — oddiy holatga yoziladi, holat o'zgarishini silliq qiladi.
- `@keyframes nom { 0% {...} 100% {...} }` + `animation: nom 2s infinite;` — o'zi ishlaydigan animatsiya.
- Unumdorlik: `transform` va `opacity` . Me'yorni saqlang.

✓ Test javoblari

1. Elementni 1.2 barobar kattalashtiruvchi kod? — **C) transform: scale(1.2)**
2. Hover effekti silliq bo'lishi uchun transition qayerga yoziladi? — **A) Elementning oddiy holatiga**
3. Animatsiyani cheksiz takrorlash uchun qaysi qiymat? — **D) infinite**
4. transition va animation ning asosiy farqi? — **B) transition holat o'zgarganda ishlaydi; animation o'zi boshlanadi va ko'p bosqichli bo'lishi mumkin**
5. Qaysi xossalarni animatsiya qilish eng tez (unumdor)? — **B) transform va opacity**



13-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Jamoa bilan ishlash va loyiha qilish. Netlify va Vercel'ga deploy

Jamoaviy loyiha bosqichlari, rollarni taqsimlash, Git branch, merge va konfliktlar, GitHub'da hamkorlik va Pull Request; saytni Netlify va Vercel'ga bepul joylash.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Haqiqiy dasturchilar jamoada qanday ishlashini;
- Loyihani rejalashtirish va vazifalarni taqsimlashni;
- Git **branch** (tarmoq), **merge** va **konflikt** tushunchalarini;
- GitHub'da hamkor qo'shish va **Pull Request** orqali kodni birlashtirishni;
- Tayyor saytni **Netlify** va **Vercel**'ga joylashni (deploy).

Nima uchun jamoada ishlash muhim?

Instagram, Telegram, Click kabi ilovalarni bitta odam yaratmagan — ularda o'nlab, yuzlab dasturchilar birga ishlaydi. Dasturchining eng qadrli ko'nikmalaridan biri — **jamoada ishlay olish**: vazifani bo'lishish, boshqalarning kodini tushunish, kelishish.



O'xshatish: futbol jamoasi

Futbolda hamma to'p ortidan yugursa, jamoa yutqazadi. Har bir o'yinchining o'z o'rni bor: darvozabon, himoyachi, hujumchi. Murabbiy reja tuzadi. Dasturlash jamoasida ham shunday: kimdir header va menyuni, kimdir galereyani, kimdir formani yozadi — va oxirida hammasi bitta saytga birlashadi.

Jamoaviy loyiha bosqichlari

- 1 **G'oya va maqsad.** Sayt nima haqida? Kimlar uchun? Asosiy sahifalari qaysilar?
- 2 **Reja (vazifalar ro'yxati).** Saytni kichik vazifalarga bo'ling: "Header va menyu", "Bosh sahifa hero bo'limi", "Galereya sahifasi", "Aloqa formasi", "Telefon versiyasi"...
- 3 **Rollarni taqsimlash.** Har bir vazifaga mas'ul belgilanadi.
- 4 **Kelishuvlar.** Ranglar palitrasi, shriftlar, fayl nomlari, CSS class nomlash qoidalari — hamma bir xil yozishi uchun.
- 5 **Ishlab chiqish.** Har kim o'z qismini alohida **branch**da yozadi.
- 6 **Tekshirish (code review).** Jamoadoshlar bir-birining kodini ko'rib chiqadi.
- 7 **Birlashtirish va deploy.** Hammasi asosiy branch'ga qo'shiladi va internetga chiqariladi.

Vazifalarni boshqarish: Kanban doskasi

Jamoalar vazifalarni **Kanban** doskasida kuzatadi — 3 ustunli jadval:

 Qilinishi kerak (To Do)	 Jarayonda (In Progress)	 Tayyor (Done)
Galereya sahifasi — <i>Madina</i> Aloqa formasi — <i>Javohir</i>	Header va menyu — <i>Sardor</i>	Ranglar va shriftlar tanlandi — <i>hamma</i> Papka tuzilmasi — <i>Sardor</i>

Buning uchun **GitHub Projects** (repozitoriyada "Projects" yorlig'i) yoki Trello'dan foydalanish mumkin. Oddiy variant — sinf doskasi va stikerlar!

Git branch — parallel ish

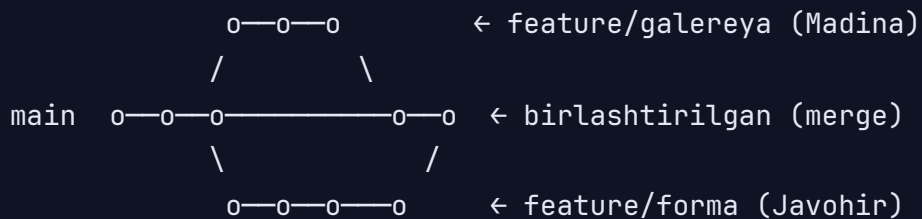
Branch (tarmoq) — loyihaning **alohida nusxasi**, unda asosiy kodga tegmasdan ishlash mumkin. Asosiy branch odatda `main` deb ataladi va unda doim **ishlaydigan** kod turadi.



O'xshatish: daraxt shoxlari

`main` — daraxtning tanasi. Har bir dasturchi o'z vazifasi uchun alohida shox (branch) o'tiradi. Ish tugagach, shox tanaga qayta "payvand" qilinadi (**merge**). Shoxda xato bo'lsa — tana zarar ko'rmaydi.

Matn



Terminal

```
# Barcha branch'larni ko'rish (* - siz hozir qaysi branch'dasiz)
git branch

# Yangi branch yaratish va unga o'tish
git switch -c feature/galereya

# ... ishlash, git add, git commit ...

# Asosiy branch'ga qaytish
git switch main

# feature/galereya ni main ga birlashtirish
git merge feature/galereya
```

Konflikt — ikki kishi bitta qatorni o'zgartirsa

Madina va Javohir ikkalasi `index.html` dagi bitta sarlavhani turlicha o'zgartirishdi. Git qaysi birini tanlashni bilmaydi va **konflikt** haqida xabar beradi. Faylda shunday belgilar paydo bo'ladi:

Matn

```
<<<<<<< HEAD
<h1>Bizning maktab</h1>
=====
<h1>5-maktab rasmiy sayti</h1>
>>>>>>> feature/forma
```

- <<<<<<< HEAD va ===== orasida — sizning (joriy) variant;
- ===== va >>>>>>> orasida — birlashtirilayotgan branch varianti.

Hal qilish: jamoa bilan kelishib, keraklisini qoldiring, **barcha belgilarni o'chiring**, so'ng `git add .` va `git commit`. VS Code konfliktlarni rangli ko'rsatadi va "Accept Current / Incoming / Both" tugmalarini taklif qiladi.



Konfliktlarni kamaytirish

Har kim **turli fayllarda** ishlasin (masalan, har bir sahifa alohida fayl, har bir bo'lim CSS'i alohida). Tez-tez `git pull` qiling va kichik commit'lar bilan ishleng.

GitHub'da hamkorlik

1. Hamkor qo'shish

Jamoa sardori repozitoriyanı yaratadi → **Settings** → **Collaborators** → **Add people** → jamoadoshlarning GitHub nomlarini qo'shadi. Ular emailga kelgan taklifni qabul qiladi va endi `git clone` qilib, `push` qila oladi.

2. Pull Request (PR) — kodni birlashtirishni so'rash

Professional jamoalarda hech kim to'g'ridan-to'g'ri `main` ga push qilmaydi. Buning o'rniga:

- 1 O'z branch'ingizda ishlaysiz va uni GitHub'ga yuborasiz: `git push -u origin feature/galereya`
- 2 GitHub'da sariq **"Compare & pull request"** tugmasi chiqadi — bosing.
- 3 Sarlavha va tavsif yozing: nima qildingiz, qanday tekshirish mumkin, skrinshot.

- 4 Jamoadoshlar **Files changed** yorlig'ida kodingizni ko'rib, izoh qoldiradi ("Bu rasmga alt qo'shish kerak").
- 5 Izohlarni tuzatib, yana push qilasiz — PR avtomatik yangilanadi.
- 6 Hamma rozi bo'lsa — **Merge pull request**. Kod `main` ga qo'shildi!
- 7 Hamma o'z kompyuterida yangilanadi: `git switch main` → `git pull`.

Code review madaniyati

- Kodni tanqid qiling, odamni emas: "Bu yerda `alt` yo'q" ✓, "Sen hech narsa bilmaysan" ✗.
- Maqtashni ham unutmang: "Animatsiya zo'r chiqibdi!" 👍
- Taklif sifatida yozing: "Bu yerda Grid ishlatsak qanday bo'ladi?"

Deploy — saytni internetga chiqarish

Deploy — tayyor loyihani foydalanuvchilar ochishi uchun serverga joylash. 11-mavzuda GitHub Pages'ni ko'rdik. Endi ikkita mashhur platforma:

	Netlify	Vercel
Narxi	Shaxsiy loyihalar uchun bepul	Shaxsiy loyihalar uchun bepul
Manzil	<code>nom.netlify.app</code>	<code>nom.vercel.app</code>
HTTPS	Avtomatik	Avtomatik
GitHub bilan ulanish	Ha — har push'da avtomatik yangilanadi	Ha — har push'da avtomatik yangilanadi
O'ziga xosligi	Papkani sudrab tashlab deploy (Drag & Drop), formalarni qabul qilish	React/Next.js loyihalari uchun ayniqsa qulay

Netlify — 1-usul: sudrab tashlash (eng oson)

- 1 `app.netlify.com/drop` sahifasini oching.

- 2 Sayt papkangizni (ichida `index.html` bor) sahifadagi maydonga sichqoncha bilan **sudrab tashlang**.
- 3 Bir necha soniyada sayt tayyor: `https://tasodifiy-nom-123.netlify.app`. Saqlab qolish uchun ro'yxatdan o'ting.
- 4 **Site configuration** → **Change site name** orqali chiroyli nom bering: `maktab5-samarqand.netlify.app`.

Netlify yoki Vercel — 2-usul: GitHub orqali (professional)

- 1 Netlify yoki Vercel'ga **GitHub akkauntingiz bilan** kiring.
- 2 **Add new site** → **Import an existing project** (Netlify) yoki **Add New** → **Project** (Vercel).
- 3 GitHub'dagi repozitoriyangizni tanlang.
- 4 Oddiy HTML/CSS sayt uchun hech qanday sozlama kerak emas (Build command — bo'sh, Publish directory — `/` yoki loyiha ildizi). **Deploy** bosing.
- 5 Endi har safar `git push` qilganingizda sayt **avtomatik yangilanadi**. Bu — **CI/CD** (uzluksiz yetkazib berish) degan professional usulning eng oddiy ko'rinishi.



O'z domeningiz

Agar `.uz` domen sotib olsangiz, Netlify/Vercel'da **Domain settings** → **Add custom domain** qilib, domen sozlamalarida (DNS, 1-mavzu) ular ko'rsatgan yozuvlarni kiritasiz.

Amaliy mashg'ulot: jamoaviy loyiha (8-loyiha)

3–4 kishilik jamoalarda **"Bizning sinf"** sayтини yarating.

- 1 **Sardor** (jamoarahbari) GitHub'da `bizning-sinf` repozitoriyasini yaratadi, boshlang'ich tuzilmani push qiladi va hamkorlarni qo'shadi:

Matn

```
bizning-sinf/  
├── index.html           ← bosh sahifa (Sardor)  
├── oquvchilar.html     ← o'quvchilar kartalari (2-a'zo)  
├── galereya.html       ← rasmlar galereyasi (3-a'zo)  
├── aloqa.html          ← forma va xarita (4-a'zo)  
├── css/  
│   ├── umumiy.css      ← ranglar, shriftlar, header, footer (kelishib  
│   │   yoziladi)  
│   └── ...             ← har kimning sahifasi uchun alohida fayl  
└── rasmlar/
```

- 2 Jamoa **kelishuvlar** faylini (`README.md`) yozadi: ranglar (HEX), shriftlar, class nomlash qoidasi, kim nimaga mas'ul.
- 3 Har bir a'zo `git clone` qiladi va o'z branch'ini yaratadi: `git switch -c feature/galereya`.
- 4 O'z sahifasini yozadi, kamida 3 ta commit qiladi va `git push -u origin feature/galereya`.
- 5 GitHub'da Pull Request ochadi; boshqa a'zo uni ko'rib, kamida bitta izoh qoldiradi; so'ng merge qilinadi.
- 6 Barcha PR'lar birlashgach, sardor loyihani Netlify yoki Vercel'ga GitHub orqali ulaydi.
- 7 Har bir a'zo telefonida saytni ochib tekshiradi; topilgan xatolar uchun GitHub'da **Issue** ochiladi va tuzatiladi.
- 8 **Taqdimot:** jamoa saytni sinfga 3 daqiqada tanishtiradi: kim nima qildi, qanday qiyinchiliklar bo'ldi.

Ko'p uchraydigan xatolar

- ✗ Hamma `main` ga to'g'ridan-to'g'ri push qiladi — bir-birining ishini o'chirib yuboradi.
- ✗ Ishni boshlashdan oldin `git pull` qilmaslik — eski kod ustida ishlab, keyin katta konflikt chiqadi.
- ✗ Konflikt belgilarini (`<<<<<<`) faylda qoldirib yuborish — sahifada g'alati matn ko'rinadi.
- ✗ Netlify'da "Page not found" — papka ichida yana papka (`sayt/sayt/index.html`); `index.html` yuklangan papkaning **ildizida** bo'lishi kerak.

O'zingizni tekshiring

? Git'da asosiy koddan alohida, mustaqil ishlash uchun nima yaratiladi?

- A. Commit
- B. Branch
- C. Clone
- D. Remote

? GitHub'da o'z branch'ingizni asosiy kodga qo'shishni so'rash nima deyiladi?

- A. Push Request
- B. Merge Commit
- C. Issue
- D. Pull Request

? Konflikt qachon yuzaga keladi?

- A. Ikki branch'da bitta faylning bitta qatori turlicha o'zgartirilganda
- B. Internet uzilganda
- C. Fayl o'chirilganda
- D. Juda ko'p commit qilinganda

? **Netlify'ga GitHub orqali ulangan saytni yangilash uchun nima qilish kerak?**

- A. Har safar papkani qayta yuklash
- B. Netlify'ga xat yozish
- C. Shunchaki `git push` qilish — sayt avtomatik yangilanadi
- D. Saytni o'chirib, qaytadan yaratish

? **Kanban doskasining odatiy ustunlari qaysilar?**

- A. HTML, CSS, JS
- B. Qilinishi kerak, Jarayonda, Tayyor
- C. Dushanba, Seshanba, Chorshanba
- D. Oson, O'rta, Qiyin



Uyga vazifa

1. Jamoaviy loyihadagi o'z qismingizni yakunlang va PR orqali birlashtiring.
2. Shaxsiy portfoliingizni Netlify'ga (GitHub orqali) ham joylang va manzilini GitHub profilingizga qo'shing.
3. Biror sinfdoshingizning repozitoriyasida "Issue" oching: saytida topgan bitta xato yoki taklifingizni hurmat bilan yozing.



Yodda tuting

- Jamoaviy ish: reja → rollar → kelishuvlar → branch'larda ishlash → code review → merge → deploy.
- `git switch -c nom` — yangi branch; `git merge nom` — birlashtirish; konfliktida belgilarni o'chirib, add + commit.
- GitHub: Collaborators, Pull Request, Issues, Projects.

- Deploy: Netlify (Drop yoki GitHub), Vercel (GitHub) — bepul, HTTPS, har push'da avtomatik yangilanish.

✓ Test javoblari

1. Git'da asosiy koddan alohida, mustaqil ishlash uchun nima yaratiladi? — **B) Branch**
2. GitHub'da o'z branch'ingizni asosiy kodga qo'shishni so'rash nima deyiladi? — **D) Pull Request**
3. Konflikt qachon yuzaga keladi? — **A) Ikki branch'da bitta faylning bitta qatori turlicha o'zgartirilganda**
4. Netlify'ga GitHub orqali ulangan saytni yangilash uchun nima qilish kerak? — **C) Shunchaki git push qilish — sayt avtomatik yangilanadi**
5. Kanban doskasining odatiy ustunlari qaysilar? — **B) Qilinishi kerak, Jarayonda, Tayyor**



Yakuniy nazorat ishi — I bob (HTML va CSS)

I bob bo'yicha nazorat: 25 ta yopiq test (3 balldan) va 5 ta ochiq savol/amaliy topshiriq (5 balldan) — jami 100 ball. Takrorlash rejasi, javoblar va baholash mezonlari.

1 soat · nazorat ishi

Nazorat ishi tuzilishi

- **1-qism:** 25 ta yopiq test — har biri **3 ball** (jami 75 ball). Har bir savolda bitta to'g'ri javob.
- **2-qism:** 5 ta ochiq savol va amaliy topshiriq — har biri **5 ball** (jami 25 ball).
- **Jami: 100 ball.** Vaqt: 80 daqiqa (test — 30 daqiqa, amaliy qism — 50 daqiqa).

Bu sahifadan **tayyorgarlik** uchun foydalaning: testlarni yeching, natijangizni ko'ring, xato qilgan mavzularingizni qayta o'qing.

Takrorlash rejasi

Mavzu	Bilishingiz kerak
1. Internet, DNS, hosting	Mijoz-server, IP, domen, DNS, hosting, HTTP/HTTPS, status kodlar, front-end va back-end
2. HTML tuzilishi	Teg, element, atribut, skelet, head/body, h1-h6, p, matn teglari
3. Jadval, ro'yxat, forma	ul/ol/li, table/tr/td/th, colspan/rowspan, form, input turlari, name, label, select, textarea
4. Havolalar, media	a href, target, absolyut/nisbiy yo'l, ../, #id, img src alt, audio/video controls, iframe

5. Semantik teglar	header, nav, main, section, article, aside, footer; blok va inline
6. CSS asoslari	3 ulash usuli, selektorlar, ustuvorlik, ranglar, matn va shrift xossalari
7. Fon va chegara	background-*, cover/contain, gradient, border, border-radius, box-shadow
8. Box model, Flex, Grid	margin/padding/border, box-sizing, birliklar, display:flex va xossalari, grid-template-columns, fr
9. Pseudo va position	:hover, :nth-child, ::before/::after, relative/absolute/fixed/sticky, z-index
10. Responsiv dizayn	viewport, media query, min/max-width, mobile-first, Figma asoslari
11. Git, GitHub, SEO	init/add/commit/push, GitHub Pages, title, description, alt
12. Animatsiya	transform, transition, @keyframes, animation
13. Jamoa va deploy	branch, merge, konflikt, Pull Request, Netlify/Vercel

1-qism: Yopiq testlar (25 × 3 = 75 ball)

? 1. Domen nomini IP-manzilga aylantiruvchi tizim qaysi?

- A. HTTP
- B. DNS
- C. HTML
- D. Hosting

? 2. Server "500" kodini qaytardi. Bu nimani bildiradi?

- A. Sahifa topilmadi
- B. Muvaffaqiyatli
- C. Sahifa ko'chirilgan
- D. Serverda ichki xato

? 3. Foydalanuvchiga ko'rinadigan mazmun qaysi teg ichida yoziladi?

- A. `<body>`
- B. `<head>`
- C. `<title>`
- D. `<meta>`

? 4. Qaysi teg yolg'iz (yopuvchi tegi yo'q)?

- A. `<p>`
- B. `<li>`
- C. `<img>`
- D. `<a>`

? 5. Tartibli (raqamlangan) ro'yxat qaysi teg bilan yaratiladi?

- A. `<ul>`
- B. `<ol>`
- C. `<dl>`
- D. `<list>`

? 6. Jadval katagini ikki qator balandligida birlashtiruvchi atribut?

- A. `colspan="2"`
- B. `merge="2"`
- C. `height="2"`
- D. `rowspan="2"`

? 7. Forma maydonidagi ma'lumot serverga qaysi atribut nomi bilan yuboriladi?

- A. `name`
- B. `id`
- C. `class`
- D. `value`

? 8. Bir guruhdan faqat bitta variant tanlash uchun qaysi input turi?

- A. `checkbox`
- B. `select`
- C. `radio`
- D. `option`

? 9. `sahifalar/aloqa.html` dan `rasmlar/logo.png` ga to'g'ri yo'l?

- A. `rasmlar/logo.png`
- B. `../rasmlar/logo.png`
- C. `/sahifalar/rasmlar/logo.png`
- D. `C:/rasmlar/logo.png`

? 10. Rasmning matnli tavsifi qaysi atributda yoziladi?

- A. `title`
- B. `src`
- C. `desc`
- D. `alt`

? 11. Mustaqil maqola yoki yangilik uchun eng mos semantik teg?

- A. `<article>`
- B. `<aside>`
- C. `<nav>`
- D. `<div>`

? 12. Butun saytga yagona dizayn berishning eng to'g'ri usuli?

- A. Inline CSS
- B. Har sahifada `<style>`
- C. External CSS fayl
- D. HTML atributlari

? 13. `id="menyu"` elementni tanlovchi selektor?

- A. `.menyu`
- B. `#menyu`
- C. `menyu`
- D. `*menyu`

? 14. Qaysi selektor eng kuchli (ustuvor)?

- A. `p`
- B. `.matn`
- C. `div p`
- D. `#asosiy`

? 15. Fon rasmi butun maydonni to'ldirishi uchun?

- A. `background-size: cover`
- B. `background-size: contain`
- C. `background-repeat: repeat`
- D. `background: fill`

? 16. Kvadrat elementni doiraga aylantirish uchun?

- A. `border: round`
- B. `radius: 100px`
- C. `border-radius: 50%`
- D. `shape: circle`

? 17. Element chegarasi va mazmuni orasidagi ichki bo'shliq?

- A. `margin`
- B. `padding`
- C. `gap`
- D. `outline`

? 18. `box-sizing: border-box` nima qiladi?

- A. Chegarani yashiradi
- B. Elementni markazga qo'yadi
- C. Margin'ni nolga tenglaydi
- D. width ichiga padding va border'ni ham qo'shadi

? 19. Flex konteynerda elementlarni asosiy o'q bo'yicha markazlash?

- A. `justify-content: center`
- B. `align-items: center`
- C. `text-align: center`
- D. `flex: center`

? 20. 3 ta teng ustunli Grid qaysi?

- A. `grid-columns: 3`
- B. `grid-template-columns: 3fr`
- C. `grid-template-columns: repeat(3, 1fr)`
- D. `columns: 1fr, 1fr, 1fr`

? 21. Ichidagi `absolute` elementlar unga nisbatan joylashishi uchun ota elementga nima beriladi?

- A. `position: absolute`
- B. `position: relative`
- C. `position: static`
- D. `display: block`

? 22. `@media (max-width: 768px)` qoidalari qachon ishlaydi?

- A. Ekran 768px dan katta bo'lsa
- B. Faqat planshetda
- C. Doim
- D. Ekran 768px yoki undan kichik bo'lsa

? 23. Hover effekti silliq bo'lishi uchun ishlatiladigan xossa?

- A. `transition`
- B. `transform`
- C. `animation-name`
- D. `@keyframes`

? 24. Lokal o'zgarishlarni GitHub'ga yuboruvchi Git buyrug'i?

- A. `git commit`
- B. `git pull`
- C. `git push`
- D. `git add`

? 25. Netlify yoki Vercel nima?

- A. Kod muharrirlari
- B. Saytlarni bepul joylash (hosting) platformalari
- C. CSS freymvorklari
- D. Brauzerlar

2-qism: Ochiq savollar va amaliy topshiriqlar (5 × 5 = 25 ball)

1-topshiriq (5 ball). Nazariy savol

Brauzerga `https://maktab.uz` deb yozilganda sahifa ekranda paydo bo'lgunicha qanday bosqichlar bo'lishini ketma-ket tushuntiring. Javobda DNS, server, so'rov, javob tushunchalari bo'lsin.

▼ Namuna javob va baholash

(1) DNS `maktab.uz` ni IP-manzilga aylantiradi; (2) brauzer shu IP'dagi serverga xavfsiz (HTTPS) ulanadi; (3) brauzer HTTP so'rov yuboradi; (4) server (hosting) sahifani topib, status 200 va HTML bilan javob qaytaradi; (5) brauzer HTML'ni o'qiydi, CSS, rasm va JS fayllarni qo'shimcha so'raydi; (6) hammasini birlashtirib, sahifani chizadi.

Baholash: 5 — barcha bosqichlar to'g'ri tartibda; 3–4 — 1–2 bosqich tushib qolgan; 1–2 — faqat umumiy tasavvur; 0 — javob yo'q yoki noto'g'ri.

2-topshiriq (5 ball). Kodni tuzating

Quyidagi kodda **5 ta xato** bor. Ularni toping va to'g'rilangan kodni yozing.

HTML

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <title>Kutubxona<title>
</head>
<body>
  <h1>Kitoblarimiz</h1>
  
  <ul>
    <li>O'tkan kunlar</li>
    <li>Mehrobdan chayon
  </ul>
  <a href="www.kitob.uz">Kitob do'koni</a>
  <p>Telefon: <strong>+998 90 123 45 67</p></strong>
</body>
</html>
```

▼ Javob

1. `<title>` yopilgan: `</title>` bo'lishi kerak.
2. `<img>` da `alt` atributi yo'q: `alt="Kitob muqovasi"`.
3. Ikkinchi `<li>` yopilgan.
4. Tashqi havolada protokol yo'q: `href="https://www.kitob.uz"`.
5. Teglar noto'g'ri yopilgan: `<strong>... </strong></p>`.

Qo'shimcha: `<meta charset="UTF-8">` va viewport meta-tegi ham yo'q (topgan o'quvchiga rag'bat).

Baholash: har bir topilgan va to'g'ri tuzatilgan xato — 1 ball.

3-topshiriq (5 ball). Forma yarating

"Olimpiadaga ro'yxatdan o'tish" formasini yozing: to'liq ism (majburiy), sinf (select: 7, 8, 9), fan (radio: matematika, informatika, fizika), email (majburiy), izoh (textarea) va "Yuborish" tugmasi. Har bir maydonda `label` bo'lsin.

▼ Namuna javob

HTML

```
<form action="/olimpiada" method="POST">
  <p><label for="ism">To'liq ism</label><br>
    <input type="text" id="ism" name="ism" required></p>
  <p><label for="sinf">Sinf</label><br>
    <select id="sinf" name="sinf"><option>7</option>
<option>8</option><option>9</option></select></p>
  <p>Fan:<br>
    <label><input type="radio" name="fan" value="matematika"
required> Matematika</label>
    <label><input type="radio" name="fan" value="informatika">
Informatika</label>
    <label><input type="radio" name="fan" value="fizika">
Fizika</label></p>
  <p><label for="email">Email</label><br>
    <input type="email" id="email" name="email" required></p>
  <p><label for="izoh">Izoh</label><br>
    <textarea id="izoh" name="izoh" rows="3"></textarea></p>
  <button type="submit">Yuborish</button>
</form>
```

Baholash: forma va method — 1; barcha maydonlar to'g'ri turda — 2; label va name'lar — 1; required va radio'larda bir xil name — 1.

4-topshiriq (5 ball). Flexbox maket

Header yarating: chapda logotip, o'ngda 3 ta havoladan iborat menyu, vertikal markazda. Menyu havolalariga sichqoncha borganda rang silliq o'zgarsin.

▼ Namuna javob

HTML

```
<style>
  .header { display: flex; justify-content: space-between; align-items: center; padding: 16px 32px; background: #1e1b4b; }
  .logo { color: white; font-weight: bold; font-size: 1.4rem; }
  .menyu { display: flex; gap: 24px; }
  .menyu a { color: #c7d2fe; text-decoration: none; transition: color .3s; }
  .menyu a:hover { color: #fbbf24; }
</style>
<header class="header">
  <div class="logo">Kutubxona</div>
  <nav class="menyu"><a href="#">Bosh sahifa</a><a href="#">Kitoblar</a><a href="#">Aloqa</a></nav>
</header>
```

Baholash: semantik teglar — 1; display:flex + space-between — 1; align-items: center — 1; menyu ichida flex/gap — 1; transition + :hover — 1.

5-topshiriq (5 ball). Responsiv kartalar

3 ta mahsulot kartasidan iborat bo'lim yarating: kompyuterda 3 ustun, 768px va undan kichik ekranda 1 ustun. Kartalarda rasm, nom, narx, yumaloq burchak va soya bo'lsin.

▼ Namuna javob

HTML

```
<style>
  * { box-sizing: border-box; }
  .kartalar { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 20px; padding: 20px; }
  .karta { border-radius: 14px; box-shadow: 0 6px 18px rgba(0,0,0,.1);
overflow: hidden; font-family: sans-serif; }
  .karta img { width: 100%; height: 150px; object-fit: cover; display:
block; }
  .karta h3, .karta p { margin: 10px 14px; }
  .narx { color: #4f46e5; font-weight: bold; }
  @media (max-width: 768px) {
    .kartalar { grid-template-columns: 1fr; }
  }
</style>
<section class="kartalar">
  <article class="karta">
<h3>Daftar</h3><p class="narx">5 000 so'm</p></article>
  <article class="karta">
<h3>Ruchka</h3><p class="narx">3 000 so'm</p></article>
  <article class="karta"><h3>Sumka</h3>
<p class="narx">150 000 so'm</p></article>
</section>
```

Baholash: Grid 3 ustun — 1; media query to'g'ri — 2; karta dizayni (radius, soya) — 1; rasm alt va o'lchami to'g'ri — 1.

Baholash shkalasi

Ball	Natija	Tavsiya
86–100	A'lo	II bobga tayyorsiz! Portfoliingizni boyitishda davom eting.

71–85	Yaxshi	Xato qilgan mavzularni qayta ko'rib chiqing.
60–70	Qoniqarli	Amaliy topshiriqlarni qayta bajaring, ayniqsa 8 va 10-mavzular.
0–59	Qoniqarsiz	Mentor bilan qo'shimcha mashg'ulot va bobni qayta takrorlash tavsiya etiladi.



O'quv dasturiga ko'ra, chorak yakunida joriy nazorat (40 ball) va oraliq nazorat (20 ball) jamlanadi; jami 60 balldan kamida **36 ball** to'plagan o'quvchi yakuniy nazoratga qo'yiladi. Yakuniy nazoratda 100 balldan kamida **60 ball** to'plash keyingi chorakda o'qishni davom ettirish imkonini beradi.



I bob yakunlandi!

Siz endi HTML va CSS yordamida semantik, chiroyli, animatsiyali va telefonga moslashuvchan veb-sahifalar yarata olasiz, ularni GitHub'da saqlab, Netlify/Vercel'ga joylay olasiz. Keyingi bobda sahifalarga **"jon" beramiz** — JavaScript dasturlash tilini o'rganamiz.

✓ Test javoblari

1. 1. Domen nomini IP-manzilga aylantiruvchi tizim qaysi? — **B) DNS**
2. 2. Server "500" kodini qaytardi. Bu nimani bildiradi? — **D) Serverda ichki xato**
3. 3. Foydalanuvchiga ko'rinadigan mazmun qaysi teg ichida yoziladi? — **A) <body>**
4. 4. Qaysi teg yolg'iz (yopuvchi tegi yo'q)? — **C) **
5. 5. Tartibli (raqamlangan) ro'yxat qaysi teg bilan yaratiladi? — **B) **
6. 6. Jadval katagini ikki qator balandligida birlashtiruvchi atribut? — **D) rowspan="2"**
7. 7. Forma maydonidagi ma'lumot serverga qaysi atribut nomi bilan yuboriladi? — **A) name**
8. 8. Bir guruhdan faqat bitta variant tanlash uchun qaysi input turi? — **C) radio**
9. 9. sahifalar/aloqa.html dan rasmlar/logo.png ga to'g'ri yo'l? — **B) ../rasmlar/logo.png**
10. 10. Rasmning matnli tavsifi qaysi atributda yoziladi? — **D) alt**
11. 11. Mustaqil maqola yoki yangilik uchun eng mos semantik teg? — **A) <article>**
12. 12. Butun saytga yagona dizayn berishning eng to'g'ri usuli? — **C) External CSS fayl**
13. 13. id="menyu" elementni tanlovchi selektor? — **B) #menyu**
14. 14. Qaysi selektor eng kuchli (ustuvor)? — **D) #asosiy**
15. 15. Fon rasmi butun maydonni to'ldirishi uchun? — **A) background-size: cover**
16. 16. Kvadrat elementni doiraga aylantirish uchun? — **C) border-radius: 50%**
17. 17. Element chegarasi va mazmuni orasidagi ichki bo'shliq? — **B) padding**
18. 18. box-sizing: border-box nima qiladi? — **D) width ichiga padding va border'ni ham qo'shadi**
19. 19. Flex konteynerda elementlarni asosiy o'q bo'yicha markazlash? — **A) justify-content: center**
20. 20. 3 ta teng ustunli Grid qaysi? — **C) grid-template-columns: repeat(3, 1fr)**
21. 21. Ichidagi absolute elementlar unga nisbatan joylashishi uchun ota elementga nima beriladi? — **B) position: relative**
22. 22. @media (max-width: 768px) qoidalari qachon ishlaydi? — **D) Ekran 768px yoki undan kichik bo'lsa**
23. 23. Hover effekti silliq bo'lishi uchun ishlatiladigan xossa? — **A) transition**
24. 24. Lokal o'zgarishlarni GitHub'ga yuboruvchi Git buyrug'i? — **C) git push**
25. 25. Netlify yoki Vercel nima? — **B) Saytlarni bepul joylash (hosting) platformalari**



15-MAVZU · 42 TADAN · II BOB. JAVASCRIPT ASOSLARI

JavaScript'ga kirish. O'zgaruvchilar va ma'lumot turlari

JavaScript nima va qayerda ishlatiladi, kodni ulash usullari, konsol, let/const/var, o'zgaruvchilarni nomlash qoidalari, string, number, boolean, undefined, null turlari va typeof.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- JavaScript nima va u HTML/CSS'dan nimasi bilan farq qilishini;
- JavaScript kodini sahifaga ulash va brauzer konsolidan foydalanishni;
- **O'zgaruvchi** nima ekanini va `let`, `const` bilan yaratishni;
- Asosiy ma'lumot turlarini: matn, son, mantiqiy qiymat va boshqalar;
- Birinchi interaktiv dasturingizni yozishni.

JavaScript nima?

JavaScript (JS) — veb-sahifalarga **harakat va mantiq** qo'shuvchi **dasturlash tili**. HTML va CSS'dan farqli o'laroq, unda hisob-kitob, shartlar, takrorlanishlar bor — ya'ni u haqiqiy dasturlash tili.



O'xshatish: uy (davomi)

HTML — devorlar, CSS — bo'yoq va mebel. **JavaScript** — elektr tarmog'i va "aqlli uy" tizimi: tugma bosilsa chiroq yonadi, eshik oldiga kelsangiz qo'ng'iroq chalinadi, harorat pasaysa isitish yoqiladi.

JavaScript bilan nimalar qilinadi?

- Tugma bosilganda biror narsa sodir bo'lishi (menyu ochilishi, rasm almashishi);
- Formani yuborishdan oldin tekshirish ("Parol juda qisqa!");

- Kalkulyator, o'yinlar, testlar, taymerlar;
- Sahifani qayta yuklamasdan yangi ma'lumot olish (Instagram lentasi, ob-havo).



JavaScript ≠ Java

Nomlari o'xshash, lekin bu ikki butunlay boshqa til — xuddi "olma" va "olmos" kabi. JavaScript 1995-yilda atigi 10 kunda yaratilgan va hozir dunyodagi eng mashhur dasturlash tillaridan biri. U nafaqat brauzerda, balki serverda (Node.js) va mobil ilovalarda ham ishlaydi.

JavaScript'ni ishga tushirish

1. Brauzer konsoli — eng tez usul

Chrome'da **F12** → **Console** yorlig'i. Bu yerga kod yozib **Enter** bossangiz, darhol bajariladi:

JavaScript

```
console.log("Salom, dunyo!");  
console.log(2 + 3);  
console.log("Men", 14, "yoshdaman");
```

`console.log()` — qavs ichidagini konsolga chiqaradi. Dasturchilar undan kodni tekshirish uchun doimo foydalanadi. (Bu sahifadagi ► tugmasi natijani o'ng tomonda ko'rsatadi.)

2. HTML sahifada — `<script>` tegi

HTML

```
<!DOCTYPE html>
<html lang="uz">
<head><meta charset="UTF-8"><title>JS</title></head>
<body>
  <h1>Birinchi dasturim</h1>

  <script>
    alert("Saytga xush kelibsiz!");           // oyna chiqaradi
    console.log("Bu konsolda ko'rinadi");
    document.write("<p>Bu sahifaga yozildi</p>");
  </script>
</body>
</html>
```

3. Alohida `.js` fayl

index.html

```
<body>
  <h1>Sahifa</h1>
  <!-- Skript body OXIRIDA: avval HTML yuklansin, keyin JS -->
  <script src="app.js"></script>
</body>
```

app.js

```
console.log("app.js ishladi!");
```

CSS kabi, eng to'g'ri usul — alohida fayl. `<script>` ni `</body>` dan oldin qo'ying — shunda JavaScript ishlay boshlaganda sahifa elementlari allaqachon mavjud bo'ladi.

Izohlar

JavaScript

```
// Bir qatorli izoh

/* Ko'p qatorli
   izoh */
```

O'zgaruvchilar

O'zgaruvchi — ma'lumotni saqlash uchun **nomlangan "quti"**. Unga qiymat qo'yasiz va keyinroq nomi orqali ishlatasiz.



O'xshatish: yorliqli qutilar

Javonda qutilar turibdi. Har biriga yorliq yopishtirilgan: "ism", "yosh", "ball". "ism" qutisiga "Malika" qog'ozini, "yosh" qutisiga 14 raqamini soldingiz. Endi "yosh qutisida nima bor?" deb so'rasangiz — 14. Yorliq — o'zgaruvchi **nomi**, ichidagi narsa — uning **qiymati**.

JavaScript

```
let ism = "Malika";           // "ism" qutisini yaratdik va ichiga "Malika" ni
                                soldik
let yosh = 14;
console.log(ism);              // Malika
console.log(yosh);             // 14

yosh = 15;                     // qiymatni o'zgartirish (let ni qayta yozmaymiz!)
console.log(yosh);             // 15

let shahar;                    // e'lon qilindi, lekin qiymat berilmadi
console.log(shahar);           // undefined
```

- `let` — o'zgaruvchi e'lon qilish kalit so'zi.
- `=` — **tayinlash** belgisi ("tenglik" emas!): "o'ngdagi qiymatni chapdagi qutiga sol".
- Satr oxirida `;` (nuqta-vergul) — buyruq tugadi.

let, const va var

JavaScript

```
const PI = 3.14;           // const — o'zgarmas (constant)
const tugilganYil = 2012;
// PI = 5;                 // ❌ Xato! const qiymatini o'zgartirib bo'lmaydi

let ball = 0;              // let — o'zgaradigan qiymat
ball = ball + 10;
console.log(ball);         // 10
```

	const	let	var
Qiymatni o'zgartirish	❌ Yo'q	✅ Ha	✅ Ha
Qachon ishlatish	Doim birinchi tanlov	Qiymat o'zgarishi kerak bo'lsa	Eski kodlarda uchraydi. Ishlatmang



Oltin qoida

Har doim `const` bilan boshlang. Agar keyinroq qiymatni o'zgartirish kerak bo'lsa — `let` ga almashtiring. `var` eski usul va kutilmagan xatolarga olib keladi.

O'zgaruvchilarni nomlash qoidalari

Qoida	✅ To'g'ri	❌ Noto'g'ri
Harf, <code>\$</code> yoki <code>_</code> bilan boshlanadi	<code>ism</code> , <code>_soni</code>	<code>1ism</code>
Bo'sh joy bo'lmaydi	<code>sinfRahbari</code>	<code>sinf rahbari</code>
Defis bo'lmaydi	<code>umumiyBall</code>	<code>umumiy-ball</code>
Maxsus so'zlar bo'lmaydi	<code>mening_let</code>	<code>let</code> , <code>const</code> , <code>if</code>
Apostrof (') bo'lmaydi	<code>ogil</code> , <code>yosh</code>	<code>o'g'il</code>

- JS **katta-kichik harfni farqlaydi**: `ism` va `Ism` — ikki xil o'zgaruvchi!
- Bir necha so'zli nomlar **camelCase** uslubida yoziladi: birinchi so'z kichik, keyingilari katta harf bilan — `oquvchiSoni`, `dasturlashTili`.
- Nom **mazmunli** bo'lsin: `x`, `a1` ❌ → `narx`, `chegirmaFoizi` ✅.

Ma'lumot turlari

Har bir qiymatning **turi** bor. Tur — bu qiymat bilan nima qilish mumkinligini belgilaydi: sonlarni qo'shish mumkin, matnni katta harfga aylantirish mumkin.

1. String — matn (satr)

JavaScript

```
const ism = "Aziz";           // qo'sh tirnoq
const familiya = 'Karimov';   // bitta tirnoq
const shahar = `Toshkent`;    // teskari tirnoq (backtick) — eng qulayi

// Satrlarni birlashtirish
console.log(ism + " " + familiya);           // Aziz Karimov

// Shablon satr (template literal): ${} ichiga o'zgaruvchi qo'yiladi
const yosh = 14;
console.log(`Mening ismim ${ism}, ${yosh} yoshdaman, ${shahar}da yashayman.`);

// Foydali imkoniyatlar
console.log(ism.length);           // 4 — harflar soni
console.log(ism.toUpperCase());     // AZIZ
console.log(ism.toLowerCase());     // aziz
console.log(familiya[0]);           // K — birinchi harf (sanash 0 dan boshlanadi!)
console.log("salom dunyo".includes("dunyo")); // true
```



O'zbekcha apostrof bilan ehtiyot bo'ling

`'0'zbekiston'` ❌ — birinchi `'` satrni ochadi, ikkinchisi yopadi va xato chiqadi. Yechim: qo'sh tirnoq yoki backtick ishlatib: `"0'zbekiston"` yoki ``0'zbekiston`` ✅.

2. Number — son

JavaScript

```
const butun = 42;
const kasr = 3.14;           // o'nlik kasr NUQTA bilan (vergul emas!)
const manfiy = -10;
console.log(butun + kasr);    // 45.14
console.log(10 / 3);          // 3.3333333333333335
console.log(10 / 0);          // Infinity — cheksizlik
console.log("olma" * 2);      // NaN — "Not a Number" (son emas)
```

3. Boolean — mantiqiy qiymat

Faqat ikki qiymat: `true` (rost, ha) yoki `false` (yolg'on, yo'q). Shartlarda ishlatiladi (16-mavzu).

JavaScript

```
const tizimgaKirgan = true;
const darsTugadi = false;
console.log(5 > 3);          // true
console.log(10 === 7);       // false
```

4. Undefined va null

- `undefined` — "qiymat hali **berilmagan**" (bo'sh quti, ichiga hech narsa solinmagan).
- `null` — "qiymat **ataylab bo'sh**" (dasturchi qutini o'zi bo'shatib qo'ygan).

Murakkab turlar (oldinda)

Array (massiv, ro'yxat) `[1, 2, 3]` va **Object** (obyekt) `{ ism: "Ali" }` — 18-mavzuda batafsil.

Turni aniqlash: `typeof`

JavaScript

```
console.log(typeof "Salom");    // string
console.log(typeof 25);         // number
console.log(typeof true);       // boolean
console.log(typeof undefined);  // undefined
console.log(typeof null);       // object ← JavaScript'ning mashhur "tarixiy xatosi"!
console.log(typeof [1, 2]);     // object
```

Turlarni o'zgartirish

JavaScript

```
// Matn + son = matn (son matnga aylanadi)
console.log("5" + 3);           // "53" 😬
console.log("5" - 3);           // 2    (ayirishda matn songa aylanadi)

// Aniq o'zgartirish
console.log(Number("42") + 8);  // 50
console.log(String(100) + "%"); // "100%"
console.log(parseInt("15px"));  // 15
console.log(parseFloat("3.5")); // 3.5
```



Foydalanuvchi kiritgan har qanday ma'lumot (`prompt` , forma) **doim matn** bo'lib keladi. Hisoblashdan oldin `Number()` bilan songa aylantiring!

Foydalanuvchi bilan muloqot

JavaScript

```
const ism = prompt("Ismingiz nima?"); // matn kiritish oynasi
const tugilganYil = Number(prompt("Qaysi yilda tug'ilgansiz?"));
const yosh = 2026 - tugilganYil;

alert(`Salom, ${ism}! Siz ${yosh} yoshdasiz.`);
const tasdiq = confirm("Dasturlashni yoqtirasizmi?"); // OK → true, Cancel → false
console.log("Javob:", tasdiq);
```

Funksiya	Vazifasi	Nima qaytaradi
alert("...")	Xabar oynasi	Hech narsa
prompt("...")	Matn kiritish oynasi	Kiritilgan matn (yoki <code>null</code>)
confirm("...")	OK / Bekor qilish oynasi	<code>true</code> yoki <code>false</code>

Bu oynalar eski va unchalik chiroyli emas — o'rganish uchun qulay. 19-mavzudan boshlab sahifaning o'zida input va tugmalar bilan ishlaymiz.

Amaliy mashg'ulot

- 1 `js-darslar/` papkasida `index.html` va `app.js` yarating, skriptni `</body>` oldidan ulang.
- 2 **"Men haqimda" dasturi.** O'zingiz haqingizda 5 ta o'zgaruvchi yarating (ism, familiya, yosh, sinf, sevimli fan) va shablon satr bilan bitta chiroyli gapni konsolga chiqaring.
- 3 **Yosh kalkulyatori.** `prompt` bilan tug'ilgan yilni so'rang va: yoshni, 2030-yilda necha yoshda bo'lishini, necha kun yashaganini (taxminan, $\times 365$) hisoblang.
- 4 **Do'kon cheki.**

JavaScript

```
const mahsulot = "Daftar";
const narx = 5000;
const soni = 4;
const jami = narx * soni;

console.log("==== CHEK =====");
console.log(`Mahsulot: ${mahsulot}`);
console.log(`Narxi: ${narx} so'm × ${soni} dona`);
console.log(`Jami: ${jami} so'm`);
console.log("=====");
console.log("Xaridingiz uchun rahmat!");
```

Kodni o'zgartiring: yana ikkita mahsulot qo'shing va umumiy summani hisoblang.

- 5 **Tur detektivi.** 8 ta turli qiymat (matn, son, kasr, true, undefined, null, "123", 0) uchun `typeof` natijalarini oldindan daftarga yozing, so'ng konsolda tekshiring.

Ko'p uchraydigan xatolar

- ✗ `Uncaught SyntaxError` — qo'shtirnoq yopilgan yoki o'zbekcha apostrof satrni "sindirib" qo'ygan.
- ✗ `ReferenceError: x is not defined` — o'zgaruvchi nomi noto'g'ri yozilgan (katta-kichik harf!) yoki e'lon qilinmagan.
- ✗ `TypeError: Assignment to constant variable` — `const` qiymatini o'zgartirishga urinish.
- ✗ `prompt` dan olingan sonni qo'shish: `"10" + 5 = "105"`. `Number()` unutilgan.
- ✗ Bitta o'zgaruvchini ikki marta `let` bilan e'lon qilish: `let a = 1; let a = 2;`



Xatolarni o'qishni o'rganing

Konsoldagi qizil xabar — dushman emas, yordamchi! U xato turini va qaysi faylning **qaysi qatorida** ekanini aytadi (masalan, `app.js:12`). Ustiga bossangiz, o'sha qatorga olib boradi.

O'zingizni tekshiring

? Qiymati keyinchalik o'zgarmaydigan o'zgaruvchi qaysi kalit so'z bilan e'lon qilinadi?

- A. `let`
- B. `const`
- C. `var`
- D. `fix`

? Qaysi o'zgaruvchi nomi to'g'ri?

- A. `2sinf`
- B. `umumiy-ball`
- C. `sinf rahbari`
- D. `oquvchilarSoni`

? `console.log("7" + 3)` natijasi?

- A. 10
- B. 4
- C. "73"
- D. Xato

? `typeof true` nima qaytaradi?

- A. `"boolean"`
- B. `"string"`
- C. `"true"`
- D. `"number"`

? `const ism = "Ali"; console.log(`Salom, ${ism}!`)` nima chiqaradi?

- A. `Salom, ${ism}!`
- B. `Salom, Ali!`
- C. `Salom, ism!`
- D. Xato

? `prompt()` qanday turdagi qiymat qaytaradi?

- A. Doim son
- B. Boolean
- C. Matn (string) yoki null
- D. Massiv



Uyga vazifa

1. **Valyuta hisoblagichi (oddiy):** foydalanuvchidan dollar miqdorini so'rang va kurs (`const KURS = 12700`) bo'yicha so'mga aylantirib, `alert` bilan ko'rsating.
2. **Doira:** radiusni so'rang, yuzasi ($\pi \cdot r^2$) va uzunligini ($2 \cdot \pi \cdot r$) hisoblang. `Math.PI` dan foydalaning.
3. Ismingizni so'rab, quyidagilarni chiqaring: harflar soni, KATTA harflarda, birinchi va oxirgi harfi.

✓ Yodda tuting

- JavaScript — sahifaga mantiq va harakat qo'shuvchi dasturlash tili; alohida `.js` fayl `</body>` oldidan ulanadi.
- `console.log()` — natijani konsolga chiqarish (`F12` → Console).
- `const` — o'zgarmas, `let` — o'zgaruvchan; `var` ishlatmang. Nomlar — camelCase.
- Turlar: string, number, boolean, undefined, null (+ array, object). `typeof` — turini aniqlash.
- Shablon satr: ``Salom, ${ism}!``. Kiritilgan ma'lumot matn — `Number()` bilan songa aylantiring.

✓ Test javoblari

1. Qiymati keyinchalik o'zgarmaydigan o'zgaruvchi qaysi kalit so'z bilan e'lon qilinadi? — **B) const**
2. Qaysi o'zgaruvchi nomi to'g'ri? — **D) oquvchilarSoni**
3. `console.log("7" + 3)` natijasi? — **C) "73"**
Matnga son qo'shilsa, son matnga aylanib, ular yopishtiriladi.
4. `typeof true` nima qaytaradi? — **A) "boolean"**
5. `const ism = "Ali"; console.log(`Salom, ${ism}!`)` nima chiqaradi? — **B) Salom, Ali!**
6. `prompt()` qanday turdagi qiymat qaytaradi? — **C) Matn (string) yoki null**



16-MAVZU · 42 TADAN · II BOB. JAVASCRIPT ASOSLARI

JavaScript'da operatorlar va ifodalar. Shart operatorlari

Arifmetik, tayinlash, taqqoslash va mantiqiy operatorlar; `==` va `===` farqi; `if`, `else if`, `else`; ternary operator; `switch`; amaliy dasturlar: baho aniqlash, juft-toq, kirish tizimi.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Arifmetik amallar va qoldiqli bo'lish (`%`);
- Qiymatlarni taqqoslash va `=` bilan `===` farqini;
- Mantiqiy operatorlar: VA (`&&`), YOKI (`||`), EMAS (`!`);
- Dasturga "qaror qabul qildirish": `if`, `else if`, `else`, `switch` ;
- Baho hisoblovchi, parolni tekshiruvchi va boshqa amaliy dasturlarni.

Operator va ifoda

Operator — qiymatlar ustida amal bajaruvchi belgi (`+`, `>`, `&&`). **Ifoda** — natijada qiymat beradigan kod bo'lagi: `5 + 3` → `8`, `yosh ≥ 18` → `true` .

Arifmetik operatorlar

JavaScript

```
const a = 17;
const b = 5;
console.log(a + b);    // 22  qo'shish
console.log(a - b);    // 12  ayirish
console.log(a * b);    // 85  ko'paytirish
console.log(a / b);    // 3.4 bo'lish
console.log(a % b);    // 2   QOLDIQ: 17 = 5·3 + 2
console.log(a ** 2);   // 289 daraja: 172
console.log((a + b) * 2); // 44 – qavslar avval bajariladi
```

Qoldiq operatori `%` — juda foydali!

- **Juft yoki toq?** `son % 2 === 0` bo'lsa — juft.
- **Bo'linadimi?** `son % 3 === 0` — 3 ga qoldiqsiz bo'linadi.
- **Oxirgi raqam:** `1234 % 10` → `4`.
- **Vaqt:** 135 daqiqa = `Math.floor(135 / 60)` soat va `135 % 60` daqiqa → 2 soat 15 daqiqa.

Math obyekti

JavaScript

```
console.log(Math.round(4.6)); // 5   – yaxlitlash
console.log(Math.floor(4.9)); // 4   – pastga yaxlitlash
console.log(Math.ceil(4.1));  // 5   – yuqoriga yaxlitlash
console.log(Math.max(3, 9, 2)); // 9   – eng kattasi
console.log(Math.min(3, 9, 2)); // 2
console.log(Math.sqrt(81));   // 9   – kvadrat ildiz
console.log(Math.abs(-7));    // 7   – modul
console.log(Math.random());   // 0 va 1 orasidagi tasodifiy son
console.log(Math.floor(Math.random() * 6) + 1); // 1 dan 6 gacha – o'yin
kubigi 🎲
```

Tayinlash operatorlari

JavaScript

```
let ball = 10;
ball += 5;    // ball = ball + 5 → 15
ball -= 3;    // ball = ball - 3 → 12
ball *= 2;    // ball = ball * 2 → 24
ball /= 4;    // ball = ball / 4 → 6
ball++;       // ball = ball + 1 → 7 (inkrement)
ball--;       // ball = ball - 1 → 6 (dekrement)
console.log(ball);
```

Taqqoslash operatorlari

Natijasi doim `true` yoki `false` :

Operator	Ma'nosi	Misol	Natija
>	katta	10 > 7	true
<	kichik	10 < 7	false
≥	katta yoki teng	18 ≥ 18	true
≤	kichik yoki teng	5 ≤ 4	false
≡	qat'iy teng (qiymat va tur)	5 ≡ 5	true
≢	qat'iy teng emas	5 ≢ 3	true

`=` va `===` — muhim farq!

JavaScript

```
console.log(5 = "5");    // true 🤪 — turni o'zgartirib taqqoslaydi
console.log(5 === "5");  // false ✅ — son va matn har xil narsa
console.log(0 = false);  // true 🤪
console.log(0 === false); // false ✅
```



Qoida

Doim `===` va `!==` ishlatib. `=` turlarni "o'zicha" o'zgartiradi va kutilmagan xatolarga olib keladi. Va yana: `=` — tayinlash, `===` — taqqoslash. Ularni adashtirmang!

Mantiqiy operatorlar

Operator	Ma'nosi	Qachon true
<code>&&</code>	VA	Ikkala shart ham rost bo'lsa
<code> </code>	YOKI	Kamida bittasi rost bo'lsa
<code>!</code>	EMAS	Qiymatni teskarisiga aylantiradi



Hayotiy misollar

VA: "Kinoga borasan — agar uy vazifasini bajargan **va** xonangni yig'ishtirgan bo'lsang." Ikkalasi ham kerak.

YOKI: "Muzqaymoq olasan — agar a'lo baho olgan **yoki** tug'ilgan kuning bo'lsa." Bittasi yetarli.

EMAS: "Agar yomg'ir yog'**mayotgan** bo'lsa, futbolga chiqamiz."

JavaScript

```
const yosh = 15;
const ruxsatnoma = true;

console.log(yosh ≥ 14 && ruxsatnoma); // true – ikkalasi ham rost
console.log(yosh ≥ 18 && ruxsatnoma); // false – birinchisi yolg'on
console.log(yosh ≥ 18 || ruxsatnoma); // true – ikkinchisi rost
console.log(!ruxsatnoma);           // false

// Oraliqni tekshirish: 10 va 20 orasidami?
const son = 15;
console.log(son ≥ 10 && son ≤ 20);    // true
```

Shart operatori: `if`

Dastur hamma vaqt bir xil ishlamaydi — u **vaziyatga qarab qaror qabul qiladi**.

JavaScript

```
if (shart) {
  // shart TRUE bo'lsa bajariladi
} else {
  // shart FALSE bo'lsa bajariladi
}
```

JavaScript

```
const harorat = 28;

if (harorat > 25) {
  console.log("Issiq! Kepka kiying 🧢");
} else {
  console.log("Havo salqin 🧣");
}
```

else if — bir nechta variant

Shartlar **yuqoridan pastga** tekshiriladi va **birinchi** rost shart bajariladi — qolganlari tekshirilmaydi.

JavaScript

```
const ball = 78;
let baho;

if (ball ≥ 86) {
  baho = 5;
} else if (ball ≥ 71) {
  baho = 4;
} else if (ball ≥ 56) {
  baho = 3;
} else {
  baho = 2;
}

console.log(`${ball} ball → ${baho} baho`);
```

ball = 78



78 ≥ 86?
yo'q



78 ≥ 71?
HA ✓



baho = 4
qolganlari tekshirilmaydi



Tartib muhim!

Agar `ball ≥ 56` shartini birinchi yozsangiz, 95 ball ham "3" bo'lib qoladi — chunki $95 \geq 56$ rost va dastur shu yerda to'xtaydi. Shartlarni **kattadan kichikka** (yoki aniqroqdan umumiyroqqa) yozing.

Ichma-ich shartlar

JavaScript

```
const login = "admin";
const parol = "12345";

if (login === "admin") {
  if (parol === "12345") {
    console.log("✅ Xush kelibsiz, admin!");
  } else {
    console.log("❌ Parol noto'g'ri");
  }
} else {
  console.log("❌ Bunday foydalanuvchi yo'q");
}
```

Ternary (uchlik) operator — qisqa if

JavaScript

```
const yosh = 16;
// shart ? rost_bo'lsa : yolg'on_bo'lsa
const holat = yosh ≥ 18 ? "katta" : "voyaga yetmagan";
console.log(holat);

const son = 7;
console.log(`${son} - ${son % 2 === 0 ? "juft" : "toq"} son`);
```

Oddiy, ikki variantli tanlovlar uchun qulay. Murakkab shartlar uchun oddiy `if` o'qilishi osonroq.

switch — ko'p variantli tanlov

Bitta qiymatni ko'p aniq variantlar bilan solishtirish kerak bo'lganda `if...else if` zanjiridan qulayroq:

```
const kun = 3;
let kunNomi;

switch (kun) {
  case 1:
    kunNomi = "Dushanba";
    break;
  case 2:
    kunNomi = "Seshanba";
    break;
  case 3:
    kunNomi = "Chorshanba";
    break;
  case 4:
    kunNomi = "Payshanba";
    break;
  case 5:
    kunNomi = "Juma";
    break;
  case 6:
  case 7:
    kunNomi = "Dam olish kuni 🎉"; // 6 va 7 uchun bir xil natija
    break;
  default:
    kunNomi = "Bunday kun yo'q"; // hech biri mos kelmasa
}

console.log(kunNomi);
```



`break` ni unutsangiz, dastur keyingi `case` larni ham bajarib ketadi! `switch` qat'iy taqqoslash (`===`) bilan ishlaydi.

Amaliy mashg'ulot

- 1 **Juft yoki toq.** Foydalanuvchidan son so'rang va uning juft/toq, musbat/manfiy/nol ekanini aniqlang.

2 **Baho aniqlovchi.** 0–100 oralig'idagi ballni so'rang. Agar ball shu oraliqda bo'lmasa — "Noto'g'ri ball!" deb chiqaring. Aks holda bahoni va izohni ("A'lo!", "Yaxshi", ...) chiqaring.

3 **Kirish tizimi:**

JavaScript

```
const TOGRI_LOGIN = "malika";
const TOGRI_PAROL = "qwerty123";

const login = prompt("Login:");
const parol = prompt("Parol:");

if (login === null || parol === null) {
  alert("Kirish bekor qilindi");
} else if (login === "" || parol === "") {
  alert("⚠ Login va parol bo'sh bo'lmasligi kerak");
} else if (login.toLowerCase() === TOGRI_LOGIN && parol === TOGRI_PAROL) {
  alert(`✅ Xush kelibsiz, ${login}!`);
} else {
  alert("❌ Login yoki parol noto'g'ri");
}
```

Nima uchun loginni `toLowerCase()` bilan, parolni esa aynan solishtiryapmiz? Muhokama qiling.

4 **Kabisa yili.** Yil 4 ga bo'linsa va 100 ga bo'linmasa, yoki 400 ga bo'linsa — kabisa yili. Foydalanuvchi kiritgan yilni tekshiring (2024 ✅, 1900 ❌, 2000 ✅).

5 **Chipta narxi.** Kinoteatr: 7 yoshgacha — bepul, 7–17 — 20 000, 18–59 — 35 000, 60+ — 15 000 so'm. Seshanba kuni hammaga 50% chegirma. Yosh va hafta kunini so'rab, narxni hisoblang.

6 **Hafta kuni (switch).** 1–12 raqamini so'rab, oy nomini va qaysi faslga tegishli ekanini chiqaring.

Ko'p uchraydigan xatolar

- ❌ `if (ball = 100)` — bu taqqoslash emas, **tayinlash!** To'g'risi: `if (ball === 100)`.
- ❌ `if (10 < son < 20)` — JS'da bunday yozib bo'lmaydi. To'g'risi: `if (son > 10 && son < 20)`.
- ❌ `prompt` natijasini `Number()` siz taqqoslash: `"9" > "10" → true` (matnlar alifbo bo'yicha taqqoslanadi!).

- ❌ `switch` da `break` unutilgan.
- ❌ Jinalak qavslarni yopishni unutish — VS Code qavslarni rangli qilib juftlab ko'rsatadi, kuzating.

O'zingizni tekshiring

? `17 % 5` natijasi?

- A. 3.4
- B. 3
- C. 2
- D. 85

? `5 === "5"` natijasi?

- A. true
- B. false
- C. Xato
- D. undefined

? `true && false` natijasi?

- A. false
- B. true
- C. null
- D. Xato

? `let x = 10; x += 5; x *= 2;` dan keyin x nechaga teng?

- A. 17
- B. 20
- C. 25
- D. 30

? Ball 92 bo'lsa, quyidagi kod nima chiqaradi?

`if (ball ≥ 56) console.log("3"); else if (ball ≥ 86) console.log("5");`

- A. 5
- B. 3
- C. Ikkalasi ham
- D. Hech narsa

? `const t = 20 > 10 ? "ha" : "yo'q";` — t ning qiymati?

- A. `true`
- B. "yo'q"
- C. "ha"
- D. 20



Uyga vazifa

1. **BMI kalkulyatori:** vazn (kg) va bo'y (m) so'rab, $BMI = \text{vazn} / \text{bo'y}^2$ ni hisoblang va natijani izohlang (kam vazn, me'yorda, ortiqcha).
2. **Uchburchak:** uch tomon uzunligini so'rab, bunday uchburchak mavjudmi (har bir tomon qolgan ikkitasi yig'indisidan kichik) va qanday turda (teng tomonli, teng yonli, turli tomonli) ekanini aniqlang.

3. **Tosh-qaychi-qog'oz:** foydalanuvchi tanlaydi, kompyuter `Math.random()` bilan tanlaydi, g'olibni aniqlang.

✓ Yodda tuting

- Arifmetika: `+` `-` `*` `/` `%` `**`; `%` — qoldiq (juft/toq tekshirish).
- Taqqoslash: `>` `<` `>=` `<=` `==` `!=` — doim `==`.
- Mantiq: `&&` (ikkalasi), `||` (bittasi), `!` (teskari).
- `if / else if / else` — birinchi rost shart bajariladi; tartibga e'tibor bering.
- Ternary: `shart ? a : b`; `switch` — ko'p aniq variantlar uchun, `break` bilan.

✓ Test javoblari

1. $17 \% 5$ natijasi? — **C) 2**

$17 = 5 \times 3 + 2$. % — qoldiq.

2. $5 === "5"$ natijasi? — **B) false**

$===$ turini ham tekshiradi: son va matn har xil.

3. $true \&\& false$ natijasi? — **A) false**

4. `let x = 10; x += 5; x *= 2;` dan keyin x nechaga teng? — **D) 30**

5. Ball 92 bo'lsa, quyidagi kod nima chiqaradi?`if (ball >= 56) console.log("3"); else if (ball >= 86) console.log("5");` — **B) 3**

Birinchi rost shart bajariladi. Tartib noto'g'ri yozilgani uchun xato natija chiqdi.

6. `const t = 20 > 10 ? "ha" : "yo'q";` — t ning qiymati? — **C) "ha"**



17-MAVZU · 42 TADAN · II BOB. JAVASCRIPT ASOSLARI

JavaScript'da takrorlanish operatorlari va funksiyalar

for, while, do...while sikllari, break va continue, cheksiz sikl xavfi; funksiya e'lon qilish, parametrlar, return, arrow funksiyalar, ko'rinish doirasi; kalkulyator va son topish o'yini.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- Bir xil ishni ko'p marta bajaruvchi **sikllarni**: `for`, `while`, `do...while`;
- Siklni to'xtatish (`break`) va qadamni o'tkazib yuborish (`continue`);
- **Funksiya** — qayta ishlatiladigan kod bloki yaratishni;
- Parametrlar, `return` va arrow funksiyalarni;
- Kalkulyator va "Sonni top" o'yinini (9 va 10-loyihalar).

Nima uchun sikl kerak?

1 dan 100 gacha sonlarni chiqarish kerak. 100 qator `console.log` yozasizmi? Albatta yo'q! **Sikl** — kodni berilgan shart bajarilguncha **qayta-qayta** bajaruvchi konstruktsiya.



O'xshatish: sport mashg'uloti

Murabbiy: "20 marta o'tirib-tur!" Siz har safar sanaysiz: 1, 2, 3... 20 ga yetganda to'xtaysiz. Bu — `for` sikli: boshlang'ich qiymat (1), shart (20 gacha) va qadam (har safar +1).

for sikli

JavaScript

```
for (boshlanish; shart; qadam) {  
  // takrorlanadigan kod (siklning "tanasi")  
}
```

JavaScript

```
for (let i = 1; i ≤ 5; i++) {  
  console.log(`${i}-marta o'tirib-turdim`);  
}  
console.log("Mashq tugadi! 💪");
```

Qism	Kod	Ma'nosi
Boshlanish	<code>let i = 1</code>	Hisoblagich 1 dan boshlanadi (bir marta bajariladi)
Shart	<code>i ≤ 5</code>	Har qadam oldidan tekshiriladi; <code>false</code> bo'lsa sikl tugaydi
Qadam	<code>i++</code>	Har aylanishdan keyin i bittaga oshadi

Qadamma-qadam: $i=1 \rightarrow 1 \leq 5?$ ha $\rightarrow$ chiqar $\rightarrow i=2 \rightarrow \dots \rightarrow i=5 \rightarrow$ chiqar $\rightarrow i=6 \rightarrow 6 \leq 5?$ yo'q $\rightarrow$ **tugadi**.

Turli sikllar

JavaScript

```
// Teskari sanash
for (let i = 10; i ≥ 1; i--) {
  console.log(i);
}
console.log("🚀 Start!");

// 2 qadam bilan: faqat juft sonlar
for (let i = 2; i ≤ 20; i += 2) {
  console.log(i);
}
```

Yig'indi hisoblash — eng muhim namuna

JavaScript

```
let yigindi = 0; // "hamyon" — avval bo'sh
for (let i = 1; i ≤ 100; i++) {
  yigindi += i; // har bir sonni hamyonga qo'shamiz
}
console.log(`1 dan 100 gacha yig'indi: ${yigindi}`); // 5050
```

Bu **akkumulyator** (yig'uvchi) namunasi: sikldan oldin 0 yoki bo'sh qiymatli o'zgaruvchi, sikl ichida unga qo'shib borish. Ko'p masalalarda kerak bo'ladi.

Ko'paytirish jadvali — sikl ichida sikl

JavaScript

```
for (let i = 1; i ≤ 5; i++) {
  let qator = "";
  for (let j = 1; j ≤ 5; j++) {
    qator += (i * j) + "\t"; // \t — tabulyatsiya (bo'sh joy)
  }
  console.log(qator);
}
```

while va do...while

`while` — **nechta** qadam bo'lishini oldindan bilmaganimizda ishlatiladi: "shart rost ekan, takrorla".

JavaScript

```
// Pul jamg'arish: har oy 50 000 so'm, velosiped 400 000 so'm
let jamgarma = 0;
let oy = 0;
while (jamgarma < 400000) {
  jamgarma += 50000;
  oy++;
}
console.log(`${oy} oyda velosipedga pul yig'iladi 🚲`);
```

JavaScript

```
// do...while — kamida BIR MARTA bajariladi, shart keyin tekshiriladi
let parol;
do {
  parol = prompt("Kamida 6 belgili parol kiriting:");
} while (parol !== null && parol.length < 6);
console.log(parol ? "Parol qabul qilindi ✅" : "Bekor qilindi");
```



Cheksiz sikl!

Agar shart **hech qachon** `false` bo'lmasa, sikl to'xtamaydi va brauzer "qotib" qoladi: `while (true) { }` yoki hisoblagichni oshirishni unutish: `let i = 0; while (i < 5) { console.log(i); } — i++` yo'q! Shunday bo'lsa, brauzer yorlig'ini yoping.

break va continue

JavaScript

```
// break — siklni darhol TO'XTATISH
for (let i = 1; i ≤ 100; i++) {
  if (i % 7 === 0 && i % 5 === 0) {
    console.log(`5 ga ham, 7 ga ham bo'linadigan birinchi son: ${i}`);
    break;
  }
}

// continue — joriy qadamni O'TKAZIB, keyingisiga o'tish
for (let i = 1; i ≤ 10; i++) {
  if (i % 3 === 0) continue; // 3 ga bo'linadiganlarni tashlab ketamiz
  console.log(i);           // 1 2 4 5 7 8 10
}
```

Funksiyalar

Funksiya — nom berilgan, **qayta ishlatiladigan** kod bloki. Bir marta yozasiz — xohlagancha chaqirasiz.



O'xshatish: blender (mikser)

Blenderga meva **solasiz** (parametrlar), tugmani **bosasiz** (chaqirish), ichida maydalanadi (funksiya tanasi), va sizga **sharbat qaytaradi** (return). Har safar blender yasash shart emas — tayyor blenderdan foydalanasiz. Olma solsangiz — olma sharbati, banan solsangiz — banan sharbati.

Funksiya e'lon qilish va chaqirish

JavaScript

```
// 1. E'lon qilish (yaratish) — hali hech narsa bajarilmaydi
function salomlash() {
  console.log("Assalomu alaykum!");
  console.log("Darsga xush kelibsiz!");
}

// 2. Chaqirish — endi bajariladi
salomlash();
salomlash(); // xohlagancha marta
```

Parametrlar — funksiyaga ma'lumot berish

JavaScript

```
function salomlash(ism, sinf) { // ism, sinf — parametrlar
  console.log(`Salom, ${ism}! Siz ${sinf}-sinfdasiz.`);
}

salomlash("Aziza", 8); // "Aziza", 8 — argumentlar
salomlash("Bobur", 9);

// Standart qiymat
function xabar(matn, belgi = " !") {
  console.log(belgi + " " + matn);
}

xabar("Ertaga nazorat ishi"); // ! Ertaga nazorat ishi
xabar("Yangi dars", "📢"); // 📢 Yangi dars
```

return — natija qaytarish

`console.log` faqat ekranga chiqaradi. `return` esa natijani **qaytaradi** — uni o'zgaruvchiga saqlash va boshqa hisoblarda ishlatish mumkin.

JavaScript

```
function kvadrat(son) {  
    return son * son;  
}  
  
const a = kvadrat(5);           // 25  
const b = kvadrat(3) + kvadrat(4); // 9 + 16 = 25  
console.log(a, b);  
  
function bahoAniqla(ball) {  
    if (ball ≥ 86) return 5;    // return funksiyani darhol TUGATADI  
    if (ball ≥ 71) return 4;  
    if (ball ≥ 56) return 3;  
    return 2;  
}  
console.log(bahoAniqla(92), bahoAniqla(60), bahoAniqla(30));
```



`return` dan keyingi kod bajarilmaydi. `return` siz funksiya `undefined` qaytaradi.

Arrow (strelkali) funksiyalar — qisqa yozuv

JavaScript

```
// Oddiy funksiya  
function qoshish(a, b) {  
    return a + b;  
}  
  
// Arrow funksiya — xuddi shu narsa  
const qoshish2 = (a, b) ⇒ {  
    return a + b;  
};  
  
// Bitta ifoda bo'lsa — eng qisqa (return avtomatik)  
const qoshish3 = (a, b) ⇒ a + b;  
const ikkilantir = son ⇒ son * 2;    // bitta parametr — qavs ixtiyoriy  
  
console.log(qoshish(2, 3), qoshish2(2, 3), qoshish3(2, 3), ikkilantir(21));
```

Zamonaviy kodda arrow funksiyalar juda ko'p uchraydi, ayniqsa 18-mavzudagi massiv metodlari bilan.

Ko'rinish doirasi (scope)

JavaScript

```
const maktab = "5-maktab";           // global — hamma joyda ko'rinadi

function tanishtir() {
  const ism = "Laylo";                // lokal — faqat funksiya ichida
  console.log(`${ism}, ${maktab}`); // ✅ ikkalasi ham ko'rinadi
}

tanishtir();
// console.log(ism);                // ❌ ReferenceError: ism is not defined
```

Funksiya ichida yaratilgan o'zgaruvchi — uning "shaxsiy xonasi". Tashqaridan unga kirib bo'lmaydi. Bu yaxshi: turli funksiyalar bir xil nomli o'zgaruvchilar bilan bir-biriga xalaqit bermaydi.

Yaxshi funksiya qoidalari

- **Bitta funksiya — bitta vazifa.** `yuzaHisobla()` yuzani hisoblasin, chiqarmasin.
- **Nomi fe'l bilan:** `hisobla`, `tekshir`, `korsat`, `getBaho`.
- **Takrorlanadigan kodni funksiyaga chiqaring** (DRY — Don't Repeat Yourself).

Amaliy mashg'ulot

1. Kalkulyator (9-loyiha, konsol versiyasi)

JavaScript

```
function hisobla(a, amal, b) {
  switch (amal) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/":
      if (b === 0) return "Nolga bo'lib bo'lmaydi!";
      return a / b;
    case "%": return a % b;
    case "^": return a ** b;
    default: return "Noma'lum amal";
  }
}

let davom = true;
while (davom) {
  const a = Number(prompt("Birinchi son:"));
  const amal = prompt("Amal (+ - * / % ^):");
  const b = Number(prompt("Ikkinchi son:"));

  if (Number.isNaN(a) || Number.isNaN(b)) {
    alert("Iltimos, son kiriting!");
  } else {
    alert(`${a} ${amal} ${b} = ${hisobla(a, amal, b)}`);
  }
  davom = confirm("Yana hisoblaymizmi?");
}
```

2. "Sonni top" o'yini (10-loyiha)

JavaScript

```
const yashirinSon = Math.floor(Math.random() * 100) + 1; // 1..100
const MAKS_URINISH = 7;
let urinish = 0;
let topildi = false;

alert("Men 1 dan 100 gacha son o'yladim. Topa olasizmi? Sizda 7 ta urinish bor!");

while (urinish < MAKS_URINISH && !topildi) {
  const javob = prompt(`${urinish + 1}-urinish. Sizning taxminingiz:`);
  if (javob === null) break; // "Bekor qilish" bosildi
  const son = Number(javob);
  urinish++;

  if (son === yashirinSon) {
    topildi = true;
    alert("🎉 Topdingiz! Bu ${yashirinSon} edi. Urinishlar: ${urinish}");
  } else if (son < yashirinSon) {
    alert("⬆ Kattaroq son o'ylang");
  } else {
    alert("⬇ Kichikroq son o'ylang");
  }
}

if (!topildi) alert("😞 Urinishlar tugadi. Javob: ${yashirinSon}");
```

Savol: nima uchun 7 ta urinish 1–100 oralig'i uchun doim yetarli? (Maslahat: har safar o'rtadagi sonni ayting — oraliq ikki baravar qisqaradi. Bu **ikkilik qidiruv** algoritmi.)

3. Mustaqil topshiriqlar

- 1 `faktorial(n)` funksiyasini yozing: $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$.
- 2 `tubmi(n)` — son tub ekanini tekshiruvchi funksiya (faqat 1 ga va o'ziga bo'linadi). Uning yordamida 1–100 oralig'idagi barcha tub sonlarni chiqaring.
- 3 **FizzBuzz** (dasturchilar suhbatida mashhur savol): 1 dan 30 gacha sonlarni chiqaring, lekin 3 ga bo'linsa "Fizz", 5 ga — "Buzz", ikkalasiga — "FizzBuzz".

Ko'p uchraydigan xatolar

- ❌ Funksiyani e'lon qilib, chaqirishni unutish (yoki `salomlash` deb, qavssiz yozish).
- ❌ `return` o'rniga `console.log` — natijani boshqa hisoblarda ishlatib bo'lmaydi (`undefined`).
- ❌ Sikl shartida xato: `for (let i = 0; i ≤ 10; i--)` — i kamayib boradi, shart doim rost → cheksiz sikl.
- ❌ `for (let i = 1; i < 10; i++)` 10 ni ham chiqaradi deb o'ylash — `<` va `≤` farqiga e'tibor bering.

O'zingizni tekshiring

? `for (let i = 0; i < 5; i++)` sikli necha marta bajariladi?

- A. 4
- B. 6
- C. 5
- D. Cheksiz

? Siklni darhol to'xtatuvchi kalit so'z?

- A. `break`
- B. `continue`
- C. `stop`
- D. `return`

? Kamida bir marta bajarilishi kafolatlangan sikl qaysi?

- A. `for`
- B. `while`
- C. `if`
- D. `do...while`

? `const f = x  $\Rightarrow$  x * 3; console.log(f(4));` natijasi?

- A. 7
- B. 12
- C. 43
- D. undefined

? Funksiya natijasini chaqirgan joyga qaytaruvchi kalit so'z?

- A. `console.log`
- B. `break`
- C. `return`
- D. `output`

? Quyidagi kod nima chiqaradi? `let s = 0; for (let i = 1; i ≤ 4; i++) s += i; console.log(s);`

- A. 4
- B. 10
- C. 24
- D. 1234

Uyga vazifa

1. Kalkulyatorga "tarix" qo'shing: barcha hisob-kitoblar bir satrga yig'ilib, oxirida ko'rsatilsin.
2. "Sonni top" o'yiniga qiyinlik darajasini qo'shing: oson (1–20, 5 urinish), o'rta (1–100, 7), qiyin (1–1000, 10).
3. `raqamlarYigindisi(n)` funksiyasini yozing: $1234 \rightarrow 1+2+3+4 = 10$. (Maslahat: `% 10` va `Math.floor(n / 10)`.)

Yodda tuting

- `for (let i = 1; i ≤ n; i++)` — necha marta takrorlash ma'lum bo'lsa; `while (shart)` — noma'lum bo'lsa; `do...while` — kamida bir marta.
- `break` — to'xtatish, `continue` — keyingi qadamga o'tish. Cheksiz sikldan ehtiyot bo'ling.
- Funksiya: `function nom(param) { return natija; }` yoki `const nom = (param) ⇒ natija;`
- `return` natija qaytaradi va funktsiyani tugatadi. Funksiya ichidagi o'zgaruvchilar — lokal.

✓ Test javoblari

1. `for (let i = 0; i < 5; i++)` sikli necha marta bajariladi? — **C) 5**
`i = 0, 1, 2, 3, 4 → 5 marta.`
2. Siklni darhol to'xtatuvchi kalit so'z? — **A) break**
3. Kamida bir marta bajarilishi kafolatlangan sikl qaysi? — **D) do...while**
4. `const f = x ⇒ x * 3; console.log(f(4));` natijasi? — **B) 12**
5. Funksiya natijasini chaqirgan joyga qaytaruvchi kalit so'z? — **C) return**
6. Quyidagi kod nima chiqaradi? `let s = 0; for (let i = 1; i <= 4; i++) s += i; console.log(s);` — **B) 10**
`1 + 2 + 3 + 4 = 10.`



Arraylar (massivlar) va obyektlar bilan ishlash

Massiv yaratish, indekslar, length, push/pop/shift/unshift, includes/indexOf, for...of; forEach, map, filter, find, reduce; obyektlar, xossalar, metodlar, obyektlar massivi; o'quvchilar bahosini hisoblovchi dastur.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- **Massiv (array)** — ko'p qiymatni bitta o'zgaruvchida saqlashni;
- Massivga qo'shish, o'chirish, qidirish va aylanib chiqishni;
- Kuchli metodlar: `map`, `filter`, `find`, `reduce`;
- **Obyekt** — bitta narsaning bir nechta xususiyatini saqlashni;
- Obyektlar massivi bilan real ma'lumotlarni qayta ishlashni (14-loyiha).

Massiv nima?

Sinfdagi 25 o'quvchi ismini saqlash uchun 25 ta o'zgaruvchi yaratish (`ism1`, `ism2` ...) noqulay. **Massiv** — qiymatlarning **tartiblangan ro'yxati**, bitta o'zgaruvchi ichida.



O'xshatish: poyezd vagonlari

Massiv — poyezd. Har bir vagonda bitta qiymat. Vagonlar raqamlangan, lekin raqamlash **0 dan** boshlanadi: birinchi vagon — 0-vagon. Bu raqam **indeks** deyiladi. Vagonlar soni — `length`.

JavaScript

```
const mevalar = ["olma", "nok", "uzum", "anor"];
//           0       1       2       3       ← indekslar

console.log(mevalar);
console.log(mevalar[0]); // olma - birinchi element
console.log(mevalar[2]); // uzum
console.log(mevalar.length); // 4 - elementlar soni
console.log(mevalar[mevalar.length - 1]); // anor - OXIRGI element
console.log(mevalar[10]); // undefined - bunday element yo'q

mevalar[1] = "shaftoli"; // elementni o'zgartirish
console.log(mevalar);

const aralash = [14, "Ali", true, null]; // turli turlar ham bo'lishi mumkin
const bosh = []; // bo'sh massiv
```

Massiv metodlari

Qo'shish va o'chirish

JavaScript

```
const navbat = ["Ali", "Vali"];

navbat.push("Gani");           // OXIRIGA qo'shish      → ["Ali", "Vali", "Gani"]
navbat.unshift("Sobir");      // BOSHIGA qo'shish      → ["Sobir", "Ali", "Vali", "Gani"]
console.log(navbat);

const oxirgi = navbat.pop();   // OXIRIDAN olish      → "Gani"
const birinchi = navbat.shift(); // BOSHIDAN olish  → "Sobir"
console.log(oxirgi, birinchi, navbat); // Gani Sobir ["Ali", "Vali"]

// splice – o'rtadan o'chirish: (qaysi indeksdan, nechta)
const sonlar = [10, 20, 30, 40, 50];
sonlar.splice(1, 2);          // 1-indeksdan 2 ta element o'chdi
console.log(sonlar);          // [10, 40, 50]
```

Metod	Nima qiladi	Eslab qolish
push(x)	Oxiriga qo'shadi	Navbat oxiriga turish
pop()	Oxirgisini olib tashlaydi va qaytaradi	Oxirgisi chiqib ketdi
unshift(x)	Boshiga qo'shadi	Navbatsiz oldinga o'tdi
shift()	Birinchisini olib tashlaydi	Navbatdagi birinchi kirdi
splice(i, n)	i-indeksdan n tasini o'chiradi	Kesib olish

Qidirish va boshqa foydali metodlar

JavaScript

```
const fanlar = ["matematika", "fizika", "informatika", "tarix"];

console.log(fanlar.includes("fizika"));           // true  – bormi?
console.log(fanlar.indexOf("informatika"));       // 2    – qaysi indeksda?
console.log(fanlar.indexOf("kimyo"));             // -1    – topilmadi
console.log(fanlar.join(", "));                   // matnga: "matematika, fizika,
... "
console.log("a,b,c".split(","));                  // matndan massivga: ["a","b","c"]
console.log(fanlar.slice(1, 3));                   // nusxa: ["fizika",
"informatika"]

const sonlar = [5, 1, 10, 3];
sonlar.sort((a, b) => a - b);                       // o'sish tartibida saralash
console.log(sonlar);                                // [1, 3, 5, 10]
console.log(fanlar.sort());                          // matnlarni alifbo bo'yicha
```



Sonlarni `sort()` bilan qavssiz saralasangiz, ular **matn sifatida** saralanadi: `[5, 1, 10, 3].sort()` → `[1, 10, 3, 5]` 🤖. Sonlar uchun doim `sort((a, b) => a - b)` yozing (kamayish tartibi uchun `b - a`).

Massivni aylanib chiqish

JavaScript

```
const ballar = [85, 92, 78, 95, 60];

// 1. Oddiy for – indeks kerak bo'lsa
for (let i = 0; i < ballar.length; i++) {
  console.log(`${i + 1}-o'quvchi: ${ballar[i]} ball`);
}

// 2. for...of – eng sodda va o'qilishi oson
for (const ball of ballar) {
  console.log(ball);
}

// 3. forEach – har bir element uchun funksiya
ballar.forEach((ball, index) => {
  console.log(`#${index}: ${ball}`);
});
```

Amaliy masala: eng katta, o'rtacha, yig'indi

JavaScript

```
const ballar = [85, 92, 78, 95, 60];

let yigindi = 0;
let eng = ballar[0];
for (const b of ballar) {
  yigindi += b;
  if (b > eng) eng = b;
}
const ortacha = yigindi / ballar.length;

console.log(`Yig'indi: ${yigindi}`);
console.log(`O'rtacha: ${ortacha.toFixed(1)}); // toFixed(1) – 1 xona kasr
console.log(`Eng yuqori: ${eng}`);
console.log(`Math bilan: ${Math.max(...ballar)}); // ... – massivni "yoyish"
```

Kuchli massiv metodlari

Bu metodlar professional JavaScript'ning yuragi. Ular asl massivni o'zgartirmaydi — **yangi** natija qaytaradi.

Metod	Savol	Natija
<code>map</code>	"Har birini qanday o'zgartiray?"	Xuddi shuncha elementli yangi massiv
<code>filter</code>	"Qaysilari shartga mos?"	Faqat mos kelganlardan iborat massiv
<code>find</code>	"Shartga mos birinchisi qaysi?"	Bitta element (yoki <code>undefined</code>)
<code>some</code> / <code>every</code>	"Kamida bittasi / hammasi mosmi?"	<code>true</code> / <code>false</code>
<code>reduce</code>	"Hammasini bitta qiymatga qanday yig'ay?"	Bitta qiymat (yig'indi, ko'paytma...)



O'xshatish: mevalar savati

map — har bir mevani to'g'rash (5 meva → 5 to'g'ralgan meva). **filter** — faqat pishganlarini tanlab olish. **find** — birinchi uchragan olmani olish. **reduce** — hammasidan bitta sharbat tayyorlash.


```
const narxlar = [12000, 45000, 8000, 30000];

const chegirmali = narxlar.map(n => n * 0.9);           // har biriga 10%
chegirma
console.log(chegirmali);                               // [10800, 40500, 7200,
27000]

const arzonlar = narxlar.filter(n => n < 20000);        // 20 000 dan arzonlari
console.log(arzonlar);                                 // [12000, 8000]

const birinchiQiimmat = narxlar.find(n => n > 25000);   // 45000
console.log(birinchiQiimmat);

console.log(narxlar.some(n => n > 40000));              // true – kamida bittasi
console.log(narxlar.every(n => n > 5000));              // true – hammasi

const jami = narxlar.reduce((yigindi, n) => yigindi + n, 0); // 0 –
boshlang'ich qiymat
console.log(jami);                                     // 95000
```

Obyektlar

Massiv — ko'p **bir xil** narsalar ro'yxati. **Obyekt** esa **bitta narsaning** turli xususiyatlarini nomlari bilan saqlaydi.



O'xshatish: o'quvchi anketasi

Anketada: "Ism: Dilnoza", "Yosh: 14", "Sinf: 8-A", "To'garak: shaxmat". Har bir qator — **kalit: qiymat** jufti. Obyekt — aynan shunday anketa.

```
const oquvchi = {
  ism: "Dilnoza",          // kalit (xossa nomi): qiymat
  yosh: 14,
  sinf: "8-A",
  alochi: true,
  fanlar: ["matematika", "ingliz tili"], // qiymat massiv ham bo'lishi
  mumkin
};

// Xossani o'qish – ikki usul
console.log(oquvchi.izm);          // nuqta orqali – ko'p ishlatiladi
console.log(oquvchi["sinf"]);      // kvadrat qavs orqali

// O'zgartirish, qo'shish, o'chirish
oquvchi.yosh = 15;
oquvchi.shahar = "Namangan";      // yangi xossa
delete oquvchi.alochi;
console.log(oquvchi);

// Barcha kalitlar va qiymatlar
console.log(Object.keys(oquvchi));
console.log(Object.values(oquvchi));
for (const kalit in oquvchi) {
  console.log(`${kalit}: ${oquvchi[kalit]}`);
}
```

Metodlar — obyekt ichidagi funksiyalar

JavaScript

```
const hisob = {
  egasi: "Jasur",
  balans: 50000,
  toldirish(summa) {
    this.balans += summa;          // this — "shu obyektning o'zi"
    console.log(`+${summa}. Balans: ${this.balans}`);
  },
  yechish(summa) {
    if (summa > this.balans) {
      console.log("✗ Mablag' yetarli emas");
      return;
    }
    this.balans -= summa;
    console.log(`-${summa}. Balans: ${this.balans}`);
  },
};

hisob.toldirish(20000);
hisob.yechish(100000);
hisob.yechish(30000);
```

Obyektlar massivi — real ma'lumotlar

Haqiqiy dasturlarda ma'lumotlar deyarli doim shunday ko'rinishda bo'ladi: mahsulotlar ro'yxati, o'quvchilar ro'yxati, xabarlar ro'yxati. Serverdan keladigan ma'lumot ham (JSON, 39-mavzu) aynan shunday tuzilgan.

```
const mahsulotlar = [  
  { nomi: "Daftar", narx: 5000, soni: 120, toifa: "kanselyariya" },  
  { nomi: "Ruchka", narx: 3000, soni: 0, toifa: "kanselyariya" },  
  { nomi: "Ryukzak", narx: 180000, soni: 7, toifa: "sumka" },  
  { nomi: "Kalkulyator", narx: 95000, soni: 15, toifa: "texnika" },  
];  
  
// Omborda bor mahsulotlar nomlari  
const mavjud = mahsulotlar.filter(m => m.soni > 0).map(m => m.nomi);  
console.log("Mavjud:", mavjud);  
  
// Nomi bo'yicha qidirish  
const topilgan = mahsulotlar.find(m => m.nomi === "Ryukzak");  
console.log(topilgan.narx);  
  
// Ombordagi umumiy qiymat  
const omborQiymati = mahsulotlar.reduce((jami, m) => jami + m.narx * m.soni,  
0);  
console.log(`Ombor qiymati: ${omborQiymati.toLocaleString()} so'm`);  
  
// Narx bo'yicha saralash (qimmatdan arzonaga)  
const saralangan = [...mahsulotlar].sort((a, b) => b.narx - a.narx);  
saralangan.forEach(m => console.log(`${m.nomi}: ${m.narx} so'm`));
```

Amaliy mashg'ulot: o'quvchilar bahosini hisoblovchi

dastur (14-loyiha)

JavaScript

```
const oquvchilar = [
  { ism: "Akmal", ballar: [92, 85, 78] },
  { ism: "Barno", ballar: [67, 88, 95] },
  { ism: "Davron", ballar: [45, 52, 60] },
  { ism: "Gulnoza", ballar: [98, 94, 91] },
  { ism: "Erkin", ballar: [73, 39, 70] },
];

// 1. Yordamchi funksiyalar
const ortacha = (sonlar) => sonlar.reduce((a, b) => a + b, 0) / sonlar.length;

function baho(ball) {
  if (ball ≥ 86) return 5;
  if (ball ≥ 71) return 4;
  if (ball ≥ 56) return 3;
  return 2;
}

// 2. Har bir o'quvchi uchun natija
const natijalar = oquvchilar.map(o => {
  const ort = Math.round(ortacha(o.ballar));
  return { ism: o.ism, ortacha: ort, baho: baho(ort) };
});

// 3. Reyting (o'rtacha ball bo'yicha)
natijalar.sort((a, b) => b.ortacha - a.ortacha);

console.log("🏆 REYTING");
natijalar.forEach((n, i) => {
  console.log(`${i + 1}. ${n.ism} - ${n.ortacha} ball, baho: ${n.baho}`);
});

// 4. Statistika
const alochilar = natijalar.filter(n => n.baho === 5).map(n => n.ism);
const sinfOrtachasi = ortacha(natijalar.map(n => n.ortacha));
console.log(`\nA'lochilar: ${alochilar.join(", ") || "yo'q"}`);
console.log(`Sinf o'rtachasi: ${sinfOrtachasi.toFixed(1)}`);
```

```
console.log(`Qoniqarsiz baho olganlar: ${natijalar.filter(n => n.baho >= 2).length} nafar`);
```

Topshiriqlar:

- 1 Har bir o'quvchiga 4-fan bahosi qo'shing va natijani qayta hisoblang.
- 2 Eng past ball olgan o'quvchini `reduce` yoki saralash bilan toping.
- 3 "Yangi o'quvchi qo'shish" funksiyasini yozing: `qosh(ism, ballar)` u `push` qilsin.
- 4 Ismi bo'yicha qidirish funksiyasi: `izla("barno")` — katta-kichik harfga e'tibor bermasin.

Ko'p uchraydigan xatolar

- ✗ Indeks 1 dan boshlanadi deb o'ylash: `arr[1]` — bu **ikkinchi** element!
- ✗ Oxirgi elementga `arr[arr.length]` deb murojaat — to'g'risi `arr[arr.length - 1]`.
- ✗ `map` ichida `return` ni unutish (jingalak qavs bilan yozilganda) — natija `[undefined, undefined...]`.
- ✗ Obyekt xossasi nomidagi xato: `oquvchi.Ism` — `undefined`, chunki kalit `ism`.
- ✗ `reduce` da boshlang'ich qiymatni `(, 0)` unutish.

O'zingizni tekshiring

? `const a = [10, 20, 30];` — `a[1]` nimaga teng?

- A. 10
- B. 20
- C. 30
- D. undefined

? **Massiv oxiriga element qo'shuvchi metod?**

- A. `push()`
- B. `pop()`
- C. `shift()`
- D. `add()`

? **`[1, 2, 3, 4].filter(x ⇒ x % 2 ≡ 0)` natijasi?**

- A. `[1, 3]`
- B. `[2, 4, 6, 8]`
- C. `[2, 4]`
- D. `2`

? **`[1, 2, 3].map(x ⇒ x * 10)` natijasi?**

- A. `60`
- B. `[1, 2, 3]`
- C. `30`
- D. `[10, 20, 30]`

? **`const k = { ism: "Ali", yosh: 14 };` — yoshni o'qishning to'g'ri usuli?**

- A. `k[yosh]`
- B. `k.yosh`
- C. `k(yosh)`
- D. `k→yosh`

? **Massivdagi barcha sonlar yig'indisini bitta qiymatga yig'uvchi metod?**

- A. `map`
- B. `find`
- C. `reduce`
- D. `join`

 Uyga vazifa

1. **Kitoblar javoni:** 6 ta kitobdan iborat obyektlar massivi (nomi, muallif, yil, sahifalar, o'qilganmi). Chiqaring: o'qilmagan kitoblar, eng qalin kitob, 2000-yildan keyin chiqqanlar, umumiy sahifalar soni.
2. **Takrorlarni olib tashlash:** `[1, 2, 2, 3, 1, 4]` → `[1, 2, 3, 4]` (`filter` + `indexOf` yoki `new Set()`).
3. **So'zlar statistikasi:** gapni so'zlarga ajratib (`split(" ")`), eng uzun so'zni va so'zlar sonini toping.

✓ **Yodda tuting**

- Massiv: `[a, b, c]`, indeks 0 dan; `length`, `push/pop/shift/unshift`, `includes`, `indexOf`, `sort((a,b)⇒a-b)`.
- Aylanish: `for`, `for...of`, `forEach`.
- `map` — o'zgartirish, `filter` — tanlash, `find` — birinchisi, `reduce` — yig'ish.
- Obyekt: `{ kalit: qiymat }`; `obj.kalit` yoki `obj["kalit"]`; metodlarda `this`.
- Obyektlar massivi — real dasturlardagi ma'lumotlarning asosiy shakli.

✓ Test javoblari

1. `const a = [10, 20, 30];` — `a[1]` nimaga teng? — **B) 20**
2. Massiv oxiriga element qo'shuvchi metod? — **A) push()**
3. `[1, 2, 3, 4].filter(x ⇒ x % 2 === 0)` natijasi? — **C) [2, 4]**
4. `[1, 2, 3].map(x ⇒ x * 10)` natijasi? — **D) [10, 20, 30]**
5. `const k = { ism: "Ali", yosh: 14 };` — yoshni o'qishning to'g'ri usuli? — **B) k.yosh**
`k["yosh"]` ham to'g'ri — qo'shtirnoq bilan.
6. Massivdagi barcha sonlar yig'indisini bitta qiymatga yig'uvchi metod? — **C) reduce**



DOM va BOM bilan ishlash. JavaScript'da eventlar

DOM daraxti, elementlarni topish (querySelector), matn, atribut, uslub va class'larni o'zgartirish, element yaratish va o'chirish; BOM: window, location, localStorage, setTimeout; hodisalar va addEventListener; valyuta konvertori.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- **DOM** nima va JavaScript sahifadagi elementlarni qanday "ko'radi";
- Elementlarni topish, matnini, rangini, class'ini o'zgartirishni;
- Yangi elementlar yaratish va o'chirishni;
- **BOM**: brauzer oynasi, manzil, xotira (`localStorage`), taymerlar;
- **Hodisalar (events)**: tugma bosilishi, matn kiritilishi va boshqalarga javob berishni;
- Haqiqiy interaktiv dastur — valyuta konvertorini (13-loyiha).

DOM nima?

Brauzer HTML faylni o'qiganda, uni xotirada **daraxt** ko'rinishidagi obyektlarga aylantiradi. Bu daraxt **DOM (Document Object Model)** deyiladi. JavaScript sahifani aynan DOM orqali o'zgartiradi.

HTML

```
<body>
  <h1 id="sarlavha">Salom</h1>
  <ul class="royxat">
    <li>Olma</li>
    <li>Nok</li>
  </ul>
</body>
```

Matn

```
document
├── html
│   └── body
│       ├── h1#sarlavha → "Salom"
│       └── ul.royxat
│           ├── li → "Olma"
│           └── li → "Nok"
```



O'xshatish: oila shajarasi

DOM — oila daraxti. `body` — bobo, `ul` — ota, `li` lar — bolalar (va o'zaro aka-uka). JavaScript shu daraxtdagi istalgan "qarindosh"ni topib, uning kiyimini (uslub), ismini (matn) o'zgartirishi, yangi farzand qo'shishi yoki olib tashlashi mumkin.

`document` — butun sahifani ifodalovchi obyekt, DOM bilan ishlashning kirish nuqtasi.

Elementlarni topish

JavaScript

```
// querySelector – CSS selektori bilan BIRINCHI mos elementni topadi
const sarlavha = document.querySelector("#sarlavha"); // id bo'yicha
const royxat    = document.querySelector(".royxat");  // class bo'yicha
const birinchiLi = document.querySelector("li");      // teg bo'yicha

// querySelectorAll – BARCHA mos elementlar (ro'yxat)
const hammaLi = document.querySelectorAll("li");
hammaLi.forEach(li ⇒ console.log(li.textContent));

// Eski, lekin ko'p uchraydigan usul
const s = document.getElementById("sarlavha");      // # belgisisiz!
```



CSS'da qanday selektor yozsangiz (`#id` , `.class` , `nav a`), `querySelector` ga ham xuddi shunday yozasiz. 6-mavzudagi bilimlar shu yerda ham ishlaydi!

Elementlarni o'zgartirish

HTML

```
<h1 id="sarlavha">Eski sarlavha</h1>
<p class="matn">Oddiy paragraf</p>

<a id="havola" href="#">Havola</a>

<script>
  const h1 = document.querySelector("#sarlavha");
  h1.textContent = "Yangi sarlavha ✨";           // matnni almashtirish

  const p = document.querySelector(".matn");
  p.innerHTML = "Endi <b>qalin</b> so'z bor";    // HTML bilan almashtirish

  // Uslub (inline CSS) — xossa nomlari camelCase: background-color →
  backgroundColor
  h1.style.color = "white";
  h1.style.backgroundColor = "#4f46e5";
  h1.style.padding = "10px";
  h1.style.borderRadius = "8px";

  // Atributlar
  document.querySelector("#rasm").src =
"https://picsum.photos/seed/ikki/200/120";
  const a = document.querySelector("#havola");
  a.href = "https://uz.wikipedia.org";
  a.textContent = "Vikipediya";
  a.setAttribute("target", "_blank");
</script>
```



textContent yoki innerHTML?

Foydalanuvchi kiritgan matnni (ism, izoh) **doim** `textContent` bilan qo'ying. `innerHTML` esa matn ichidagi HTML va skriptlarni ham bajaradi — yomon niyatli odam izoh sifatida zararli kod yozishi mumkin (bu **XSS hujumi** deyiladi).

Class'lar bilan ishlash — eng to'g'ri usul

Uslubni `style` orqali birma-bir o'zgartirgandan ko'ra, CSS'da tayyor class yozib, JavaScript bilan uni qo'shish/olib tashlash yaxshiroq:

HTML

```
<style>
  .quti { padding: 20px; border-radius: 10px; background: #e5e7eb; transition:
.3s; font-family: sans-serif; }
  .faol { background: #22c55e; color: white; transform: scale(1.05); }
  .yashirin { display: none; }
</style>
<div class="quti" id="q">Men qutiman</div>
<p id="sir" class="yashirin">🙄 Sir matn topildi!</p>

<script>
  const q = document.querySelector("#q");
  q.classList.add("faol");           // class qo'shish
  // q.classList.remove("faol");      // olib tashlash
  // q.classList.toggle("faol");      // bor bo'lsa olib tashlaydi, yo'q
bo'lsa qo'shadi
  console.log(q.classList.contains("faol")); // true — bormi?

  document.querySelector("#sir").classList.remove("yashirin");
</script>
```

Element yaratish va o'chirish

HTML

```
<ul id="royxat"><li>Non</li></ul>

<script>
  const royxat = document.querySelector("#royxat");
  const mahsulotlar = ["Sut", "Tuxum", "Olma"];

  mahsulotlar.forEach(nom => {
    const li = document.createElement("li"); // 1. yaratish (hali sahifada
yo'q)
    li.textContent = nom; // 2. to'ldirish
    royxat.append(li); // 3. sahifaga qo'shish
  });

  royxat.firstChild.remove(); // birinchi elementni
o'chirish
</script>
```

BOM — brauzer bilan ishlash

BOM (Browser Object Model) — sahifaning o'zi emas, **brauzer oynasi** bilan ishlash vositalari. Ularning "boshlig'i" — `window` obyekti.

JavaScript

```
console.log(window.innerWidth, window.innerHeight); // oyna o'lchami
console.log(location.href); // joriy sahifa manzili
// location.href = "https://kun.uz"; // boshqa sahifaga o'tish
// location.reload(); // sahifani yangilash
console.log(navigator.language); // brauzer tili, masalan "uz" yoki "ru"
// history.back(); // "Orqaga" tugmasi

// Taymerlar
setTimeout(() => {
  console.log("3 soniya o'tdi 🕒");
}, 3000); // bir marta, 3000 ms dan keyin

let soniya = 0;
const taymer = setInterval(() => {
  soniya++;
  console.log(`${soniya} soniya`);
  if (soniya === 5) clearInterval(taymer); // to'xtatish
}, 1000); // har 1000 ms da
```

localStorage — brauzer xotirasi

Sahifa yangilansa, JavaScript o'zgaruvchilari "unutiladi". `localStorage` esa ma'lumotni brauzerda **doimiy** saqlaydi (brauzer yopilsa ham).

JavaScript

```
localStorage.setItem("ism", "Malika"); // saqlash (faqat matn!)
console.log(localStorage.getItem("ism")); // o'qish → "Malika"
localStorage.removeItem("ism"); // o'chirish

// Massiv yoki obyektini saqlash uchun – JSON ga aylantirish
const vazifalar = ["Dars qilish", "Kitob o'qish"];
localStorage.setItem("vazifalar", JSON.stringify(vazifalar));
const qaytaOlingan = JSON.parse(localStorage.getItem("vazifalar"));
console.log(qaytaOlingan[0]);
```


Hodisalar (events)

Hodisa — sahifada sodir bo'ladigan narsa: tugma bosildi, klavish bosildi, sichqoncha ustiga keldi, forma yuborildi. JavaScript hodisani **"tinglab"** turadi va sodir bo'lganda belgilangan funksiyani bajaradi.



O'xshatish: eshik qo'ng'irog'i

Siz eshik oldida doim kutib turmaysiz. Qo'ng'iroq **chalinganda** (hodisa) eshikni ochishga borasiz (funksiya). `addEventListener` — "qo'ng'iroq chalinsa, menga ayt" degan kelishuv.

HTML

```
<button id="tugma">Meni bosing!</button>
<p id="natija">Hali bosilmadi</p>

<script>
  const tugma = document.querySelector("#tugma");
  const natija = document.querySelector("#natija");
  let soni = 0;

  tugma.addEventListener("click", () => {
    soni++;
    natija.textContent = `Siz ${soni} marta bosdingiz 🎉`;
  });
</script>
```

JavaScript

```
element.addEventListener("hodisa_turi", funksiya);
```

Hodisa	Qachon sodir bo'ladi
<code>click</code>	Element bosilganda
<code>dblclick</code>	Ikki marta bosilganda
<code>mouseenter</code> / <code>mouseleave</code>	Sichqoncha ustiga kelganda / ketganda
<code>input</code>	Maydonga har bir belgi kiritilganda

change	Qiymat o'zgarib, maydondan chiqilganda (select, checkbox)
submit	Forma yuborilganda
keydown	Klaviaturada tugma bosilganda
load	Sahifa to'liq yuklanganda
scroll / resize	Sahifa aylantirilganda / oyna o'lchami o'zgarganda

Hodisa obyкти (event)

HTML

```
<input id="matn" placeholder="Bu yerga yozing...">
<p>Belgilar soni: <b id="soni">0</b></p>
<p id="oxirgi"></p>

<script>
  const input = document.querySelector("#matn");

  input.addEventListener("input", (event) => {
    // event.target - hodisa sodir bo'lgan element
    document.querySelector("#soni").textContent = event.target.value.length;
  });

  document.addEventListener("keydown", (e) => {
    document.querySelector("#oxirgi").textContent = `Oxirgi bosilgan tugma:
    ${e.key}`;
    if (e.key === "Escape") input.value = "";
  });
</script>
```

Forma yuborilishini ushlash

HTML

```
<form id="forma">
  <input name="ism" placeholder="Ismingiz" required>
  <button>Yuborish</button>
</form>
<p id="javob"></p>

<script>
  document.querySelector("#forma").addEventListener("submit", (e) => {
    e.preventDefault(); // ← sahifa qayta yuklanishini
    TO'XTATADI
    const ism = e.target.ism.value.trim();
    document.querySelector("#javob").textContent = `Rahmat, ${ism}! Ma'lumot
qabul qilindi.`;
    e.target.reset(); // formani tozalash
  });
</script>
```



`e.preventDefault()` — brauzerning odatiy harakatini bekor qiladi. Forma uchun bu sahifa qayta yuklanishining oldini oladi va ma'lumotni JavaScript bilan o'zimiz qayta ishlaymiz.

Amaliy mashg'ulot: valyuta konvertori (13-loyiha)

konvertor.html

```
<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Valyuta konvertori</title>
<style>
  * { box-sizing: border-box; }
  body { font-family: "Segoe UI", sans-serif; background: linear-
gradient(135deg, #e0e7ff, #fce7f3);
    min-height: 100vh; display: grid; place-items: center; margin: 0; }
  .karta { background: white; padding: 28px; border-radius: 20px; width: 340px;
    box-shadow: 0 20px 40px rgba(79,70,229,.15); }
  h1 { margin: 0 0 18px; font-size: 1.4rem; text-align: center; }
  label { font-size: 14px; color: #6b7280; }
  input, select { width: 100%; padding: 12px; margin: 6px 0 14px; font-size:
16px;
    border: 2px solid #e5e7eb; border-radius: 10px; }
  input:focus, select:focus { border-color: #4f46e5; outline: none; }
  .qator { display: flex; gap: 10px; align-items: center; }
  .qator select { flex: 1; }
  #almashtir { border: none; background: #eef2ff; border-radius: 50%; width:
42px; height: 42px;
    font-size: 18px; cursor: pointer; transition: transform .3s; }
  #almashtir:hover { transform: rotate(180deg); }
  .natija { margin-top: 6px; padding: 16px; background: #4f46e5; color: white;
border-radius: 12px;
    text-align: center; font-size: 1.3rem; font-weight: bold; }
  .kurs { text-align: center; color: #6b7280; font-size: 13px; margin-top: 8px;
}
</style>
</head>
<body>
  <div class="karta">
    <h1>💰 Valyuta konvertori</h1>
    <label for="miqdor">Miqdor</label>
    <input id="miqdor" type="number" value="100" min="0">
    <div class="qator">
      <select id="dan"></select>
```

```

    <button id="almashtir" title="Almashtirish">↔</button>
    <select id="ga"></select>
  </div>
  <div class="natija" id="natija">--</div>
  <div class="kurs" id="kurs"></div>
</div>

<script>
  // 1 birlik valyuta necha so'm (namuna kurslar)
  const KURSLAR = {
    UZS: { nomi: "🇺🇿 So'm", kurs: 1 },
    USD: { nomi: "🇺🇸 Dollar", kurs: 12700 },
    EUR: { nomi: "🇪🇺 Yevro", kurs: 13900 },
    RUB: { nomi: "🇷🇺 Rubl", kurs: 155 },
    KZT: { nomi: "🇰🇿 Tenge", kurs: 25 },
  };

  const miqdorEl = document.querySelector("#miqdor");
  const danEl = document.querySelector("#dan");
  const gaEl = document.querySelector("#ga");
  const natijaEl = document.querySelector("#natija");
  const kursEl = document.querySelector("#kurs");

  // Select'larni JavaScript bilan to'ldiramiz
  for (const kod in KURSLAR) {
    danEl.append(new Option(`${KURSLAR[kod].nomi} (${kod})`, kod));
    gaEl.append(new Option(`${KURSLAR[kod].nomi} (${kod})`, kod));
  }

  // Oxirgi tanlovni eslab qolish (localStorage)
  danEl.value = localStorage.getItem("dan") || "USD";
  gaEl.value = localStorage.getItem("ga") || "UZS";

  function hisobla() {
    const miqdor = Number(miqdorEl.value);
    const dan = danEl.value;
    const ga = gaEl.value;

    if (miqdor < 0 || miqdorEl.value === "") {
      natijaEl.textContent = "Miqdorni kiriting";
      return;
    }

    const somda = miqdor * KURSLAR[dan].kurs; // avval so'mga

```

```

    const natija = somda / KURSLAR[ga].kurs; // keyin kerakli
    valyutaga

    natijaEl.textContent = `${natija.toLocaleString("uz-UZ", {
    maximumFractionDigits: 2 })} ${ga}`;
    kursEl.textContent = `1 ${dan} = ${(KURSLAR[dan].kurs /
    KURSLAR[ga].kurs).toLocaleString("uz-UZ", { maximumFractionDigits: 4 })}
    ${ga}`;

    localStorage.setItem("dan", dan);
    localStorage.setItem("ga", ga);
  }

  miqdorEl.addEventListener("input", hisobla);
  danEl.addEventListener("change", hisobla);
  gaEl.addEventListener("change", hisobla);

  document.querySelector("#almashtir").addEventListener("click", () => {
    [danEl.value, gaEl.value] = [gaEl.value, danEl.value]; // qiymatlarni
    almashtirish
    hisobla();
  });

  hisobla(); // sahifa ochilganda birinchi hisob
</script>
</body>
</html>

```

Kodni tahlil qiling va kengaytiring:

- 1 Har bir qatorga izoh yozing: bu qator nima qiladi?
- 2 Yana 2 ta valyuta qo'shing (masalan, GBP, CNY).
- 3 "Tarix" bo'limi qo'shing: oxirgi 5 ta hisob ro'yxat sifatida natija ostida ko'rsatilsin (`createElement` + `prepend`).
- 4 Miqdor 1 000 000 dan katta bo'lsa, natija qizil rangda ko'rinsin (`classList`).

Qo'shimcha mashqlar

- **Tungi rejim tugmasi:** bosilganda `body` ga `dark` class qo'shilsin/olinsin, tanlov `localStorage` da saqlansin.

- **Burger menyu:** 10-mavzudagi menyuni endi JavaScript bilan oching: ☰ bosilganda `.menu` ga `ochiq` class.
- **Sekundomer:** Start/Stop/Reset tugmalari va `setInterval`.

Ko'p uchraydigan xatolar

- ❌ `Cannot read properties of null` — `querySelector` elementni topmadi: selektorda xato (`#` yoki `.` unutilgan) yoki skript HTML'dan **oldin** ulangan.
- ❌ `addEventListener("click", hisobla())` — qavs qo'yilsa funksiya darhol bajariladi. To'g'risi: `hisobla` (qavssiz).
- ❌ `input.value` ni son deb o'ylash — u doim matn. `Number()` !
- ❌ Formada `e.preventDefault()` unutilgan — sahifa yangilanib, natija bir lahzada yo'qoladi.
- ❌ `style.background-color` — JS'da defisli nomlar camelCase bo'ladi: `style.backgroundColor`.

O'zingizni tekshiring

? `class="karta"` bo'lgan birinchi elementni topuvchi kod?

- A. `document.querySelector("karta")`
- B. `document.querySelector(".karta")`
- C. `document.getElementById(".karta")`
- D. `document.find("karta")`

? Element matnini xavfsiz o'zgartiruvchi xossa?

- A. `innerHTML`
- B. `value`
- C. `src`
- D. `textContent`

? Class bor bo'lsa olib tashlaydigan, yo'q bo'lsa qo'shadigan metod?

- A. `classList.toggle()`
- B. `classList.add()`
- C. `classList.switch()`
- D. `className.change()`

? Maydonga har bir belgi kiritilganda sodir bo'ladigan hodisa?

- A. `click`
- B. `submit`
- C. `input`
- D. `load`

? Ma'lumotni brauzer yopilsa ham saqlab qoluvchi BOM vositasi?

- A. `setTimeout`
- B. `localStorage`
- C. `console`
- D. `location`

? Forma yuborilganda sahifa qayta yuklanmasligi uchun nima chaqiriladi?

- A. `e.stop()`
- B. `return false`
- C. `form.reset()`
- D. `e.preventDefault()`



Uyga vazifa

1. Valyuta konvertorini yakunlab, GitHub'ga joylang va Netlify'da e'lon qiling.
2. **Rang tanlagich:** 3 ta `input type="range"` (R, G, B). Ular o'zgarganda katta quti rangi o'zgarsin va HEX kodi ko'rsatilsin.
3. **Rasm galereyasi:** kichik rasmlar ustiga bosilganda katta rasm almashsin.



Yodda tuting

- DOM — sahifaning obyektlar daraxti; `document.querySelector()` / `querySelectorAll()` bilan topiladi.
- O'zgartirish: `textContent`, `innerHTML`, `style.xossa`, `classList.add/remove/toggle`, atributlar.
- Yaratish: `createElement` → `append`; o'chirish: `remove()`.
- BOM: `window`, `location`, `setTimeout/setInterval`, `localStorage` (JSON bilan).
- Hodisalar: `e1.addEventListener("click", funksiya)`; formada `e.preventDefault()`.

Test javoblari

1. class="karta" bo'lgan birinchi elementni topuvchi kod? — **B) document.querySelector(".karta")**
2. Element matnini xavfsiz o'zgartiruvchi xossa? — **D) textContent**
3. Class bor bo'lsa olib tashlaydigan, yo'q bo'lsa qo'shadigan metod? — **A) classList.toggle()**
4. Maydonga har bir belgi kiritilganda sodir bo'ladigan hodisa? — **C) input**
5. Ma'lumotni brauzer yopilsa ham saqlab qoluvchi BOM vositasi? — **B) localStorage**
6. Forma yuborilganda sahifa qayta yuklanmasligi uchun nima chaqiriladi? — **D) e.preventDefault()**



20-MAVZU · 42 TADAN · II BOB. JAVASCRIPT ASOSLARI

HTML eventlari va sahifaga interaktivlik. JavaScript mini loyiha

Loyihani rejalashtirish, "holat → chizish" usuli, forma tekshirish; To-Do List (localStorage bilan), mini test tizimi va interaktiv komponentlar — bosqichma-bosqich.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Loyihani boshlashdan oldin **rejalashtirishni**;
- Professional usul: ma'lumotni **massivda** saqlab, sahifani undan **chizish** (render);
- Formani JavaScript bilan tekshirish (validatsiya);
- To'liq **To-Do List** ilovasini (11-loyiha);
- **Mini test tizimini** (12-loyiha) yaratishni.

Bu dars — II bobning yakuni: o'zgaruvchilar, shartlar, sikllar, funksiyalar, massivlar, DOM va hodisalar bitta loyihada birlashadi.

Loyihani qanday boshlash kerak?

Yangi boshlovchilarning eng katta xatosi — darhol kod yozishga tushish. Tajribali dasturchi avval o'ylaydi:

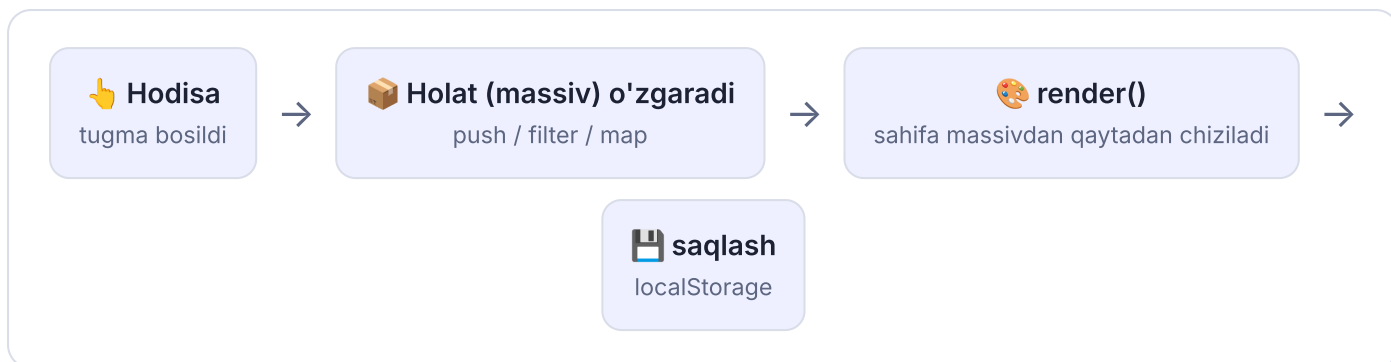
- 1 **Nima qilishi kerak?** Funksiyalar ro'yxatini yozing: "vazifa qo'shish", "bajarildi deb belgilash", "o'chirish"...
- 2 **Qanday ko'rinadi?** Qog'ozga chizib oling: input, tugma, ro'yxat.
- 3 **Qanday ma'lumot saqlanadi?** Har bir vazifa — obyekt: `{ id, matn, bajarildi }`; hammasi — massiv.

4 **Qaysi hodisalar bor?** Forma yuborildi → qo'shish; checkbox bosildi → belgilash; 🗑️ bosildi → o'chirish.

5 **Kichik qadamlar bilan yozing** va har qadamdan keyin tekshiring.

"Holat → Chizish" usuli

Ro'yxatga element qo'shganda ham, o'chirganda ham DOM'ni qo'lda "yamash" chalkashlikka olib keladi. Professional usul:



O'xshatish: doska va daftar

Daftar — **holat** (haqiqiy ma'lumot). Doska — **ekran**. O'qituvchi doskaga to'g'ridan-to'g'ri yozib-o'chirib o'tirmaydi: avval daftarni yangilaydi, keyin doskani artib, daftardagini qaytadan yozadi. Shunda doska va daftar hech qachon farq qilmaydi.

Aynan shu g'oya React kabi zamonaviy kutubxonalarning asosida yotadi.

1-loyiha: To-Do List (11-loyiha)

1-qadam: HTML va CSS

todo/index.html

```
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vazifalarim</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="ilova">
    <h1>📝 Vazifalarim</h1>

    <form id="forma">
      <input id="kiritish" placeholder="Yangi vazifa..." maxlength="80"
autocomplete="off">
      <button>Qo'shish</button>
    </form>
    <p id="xato" class="xato"></p>

    <div class="filtrlar">
      <button data-filtr="hammasi" class="faol">Hammasi</button>
      <button data-filtr="faol">Faol</button>
      <button data-filtr="bajarilgan">Bajarilgan</button>
    </div>

    <ul id="royxat"></ul>

    <footer class="pastki">
      <span id="hisob"></span>
      <button id="tozalash">Bajarilganlarni o'chirish</button>
    </footer>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

2-qadam: JavaScript — bosqichma-bosqich

todo/app.js

```
// ===== 1. HOLAT (ma'lumotlar) =====
let vazifalar = JSON.parse(localStorage.getItem("vazifalar")) || [];
let filtr = "hammasi";

// ===== 2. ELEMENTLAR =====
const forma = document.querySelector("#forma");
const kiritish = document.querySelector("#kiritish");
const royxat = document.querySelector("#royxat");
const xato = document.querySelector("#xato");
const hisob = document.querySelector("#hisob");

// ===== 3. AMALLAR (holatni o'zgartiradi) =====
function qoshish(matn) {
  vazifalar.push({ id: Date.now(), matn: matn, bajarildi: false });
  yangilash();
}

function belgilash(id) {
  vazifalar = vazifalar.map(v => v.id === id ? { ...v, bajarildi: !v.bajarildi } : v);
  yangilash();
}

function ochirish(id) {
  vazifalar = vazifalar.filter(v => v.id !== id);
  yangilash();
}

function yangilash() {
  localStorage.setItem("vazifalar", JSON.stringify(vazifalar)); // saqlash
  render(); // chizish
}

// ===== 4. CHIZISH (holatdan sahifa) =====
function render() {
  royxat.innerHTML = ""; // eski ro'yxatni tozalash

  const korinadigan = vazifalar.filter(v => {
    if (filtr === "faol") return !v.bajarildi;
```

```

    if (filtr === "bajarilgan") return v.bajarildi;
    return true;
  });

  if (korinadigan.length === 0) {
    royxat.innerHTML = '<li class="bosh">Bu yerda hozircha hech narsa yo\'q 🤖';
  }

  korinadigan.forEach(v => {
    const li = document.createElement("li");
    if (v.bajarildi) li.classList.add("bajarildi");

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = v.bajarildi;
    checkbox.addEventListener("change", () => belgilash(v.id));

    const span = document.createElement("span");
    span.textContent = v.matn; // ✅ textContent – xavfsiz

    const ochirishTugma = document.createElement("button");
    ochirishTugma.textContent = "🗑️";
    ochirishTugma.title = "O'chirish";
    ochirishTugma.addEventListener("click", () => ochirish(v.id));

    li.append(checkbox, span, ochirishTugma);
    royxat.append(li);
  });

  const qolgan = vazifalar.filter(v => !v.bajarildi).length;
  hisob.textContent = `${qolgan} ta vazifa qoldi`;
}

// ===== 5. HODISALAR =====
forma.addEventListener("submit", (e) => {
  e.preventDefault();
  const matn = kiritish.value.trim();

  // Validatsiya
  if (matn === "") {
    xato.textContent = "⚠️ Vazifa matnini kiriting";
    return;
  }

```

```

    if (vazifalar.some(v => v.matn.toLowerCase() === matn.toLowerCase())) {
      xato.textContent = "⚠ Bunday vazifa allaqachon bor";
      return;
    }

    xato.textContent = "";
    qoshish(matn);
    kiritish.value = "";
    kiritish.focus();
  });

  document.querySelectorAll("[data-filtr]").forEach(tugma => {
    tugma.addEventListener("click", () => {
      filtr = tugma.dataset.filtr; // data-filtr atributining
      qiymati
      document.querySelectorAll("[data-filtr]").forEach(t =>
t.classList.remove("faol"));
      tugma.classList.add("faol");
      render();
    });
  });

  document.querySelector("#tozalash").addEventListener("click", () => {
    vazifalar = vazifalar.filter(v => !v.bajarildi);
    yangilash();
  });

  // ===== 6. BOSHLAGH =====
  render();

```


3-qadam: dizayn

todo/style.css

```
* { box-sizing: border-box; }
body { font-family: "Segoe UI", sans-serif; background: #eef2ff; margin: 0;
padding: 40px 16px; }
.ilova { max-width: 480px; margin: 0 auto; background: white; padding: 28px;
border-radius: 18px;
        box-shadow: 0 15px 35px rgba(79,70,229,.12); }
h1 { margin-top: 0; text-align: center; }
#forma { display: flex; gap: 8px; }
#forma input { flex: 1; padding: 12px; border: 2px solid #e5e7eb; border-
radius: 10px; font-size: 16px; }
#forma input:focus { border-color: #4f46e5; outline: none; }
#forma button { padding: 0 18px; border: none; border-radius: 10px; background:
#4f46e5; color: white; font-weight: 600; cursor: pointer; }
.xato { color: #dc2626; min-height: 1.2em; margin: 6px 0; font-size: 14px; }
.filtrlar { display: flex; gap: 6px; margin-bottom: 10px; }
.filtrlar button { border: 1px solid #e5e7eb; background: white; padding: 6px
12px; border-radius: 999px; cursor: pointer; }
.filtrlar button.faol { background: #4f46e5; color: white; border-color:
#4f46e5; }
#royxat { list-style: none; padding: 0; margin: 0; }
#royxat li { display: flex; align-items: center; gap: 10px; padding: 12px 4px;
border-bottom: 1px solid #f3f4f6; }
#royxat li span { flex: 1; }
#royxat li.bajarildi span { text-decoration: line-through; color: #9ca3af; }
#royxat li button { border: none; background: none; cursor: pointer; font-size:
18px; opacity: .4; transition: opacity .2s; }
#royxat li:hover button { opacity: 1; }
#royxat li.bosh { justify-content: center; color: #9ca3af; }
.pastki { display: flex; justify-content: space-between; margin-top: 14px;
font-size: 14px; color: #6b7280; }
.pastki button { border: none; background: none; color: #6b7280; text-
decoration: underline; cursor: pointer; }
```

Uchala faylni bitta papkada yarating va Live Server bilan oching. Vazifalar qo'shing, sahifani yangilang — ular **saqlanib qoladi** (localStorage).



Tushunish uchun tajriba

DevTools → **Application** → **Local Storage** ni oching. Vazifa qo'shganingizda `vazifalar` kaliti ostidagi JSON qanday o'zgarishini kuzating.

2-loyiha: Mini test tizimi (12-loyiha)

Savollar **ma'lumot** sifatida massivda saqlanadi — yangi savol qo'shish uchun kodni o'zgartirish shart emas, faqat massivga qo'shish kifoya.

```

<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Mini test</title>
<style>
  body { font-family: "Segoe UI", sans-serif; background: #f1f5f9; display:
grid; place-items: center; min-height: 100vh; margin: 0; }
  .test { background: white; width: min(480px, 92vw); padding: 28px; border-
radius: 18px; box-shadow: 0 10px 30px rgba(0,0,0,.08); }
  .progress { height: 8px; background: #e5e7eb; border-radius: 4px; overflow:
hidden; margin-bottom: 18px; }
  .progress div { height: 100%; background: #4f46e5; width: 0; transition:
width .4s; }
  .raqam { color: #6b7280; font-size: 14px; }
  h2 { margin: 6px 0 16px; font-size: 1.25rem; }
  .variant { display: block; width: 100%; text-align: left; padding: 12px 14px;
margin: 8px 0; border: 2px solid #e5e7eb;
    border-radius: 10px; background: white; font-size: 16px; cursor:
pointer; transition: .2s; }
  .variant:hover:not(:disabled) { border-color: #4f46e5; background: #eef2ff; }
  .variant.togri { border-color: #22c55e; background: #dcfce7; }
  .variant.notogri { border-color: #ef4444; background: #fee2e2; }
  #keyingi { margin-top: 14px; padding: 12px 22px; border: none; border-radius:
10px; background: #4f46e5; color: white; font-size: 16px; cursor: pointer; }
  .yashirin { display: none; }
  .natija { text-align: center; }
  .natija .ball { font-size: 3rem; font-weight: 800; color: #4f46e5; }
</style>
</head>
<body>
<div class="test">
  <div class="progress"><div id="chiziq"></div></div>
  <div id="savolBloki">
    <div class="raqam" id="raqam"></div>
    <h2 id="savol"></h2>
    <div id="variantlar"></div>
    <button id="keyingi" class="yashirin">Keyingi →</button>
  </div>
  <div id="natijaBloki" class="natija yashirin"></div>
</div>

```

```

<script>
  const SAVOLLAR = [
    { savol: "Sahifa tuzilishini yaratuvchi til qaysi?", variantlar: ["CSS",
"HTML", "JavaScript", "Python"], togri: 1 },
    { savol: "Doim ishlatilishi kerak bo'lgan taqqoslash operatori?",
variantlar: ["=", "==", "===", "⇒"], togri: 2 },
    { savol: "Massiv oxiriga element qo'shuvchi metod?", variantlar: ["pop()",
"push()", "shift()", "add()"], togri: 1 },
    { savol: "Elementni CSS selektori bilan topuvchi metod?", variantlar:
["querySelector", "findElement", "getCSS", "select"], togri: 0 },
    { savol: "17 % 5 nechaga teng?", variantlar: ["3", "3.4", "2", "12"],
togri: 2 },
  ];

  let joriy = 0;
  let ball = 0;

  const raqamEl = document.querySelector("#raqam");
  const savolEl = document.querySelector("#savol");
  const variantlarEl = document.querySelector("#variantlar");
  const keyingiEl = document.querySelector("#keyingi");
  const chiziqEl = document.querySelector("#chiziq");

  function savolniKorsat() {
    const s = SAVOLLAR[joriy];
    raqamEl.textContent = `Savol ${joriy + 1} / ${SAVOLLAR.length}`;
    savolEl.textContent = s.savol;
    chiziqEl.style.width = `${(joriy / SAVOLLAR.length) * 100}%`;
    keyingiEl.classList.add("yashirin");
    variantlarEl.innerHTML = "";

    s.variantlar.forEach((matn, i) => {
      const tugma = document.createElement("button");
      tugma.className = "variant";
      tugma.textContent = matn;
      tugma.addEventListener("click", () => javobBer(i, tugma));
      variantlarEl.append(tugma);
    });
  }

  function javobBer(tanlangan, tugma) {
    const togri = SAVOLLAR[joriy].togri;
    const hammasi = variantlarEl.querySelectorAll(".variant");
  }

```

```

    hammasi.forEach(t => t.disabled = true); // qayta bosib
    bo'lmasin

    if (tanlangan === to'g'ri) {
        ball++;
        tugma.classList.add("togri");
    } else {
        tugma.classList.add("notogri");
        hammasi[togri].classList.add("togri"); // to'g'ri javobni
    }
    ko'rsatamiz
    }
    keyingiEl.classList.remove("yashirin");
}

keyingiEl.addEventListener("click", () => {
    joriy++;
    if (joriy < SAVOLLAR.length) savolniKorsat();
    else natijaniKorsat();
});

function natijaniKorsat() {
    chiziqEl.style.width = "100%";
    document.querySelector("#savolBlok").classList.add("yashirin");
    const foiz = Math.round((ball / SAVOLLAR.length) * 100);
    const izoh = foiz >= 86 ? "A'lo! 🏆" : foiz >= 60 ? "Yaxshi! 👍" : "Yana
    mashq qiling 📖";
    const blok = document.querySelector("#natijaBlok");
    blok.classList.remove("yashirin");
    blok.innerHTML = `<p>Natijangiz</p><div
    class="ball">${ball}/${SAVOLLAR.length}</div>
    <p>${foiz}% - ${izoh}</p><button id="qayta">Qaytadan
    boshlash</button>`;
    document.querySelector("#qayta").addEventListener("click", () => {
        joriy = 0; ball = 0;
        blok.classList.add("yashirin");
        document.querySelector("#savolBlok").classList.remove("yashirin");
        savolniKorsat();
    });
}

savlalniKorsat();
</script>
</body>
</html>

```

Kengaytirish g'oyalari:

- 1 O'zingiz 10 ta savol yozing (HTML, CSS, JS bo'yicha).
- 2 Savollar va variantlar har safar **aralashtirilib** chiqsin (Maslahat: `arr.sort(() => Math.random() - 0.5)`; variantlarni aralashtirganda to'g'ri javob indeksini ham yangilashni unutmang!).
- 3 Har bir savolga 20 soniyalik taymer qo'shing (`setInterval`). Vaqt tugasa — javob noto'g'ri hisoblanadi.
- 4 Eng yaxshi natijani `localStorage` da saqlang va "Rekord: 9/10" deb ko'rsating.

Forma validatsiyasi: real vaqtda tekshirish

HTML

```
<style>
  body { font-family: sans-serif; padding: 20px; }
  .maydon { margin-bottom: 14px; }
  input { padding: 10px; border: 2px solid #d1d5db; border-radius: 8px; width:
260px; }
  .maydon.xato input { border-color: #ef4444; }
  .maydon.ok input { border-color: #22c55e; }
  small { display: block; color: #ef4444; min-height: 1em; }
</style>
<form id="royxatdan">
  <div class="maydon"><input name="email" placeholder="Email"><small></small>
</div>
  <div class="maydon"><input name="parol" type="password" placeholder="Parol">
<small></small></div>
  <button>Ro'yxatdan o'tish</button>
</form>
<script>
  const qoidalar = {
    email: v => /^\\S+@\\S+\\.\\S+$/\\.test(v) ? "" : "Email noto'g'ri",
    parol: v => v.length < 8 ? "Kamida 8 belgi" : !\\/d/.test(v) ? "Kamida bitta
raqam bo'lsin" : "",
  };
  const forma = document.querySelector("#royxatdan");

  function tekshir(input) {
    const xato = qoidalar[input.name](input.value);
    const maydon = input.closest(".maydon");
    maydon.querySelector("small").textContent = xato;
    maydon.classList.toggle("xato", xato !== "");
    maydon.classList.toggle("ok", xato === "");
    return xato === "";
  }

  forma.addEventListener("input", e => tekshir(e.target));
  forma.addEventListener("submit", e => {
    e.preventDefault();
    const hammasiOk = [...forma.elements].filter(el =>
el.name).map(tekshir).every(Boolean);
    if (hammasiOk) alert("✅ Muvaffaqiyatli ro'yxatdan o'tdingiz!");
  });
</script>
```

```
});  
</script>
```

`/^\S+@\S+\.\S+$/` — **muntazam ifoda (regex)**: "bo'sh joydiz belgilar, @, belgilar, nuqta, belgilar".

Regex'lar matnni naqsh bo'yicha tekshirish uchun ishlatiladi.

Ko'p uchraydigan xatolar

- ✗ Holatni o'zgartirib, `render()` ni chaqirishni unutish — massiv o'zgardi, lekin ekran eski holicha.
- ✗ `localStorage` ga massivni `JSON.stringify` siz saqlash — `"[object Object]"` saqlanib qoladi.
- ✗ `render()` da ro'yxatni tozalamaslik (`innerHTML = ""`) — har safar elementlar ikki baravar ko'payadi.
- ✗ Foydalanuvchi matnini `innerHTML` bilan qo'shish — XSS xavfi. `textContent` !

O'zingizni tekshiring

? "Holat → Chizish" usulida tugma bosilganda birinchi nima o'zgaradi?

- A. Sahifadagi HTML
- B. CSS fayl
- C. Ma'lumotlar massivi (holat)
- D. `localStorage`

? **Massivni localStorage'da saqlash uchun avval nima qilinadi?**

- A. `JSON.stringify()` bilan matnga aylantiriladi
- B. `toString()` chaqiriladi
- C. Hech narsa, to'g'ridan-to'g'ri saqlanadi
- D. `Number()` bilan songa aylantiriladi

? **`<button data-filtr="faol">` dagi qiymatni JS'da qanday o'qiysiz?**

- A. `tugma.filtr`
- B. `tugma.data`
- C. `tugma.value`
- D. `tugma.dataset.filtr`

? **Vazifani o'chirishning eng qulay usuli (id bo'yicha)?**

- A. `vazifalar.pop(id)`
- B. `vazifalar = vazifalar.filter(v => v.id !== id)`
- C. `delete vazifalar[id]`
- D. `vazifalar.remove(id)`



Uyga vazifa (bob yakunlovchi loyiha)

Quyidagilardan **birini** tanlab, to'liq ilova yarating va GitHub + Netlify'ga joylang:

1. **Xarajatlar hisobchisi:** xarajat nomi, summasi va toifasi qo'shiladi; jami summa va toifalar bo'yicha yig'indi ko'rsatiladi; localStorage.
2. **Lug'at o'yini:** inglizcha so'z ko'rsatiladi, foydalanuvchi tarjimasini yozadi; ball va "qiyin so'zlar" ro'yxati.

3. **Kutubxona katalogi:** kitoblar kartochkalari, qidiruv maydoni (real vaqtda filtr), janr bo'yicha filtr, "o'qildi" belgisi.



II bob yakunlandi!

- Loyiha: reja → dizayn → ma'lumot tuzilmasi → hodisalar → kichik qadamlar.
- "Holat → Chizish": hodisa holatni o'zgartiradi, `render()` sahifani holatdan chizadi, `localStorage` saqlaydi.
- Siz endi interaktiv veb-illovalar yarata olasiz. Keyingi bobda **server tomoniga** o'tamiz — Python va ma'lumotlar bazalari!

✓ Test javoblari

1. "Holat → Chizish" usulida tugma bosilganda birinchi nima o'zgaradi? — **C) Ma'lumotlar massivi (holat)**
2. Massivni localStorage'da saqlash uchun avval nima qilinadi? — **A) JSON.stringify() bilan matnga aylantiriladi**
3. <button data-filtr="faol"> dagi qiymatni JS'da qanday o'qiysiz? — **D) tugma.dataset.filtr**
4. Vazifani o'chirishning eng qulay usuli (id bo'yicha)? — **B) vazifalar = vazifalar.filter(v ⇒ v.id !== id)**



21-MAVZU · 42 TADAN · III BOB. PYTHON VA MA'LUMOTLAR BAZASI

Back-end haqida. Python'ga kirish va o'rnatish. O'zgaruvchilar, turlar, operatorlar, kiritish-chiqarish

Back-end nima qiladi, algoritm va blok-sxema (Flowgorithm), Python'ni o'rnatish va VS Code, print va input, o'zgaruvchilar, int/float/str/bool, operatorlar, f-satrlar.

2 soat · amaliy

Bu darsda nimani o'rganasiz

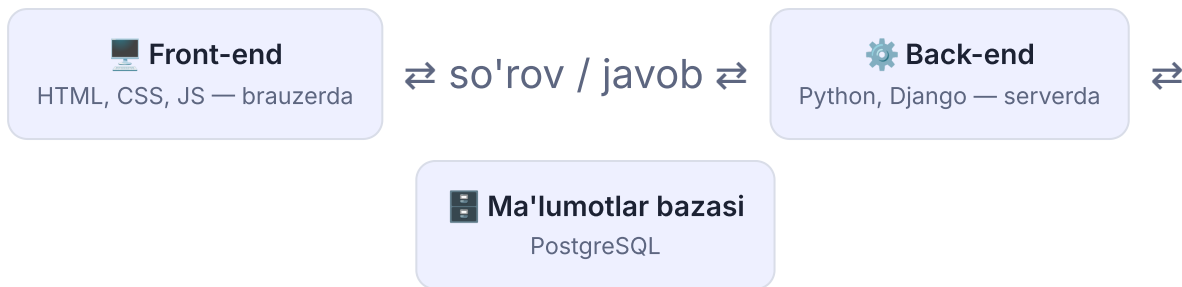
- Back-end nima va u front-end bilan qanday "gaplashishini";
- Algoritm va blok-sxema tushunchalarini;
- Python'ni o'rnatish va birinchi dasturni ishga tushirishni;
- `print()` va `input()` bilan muloqotni;
- O'zgaruvchilar, ma'lumot turlari va operatorlarni (JavaScript bilan taqqoslab).

Back-end nima?

I va II boblarda **front-end** — brauzerda ishlaydigan qismni o'rgandik. Lekin quyidagi savollarga front-end javob bera olmaydi:

- Instagram'dagi milliardlab rasmlar **qayerda saqlanadi**?
- Parolingiz to'g'riligini **kim tekshiradi**?
- Do'stingiz yuborgan xabar sizga **qanday yetib keladi**?

Bularning hammasi — **back-end**ning ishi: serverda ishlaydigan, foydalanuvchi ko'rmaydigan dastur.



O'xshatish: restoran (1-mavzudan davomi)

Front-end — zal va ofitsiant. **Back-end** — oshxona: buyurtmani qabul qiladi, retsept (biznes-mantiq) bo'yicha tayyorlaydi, omborga (ma'lumotlar bazasi) murojaat qiladi. Mijoz oshxonani ko'rmaydi, lekin usiz restoran ishlamaydi.

Back-end vazifalari: ma'lumotlarni saqlash va o'qish, foydalanuvchini tekshirish (login), hisob-kitoblar (buyurtma summasi), xavfsizlik, boshqa xizmatlar bilan aloqa (to'lov tizimlari, SMS).

Algoritm va blok-sxema

Har qanday dastur **algoritm**dan boshlanadi. **Algoritm** — masalani yechish uchun **aniq va tartibli qadamlar** ketma-ketligi.



Kundalik algoritm: choy damlash

1) Choynakka suv quy. 2) Qaynat. 3) Choynakni chay. 4) Choy sol. 5) Qaynoq suv quy. 6) 5 daqiqa kut. Har bir qadam aniq, tartibi muhim — 5-qadamni 1-qadamdan oldin bajarib bo'lmaydi.

Algoritmni chizma ko'rinishida ifodalash — **blok-sxema**:

Shakl	Ma'nosi
○ Oval	Boshlanish / Tugash
▱ Parallelogramm	Kiritish / Chiqarish (input, print)
▭ To'rtburchak	Amal (hisoblash, tayinlash)
◇ Romb	Shart (ha / yo'q)

Algoritmning uch asosiy turi: **chiziqli** (qadamlar ketma-ket), **tarmoqlanuvchi** (shart bo'yicha ikki yo'l) va **takrorlanuvchi** (sikl).



Flowgorithm

Flowgorithm — blok-sxemani chizib, uni darhol **ishga tushirish** mumkin bo'lgan bepul dastur. U chizmangizni Python kodiga ham aylantirib beradi. Murakkab masalani yechishdan oldin uning blok-sxemasini Flowgorithm'da tuzib ko'ring.

Nima uchun Python?

- **Sodda sintaksis** — ingliz tiliga yaqin, o'qish oson. Yangi boshlovchilar uchun eng mashhur til.
- **Universal** — veb-saytlar (Instagram, Pinterest back-endi Django'da), sun'iy intellekt, ma'lumotlar tahlili, avtomatlashtirish, Telegram botlar.
- **Katta jamiyat** — har qanday savolga internetda javob bor; minglab tayyor kutubxonalar.

JavaScript bilan taqqoslash

JavaScript

```
let ism = "Ali";
if (ism === "Ali") {
  console.log("Salom, " + ism);
}
```

Python

```
ism = "Ali"
if ism == "Ali":
    print("Salom, " + ism)
```

Farqlar: `let` yo'q, nuqta-vergul yo'q, jingalak qavslar o'rniga **ikki nuqta va chekinish (bo'sh joy)**.

Python'ni o'rnatish

- 1 python.org/downloads dan eng so'nggi versiyani (3.12+) yuklab oling.
- 2 O'rnatish oynasida **"Add python.exe to PATH"** belgisini **albatta** qo'ying! (Aks holda terminal Python'ni topa olmaydi.) → **Install Now**.
- 3 Terminalda (cmd yoki VS Code terminali) tekshiring:

Terminal

```
python --version
# Python 3.12.x
# macOS/Linux'da: python3 --version
```

- 4 VS Code'da **Extensions** → "Python" (Microsoft) kengaytmasini o'rnatish.
- 5 Yangi fayl: `salom.py` (kengaytma `.py`!), ichiga yozing va o'ng yuqoridagi ► tugmasini bosib (yoki terminalda `python salom.py`):

salom.py

```
print("Salom, dunyo!")
print("Men Python o'rganyapman 🐍")
```

Natija

```
Salom, dunyo!
Men Python o'rganyapman 🐍
```



Interaktiv rejim

Terminalda shunchaki `python` deb yozsangiz, `>>>` belgisi chiqadi — bu yerda har bir buyruq darhol bajariladi. Tez tajriba uchun qulay. Chiqish: `exit()`. Kompyuterda Python bo'lmasa, brauzerda ishlaydigan online-python.com dan foydalanish mumkin.

Chiqarish: `print()`

Python

```
print("Salom")           # matn
print(2 + 3)              # 5
print("Yosh:", 14)        # bir nechta qiymat – vergul bilan,
orasiga bo'sh joy qo'yiladi
print("A", "B", "C", sep="-") # A-B-C   – ajratuvchini o'zgartirish
print("Bir", end=" ")      # qator oxirida yangi qatorga o'tmaydi
print("qatorda")
print()                   # bo'sh qator
print("=" * 20)           # ===== – matnni
takrorlash
```

Natija

```
Salom
5
Yosh: 14
A-B-C
Bir qatorda

=====
```

Python'da izohlar `#` belgisi bilan yoziladi.

O'zgaruvchilar

Python'da o'zgaruvchi yaratish uchun hech qanday kalit so'z kerak emas — shunchaki nom va qiymat:

Python

```
ism = "Malika"
yosh = 14
boy = 1.62
alochi = True

print(ism, yosh, boy, alochi)

yosh = yosh + 1      # qiymatni o'zgartirish
print("Keyingi yil:", yosh)

a, b, c = 1, 2, 3    # bir nechtasini birdan
x = y = 0            # bir xil qiymat
```

Nomlash qoidalari

- Harf yoki `_` bilan boshlanadi; harf, raqam, `_` dan iborat; bo'sh joy va defis yo'q.
- Katta-kichik harf farq qiladi: `ism` ≠ `Ism`.
- Python uslubi — **snake_case**: so'zlar pastki chiziq bilan: `oquvchi_soni`, `umumiy_ball`. (JS'dagi camelCase emas!)
- O'zgarmas qiymatlar odatda KATTA harflar bilan: `PI = 3.14`, `MAKS_BALL = 100`.

Ma'lumot turlari

Tur	Nomi	Misol	JavaScript'da
<code>int</code>	Butun son	<code>14</code> , <code>-5</code> , <code>1_000_000</code>	number
<code>float</code>	Kasr son	<code>3.14</code> , <code>-0.5</code>	number
<code>str</code>	Matn (satr)	<code>"Salom"</code> , <code>'Ali'</code>	string
<code>bool</code>	Mantiqiy	<code>True</code> , <code>False</code> (bosh harf katta!)	true / false
<code>NoneType</code>	"Hech narsa"	<code>None</code>	null

Python

```
print(type(14))      # <class 'int'>
print(type(3.14))    # <class 'float'>
print(type("salom")) # <class 'str'>
print(type(True))    # <class 'bool'>
print(type(None))    # <class 'NoneType'>
```

Turlarni o'zgartirish

Python

```
print(int("42") + 8)      # 50
print(float("3.5") * 2)   # 7.0
print(str(100) + "%")     # 100%
print(int(7.9))           # 7 — kasr qismi tashlanadi (yaxlitlanmaydi!)
print(round(7.9))         # 8 — yaxlitlash
print(round(3.14159, 2))  # 3.14

# print("Yosh: " + 14)    # ❌ TypeError — Python matn va sonni avtomatik
# qo'shmaydi!
print("Yosh: " + str(14)) # ✅
```



JavaScript'dan farqi

JS'da `"5" + 3 = "53"` edi. Python esa bunday "sehr"ga yo'l qo'ymaydi va **xato** beradi — bu yaxshi, chunki xatolar yashirilib qolmaydi.

Kiritish: `input()`

Python

```
ism = input("Ismingiz nima? ")
print("Salom,", ism)

yosh = int(input("Yoshingiz nechada? "))    # ← input DOIM matn qaytaradi —
int() bilan songa!
print("10 yildan keyin siz", yosh + 10, "yoshda bo'lasiz")
```

Natija

```
Ismingiz nima? Dilshod
Salom, Dilshod
Yoshingiz nechada? 14
10 yildan keyin siz 24 yoshda bo'lasiz
```

f-satrlar — chiroyli chiqarish

JavaScript'dagi ``${}`` ning Python'dagi "akasi" — **f-string**: satr oldiga `f` harfi, o'zgaruvchilar `{}` ichida.

Python

```
ism = "Nodira"
ball = 87.456
print(f"{ism} {ball} ball to'pladi")
print(f"Yaxlitlangan: {ball:.1f}")    # 87.5 — 1 xona kasr
print(f"2 + 3 = {2 + 3}")             # ifodalar ham ishlaydi
narx = 1250000
print(f"Narxi: {narx:,.} so'm")      # 1,250,000 so'm — minglik
ajratish
```

Natija

```
Nodira 87.456 ball to'pladi
Yaxlitlangan: 87.5
2 + 3 = 5
Narxi: 1,250,000 so'm
```

Operatorlar

Python

```
a, b = 17, 5
print(a + b)    # 22
print(a - b)    # 12
print(a * b)    # 85
print(a / b)    # 3.4    - oddiy bo'lish (doim float)
print(a // b)   # 3      - BUTUN bo'lish (JS'da yo'q!)
print(a % b)    # 2      - qoldiq
print(a ** 2)   # 289    - daraja

x = 10
x += 5          # 15    (Python'da x++ YO'Q - x += 1 yoziladi)

# Taqqoslash - natija True/False
print(5 > 3, 5 == 5, 5 != 3, 5 >= 6)  # True True True False

# Mantiqiy - so'zlar bilan!
yosh = 15
print(yosh > 12 and yosh < 18)  # True    (JS: &&)
print(yosh < 7 or yosh > 60)    # False   (JS: ||)
print(not True)                 # False   (JS: !)
print(12 < yosh < 18)           # True    - Python'da shunday yozish mumkin!
```

Amal	JavaScript	Python
Teng	<code>===</code>	<code>=</code>
VA / YOKI / EMAS	<code>&&</code> / <code> </code> / <code>!</code>	<code>and</code> / <code>or</code> / <code>not</code>

Butun bo'lish	<code>Math.floor(a / b)</code>	<code>a // b</code>
Bittaga oshirish	<code>x++</code>	<code>x += 1</code>
Rost / Yolg'on	<code>true</code> / <code>false</code>	<code>True</code> / <code>False</code>

Amaliy mashg'ulot

1 Anketa. Foydalanuvchidan ism, familiya, tug'ilgan yil va shaharni so'rang va f-satr bilan chiroyli "vizitka" chiqaring (ramka uchun `"*" * 30`).

2 Do'kon:

dokon.py

```
print("≡≡≡ DAFTAR DO'KONI ≡≡≡")
narx = 5500
soni = int(input("Nechta daftar olasiz? "))
pul = int(input("Qancha pul berdingiz? "))

jami = narx * soni
qaytim = pul - jami

print(f"Jami: {jami:,} so'm")
print(f"Qaytim: {qaytim:,} so'm")
```

Kengaytiring: chegirma 10% (agar 10 tadan ko'p olsa) — hozircha shartsiz, faqat formulada hisoblab ko'ring; shartlarni keyingi darsda o'rganamiz.

3 Vaqt o'giruvchi. Daqiqalarni kiriting → soat va daqiqaga aylantiring (`//` va `%`): 135 → "2 soat 15 daqiqa".

4 Harorat. Selsiyni Farengeytga o'giring: $F = C \times 9/5 + 32$.

5 Blok-sxema. "Uchburchak yuzi (asos va balandlik bo'yicha)" algoritmining blok-sxemasini daftarga (yoki Flowgorithm'da) chizing, so'ng Python'da yozing.

Ko'p uchraydigan xatolar

- ❌ `'python' is not recognized` — o'rnatishda "Add to PATH" belgilanmagan. Python'ni qayta o'rnatish (Modify → PATH).
- ❌ `TypeError: can only concatenate str (not "int") to str` — matn va sonni `+` bilan qo'shish. `str()` yoki f-satr ishlatish.
- ❌ `input()` natijasini `int()` siz hisoblash: `"5" * 3 = "555"`.
- ❌ `true` deb kichik harf bilan yozish — Python'da `True`.
- ❌ `Print("...")` — katta P bilan. Python katta-kichik harfni farqlaydi.

O'zingizni tekshiring

? Back-end'ning vazifasi qaysi?

- A. Tugma rangini o'zgartirish
- B. Ma'lumotlarni serverda saqlash va parolni tekshirish
- C. Animatsiya yaratish
- D. Sahifa shriftini tanlash

? Blok-sxemada shart (ha/yo'q) qaysi shakl bilan belgilanadi?

- A. Oval
- B. To'rtburchak
- C. Romb
- D. Parallelogramm

? `print(17 // 5)` natijasi?

- A. 3
- B. 3.4
- C. 2
- D. 85

? `input()` funksiyasi qanday turdagi qiymat qaytaradi?

- A. int
- B. float
- C. bool
- D. str

? Python'da VA mantiqiy operatori qanday yoziladi?

- A. `&&`
- B. `and`
- C. `AND`
- D. `&`

? `ism = "Ali"; print(f"Salom, {ism}!")` natijasi?

- A. `Salom, {ism}!`
- B. `f"Salom, Ali!"`
- C. `Salom, Ali!`
- D. Xato

Uyga vazifa

1. **Shaxsiy kalkulyator:** ikki son so'rab, ularning yig'indisi, ayirmasi, ko'paytmasi, bo'linmasi, butun bo'linmasi, qoldig'i va darajasini jadval ko'rinishida chiqaring.
2. **Pul jamg'arish:** oylik jamg'arma va maqsad summasini so'rab, necha oyda maqsadga yetishni hisoblang (`//` va qoldiq bo'lsa `+`1).
3. **Uch xonali son:** foydalanuvchi uch xonali son kiritadi; uning raqamlari yig'indisini toping (345 → 12).

Yodda tuting

- Back-end — serverdagi dastur: ma'lumotlar, tekshiruv, mantiq. Algoritm — aniq qadamlar ketma-ketligi; blok-sxema — uning chizmasi.
- Python: `.py` fayl, `python fayl.py` bilan ishga tushadi; o'rnatishda "Add to PATH".
- `print()` — chiqarish, `input()` — kiritish (doim `str`; `int()` / `float()` bilan o'giring).
- Turlar: `int`, `float`, `str`, `bool` (`True/False`), `None`. Nomlar — snake_case.
- Operatorlar: `//` `%` `**`, `and` `or` `not`; f-satr: `f"{nom}"`.

✓ Test javoblari

1. Back-end'ning vazifasi qaysi? — **B) Ma'lumotlarni serverda saqlash va parolni tekshirish**
2. Blok-sxemada shart (ha/yo'q) qaysi shakl bilan belgilanadi? — **C) Romb**
3. `print(17 // 5)` natijasi? — **A) 3**
4. `input()` funksiyasi qanday turdagi qiymat qaytaradi? — **D) str**
5. Python'da VA mantiqiy operatori qanday yoziladi? — **B) and**
6. `ism = "Ali"; print(f"Salom, {ism}!")` natijasi? — **C) Salom, Ali!**



22-MAVZU · 42 TADAN · III BOB. PYTHON VA MA'LUMOTLAR BAZASI

Python control flow: shartlar va sikllar. Fayllar bilan ishlash

if, elif, else va chekinish qoidasi; match; for va range(), while, break, continue; try/except bilan xatolarni ushlash; fayllarni ochish, o'qish, yozish va qo'shish (with open). Python kalkulyatori.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Python'da shartlar: `if`, `elif`, `else` va **chekinish (indentatsiya)** qoidasi;
- Sikllar: `for` + `range()`, `while`, `break`, `continue`;
- Dastur "yiqilmasligi" uchun xatolarni ushlash: `try` / `except`;
- Fayllar bilan ishlash: yaratish, yozish, o'qish, qo'shish;
- Python kalkulyatorini (15-loyiha).

JavaScript'da shart va sikllarni o'rgangansiz — g'oyalar bir xil, faqat yozilishi boshqacha. Shuning uchun bu mavzu tez o'tadi.

Shartlar: if, elif, else

Python

```
ball = int(input("Ballingiz: "))

if ball ≥ 86:
    print("A'lo! 🏆")
    baho = 5
elif ball ≥ 71:
    print("Yaxshi 👍")
    baho = 4
elif ball ≥ 56:
    print("Qoniqarli")
    baho = 3
else:
    print("Qoniqarsiz 😞")
    baho = 2

print("Bahoingiz:", baho)
```

Chekinish (indentatsiya) — Python'ning eng muhim qoidasi

JavaScript'da kod bloklari `{ }` bilan belgilanadi. Python'da esa **ikki nuqta** `:` va undan keyingi qatorlarning **4 ta bo'sh joy** bilan ichkariga surilishi bilan.

Python

```
yosh = 15
if yosh ≥ 14:
    print("Pasport olish mumkin")
    print("Tabriklaymiz!")
print("Bu har doim chiqadi")
```

← if ichida (4 bo'sh joy)
← if ichida
← if TASHQARISIDA (chekinishsiz)



IndentationError

Chekinish noto'g'ri bo'lsa, Python ishlamaydi: `IndentationError: expected an indented block`. Bitta faylda bo'sh joy va Tab'ni aralashtirmang. VS Code `Tab` bosganda avtomatik 4 bo'sh joy qo'yadi.



O'xshatish: kitob mundarijasi

Mundarijada bo'lim nomlari chapda, ularning ichidagi kichik bo'limlar esa o'ngroqqa surilgan. Qaysi kichik bo'lim qaysi bo'limga tegishli ekanini siljishdan bilasiz. Python ham kod "ierarxiyasini" xuddi shunday — siljish orqali tushunadi.

Qo'shimcha imkoniyatlar

Python

```
yosh = 16
# Qisqa shart (ternary)
holat = "katta" if yosh ≥ 18 else "voyaga yetmagan"
print(holat)

# Oraliq
if 12 ≤ yosh ≤ 17:
    print("O'smir")

# in — ichida bormi?
mevalar = "olma nok uzum"
if "nok" in mevalar:
    print("Nok bor")

# match — JS'dagi switch (Python 3.10+)
kun = 6
match kun:
    case 1 | 2 | 3 | 4 | 5:
        print("Ish kuni")
    case 6 | 7:
        print("Dam olish kuni 🧑🏻🌴")
    case _:
        print("Noto'g'ri kun")          # _ — default
```

Sikllar

for va range()

Python

```
for i in range(5):           # 0, 1, 2, 3, 4   (5 – kirmaydi!)
    print(i, end=" ")
print()

for i in range(1, 6):        # 1, 2, 3, 4, 5
    print(i, end=" ")
print()

for i in range(10, 0, -2):    # 10, 8, 6, 4, 2   (boshlanish, tugash, qadam)
    print(i, end=" ")
print()

for harf in "Python":        # satrning har bir harfi
    print(harf, end="-")
```

Natija

```
0 1 2 3 4
1 2 3 4 5
10 8 6 4 2
P-y-t-h-o-n-
```

JavaScript

Python

```
for (let i = 0; i < 5; i++)
```

```
for i in range(5):
```

```
for (let i = 1; i ≤ 10; i += 2)
```

```
for i in range(1, 11, 2):
```

```
for (const x of arr)
```

```
for x in arr:
```

Yig'indi va ko'paytirish jadvali

Python

```
jami = 0
for i in range(1, 101):
    jami += i
print("1..100 yig'indi:", jami)          # 5050

son = int(input("Qaysi sonning jadvali? "))
for i in range(1, 11):
    print(f"{son} × {i:2} = {son * i:3}")    # :2 va :3 – ustunlarni tekislash
```

while, **break**, **continue**

Python

```
import random

yashirin = random.randint(1, 20)
urinish = 0

while True:                                # "cheksiz" sikl – break bilan
    chiqamiz
    taxmin = int(input("1-20 oralig'ida son: "))
    urinish += 1
    if taxmin == yashirin:
        print(f"🎉 Topdingiz! {urinish} ta urinishda")
        break                               # sikldan chiqish
    elif taxmin < yashirin:
        print("Kattaroq ⬆")
    else:
        print("Kichikroq ⬇")

for i in range(1, 11):
    if i % 3 == 0:
        continue                            # 3, 6, 9 ni o'tkazib yuboradi
    print(i, end=" ")
```

Xatolarni ushlash: try / except

Foydalanuvchi son o'rniga "salom" yozsa, `int("salom")` xato beradi va dastur **to'xtab qoladi**. Buning oldini olish uchun:

Python

```
try:
    son = int(input("Son kiriting: "))
    natija = 100 / son
    print("Natija:", natija)
except ValueError:
    print("❌ Bu son emas!")
except ZeroDivisionError:
    print("❌ Nolga bo'lib bo'lmaydi!")
finally:
    print("Dastur tugadi")           # har doim bajariladi
```

Natija

```
Son kiriting: abc
❌ Bu son emas!
Dastur tugadi
```

To'g'ri son kiritilmaguncha so'rash

Python

```
while True:
    try:
        yosh = int(input("Yoshingiz: "))
        if 1 ≤ yosh ≤ 120:
            break
        print("Yosh 1 dan 120 gacha bo'lishi kerak")
    except ValueError:
        print("Iltimos, raqam kiriting")

print(f"Rahmat! Siz {yosh} yoshdasiz.")
```



Bu "kiritishni tekshirish sikli" — haqiqiy dasturlarda juda ko'p ishlatiladi. Uni funksiyaga aylantirib, keyingi loyihalarda qayta ishlatib (24-mavzu).

Fayllar bilan ishlash

O'zgaruvchilardagi ma'lumot dastur yopilganda yo'qoladi. Uni saqlab qolish uchun **faylga** yozish kerak (keyinroq — ma'lumotlar bazasiga).

Faylga yozish

Python

```
with open("eslatma.txt", "w", encoding="utf-8") as f:
    f.write("Birinchi qator\n")          # \n — yangi qatorga o'tish
    f.write("Ikkinchi qator\n")
print("Fayl yaratildi!")
```

- `open(fayl_nomi, rejim, encoding)` — faylni ochadi.
- `with ... as f:` — ish tugagach faylni **avtomatik yopadi** (yopilmagan fayl ma'lumot yo'qolishiga olib kelishi mumkin).
- `encoding="utf-8"` — o'zbekcha harflar (`o'`, `g'`) va emoji to'g'ri saqlanishi uchun. Doim yozing!

Rejim	Ma'nosi	Fayl yo'q bo'lsa	Fayl bor bo'lsa
"r"	read — o'qish (standart)	✗ Xato	O'qiydi
"w"	write — yozish	Yaratadi	⚠ Eski mazmunini o'chiradi!
"a"	append — oxiriga qo'shish	Yaratadi	Oxiriga qo'shadi

Fayldan o'qish

Python

```
with open("eslatma.txt", "r", encoding="utf-8") as f:
    matn = f.read()          # butun faylni bitta satr sifatida
print(matn)

with open("eslatma.txt", encoding="utf-8") as f:
    for i, qator in enumerate(f, start=1):  # qatorma-qator (katta fayllar
        uchin qulay)
        print(i, qator.strip())             # strip() – qator oxiridagi \n ni olib
tashlaydi
```

Qo'shish va fayl borligini tekshirish

Python

```
import os

with open("kundalik.txt", "a", encoding="utf-8") as f:
    f.write(input("Bugun nima qildingiz? ") + "\n")

if os.path.exists("kundalik.txt"):
    with open("kundalik.txt", encoding="utf-8") as f:
        qatorlar = f.readlines()          # qatorlar ro'yxati
        print(f"Kundalikda {len(qatorlar)} ta yozuv bor")
```

Amaliy mashg'ulot: Python kalkulyatori (15-loyiha)

kalkulyator.py

```
from datetime import datetime

TARIX_FAYLI = "tarix.txt"

def son_sora(matn):
    """To'g'ri son kiritilmaguncha so'raydi."""
    while True:
        try:
            return float(input(matn))
        except ValueError:
            print(" ⚠ Iltimos, son kiriting (masalan: 12 yoki 3.5)")

print("=" * 32)
print(" 📖 PYTHON KALKULYATORI")
print("=" * 32)

while True:
    a = son_sora("Birinchi son: ")
    amal = input("Amal (+ - * / // % **): ").strip()
    b = son_sora("Ikkinchi son: ")

    if amal == "+":
        natija = a + b
    elif amal == "-":
        natija = a - b
    elif amal == "*":
        natija = a * b
    elif amal in ("/", "//", "%"):
        if b == 0:
            print("❌ Nolga bo'lib bo'lmaydi!")
            continue
        natija = a / b if amal == "/" else a // b if amal == "//" else a % b
    elif amal == "**":
        natija = a ** b
    else:
        print("❌ Noma'lum amal")
        continue

    # 5.0 o'rniga 5 ko'rinsin
```

```

if natija == int(natija):
    natija = int(natija)
yozuv = f"{a:g} {amal} {b:g} = {natija}"
print("✅", yozuv)

with open(TARIX_FAYLI, "a", encoding="utf-8") as f:
    f.write(f"[{datetime.now():%Y-%m-%d %H:%M}] {yozuv}\n")

javob = input("Davom etamizmi? (ha/yo'q/tarix): ").strip().lower()
if javob == "tarix":
    with open(TARIX_FAYLI, encoding="utf-8") as f:
        print(f.read())
elif javob != "ha":
    print("Xayr! 🙌")
    break

```

Topshiriqlar:

- 1 Kalkulyatorni ishga tushiring va har bir amalni sinab ko'ring, xato kiritishlarni ham.
- 2 Yangi amallar qo'shing: `sqrt` (ildiz, bitta son bilan) va `%%` (foizini hisoblash: a ning b foizi).
- 3 "tarix" buyrug'i faqat oxirgi 5 ta yozuvni ko'rsatsin (`readlines()[-5:]`).
- 4 `.txt` fayl kompyuterda qayerda yaratilganini toping va oddiy Bloknnot bilan oching.

Ko'p uchraydigan xatolar

- ❌ `if`, `for`, `while` qatori oxirida `:` unutilgan → `SyntaxError`.
- ❌ `else if` deb yozish — Python'da `elif`.
- ❌ `range(1, 10)` 10 ni ham beradi deb o'ylash — oxirgi son kirmaydi.
- ❌ `"w"` rejimida ochib, muhim faylni o'chirib yuborish. Qo'shish uchun `"a"`.
- ❌ `encoding="utf-8"` siz o'zbekcha matn — Windows'da g'alati belgilar chiqadi.

O'zingizni tekshiring

? JS'dagi `else if` Python'da qanday yoziladi?

- A. `else if`
- B. `elseif`
- C. `elif`
- D. `orif`

? `list(range(2, 8, 2))` qanday sonlarni beradi?

- A. 2, 4, 6, 8
- B. 2, 4, 6
- C. 2, 3, 4, 5, 6, 7
- D. 8, 6, 4, 2

? Python kod bloklarini qanday belgilaydi?

- A. Jinalak qavslar `{}`
- B. `begin / end` so'zlari
- C. Nuqta-vergul
- D. Ikki nuqta va chekinish (bo'sh joylar)

? Fayl mazmunini o'chirmasdan, oxiriga yozish rejimi?

- A. "a"
- B. "w"
- C. "r"
- D. "x"

? `int("abc")` qanday xato beradi?

- A. ZeroDivisionError
- B. IndentationError
- C. ValueError
- D. TypeError

? `with open(...)` ning afzalligi nimada?

- A. Faylni tezroq o'qiydi
- B. Ish tugagach faylni avtomatik yopadi
- C. Faylni shifrlaydi
- D. Faylni o'chiradi



Uyga vazifa

1. **Parol tekshiruvchi:** parol kamida 8 belgi, kamida bitta raqam va bitta katta harf bo'lsin. (`for harf in parol:` va `harf.isdigit()`, `harf.isupper()`.)
2. **Kundalik:** dastur har ishga tushganda sanani va bugungi xulosangizni `kundalik.txt` ga qo'shsin, so'ng barcha yozuvlarni ko'rsatsin.

3. **Tub sonlar faylga:** 1–200 oralig'idagi barcha tub sonlarni `tub.txt` fayliga har qatorda bittadan yozing.

✓ Yodda tuting

- `if / elif / else:` — oxirida ikki nuqta, ichidagi kod 4 bo'sh joy surilgan.
- `for i in range(boshi, oxiri, qadam):` — oxiri kirmaydi; `while shart:`; `break`, `continue`.
- `try: ... except ValueError: ...` — xatolarni ushlab, dasturni "yiqilish"dan saqlash.
- `with open("f.txt", "r/w/a", encoding="utf-8") as f:` — `read()`, `write()`, `readlines()`.

Test javoblari

1. JS'dagi else if Python'da qanday yoziladi? — **C) elif**
2. list(range(2, 8, 2)) qanday sonlarni beradi? — **B) 2, 4, 6**
3. Python kod bloklarini qanday belgilaydi? — **D) Ikki nuqta va chekinish (bo'sh joylar)**
4. Fayl mazmunini o'chirmasdan, oxiriga yozish rejimi? — **A) "a"**
5. int("abc") qanday xato beradi? — **C) ValueError**
6. with open(...) ning afzalligi nimada? — **B) Ish tugagach faylni avtomatik yopadi**



Ma'lumot tuzilmalari: list, tuple, set va dictionary

Ro'yxat (list) va uning metodlari, kesish (slicing), list comprehension; o'zgarmas tuple; takrorlanmas set; kalit-qiyamatli dictionary; qaysi birini qachon tanlash; o'quvchilar ma'lumotlarini boshqarish dasturi.

1 soat · nazariy

Bu darsda nimani o'rganasiz

- Python'ning to'rtta asosiy "konteyneri": **list**, **tuple**, **set**, **dict**;
- Har birining xususiyatlari va qachon qaysi birini tanlash kerakligini;
- Ro'yxatlarni kesish (slicing) va list comprehension;
- O'quvchilar ma'lumotlarini boshqaruvchi dasturni (16-loyiha).

To'rt tuzilma — bir qarashda

Tuzilma	Yozilishi	Tartiblangan?	O'zgartirish mumkinmi?	Takrorlanish?	JS'dagi
list (ro'yxat)	[1, 2, 3]	✓ Ha	✓ Ha	✓ Mumkin	Array
tuple (kortej)	(1, 2, 3)	✓ Ha	✗ Yo'q	✓ Mumkin	—
set (to'plam)	{1, 2, 3}	✗ Yo'q	✓ Ha	✗ Yo'q	Set
dict (lug'at)	{"ism": "Ali"}	✓ Ha	✓ Ha	Kalitlar — yo'q	Object



O'xshatishlar

list — xarid ro'yxati: tartibi bor, qo'shish-o'chirish mumkin, bir narsa ikki marta yozilishi mumkin.

tuple — tug'ilgan sana (kun, oy, yil): bir marta yoziladi va o'zgarmaydi.

set — sinfdagi o'quvchilar ismlari to'plami: har biri faqat bir marta, tartibi ahamiyatsiz.

dict — lug'at yoki telefon kitobi: so'z (kalit) bo'yicha ma'nosini (qiymatni) darhol topasiz.

List — ro'yxat

Python

```
fanlar = ["matematika", "fizika", "informatika"]
print(fanlar[0])          # matematika – indeks 0 dan
print(fanlar[-1])         # informatika – MANFIY indeks: oxiridan sanaydi
print(len(fanlar))        # 3 – elementlar soni

fanlar.append("tarix")     # oxiriga qo'shish (JS: push)
fanlar.insert(1, "kimyo")  # 1-o'ringa qo'yish
fanlar.remove("fizika")    # qiymati bo'yicha o'chirish
oxirgi = fanlar.pop()      # oxirgisini olib tashlash va qaytarish
fanlar[0] = "algebra"      # o'zgartirish
print(fanlar)              # ['algebra', 'kimyo', 'informatika']

print("kimyo" in fanlar)   # True – bormi?
print(fanlar.index("kimyo")) # 1 – qaysi o'rinda?
```

Sonlar ro'yxati bilan ishlash

Python

```
ballar = [78, 95, 62, 88, 95, 71]
print(sum(ballar))           # 489 – yig'indi
print(max(ballar), min(ballar)) # 95 62
print(sum(ballar) / len(ballar)) # 81.5 – o'rtacha
print(ballar.count(95))      # 2 – necha marta uchraydi

ballar.sort()                # o'sish tartibida (asl ro'yxatni
                              o'zgartiradi)
print(ballar)
ballar.sort(reverse=True)    # kamayish tartibida
print(sorted(["olma", "anor", "behi"])) # yangi saralangan ro'yxat qaytaradi
```

Natija

```
489
95 62
81.5
2
[62, 71, 78, 88, 95, 95]
['anor', 'behi', 'olma']
```

Kesish (slicing) — Python'ning "super kuchi"

Python

```
harflar = ["a", "b", "c", "d", "e", "f"]
#           0     1     2     3     4     5
print(harflar[1:4])    # ['b', 'c', 'd']   – 1 dan 4 gacha (4 kirmaydi)
print(harflar[:3])     # ['a', 'b', 'c']   – boshidan
print(harflar[3:])     # ['d', 'e', 'f']   – oxirigacha
print(harflar[-2:])    # ['e', 'f']       – oxirgi ikkitasi
print(harflar[::-2])   # ['a', 'c', 'e']   – har ikkinchisi
print(harflar[::-1])   # teskari tartibda!
print("Python"[::-1])  # nohtyP – satrlarda ham ishlaydi
```

Aylanib chiqish

Python

```
oquvchilar = ["Ali", "Vali", "Gani"]
for ism in oquvchilar:
    print(ism)

for i, ism in enumerate(oquvchilar, start=1):    # raqam bilan
    print(f"{i}. {ism}")
```

List comprehension — qisqa va kuchli

Mavjud ro'yxatdan yangisini bitta qatorda yaratish (JS'dagi `map` va `filter` ning birlashmasi):

Python

```
sonlar = [1, 2, 3, 4, 5, 6]

kvadratlar = [x * x for x in sonlar]           # [1, 4, 9, 16, 25, 36]
juftlar = [x for x in sonlar if x % 2 == 0]    # [2, 4, 6]
ismlar = ["ali", "vali"]
kattalar = [i.capitalize() for i in ismlar]   # ['Ali', 'Vali']

# Oddiy sikl bilan xuddi shu:
kvadratlar2 = []
for x in sonlar:
    kvadratlar2.append(x * x)
```

Tuple — o'zgarmas ro'yxat

Python

```
sana = (1, 9, 2026)           # kun, oy, yil
koordinata = (41.31, 69.28)   # Toshkent kengligi va uzunligi
print(sana[2])                # 2026
# sana[0] = 5                  # ❌ TypeError: tuple o'zgartirilmaydi

kun, oy, yil = sana           # "ochish" (unpacking)
print(f"{kun}.{oy}.{yil}")

a, b = 5, 10
a, b = b, a                    # qiymatlarni almashtirish — aslida tuple!
print(a, b)                    # 10 5
```

Qachon tuple? Ma'lumot o'zgarmasligi kerak bo'lsa: sanalar, koordinatalar, RGB ranglar `(255, 0, 0)`, haftaning kunlari. Tuple list'dan biroz tezroq va "tasodifan o'zgartirib qo'yish"dan himoya qiladi.

Set — takrorlanmas to'plam

Python

```
ranglar = {"qizil", "yashil", "qizil", "ko'k"}
print(ranglar)           # {'yashil', 'ko'k', 'qizil'} – takror o'chdi,
                          # tartib kafolatlanmaydi
ranglar.add("sariq")
ranglar.discard("yashil")
print("qizil" in ranglar) # True – set'da qidirish juda tez

# Takrorlarni olib tashlashning eng oson usuli
royxat = [1, 2, 2, 3, 3, 3, 4]
print(list(set(royxat))) # [1, 2, 3, 4]

# To'plamlar ustida amallar (matematikadagi kabi)
futbol = {"Ali", "Vali", "Sardor"}
shaxmat = {"Vali", "Malika", "Sardor"}
print(futbol & shaxmat)    # kesishma – ikkalasida ham: {'Vali', 'Sardor'}
print(futbol | shaxmat)   # birlashma – hammasi
print(futbol - shaxmat)   # ayirma – faqat futbolda: {'Ali'}
```



Bo'sh set — `set()`. `{}` yozsangiz bo'sh **dict** yaratiladi!

Dictionary — kalit: qiymat

Eng ko'p ishlatiladigan tuzilma. Ma'lumotni **nomi (kaliti)** bo'yicha saqlaydi va darhol topadi.

Python

```
oquvchi = {
    "ism": "Dilnoza",
    "yosh": 14,
    "sinf": "8-A",
    "fanlar": ["matematika", "ingliz tili"],
}

print(oquvchi["ism"])           # Dilnoza
print(oquvchi.get("telefon"))  # None — kalit yo'q bo'lsa xato
                                # bermaydi
print(oquvchi.get("telefon", "yo'q")) # standart qiymat bilan

oquvchi["yosh"] = 15           # o'zgartirish
oquvchi["shahar"] = "Andijon"  # yangi kalit qo'shish
del oquvchi["sinf"]            # o'chirish
print("ism" in oquvchi)        # True — kalit bormi?

for kalit, qiymat in oquvchi.items(): # kalit va qiymatni birga
    print(f"{kalit}: {qiymat}")

print(list(oquvchi.keys()))
print(list(oquvchi.values()))
```



`oquvchi["telefon"]` — kalit yo'q bo'lsa `KeyError` xatosi. Ishonchingiz komil bo'lmasa, `.get()` ishlatang.

Dict bilan sanash — klassik masala

Python

```
matn = "olma nok olma uzum nok olma"
sanoq = {}
for soz in matn.split():
    sanoq[soz] = sanoq.get(soz, 0) + 1
print(sanoq)           # {'olma': 3, 'nok': 2, 'uzum': 1}
```

Ichma-ich tuzilmalar — real ma'lumot

Python

```
sinf = [
    {"ism": "Ali", "baholar": {"matematika": 5, "fizika": 4}},
    {"ism": "Zarina", "baholar": {"matematika": 5, "fizika": 5}},
]
print(sinf[1]["baholar"]["fizika"])      # 5

for o in sinf:
    ortacha = sum(o["baholar"].values()) / len(o["baholar"])
    print(f"{o['ism']}: {ortacha}")
```

Bu ko'rinish JavaScript'dagi obyektlar massiviga (18-mavzu) va internetdagi **JSON** formatiga juda o'xshaydi — 39-mavzuda API yaratganimizda aynan shu ishlatiladi.

Qaysi birini tanlash kerak?

Vazifa	Tanlov
Tartibli, o'zgaradigan ro'yxat (vazifalar, xabarlar)	list
O'zgarmas guruh (sana, koordinata)	tuple
Faqat noyob qiymatlar, tez "bormi?" tekshiruvi	set
Nom bo'yicha qidirish (login → parol, so'z → tarjima)	dict
Ko'p obyekt, har birining xususiyatlari	list ichida dict

Amaliy mashg'ulot: o'quvchilar ma'lumotlarini

boshqarish (16-loyiha)

oquvchilar.py

```
oquvchilar = {
    "ali": {"ism": "Ali Valiyev", "sinf": "8-A", "baholar": [5, 4, 5]},
    "zarina": {"ism": "Zarina Karimova", "sinf": "8-B", "baholar": [5, 5, 4]},
}

def ortacha(baholar):
    return round(sum(baholar) / len(baholar), 2) if baholar else 0

def royxat():
    if not oquvchilar:
        print("Ro'yxat bo'sh")
        return
    print(f"\n{'Login':<10}{'Ism':<20}{'Sinf':<6}{'O'rtacha':>8}")
    print("-" * 44)
    for login, o in oquvchilar.items():
        print(f"{'login':<10}{'o['ism']':<20}{'o['sinf']':<6}
{'ortacha(o['baholar']):>8}")

def qoshish():
    login = input("Login: ").strip().lower()
    if login in oquvchilar:
        print("❌ Bunday login band")
        return
    ism = input("To'liq ism: ").strip()
    sinf = input("Sinf: ").strip()
    oquvchilar[login] = {"ism": ism, "sinf": sinf, "baholar": []}
    print("✅ Qo'shildi")

def baho_qoyish():
    login = input("Login: ").strip().lower()
    if login not in oquvchilar:
        print("❌ Topilmadi")
        return
    try:
        baho = int(input("Baho (2-5): "))
    except ValueError:
        print("❌ Son kiriting")
        return
    if baho not in (2, 3, 4, 5):
```

```

        print("❌ Baho 2 dan 5 gacha")
        return
    oquvchilar[login]["baholar"].append(baho)
    print("✅ Baho qo'yildi")

def statistika():
    if not oquvchilar:
        return
    eng_yaxshi = max(oquvchilar.values(), key=lambda o: ortacha(o["baholar"]))
    sinflar = {o["sinf"] for o in oquvchilar.values()} # set
    comprehension
    print(f"Jami o'quvchilar: {len(oquvchilar)}")
    print(f"Sinflar: {' '.join(sorted(sinflar))}")
    print(f"Eng yuqori o'rtacha: {eng_yaxshi['ism']}
    ({ortacha(eng_yaxshi['baholar'])})")

MENYU = {
    "1": ("Ro'yxatni ko'rish", royxat),
    "2": ("O'quvchi qo'shish", qoshish),
    "3": ("Baho qo'yish", baho_qoyish),
    "4": ("Statistika", statistika),
}

while True:
    print("\n===== O'QUVCHILAR TIZIMI =====")
    for kalit, (nom, _) in MENYU.items():
        print(f"{kalit}. {nom}")
    print("0. Chiqish")
    tanlov = input("Tanlang: ").strip()
    if tanlov == "0":
        print("Xayr!")
        break
    if tanlov in MENYU:
        MENYU[tanlov][1]() # dict'dan funksiyani olib, chaqiramiz
    else:
        print("❌ Noto'g'ri tanlov")

```

Kodni o'rganing: o'quvchilar **dict**da (login bo'yicha tez topish), baholar **list**da, menyu ham **dict** (raqam → nom va funksiya **tuple**), sinflar **set**da (takrorlanmas). To'rtala tuzilma bitta dasturda!

- 1 "O'quvchini o'chirish" va "Ism bo'yicha qidirish" menyu bandlarini qo'shing.
- 2 Dastur yopilganda ma'lumot yo'qolmasligi uchun JSON faylga saqlang (24-mavzuda `json` modulini o'rganamiz — oldindan sinab ko'ring!).

Ko'p uchraydigan xatolar

- ✗ `IndexError: list index out of range` — mavjud bo'lmagan indeks (3 elementli ro'yxatda `[3]`).
- ✗ `KeyError` — dict'da yo'q kalit; `.get()` ishlatilg.
- ✗ `x = ro'yxat.sort()` — `sort()` hech narsa qaytarmaydi (`None`); yangi ro'yxat kerak bo'lsa `sorted()`.
- ✗ Bo'sh set uchun `{}` yozish — bu dict.

O'zingizni tekshiring

? Qaysi tuzilmani yaratilgandan keyin o'zgartirib bo'lmaydi?

- A. list
- B. tuple
- C. set
- D. dict

? `[1, 2, 2, 3, 3]` dan takrorlarni olib tashlashning eng oson usuli?

- A. `sort()`
- B. `tuple()`
- C. `pop()`
- D. `list(set(...))`

? `a = [10, 20, 30, 40]; print(a[-1])` natijasi?

- A. 10
- B. Xato
- C. 40
- D. 30

? `"Salom"[1:4]` natijasi?

- A. "alo"
- B. "Sal"
- C. "alom"
- D. "Salo"

? Telefon kitobi (ism → raqam) uchun eng mos tuzilma?

- A. list
- B. dict
- C. tuple
- D. set

? `[x * 2 for x in range(4)]` natijasi?

- A. [2, 4, 6, 8]
- B. [0, 1, 2, 3]
- C. [0, 2, 4, 6]
- D. 8

Uyga vazifa

1. **Lug'at:** 10 ta inglizcha-o'zbekcha so'zli dict yarating. Foydalanuvchi so'z kiritadi — tarjimasi chiqadi; yo'q bo'lsa, tarjimasini so'rab lug'atga qo'shadi.
2. **Umumiy do'stlar:** ikki kishining do'stlari ro'yxatidan (set) umumiy do'stlarni va faqat birinchisining do'stlarini toping.
3. **Harflar statistikasi:** kiritilgan matnda har bir harf necha marta uchrashini dict bilan hisoblang va eng ko'p uchragan 3 tasini chiqaring.

Yodda tuting

- **list** `[]` — tartibli, o'zgaruvchan: `append`, `remove`, `pop`, `sort`, slicing `[a:b]`, `[-1]`.
- **tuple** `()` — o'zgarmas; unpacking: `a, b = (1, 2)`.
- **set** `{}` — takrorsiz; `&` `|` - amallari.
- **dict** `{"k": v}` — kalit bo'yicha: `d["k"]`, `d.get("k")`, `.items()`, `.keys()`, `.values()`.
- List comprehension: `[ifoda for x in to'plam if shart]`.

✓ Test javoblari

1. Qaysi tuzilmani yaratilgandan keyin o'zgartirib bo'lmaydi? — **B) tuple**
2. [1, 2, 2, 3, 3] dan takrorlarni olib tashlashning eng oson usuli? — **D) list(set(...))**
3. a = [10, 20, 30, 40]; print(a[-1]) natijasi? — **C) 40**
4. "Salom"[1:4] natijasi? — **A) "alo"**
5. Telefon kitobi (ism → raqam) uchun eng mos tuzilma? — **B) dict**
6. [x * 2 for x in range(4)] natijasi? — **C) [0, 2, 4, 6]**



24-MAVZU · 42 TADAN · III BOB. PYTHON VA MA'LUMOTLAR BAZASI

Python funksiyalar, modullar va standart kutubxonalar

def, parametrlar, standart qiymatlar, return, nomlangan argumentlar, lambda, ko'rinish doirasi, docstring; o'z modulini yaratish va import; math, random, datetime, json, os standart kutubxonalari; pip va tashqi paketlar.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Python'da funksiya yaratish (`def`), parametrlar va `return` ;
- Standart qiymatli va nomlangan argumentlarni, `lambda` ni;
- Kodni bir nechta faylga (modullarga) bo'lish va `import` qilishni;
- Tayyor "asboblarni qutisi" — standart kutubxonalarni: `math` , `random` , `datetime` , `json` , `os` ;
- `pip` orqali internetdan qo'shimcha kutubxonalar o'rnatishni.

Funksiyalar

17-mavzuda JavaScript funksiyalarini o'rgandik. G'oya bir xil: **nomlangan, qayta ishlatiladigan kod bloki**. Python'da u `def` (define — aniqlash) bilan yaratiladi.

Python

```
def salomlash(ism):  
    """Berilgan ism bilan salomlashadi.""" # docstring – funksiya hujjati  
    print(f"Assalomu alaykum, {ism}!")  
  
salomlash("Madina")  
salomlash("Jasur")
```

Python

```
def tortburchak_yuzi(eni, boyi):  
    return eni * boyi          # natijani qaytaradi  
  
yuza = tortburchak_yuzi(5, 3)  
print(yuza)                   # 15  
print(tortburchak_yuzi(2, 2) + 10)  # 14
```

JavaScript

```
function qosh(a, b) { return a + b; }
```

```
const qosh = (a, b) => a + b;
```

Python

```
def qosh(a, b):  
    return a + b
```

```
qosh = lambda a, b: a + b
```


Parametrlar turlari

Python

```
# Standart qiymat
def darajaga(son, daraja=2):
    return son ** daraja

print(darajaga(5))          # 25  - daraja=2 (standart)
print(darajaga(2, 10))      # 1024

# Nomlangan (kalitli) argumentlar - tartibni eslash shart emas
def anketa(ism, yosh, shahar="Toshkent"):
    print(f"{ism}, {yosh} yosh, {shahar}")

anketa("Ali", 14)
anketa(yosh=15, ism="Laylo", shahar="Xiva")

# Istalgan sonli argumentlar
def ortacha(*sonlar):          # *sonlar - barcha argumentlar tuple'ga
                                yig'iladi
    return sum(sonlar) / len(sonlar)

print(ortacha(4, 5, 5, 3))    # 4.25

# Bir nechta qiymat qaytarish (aslida tuple)
def min_max(royxat):
    return min(royxat), max(royxat)

eng_kichik, eng_katta = min_max([7, 2, 9, 4])
print(eng_kichik, eng_katta)  # 2 9
```

lambda — kichik, nomsiz funksiya

Python

```
kvadrat = lambda x: x * x
print(kvadrat(7))                # 49

oquvchilar = [("Ali", 85), ("Vali", 92), ("Gani", 78)]
oquvchilar.sort(key=lambda o: o[1], reverse=True)  # ball bo'yicha saralash
print(oquvchilar)                # [('Vali', 92), ('Ali', 85), ('Gani', 78)]
```

`lambda` asosan saralash (`sort(key=...)`), `max(key=...)` kabi joylarda "bir martalik" funksiya sifatida ishlatiladi.

Ko'rinish doirasi (scope)

Python

```
maktab = "5-maktab"                # global o'zgaruvchi

def tanishtir():
    ism = "Laylo"                  # lokal — faqat funksiya ichida
    print(ism, maktab)             # ✅ globalni o'qish mumkin

tanishtir()
# print(ism)                       # ❌ NameError
```



Yaxshi funksiya

Funksiya kerakli ma'lumotni **parametr orqali oladi** va natijani **return bilan qaytaradi**. Global o'zgaruvchilarni o'zgartiruvchi funksiyalar kodni chalkashtiradi. Har bir funksiya — bitta vazifa, nomi — fe'l: `hisobla_ortacha`, `tekshir_parol`.

Modullar — kodni fayllarga bo'lish

Dastur kattalashgani sari hamma narsani bitta faylga yozish noqulay bo'ladi. **Modul** — bu shunchaki `.py` fayl. Undagi funksiyalarni boshqa faylda `import` bilan ishlatish mumkin.



O'xshatish: asboblarni qutisi

Usta har safar bolg'a yasamaydi — tayyor qutidan oladi. Modul — sizning "asboblarni qutingiz": bir marta yozilgan foydali funksiyalar to'plami.

geometriya.py

```
PI = 3.14159

def doira_yuzi(r):
    return PI * r ** 2

def doira_uzunligi(r):
    return 2 * PI * r
```

main.py (xuddi shu papkada)

```
import geometriya                                # butun modulni ulash

print(geometriya.doira_yuzi(5))
print(geometriya.PI)

from geometriya import doira_uzunligi           # faqat kerakli funktsiyani
print(doira_uzunligi(5))

import geometriya as g                           # qisqa taxallus bilan
print(g.doira_yuzi(1))
```

```
if __name__ == "__main__":
```

geometriya.py (oxirida)

```
if __name__ == "__main__":  
    # Bu qism FAQAT fayl to'g'ridan-to'g'ri ishga tushirilganda bajariladi,  
    # boshqa fayl import qilganda – bajarilmaydi. Sinash (test) uchun qulay.  
    print("Sinov:", doira_yuzi(2))
```

Standart kutubxona — "batareyalar bilan birga"

Python bilan birga yuzlab tayyor modul keladi. Eng foydalilari:

math — matematika

Python

```
import math  
print(math.sqrt(144))      # 12.0 – kvadrat ildiz  
print(math.pi)           # 3.141592653589793  
print(math.ceil(4.1))     # 5 – yuqoriga yaxlitlash  
print(math.floor(4.9))    # 4 – pastga  
print(math.factorial(5))  # 120  
print(math.gcd(24, 36))   # 12 – EKUB
```

random — tasodifiylik

Python

```
import random
print(random.randint(1, 6))           # 1..6 (ikkalasi ham kiradi) –
o'yin kubigi
print(random.random())                # 0..1 oralig'idagi kasr
print(random.choice(["tosh", "qaychi", "qog'oz"])) # ro'yxatdan bittasi
print(random.sample(range(1, 50), 6)) # 6 ta takrorlanmas son –
lotereya
royxat = [1, 2, 3, 4, 5]
random.shuffle(royxat)                # aralashtirish
print(royxat)
```

datetime — sana va vaqt

Python

```
from datetime import datetime, date, timedelta

hozir = datetime.now()
print(hozir)                          # 2026-10-06 14:35:12.123456
print(hozir.strftime("%d.%m.%Y %H:%M")) # 06.10.2026 14:35 – formatlash
print(hozir.year, hozir.month, hozir.day)

tugilgan = date(2012, 3, 21)
bugun = date.today()
print("Yashagan kunlar:", (bugun - tugilgan).days)

keyingi_hafta = bugun + timedelta(days=7)
print("Bir haftadan keyin:", keyingi_hafta)
```

json — ma'lumotlarni saqlash va almashish

JSON — dasturlar o'rtasida ma'lumot almashishning eng mashhur formati. Python dict/list ↔ JSON matn.

Python

```
import json

oquvchilar = [{"ism": "Ali", "baho": 5}, {"ism": "Zarina", "baho": 4}]

# Faylga saqlash
with open("oquvchilar.json", "w", encoding="utf-8") as f:
    json.dump(oquvchilar, f, ensure_ascii=False, indent=2)

# Fayldan o'qish
with open("oquvchilar.json", encoding="utf-8") as f:
    qayta = json.load(f)
    print(qayta[1]["ism"])          # Zarina

# Matnga va matndan
matn = json.dumps({"ok": True})    # '{"ok": true}'
obj = json.loads(matn)
```

oquvchilar.json

```
[
  {
    "ism": "Ali",
    "baho": 5
  },
  {
    "ism": "Zarina",
    "baho": 4
  }
]
```



`ensure_ascii=False` — o'zbekcha harflar `'` kabi kodlarga aylanmasdan, o'qiladigan holda saqlanadi. `indent=2` — chiroyli formatlash.

os — fayllar va papkalar

Python

```
import os
print(os.getcwd())           # joriy papka
print(os.listdir("."))       # papkadagi fayllar ro'yxati
print(os.path.exists("oquvchilar.json")) # fayl bormi?
os.makedirs("hisobotlar", exist_ok=True) # papka yaratish
```

pip — tashqi kutubxonalar

Standart kutubxonadan tashqari, PyPI da 500 000 dan ortiq tayyor paket bor. Ular `pip` buyrug'i bilan o'rnatiladi:

Terminal

```
pip install requests      # internetdan ma'lumot olish
pip install django        # 33-mavzuda!
pip list                  # o'rnatilganlar ro'yxati
pip uninstall requests
```

Python

```
import requests
javob = requests.get("https://api.github.com/users/torvalds")
print(javob.json()["name"]) # Linus Torvalds
```



Virtual muhit (virtual environment)

Har bir loyiha uchun alohida "quti" yaratish tavsiya etiladi — shunda bir loyihaning paketlari boshqasiga xalaqit bermaydi. Buni 33-mavzuda Django o'rnatganda batafsil qilamiz: `python -m venv venv`.

Amaliy mashg'ulot

- 1 **O'z modulingiz.** `yordamchi.py` faylida quyidagi funksiyalarni yozing va `main.py` da import qilib sinang:

yordamchi.py

```
def son_sora(matn, eng_kichik=None, eng_katta=None):
    """To'g'ri butun son kiritilguncha so'raydi."""
    while True:
        try:
            son = int(input(matn))
        except ValueError:
            print("  Iltimos, butun son kiriting")
            continue
        if eng_kichik is not None and son < eng_kichik:
            print(f"  Son {eng_kichik} dan kichik bo'lmasin")
        elif eng_katta is not None and son > eng_katta:
            print(f"  Son {eng_katta} dan katta bo'lmasin")
        else:
            return son

def tubmi(n):
    """n tub son bo'lsa True qaytaradi."""
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

def baho(ball):
    if ball >= 86: return 5
    if ball >= 71: return 4
    if ball >= 56: return 3
    return 2

if __name__ == "__main__":
    print([n for n in range(30) if tubmi(n)])
```


- 2 **Lotereya:** `random.sample` bilan 6 ta "yutuqli" son yarating, foydalanuvchidan 6 ta son so'rang (`son_sora` bilan, 1–49) va nechitasi to'g'ri kelganini ayting (set kesishmasi!).
- 3 **Tug'ilgan kun hisoblagichi:** tug'ilgan sanani so'rab, necha kun yashaganingizni va keyingi tug'ilgan kuningizgacha necha kun qolganini `datetime` bilan hisoblang.
- 4 **23-mavzudagi o'quvchilar dasturini** JSON faylga saqlaydigan qiling: dastur ishga tushganda `json.load`, har o'zgarishdan keyin `json.dump`.

Ko'p uchraydigan xatolar

- ❌ Funksiyani e'lon qilishdan **oldin** chaqirish — Python faylni yuqoridan pastga o'qiydi.
- ❌ `return` unutilgan — funksiya `None` qaytaradi.
- ❌ O'z faylingizni standart modul nomi bilan atash: `random.py`, `math.py` — Python sizning faylingizni import qilib, xato beradi!
- ❌ `ModuleNotFoundError: No module named 'requests'` — paket o'rnatilmagan: `pip install requests`.

O'zingizni tekshiring

? Python'da funksiya qaysi kalit so'z bilan yaratiladi?

- A. `function`
- B. `def`
- C. `func`
- D. `fn`

? `def f(a, b=10): return a + b` — `f(5)` natijasi?

- A. 5
- B. Xato
- C. 15
- D. 10

? 1 dan 6 gacha tasodifiy butun son beruvchi kod?

- A. `random.randint(1, 6)`
- B. `random.random(6)`
- C. `math.random(1, 6)`
- D. `randint(6)`

? Python dict'ni JSON faylga yozuvchi funksiya?

- A. `json.load()`
- B. `json.read()`
- C. `file.json()`
- D. `json.dump()`

? Tashqi kutubxonani o'rnatish buyrug'i?

- A. `python get requests`
- B. `pip install requests`
- C. `import install requests`
- D. `npm install requests`

Uyga vazifa

1. **Parol generatori:** `random` va `string` modullari yordamida berilgan uzunlikdagi (harf, raqam, belgi aralash) parol yaratuvchi funksiya.
2. **Kunlik kvota:** "Bugungi maslahat" dasturi — 10 ta maslahatdan biri tasodifiy chiqsin, chiqqanlari JSON faylda saqlansin va bir kunda takrorlanmasin.
3. O'zingizning `matn_yordamchi.py` modulingizni yarating: so'zlar soni, unli harflar soni, palindrom tekshiruvi funksiyalari bilan.

Yodda tuting

- `def nom(param, standart=qiymat): return natija`; nomlangan argumentlar; `*args`; `lambda x: ...`.
- Modul — `.py` fayl: `import modul`, `from modul import f`, `import modul as m`; `if __name__ == "__main__":`.
- Standart kutubxona: `math`, `random`, `datetime`, `json` (`dump/load`), `os`.
- Tashqi paketlar: `pip install nom`.

✓ Test javoblari

1. Python'da funksiya qaysi kalit so'z bilan yaratiladi? — **B) def**
2. `def f(a, b=10): return a + b` — `f(5)` natijasi? — **C) 15**
3. 1 dan 6 gacha tasodifiy butun son beruvchi kod? — **A) `random.randint(1, 6)`**
4. Python dict'ni JSON faylga yozuvchi funksiya? — **D) `json.dump()`**
5. Tashqi kutubxonani o'rnatish buyrug'i? — **B) `pip install requests`**



Obyektga yo'naltirilgan dasturlash (OOP) asoslari

Klass va obyekt, `__init__` va `self`, atributlar va metodlar, `__str__`, inkapsulyatsiya, merosxo'rlik (inheritance), polimorfizm; kutubxona boshqaruv tizimi loyihasi.

2 soat · 1 nazariy + 1 amaliy

 **Bu darsda nimani o'rganasiz**

- **Klass** va **obyekt** nima va ular nima uchun kerakligini;
- `__init__`, `self`, atributlar va metodlarni;
- OOP'ning uch ustuni: **inkapsulyatsiya**, **merosxo'rlik**, **polimorfizm**;
- Kutubxona boshqaruv tizimini (17-loyiha) OOP uslubida yaratishni.

Bu mavzu Django uchun juda muhim: Django'da modellar, formalar va view'lar — hammasi klasslar!

Nima uchun OOP?

Hozirgacha ma'lumotlarni (dict) va ular bilan ishlovchi funksiyalarni **alohida** saqladik. Dastur kattalashganda bu chalkashadi: qaysi funksiya qaysi ma'lumot uchun? **Obyektga yo'naltirilgan dasturlash (OOP)** ma'lumot va unga tegishli amallarni **bitta "obyekt" ichida** birlashtiradi.

 **O'xshatish: avtomobil zavodi**

Klass — avtomobilning **chizmasi (loyihasi)**: unda "rangi bor, tezligi bor, yura oladi, to'xtay oladi" deb yozilgan. Chizmaning o'zi yurmaydi! **Obyekt** — shu chizma asosida yasalgan **haqiqiy mashina**: qizil Cobalt, oq Malibu... Bitta chizmadan minglab mashina yasash mumkin, har birining rangi va probegi o'ziniki, lekin hammasi "yura oladi".

Tushuncha	Ma'nosi	Misol
Klass (class)	Shablon, chizma	<code>0quvchi</code>

Obyekt (object, instance)	Klassdan yaratilgan aniq nusxa	ali, zarina
Atribut (attribute)	Obyektning xususiyati (ma'lumot)	ism, sinf, baholar
Metod (method)	Obyekt bajara oladigan amal (funksiya)	baho_qosh(), ortacha()

Aslida siz OOP bilan allaqachon tanishsiz: `"salom".upper()` — bu `str` klassi obyektining `upper` metodi; `royxat.append(5)` — `list` obyektining metodi.

Birinchi klass

Python

```
class Oquvchi:                                # klass nomi – KattaHarfBilan
(PascalCase)
    def __init__(self, ism, sinf):            # konstruktor: obyekt
        yaratilganda chaqiriladi
        self.ism = ism                        # atributlar
        self.sinf = sinf
        self.baholar = []                    # har bir o'quvchida o'z bo'sh
        ro'yxati

    def baho_qosh(self, baho):                # metod
        self.baholar.append(baho)

    def ortacha(self):
        if not self.baholar:
            return 0
        return round(sum(self.baholar) / len(self.baholar), 2)

    def tanishtir(self):
        print(f"Men {self.ism}, {self.sinf} sinf. O'rtacha bahom:
        {self.ortacha()}")

# Obyektlar yaratish
ali = Oquvchi("Ali", "8-A")
zarina = Oquvchi("Zarina", "8-B")

ali.baho_qosh(5)
ali.baho_qosh(4)
zarina.baho_qosh(5)

ali.tanishtir()
zarina.tanishtir()
print(ali.ism, ali.baholar)
```

Natija

```
Men Ali, 8-A sinf. O'rtacha bahom: 4.5
Men Zarina, 8-B sinf. O'rtacha bahom: 5.0
Ali [5, 4]
```

`__init__` va `self` — eng muhim ikki tushuncha

- `__init__` (initialize — boshlang'ich holatga keltirish) — **konstruktor**. `Oquvchi("Ali", "8-A")` yozilganda avtomatik chaqiriladi va yangi obyektning atributlarini "to'ldiradi".
- `self` — "**shu obyektning o'zi**". `ali.baho_qosh(5)` chaqirilganda, metod ichidagi `self` — `ali` bo'ladi; `zarina.baho_qosh(5)` da — `zarina`. Shuning uchun har bir o'quvchining baholari alohida saqlanadi.



`self` — "men"

Sinfdagi har bir o'quvchi "mening ismim", "mening daftarim" deydi. So'z bir xil — "mening", lekin har bir o'quvchi uchun u boshqa narsani bildiradi. `self.ism` — "mening ismim".



Har bir metodning **birinchi parametri** — `self`. Uni yozishni unutsangiz: `TypeError: takes 1 positional argument but 2 were given`. Chaqirganda esa `self` berilmaydi — Python uni o'zi qo'yadi.

`__str__` — obyektни chiroyli chiqarish

Python

```
class Kitob:
    def __init__(self, nomi, muallif, yil):
        self.nomi = nomi
        self.muallif = muallif
        self.yil = yil

    def __str__(self):
        # print(obyekt) chaqirilganda
        return f"«{self.nomi}» — {self.muallif} ({self.yil})"

k = Kitob("O'tkan kunlar", "Abdulla Qodiriy", 1926)
print(k)          # «O'tkan kunlar» — Abdulla Qodiriy (1926)
# __str__ bo'lmaganda: <__main__.Kitob object at 0x7f...>
```

`__init__`, `__str__` kabi ikki pastki chiziq bilan o'ralgan metodlar — "**maxsus (magic, dunder) metodlar**". Django modellarida `__str__` ni albatta yozamiz (36-mavzu) — admin panelda obyektlar shu nom bilan ko'rinadi.

OOP'ning uch ustuni

1. Inkapsulyatsiya — ichki ishlarni yashirish

Obyekt o'z ma'lumotlarini **himoya qiladi** va ularni faqat **o'z metodlari orqali** o'zgartirishga ruxsat beradi.

```

class BankHisobi:
    def __init__(self, egasi, balans=0):
        self.egasi = egasi
        self._balans = balans          # _ – "ichki, tashqaridan tegmang" degan
kelishuv

    def toldirish(self, summa):
        if summa ≤ 0:
            print("❌ Summa musbat bo'lishi kerak")
            return
        self._balans += summa

    def yechish(self, summa):
        if summa > self._balans:
            print("❌ Mablag' yetarli emas")
            return
        self._balans -= summa

    def balans(self):
        return self._balans

h = BankHisobi("Jasur", 100_000)
h.yechish(500_000)          # ❌ Mablag' yetarli emas
h.toldirish(-50)           # ❌ Summa musbat bo'lishi kerak
h.toldirish(50_000)
print(h.balans())          # 150000

```



O'xshatish: bankomat

Siz bankomatdagi pulni qo'lingiz bilan olmaysiz — faqat tugmalar (metodlar) orqali so'raysiz, bankomat esa qoidalarni tekshiradi (balans yetarlimi, PIN to'g'rimi). Ichki mexanizm yashirin — bu inkapsulyatsiya.

2. Merosxo'rlik (inheritance) — mavjud klassdan yangisini yaratish

Yangi klass boshqa klassning barcha atribut va metodlarini **meros qilib oladi** va o'zinikini qo'shadi.

```

class Odam:                                # ota (asosiy) klass
    def __init__(self, ism, yosh):
        self.ism = ism
        self.yosh = yosh

    def tanishtir(self):
        return f"Men {self.ism}, {self.yosh} yoshdaman"

class Oquvchi(Odam):                        # Odam'dan meros oladi
    def __init__(self, ism, yosh, sinf):
        super().__init__(ism, yosh)        # ota klass konstruktorini
chaqirish
        self.sinf = sinf                  # yangi atribut

    def dars_qil(self):                     # yangi metod
        return f"{self.ism} uy vazifasini bajaryapti 📖"

class Oqituvchi(Odam):
    def __init__(self, ism, yosh, fan):
        super().__init__(ism, yosh)
        self.fan = fan

    def tanishtir(self):                   # ota metodini QAYTA YOZISH
(override)
        return f"Men {self.ism}, {self.fan} fani o'qituvchisiman"

o = Oquvchi("Madina", 14, "8-A")
t = Oqituvchi("Rustam aka", 35, "informatika")
print(o.tanishtir())                     # ota klassdan meros
print(o.dars_qil())                       # o'zining metodi
print(t.tanishtir())                     # qayta yozilgan metod

```

Natija

```

Men Madina, 14 yoshdaman
Madina uy vazifasini bajaryapti 📖
Men Rustam aka, informatika fani o'qituvchisiman

```

Odam

ism, yosh, tanishtir()

→ meros →

Oquvchi

+ sinf, + dars_qil()

Oqituvchi

+ fan, tanishtir() qayta yozilgan

Django'da buni doim ko'rasiz: `class Kitob(models.Model):` — sizning modelingiz Django'ning tayyor `Model` klassidan meros oladi va bazaga saqlash kabi yuzlab imkoniyatlarni "tekinga" oladi.

3. Polimorfizm — bir xil buyruq, turli xatti-harakat

Python

```
class Hayvon:
    def __init__(self, ism):
        self.ism = ism
    def ovoz(self):
        return "... "

class Mushuk(Hayvon):
    def ovoz(self):
        return "Miyov 🐱"

class It(Hayvon):
    def ovoz(self):
        return "Vov-vov 🐶"

class Sigir(Hayvon):
    def ovoz(self):
        return "Mo-o-o 🐮"

ferma = [Mushuk("Mosh"), It("Bobik"), Sigir("Zebo")]
for h in ferma:
    print(f"{h.ism}: {h.ovoiz()}")      # bitta buyruq — har biri o'zicha javob
beradi
```

Polimorfizm (ko'p shakllilik) — turli klass obyektlari bir xil nomli metodga ega bo'lib, har biri uni o'zicha bajaradi. Dastur obyektning aniq turini bilmasa ham ishlay oladi.

Amaliy mashg'ulot: kutubxona boshqaruv tizimi (17-

loyiha)

kutubxona.py

```
from datetime import date, timedelta

class Kitob:
    def __init__(self, isbn, nomi, muallif, nusxa=1):
        self.isbn = isbn
        self.nomi = nomi
        self.muallif = muallif
        self.jami = nusxa
        self.mavjud = nusxa

    def __str__(self):
        return f"[{self.isbn}] «{self.nomi}» – {self.muallif} ({self.mavjud}/{self.jami})"

class Azo:
    MAKS_KITOB = 3 # klass atributi – barcha a'zolar uchun umumiy

    def __init__(self, id, ism):
        self.id = id
        self.ism = ism
        self.kitoblar = {} # isbn → qaytarish muddati

    def __str__(self):
        return f"#{self.id} {self.ism} (qo'lida: {len(self.kitoblar)} ta kitob)"

class Kutubxona:
    IJARA_KUNI = 14
    JARIMA_KUNIGA = 1000

    def __init__(self, nomi):
        self.nomi = nomi
        self.kitoblar = {} # isbn → Kitob
        self.azolar = {} # id → Azo
```

```

def kitob_qosh(self, kitob):
    if kitob.isbn in self.kitoblar:
        mavjud = self.kitoblar[kitob.isbn]
        mavjud.jami += kitob.jami
        mavjud.mavjud += kitob.jami
    else:
        self.kitoblar[kitob.isbn] = kitob

def azo_qosh(self, azo):
    self.azolar[azo.id] = azo

def qidir(self, soz):
    soz = soz.lower()
    return [k for k in self.kitoblar.values()
            if soz in k.nomi.lower() or soz in k.muallif.lower()]

def berish(self, azo_id, isbn, bugun=None):
    bugun = bugun or date.today()
    azo = self.azolar.get(azo_id)
    kitob = self.kitoblar.get(isbn)
    if not azo or not kitob:
        return "❌ A'zo yoki kitob topilmadi"
    if kitob.mavjud == 0:
        return f"❌ «{kitob.nomi}» hozir mavjud emas"
    if len(azo.kitoblar) ≥ Azo.MAKS_KITOB:
        return f"❌ {azo.ism} qo'lida allaqachon {Azo.MAKS_KITOB} ta kitob
bor"

    if isbn in azo.kitoblar:
        return "❌ Bu kitob allaqachon sizda"
    kitob.mavjud -= 1
    muddat = bugun + timedelta(days=self.IJARA_KUNI)
    azo.kitoblar[isbn] = muddat
    return f"✅ «{kitob.nomi}» {azo.ism}ga berildi. Qaytarish:
{muddat:%d.%m.%Y}"

def qaytarish(self, azo_id, isbn, bugun=None):
    bugun = bugun or date.today()
    azo = self.azolar.get(azo_id)
    if not azo or isbn not in azo.kitoblar:
        return "❌ Bunday ijara yo'q"
    muddat = azo.kitoblar.pop(isbn)
    self.kitoblar[isbn].mavjud += 1
    kechikish = (bugun - muddat).days
    if kechikish > 0:

```

```

        return f"⚠️ {kechikish} kun kechikdi. Jarima: {kechikish *
self.JARIMA_KUNIGA:,} so'm"
        return "✅ Kitob o'z vaqtida qaytarildi. Rahmat!"

    def hisobot(self):
        print(f"\n📖 {self.nomi}")
        for k in self.kitoblar.values():
            print("  ", k)
        for a in self.azolar.values():
            print("  ", a)

# ===== Sinov =====
lib = Kutubxona("Maktab kutubxonasi")
lib.kitob_qosh(Kitob("001", "O'tkan kunlar", "Abdulla Qodiriy", 2))
lib.kitob_qosh(Kitob("002", "Sariq devni minib", "Xudoyberdi To'xtaboyev"))
lib.kitob_qosh(Kitob("003", "Shum bola", "G'afur G'ulom"))
lib.azo_qosh(Azo(1, "Aziz"))
lib.azo_qosh(Azo(2, "Nilufar"))

print(lib.berish(1, "002", date(2026, 9, 1)))
print(lib.berish(2, "002")) # mavjud emas
print([str(k) for k in lib.qidir("qodiriy")])
print(lib.qaytarish(1, "002", date(2026, 9, 20))) # 5 kun kechikish
lib.hisobot()

```

Natija

✅ «Sariq devni minib» Azizga berildi. Qaytarish: 15.09.2026
 ❌ «Sariq devni minib» hozir mavjud emas
 ["[001] «O'tkan kunlar» – Abdulla Qodiriy (2/2)"]
 ⚠️ 5 kun kechikdi. Jarima: 5,000 so'm

📖 Maktab kutubxonasi
 [001] «O'tkan kunlar» – Abdulla Qodiriy (2/2)
 [002] «Sariq devni minib» – Xudoyberdi To'xtaboyev (1/1)
 [003] «Shum bola» – G'afur G'ulom (1/1)
 #1 Aziz (qo'lida: 0 ta kitob)
 #2 Nilufar (qo'lida: 0 ta kitob)

Topshiriqlar:

- 1 Kodni qatorma-qator o'qing va har bir klassning vazifasini o'z so'zlaringiz bilan yozing.
- 2 Azo dan meros olgan Oqituvchi klassini yarating: unda MAKS_KITOB = 10. Polimorfizm ishlashini tekshiring.
- 3 Menyu qo'shing (23-mavzudagi kabi): kitob qo'shish, qidirish, berish, qaytarish, hisobot.
- 4 Ma'lumotlarni JSON faylga saqlash va yuklash metodlarini yozing (saqla(), yukla()).

Ko'p uchraydigan xatolar

- ✗ Metodda self parametrini unutish.
- ✗ Metod ichida ism deb yozish (self.ism o'rniga) — `NameError`.
- ✗ `__init__` ni bitta pastki chiziq bilan yozish (`_init_`) — konstruktor ishlamaydi.
- ✗ Merosxo'r klassda `super().__init__(...)` ni chaqirishni unutish — ota klass atributlari yaratilmaydi.
- ✗ Obyekt yaratishda qavsni unutish: `ali = Oquvchi` — bu klassning o'zi, obyekt emas.

O'zingizni tekshiring

? **Klass va obyekt o'rtasidagi munosabat qaysi?**

- A. Obyekt — chizma, klass — mahsulot
- B. Klass — chizma (shablon), obyekt — undan yaratilgan nusxa
- C. Ular bir xil narsa
- D. Klass — funksiya, obyekt — o'zgaruvchi

? Obyekt yaratilganda avtomatik chaqiriladigan metod?

- A. `__str__`
- B. `__main__`
- C. `__init__`
- D. `create`

? `self` nimani bildiradi?

- A. Metod chaqirilayotgan obyektning o'zini
- B. Klass nomini
- C. Global o'zgaruvchini
- D. Ota klassni

? `class It(Hayvon):` — bu qaysi OOP tamoyili?

- A. Inkapsulyatsiya
- B. Polimorfizm
- C. Abstraksiya
- D. Merosxo'rlik

? `print(obyekt)` qanday matn chiqarishini qaysi metod belgilaydi?

- A. `__init__`
- B. `__str__`
- C. `__print__`
- D. `show()`

? Turli klasslar bir xil nomli metodni o'zicha bajarishi qanday ataladi?

- A. Inkapsulyatsiya
- B. Merosxo'rlik
- C. Polimorfizm
- D. Kompilyatsiya



Uyga vazifa

1. **Geometrik shakllar:** `Shakl` ota klassi va undan meros olgan `Doira`, `Tortburchak`, `Uchburchak` — har birida `yuza()` va `perimetr()`. Shakllar ro'yxatini aylanib, yuzalarni chiqaring (polimorfizm).
2. **Internet do'kon savati:** `Mahsulot` va `Savat` klasslari: qo'shish, o'chirish, jami summa, chegirma qo'llash.
3. **Sinf jurnali:** `Sinf` klassi ichida `Oquvchi` obyektlari ro'yxati; eng yaxshi o'quvchi, sinf o'rtachasi metodlari.



Yodda tuting

- `class Nom:` — chizma; `obj = Nom(...)` — obyekt.
- `def __init__(self, ...):` — konstruktor; `self.attribut` — obyekt ma'lumoti; metodlar birinchi parametri — `self`.
- `__str__` — obyektning matn ko'rinishi.
- **Inkapsulyatsiya** — ma'lumotni metodlar orqali himoyalash; **merosxo'rlik** — `class B(A):` + `super().__init__()`; **polimorfizm** — bir xil metod, turli xatti-harakat.

✓ Test javoblari

1. Klass va obyekt o'rtasidagi munosabat qaysi? — **B) Klass — chizma (shablon), obyekt — undan yaratilgan nusxa**
2. Obyekt yaratilganda avtomatik chaqiriladigan metod? — **C) __init__**
3. self nimani bildiradi? — **A) Metod chaqirilayotgan obyektning o'zini**
4. class It(Hayvon): — bu qaysi OOP tamoyili? — **D) Merosxo'rlik**
5. print(obyekt) qanday matn chiqarishini qaysi metod belgilaydi? — **B) __str__**
6. Turli klasslar bir xil nomli metodni o'zicha bajarishi qanday ataladi? — **C) Polimorfizm**



26-MAVZU · 42 TADAN · III BOB. PYTHON VA MA'LUMOTLAR BAZASI

Ma'lumotlar bazasiga kirish. Relyatsion ma'lumotlar bazasi tushunchasi

Nima uchun fayllar yetarli emas, ma'lumotlar bazasi va DBMS, jadval, ustun, qator, birlamchi kalit, tashqi kalit, bog'lanish turlari (1:1, 1:N, N:M), SQL nima, relyatsion va NoSQL bazalar.

1 soat · nazariy

Bu darsda nimani o'rganasiz

- Nima uchun ma'lumotlarni fayllarda emas, **ma'lumotlar bazasida** saqlash kerakligini;
- **Jadval, ustun, qator** tushunchalarini;
- **Birlamchi kalit** (Primary Key) va **tashqi kalit** (Foreign Key) nima ekanini;
- Jadvallar o'rtasidagi **bog'lanish** turlarini;
- SQL tili nima ekanini va relyatsion bazalar NoSQL'dan qanday farq qilishini.

Nima uchun ma'lumotlar bazasi kerak?

22–25-mavzularda ma'lumotlarni `.txt` va `.json` fayllarda saqladik. Kichik dastur uchun bu yetarli. Lekin tasavvur qiling: maktabingizdagi **2000 o'quvchi**, ularning baholari, davomati, ota-onalari, 100 o'qituvchi... Fayl bilan quyidagi muammolar paydo bo'ladi:

- 🐢 **Sekinlik**: "8-sinflar ichida matematikadan 5 olganlar" ni topish uchun butun faylni boshidan oxirigacha o'qish kerak.
- ⚡ **Bir vaqtda yozish**: ikki o'qituvchi bir vaqtda baho qo'ysa, biri ikkinchisining yozuvini o'chirib yuborishi mumkin.
- 🤖 **Tartibsizlik**: kimdir yoshni "o'n to'rt" deb, kimdir 14 deb yozadi; baho 7 bo'lib qoladi. Fayl buni tekshirmaydi.
- 🔄 **Takrorlanish**: o'quvchining manzili har bir baho yozuvida qayta yoziladi; manzil o'zgarsa — hammasini topib tuzatish kerak.

-  **Xavfsizlik va zaxira:** kim nimani ko'ra olishi, nusxa olish, qayta tiklash — barchasini qo'lda qilish kerak.

Ma'lumotlar bazasi (database) — ma'lumotlarni **tartibli** saqlash, tez **qidirish**, **xavfsiz** va **to'g'ri** boshqarish uchun mo'ljallangan tizim. Uni boshqaruvchi dastur **DBMS** (Database Management System — MBBT, ma'lumotlar bazasini boshqarish tizimi) deyiladi: PostgreSQL, MySQL, SQLite, Oracle.



O'xshatish: kutubxona

Fayl — kitoblar tartibsiz uyilgan xona: kerakli kitobni topish uchun hammasini titkilaysiz.

Ma'lumotlar bazasi — zamonaviy kutubxona: kitoblar javonlarda (jadvallar), har birining raqami bor (kalit), katalog bor (indeks), kutubxonachi (DBMS) qoidalarga rioya qilinishini nazorat qiladi va kitobni bir zumda topib beradi.

Siz har kuni ma'lumotlar bazalaridan foydalanasiz: Telegram xabarlaringiz, Instagram postlari, bank kartangiz balansi, maktabning elektron kundaligi — hammasi bazada.

Relyatsion ma'lumotlar bazasi

Eng keng tarqalgan baza turi — **relyatsion** (relation — munosabat, bog'lanish). Unda ma'lumotlar **jadvallarda** saqlanadi va jadvallar o'zaro **bog'lanadi**.

Jadval, ustun, qator

oquvchilar jadvali:

id	ism	familiya	tugilgan_sana	sinf_id
1	Ali	Valiyev	2012-03-15	1
2	Zarina	Karimova	2012-07-02	1
3	Bobur	Toshmatov	2011-11-20	2

Atama	Boshqa nomi	Ma'nosi	Misolda
Jadval (table)	munosabat	Bir turdagi obyektlar to'plami	oquvchilar

Ustun (column)	maydon, atribut	Xususiyat; har bir ustunning ma'lumot turi bor	<code>ism</code> (matn), <code>tugilgan_sana</code> (sana)
Qator (row)	yozuv, record	Bitta obyekt haqidagi ma'lumot	Ali Valiyev haqidagi qator
Katak	qiymat	Qator va ustun kesishmasi	"Zarina"

Excel jadvaliga o'xshaydi, lekin muhim farqlar bor: har bir ustunda **faqat bir turdagi** ma'lumot bo'ladi (sana ustuniga matn yozib bo'lmaydi), baza **qoidalarni majburiy tekshiradi** va millionlab qatorlar bilan tez ishlaydi.

Kalitlar

Birlamchi kalit — Primary Key (PK)

Har bir qatorni **noyob** aniqlovchi ustun. Odatda `id` deb nomlanadi va avtomatik raqamlanadi (1, 2, 3...).

- **Noyob:** ikki qatorda bir xil bo'lishi mumkin emas.
- **Bo'sh bo'lmaydi** (NULL emas).
- **O'zgarmaydi.**



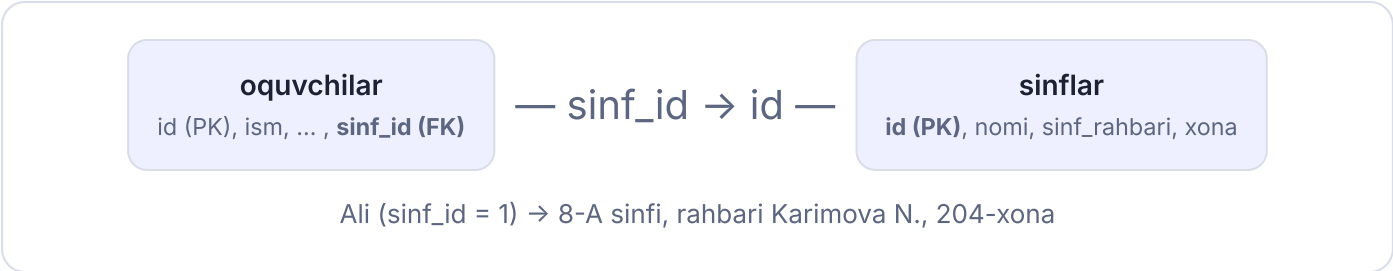
Nima uchun ism yetarli emas?

Maktabda ikkita "Ali Valiyev" bo'lishi mumkin. Lekin ularning JSHSHIR (shaxsiy raqami) yoki pasport raqami har xil. Birlamchi kalit — qatorning "pasport raqami".

Tashqi kalit — Foreign Key (FK)

Boshqa jadvalning birlamchi kalitiga **havola** qiluvchi ustun. Yuqoridagi `sinf_id` — tashqi kalit, u **sinflar** jadvaliga ishora qiladi:

id (PK)	nomi	sinf_rahbari	xona
1	8-A	Karimova N.	204
2	8-B	Rasulov A.	206



Nima uchun alohida jadval? Agar sinf rahbari ismi har bir o'quvchi qatorida yozilsa va rahbar almasha — 30 ta qatorni tuzatish kerak (va birortasini unutish oson). Alohida jadvalda esa — **bitta qator**. Bu tamoyil **"bitta fakt — bitta joyda"** deyiladi (normalizatsiya).

Baza tashqi kalitni **nazorat qiladi**: mavjud bo'lmagan `sinf_id = 99` ni yozishga yoki o'quvchilari bor sinfni o'chirishga ruxsat bermaydi. Bu **ma'lumotlar yaxlitligi** (data integrity) deyiladi — 29-mavzuda batafsil.

Bog'lanish turlari

Tur	Ma'nosi	Misol	Qanday amalga oshiriladi
Birga-bir (1:1)	Bir qatorga boshqa jadvaldan faqat bitta qator mos	O'quvchi ↔ uning shaxsiy kartochkasi	FK + noyoblik
Birga-ko'p (1:N)	Bir qatorga ko'p qator mos	Bir sinfda ko'p o'quvchi; bir muallifning ko'p kitobi	"Ko'p" tomonda FK (<code>oquvchilar.sinf_id</code>)
Ko'pga-ko'p (N:M)	Ikki tomonda ham ko'p	O'quvchi ko'p to'garakka qatnashadi, to'garakda ko'p o'quvchi	Oraliq jadval: <code>azoliklar(oquvchi_id, togarak_id)</code>

Ko'pga-ko'p misol:

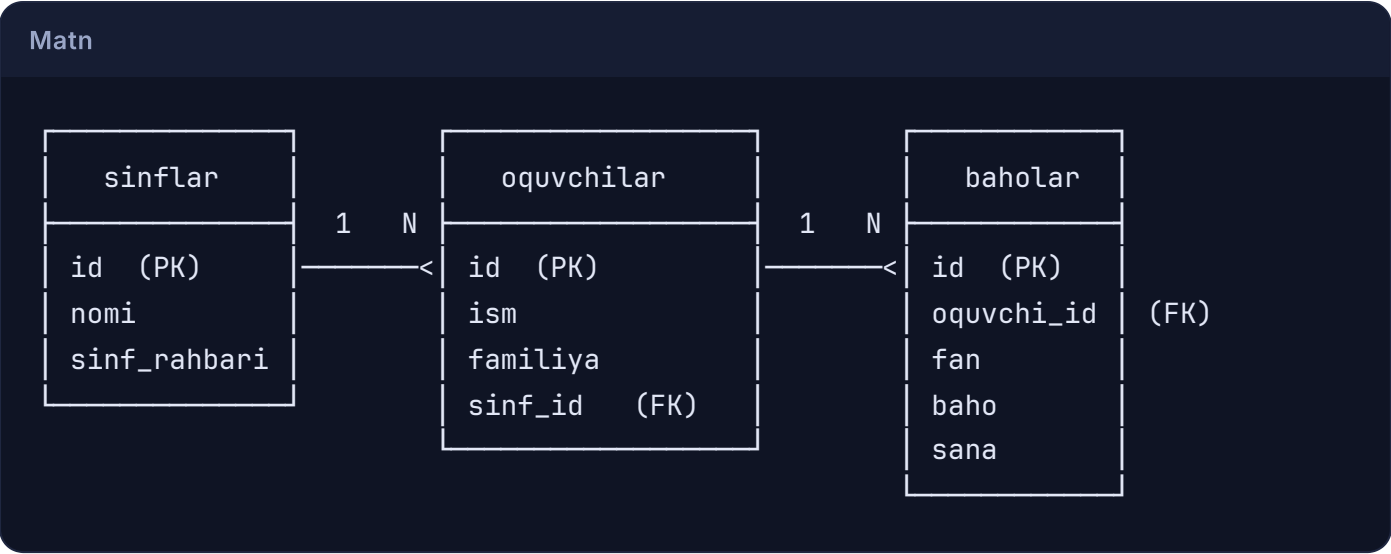
togaraklar		azoliklar (oraliq)	
id	nomi	oquvchi_id	togarak_id

1	Shaxmat	1 (Ali)	1 (Shaxmat)
2	Robototexnika	1 (Ali)	2 (Robototexnika)
		2 (Zarina)	1 (Shaxmat)

Ali ikkala to'garakda, Shaxmatda esa Ali va Zarina bor.

ER-diagramma — bazani loyihalash

Baza yaratishdan oldin uning tuzilmasi chiziladi — **ER-diagramma** (Entity-Relationship: mavjudot-bog'lanish):



Loyihalash qadamlari: (1) qanday **mavjudotlar** bor (o'quvchi, sinf, baho); (2) har birining **xususiyatlari**; (3) **kalitlar**; (4) **bog'lanishlar** va ularning turi. Bunday diagrammalarni dbdiagram.io saytida chizish qulay.

SQL — ma'lumotlar bazasi tili

SQL (Structured Query Language) — relyatsion bazalar bilan "gaplashish" tili. U ingliz tiliga juda yaqin:

SQL

```
-- 8-A sinfidagi o'quvchilarning ism va familiyalarini alifbo tartibida ber
SELECT ism, familiya
FROM oquvchilar
WHERE sinf_id = 1
ORDER BY familiya;
```

Ko'rib turganingizdek, SQL'da "**qanday** qidirish"ni emas, "**nima** kerak"ligini yozasiz — qolganini DBMS o'zi hal qiladi. Keyingi 4 ta mavzuda SQL'ni chuqur o'rganamiz.

SQL buyrug'i	Vazifasi	Mavzu
CREATE TABLE	Jadval yaratish	27
INSERT , SELECT , UPDATE , DELETE	Qo'shish, o'qish, yangilash, o'chirish (CRUD)	28
JOIN	Jadvallarni birlashtirish	29
WHERE , GROUP BY , COUNT , AVG	Filtrlash va statistika	30

Relyatsion va NoSQL bazalar

	Relyatsion (SQL)	NoSQL
Misollar	PostgreSQL, MySQL, SQLite, Oracle	MongoDB, Redis, Firebase
Tuzilma	Qat'iy jadvallar va ustunlar	Moslashuvchan hujjatlar (JSON'ga o'xshash)
Bog'lanishlar	Kuchli (kalitlar, JOIN)	Cheklangan
Qachon	Aksariyat ilovalar: maktab, bank, do'kon, ijtimoiy tarmoq	Tuzilmasi tez o'zgaradigan ma'lumotlar, kesh, real vaqt

Bu kursda **PostgreSQL** bilan ishlaymiz: u bepul, ochiq kodli, juda ishonchli va Django bilan eng yaxshi ishlaydigan baza.

Amaliy mashg'ulot (qog'ozda yoki dbdiagram.io'da)

- 1

Kutubxona bazasi. Mavjudotlar: kitoblar, mualliflar, o'quvchilar, ijaralar. Har biri uchun ustunlarni yozing, PK va FK'larni belgilang. Savol: kitob va muallif o'rtasida qanday bog'lanish? (Bir kitobning bir nechta muallifi bo'lishi mumkinmi?)
- 2

Internet-do'kon. Mijozlar, mahsulotlar, toifalar, buyurtmalar. Buyurtma va mahsulot o'rtasidagi bog'lanish qanday? Qanday oraliq jadval kerak?
- 3

Xatoni toping. Quyidagi jadvalda qanday muammolar bor?

ism	sinf	sinf_rahbari	togaraklar
Ali	8-A	Karimova N.	shaxmat, robototexnika
Zarina	8-A	Karimova N	shaxmat
Ali	8-B	Rasulov A.	

▼ Javob

(1) Birlamchi kalit yo'q — ikki "Ali"ni ajratib bo'lmaydi. (2) Sinf rahbari takrorlanadi va har xil yozilgan ("Karimova N." va "Karimova N") — **sinf**lar jadvali kerak. (3) Bir katakda bir nechta to'garak — qidirish qiyin; **togaraklar** va **azoliklar** jadvallari kerak (N:M).

O'zingizni tekshiring

?

Jadvaldagi har bir qatorni noyob aniqlovchi ustun nima deyiladi?

A.

Tashqi kalit

B.

Indeks

C.

Birlamchi kalit (Primary Key)

D.

Ustun nomi

? `oquvchilar.sinf_id` ustuni `sinfLAR.id` ga ishora qiladi. U qanday kalit?

- A. Birlamchi kalit
- B. Tashqi kalit (Foreign Key)
- C. Noyob kalit
- D. Parol

? "O'quvchi ko'p to'garakka, to'garakda ko'p o'quvchi" — bu qanday bog'lanish?

- A. 1:1
- B. 1:N
- C. N:1
- D. N:M (ko'pga-ko'p)

? Ko'pga-ko'p bog'lanish qanday amalga oshiriladi?

- A. Oraliq (bog'lovchi) jadval orqali
- B. Bitta katakka vergul bilan yozib
- C. Ikki jadvalni bitta qilib
- D. Faqat NoSQL'da mumkin

? Jadvaldagi bitta yozuv (masalan, bitta o'quvchi haqidagi ma'lumot) nima deyiladi?

- A. Ustun
- B. Kalit
- C. Qator (row)
- D. Sxema

? SQL nima?

- A. Ma'lumotlar bazasi dasturi
- B. Relyatsion bazalar bilan ishlash tili
- C. Python kutubxonasi
- D. Hosting turi



Uyga vazifa

1. Maktab elektron kundaligi uchun ER-diagramma chizing: o'quvchilar, sinflar, fanlar, o'qituvchilar, baholar, dars jadvali. Barcha PK, FK va bog'lanish turlarini ko'rsating.
2. Sevimli 2 ta ilovangiz (masalan, Telegram, YouTube) qanday jadvallarda ma'lumot saqlashi mumkinligini taxmin qilib yozing.



Yodda tuting

- Ma'lumotlar bazasi — tartibli, tez, xavfsiz saqlash; DBMS — uni boshqaruvchi dastur (PostgreSQL).
- Relyatsion baza: jadval (table) → ustun (column, turi bor) → qator (row).
- PK — qatorning noyob identifikatori; FK — boshqa jadval PK'siga havola.
- Bog'lanishlar: 1:1, 1:N (FK "ko'p" tomonda), N:M (oraliq jadval).
- "Bitta fakt — bitta joyda"; SQL — bazalar tili.

✓ Test javoblari

1. Jadvaldagi har bir qatorni noyob aniqlovchi ustun nima deyiladi? — **C) Birlamchi kalit (Primary Key)**
2. oquvchilar.sinf_id ustuni sinflar.id ga ishora qiladi. U qanday kalit? — **B) Tashqi kalit (Foreign Key)**
3. "O'quvchi ko'p to'garakka, to'garakda ko'p o'quvchi" — bu qanday bog'lanish? — **D) N:M (ko'pga-ko'p)**
4. Ko'pga-ko'p bog'lanish qanday amalga oshiriladi? — **A) Oraliq (bog'lovchi) jadval orqali**
5. Jadvaldagi bitta yozuv (masalan, bitta o'quvchi haqidagi ma'lumot) nima deyiladi? — **C) Qator (row)**
6. SQL nima? — **B) Relyatsion bazalar bilan ishlash tili**



PostgreSQL bilan tanishish. Jadval yaratish va ma'lumot turlari

PostgreSQL va pgAdmin'ni o'rnatish, psql konsoli, ma'lumotlar bazasi yaratish, CREATE TABLE, asosiy ma'lumot turlari (INTEGER, NUMERIC, VARCHAR, TEXT, BOOLEAN, DATE, TIMESTAMP), SERIAL, ALTER TABLE va DROP TABLE.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- PostgreSQL va uning grafik dasturi pgAdmin'ni o'rnatishni;
- Yangi ma'lumotlar bazasi yaratishni;
- `CREATE TABLE` bilan jadval yaratish va ustun turlarini to'g'ri tanlashni;
- Jadval tuzilmasini o'zgartirish (`ALTER`) va o'chirishni (`DROP`).

PostgreSQL nima?

PostgreSQL (qisqacha "Postgres", belgisi — fil ) — dunyodagi eng ishonchli va kuchli **bepul, ochiq kodli** relyatsion DBMS. Uni Apple, Instagram, Spotify, Reddit kabi kompaniyalar ishlatadi. U 35 yildan ortiq rivojlanib kelmoqda.

PostgreSQL **server** sifatida ishlaydi: kompyuteringizda fonda yoqiq turadi va dasturlar (pgAdmin, Python, Django) unga **ulanib**, SQL so'rovlar yuboradi.



O'rnatish

- 1 postgresql.org/download → operatsion tizimingizni tanlang → **Download the installer** (EDB) → eng so'nggi versiya.
- 2 O'rnatish oynasida barcha komponentlarni qoldiring: **PostgreSQL Server, pgAdmin 4, Command Line Tools**.
- 3 **Parol** so'raladi — bu `postgres` (bosh administrator) foydalanuvchisi paroli. **Uni yozib qo'ying!** Unutsangiz, qayta o'rnatishga to'g'ri keladi.
- 4 Port: `5432` (standart) — o'zgartirmang.
- 5 O'rnatish tugagach, "Stack Builder" ni ishga tushirish shart emas.



O'rnatib bo'lmasa

Maktab kompyuterida ruxsat bo'lmasa, brauzerda ishlaydigan bepul onlayn muharrirlardan foydalaning: db-fiddle.com (PostgreSQL'ni tanlang) yoki onecompiler.com/postgresql. Bu darsdagi barcha SQL'lar ularda ishlaydi.

pgAdmin bilan ishlash

- 1 pgAdmin 4 ni oching (birinchi marta pgAdmin uchun "master parol" so'rashi mumkin).
- 2 Chap panelda **Servers** → **PostgreSQL** ni oching — o'rnatishdagi parolni kiriting.
- 3 **Databases** ustida o'ng tugma → **Create** → **Database...** → nom: `maktab` → **Save**.
- 4 `maktab` bazasini tanlang va yuqoridagi **Query Tool** (↗ yoki SQL belgisi) tugmasini bosing — SQL yozish oynasi ochiladi.
- 5 SQL yozib, `F5` (yoki ▶ Execute) bilan bajaring. Natija pastki panelda ko'rinadi.

psql — buyruqlar qatori orqali

Terminal

```
psql -U postgres # ulanish (parol so'raladi)
```


SQL

```
CREATE DATABASE maktab;           -- baza yaratish
\l                                -- barcha bazalar ro'yxati
\c maktab                         -- maktab bazasiga o'tish
\dt                               -- jadvallar ro'yxati
\d oquvchilar                     -- jadval tuzilmasini ko'rish
\q                                -- chiqish
```



SQL yozish qoidalar

- Har bir buyruq `;` (nuqta-vergul) bilan tugaydi.
- SQL kalit so'zlari katta-kichik harfni farqlamaydi (`select` = `SELECT`), lekin **kalit so'zlarni KATTA**, nomlarni kichik yozish odat.
- Matnlar **bitta qo'shtirnoqda**: `'Ali'`. O'zbekcha apostrof ikkilantiriladi: `'O''zbekiston'`.
- Izoh: `-- bir qatorli` yoki `/* ko'p qatorli */`.
- Jadval va ustun nomlari: kichik lotin harflar va pastki chiziq — `oquvchilar`, `tugilgan_sana`.

Ma'lumot turlari

Jadval yaratishda har bir ustunning **turini** ko'rsatish shart. To'g'ri tanlangan tur xatolarning oldini oladi va joyini tejaydi.

Tur	Nima uchun	Misol qiymat
<code>INTEGER</code> (<code>INT</code>)	Butun son (± 2 mlrd gacha)	<code>14</code> , <code>-5</code>
<code>SMALLINT</code>	Kichik butun son ($\pm 32\,767$)	baho: <code>5</code>
<code>BIGINT</code>	Juda katta butun son	telefon raqami yoki katta id
<code>SERIAL</code>	Avtomatik o'suvchi butun son (1, 2, 3...) — id uchun	—

<code>NUMERIC(10, 2)</code>	Aniq o'nlik son — pul uchun (10 xona, shundan 2 tasi kasr)	<code>12500.50</code>
<code>REAL</code> , <code>DOUBLE PRECISION</code>	Taqribiy kasr son (ilmiy hisoblar)	<code>3.14159</code>
<code>VARCHAR(n)</code>	Uzunligi n gacha bo'lgan matn	<code>VARCHAR(50)</code> — ism
<code>TEXT</code>	Cheksiz uzunlikdagi matn	maqola, izoh
<code>BOOLEAN</code>	Rost / yolg'on	<code>TRUE</code> , <code>FALSE</code>
<code>DATE</code>	Sana	<code>'2012-03-15'</code> (yil-oy-kun)
<code>TIME</code>	Vaqt	<code>'08:30:00'</code>
<code>TIMESTAMP</code>	Sana va vaqt	<code>'2026-10-06 14:30:00'</code>



Turni tanlashdagi muhim qoidalar

- **Pul** uchun `REAL` emas, `NUMERIC` — kasr sonlar kompyuterda aniq saqlanmaydi ($0.1 + 0.2 \neq 0.3$!).
- **Telefon raqami** uchun `INTEGER` emas, `VARCHAR(20)` — unda `+` belgisi va boshidagi 0 bo'lishi mumkin, u bilan hisob-kitob qilinmaydi.
- **Sanani** matn sifatida saqlamang — `DATE` bilan sanalarni solishtirish va hisoblash mumkin.

Jadval yaratish: CREATE TABLE

SQL

```
CREATE TABLE sinflar (  
  id          SERIAL PRIMARY KEY,  
  nomi        VARCHAR(10) NOT NULL UNIQUE,  
  sinf_rahbari VARCHAR(100),  
  xona        SMALLINT  
);  
  
CREATE TABLE oquvchilar (  
  id          SERIAL PRIMARY KEY,          -- avtomatik id,  
  birlamchi kalit  
  ism         VARCHAR(50) NOT NULL,        -- bo'sh bo'lishi mumkin  
  emas  
  familiya    VARCHAR(50) NOT NULL,  
  tugilgan_sana DATE,  
  telefon     VARCHAR(20),  
  email       VARCHAR(100) UNIQUE,         -- takrorlanmasin  
  alochi      BOOLEAN DEFAULT FALSE,       -- standart qiymat  
  sinf_id     INTEGER REFERENCES sinflar(id), -- tashqi kalit!  
  yaratilgan  TIMESTAMP DEFAULT NOW()      -- qo'shilgan vaqt  
  avtomatik  
);
```

Cheklovlar (constraints)

Cheklov	Ma'nosi
PRIMARY KEY	Birlamchi kalit: noyob va bo'sh emas
NOT NULL	Majburiy maydon — bo'sh qoldirib bo'lmaydi
UNIQUE	Takrorlanmas qiymat (ikki o'quvchida bir xil email bo'lmaydi)
DEFAULT qiymat	Qiymat berilmasa — shu qo'yiladi
CHECK (shart)	Qiymat shartga mos bo'lishi kerak: CHECK (baho BETWEEN 2 AND 5)

SQL

```
CREATE TABLE baholar (  
    id          SERIAL PRIMARY KEY,  
    oquvchi_id  INTEGER NOT NULL REFERENCES oquvchilar(id) ON DELETE CASCADE,  
    fan         VARCHAR(50) NOT NULL,  
    baho        SMALLINT NOT NULL CHECK (baho BETWEEN 2 AND 5),  
    sana        DATE NOT NULL DEFAULT CURRENT_DATE  
);
```

ON DELETE CASCADE — o'quvchi o'chirilsa, uning barcha baholari ham avtomatik o'chadi (yetim baholar qolmaydi).



Jadvallar **tartib** bilan yaratiladi: avval bog'lanish ko'rsatiladigan (`sinf`lar), keyin unga ishora qiluvchi (`oquvchilar`), keyin `baholar`. Aks holda "relation does not exist" xatosi chiqadi.

Jadvalni o'zgartirish va o'chirish

SQL

```
-- Yangi ustun qo'shish
ALTER TABLE oquvchilar ADD COLUMN manzil TEXT;

-- Ustun nomini o'zgartirish
ALTER TABLE oquvchilar RENAME COLUMN manzil TO yashash_joyi;

-- Ustun turini o'zgartirish
ALTER TABLE oquvchilar ALTER COLUMN telefon TYPE VARCHAR(30);

-- Ustunni o'chirish
ALTER TABLE oquvchilar DROP COLUMN yashash_joyi;

-- Jadval nomini o'zgartirish (umumiy ko'rinishi)
-- ALTER TABLE eski_nom RENAME TO yangi_nom;

-- Jadvalni BUTUNLAY o'chirish (ehtiyot bo'ling — qaytarib bo'lmaydi!)
DROP TABLE IF EXISTS vaqtinchalik;
```



`DROP TABLE` va `DROP DATABASE` jadval/bazani **barcha ma'lumotlari bilan birga** o'chiradi.

"Ctrl+Z" yo'q! Bajarishdan oldin ikki marta o'ylang (va zaxira nusxangiz borligiga ishonch hosil qiling — 29-mavzu).

Amaliy mashg'ulot

- 1 PostgreSQL va pgAdmin'ni o'rnatish (yoki db-fiddle.com'ni oching).
- 2 `maktab` bazasini yarating.
- 3 Yuqoridagi `sinflar`, `oquvchilar`, `baholar` jadvallarini yarating. pgAdmin'da **Schemas** → **public** → **Tables** ni yangilab (`F5`), jadvallarni ko'ring.
- 4 `oquvchilar` jadvali ustida o'ng tugma → **Properties** → **Columns**: ustunlar va turlarini tekshiring.
- 5 **Mustaqil**: `togaraklar` (`id`, `nomi`, `rahbar`, `joylar_soni` — musbat bo'lsin) va ko'pga-ko'p uchun `azoliklar` (`oquvchi_id`, `togarak_id`, `qoshilgan_sana`) jadvallarini yarating. `azoliklar` da bitta

o'quvchi bir to'garakka ikki marta yozilmasligi uchun `PRIMARY KEY (oquvchi_id, togarak_id)` qo'llang.

6 oquvchilar jadvaliga jinsi ustunini qo'shing: faqat 'o'g'il' yoki 'qiz' bo'lishi mumkin (`CHECK ... IN (...)`).

▼ 5 va 6-topshiriqlar javobi

SQL

```
CREATE TABLE togaraklar (  
    id SERIAL PRIMARY KEY,  
    nomi VARCHAR(100) NOT NULL UNIQUE,  
    rahbar VARCHAR(100),  
    joylar_soni INTEGER CHECK (joylar_soni > 0)  
);  
  
CREATE TABLE azoliklar (  
    oquvchi_id INTEGER REFERENCES oquvchilar(id) ON DELETE CASCADE,  
    togarak_id INTEGER REFERENCES togaraklar(id) ON DELETE CASCADE,  
    qoshilgan_sana DATE DEFAULT CURRENT_DATE,  
    PRIMARY KEY (oquvchi_id, togarak_id)  
);  
  
ALTER TABLE oquvchilar ADD COLUMN jinsi VARCHAR(10) CHECK (jinsi IN  
('o'g'il', 'qiz'));
```

Ko'p uchraydigan xatolar

- ✗ `relation "oquvchilar" does not exist` — jadval yaratilmagan yoki boshqa bazaga ulangansiz.
- ✗ `syntax error at or near ")"` — oxirgi ustundan keyin ortiqcha vergul qoldirilgan.
- ✗ Matnni qo'sh qo'shtirnoqda yozish: `"Ali"` — SQL'da qo'sh qo'shtirnoq ustun nomlari uchun; matn — `'Ali'`.
- ✗ `password authentication failed` — o'rnatishdagi parol noto'g'ri kiritilgan.

O'zingizni tekshiring

? Pul summasini saqlash uchun eng to'g'ri tur?

- A. `REAL`
- B. `VARCHAR`
- C. `NUMERIC(12, 2)`
- D. `BOOLEAN`

? Avtomatik 1, 2, 3... deb o'suvchi id ustuni uchun tur?

- A. `SERIAL`
- B. `AUTO`
- C. `COUNTER`
- D. `TEXT`

? Ustunni majburiy (bo'sh bo'lmaydigan) qiluvchi cheklov?

- A. `UNIQUE`
- B. `DEFAULT`
- C. `CHECK`
- D. `NOT NULL`

? Mavjud jadvalga yangi ustun qo'shuvchi buyruq?

- A. `UPDATE TABLE ... ADD`
- B. `ALTER TABLE ... ADD COLUMN`
- C. `INSERT COLUMN`
- D. `CREATE COLUMN`

? Telefon raqami (+998901234567) uchun eng mos tur?

- A. `INTEGER`
- B. `NUMERIC`
- C. `VARCHAR(20)`
- D. `DATE`

? SQL'da matn qiymati qanday yoziladi?

- A. `'Ali'`
- B. `"Ali"`
- C. ``Ali``
- D. `Ali`



Uyga vazifa

- `dokon` bazasini yarating va jadvallarni tuzing: `toifalar`, `mahsulotlar` (narx — NUMERIC, soni — 0 dan kichik emas, `toifa_id` — FK), `mijozlar` (email UNIQUE).
- 26-mavzudagi kutubxona ER-diagrammangiz asosida haqiqiy jadvallarni SQL'da yarating.

✓ Yodda tuting

- PostgreSQL — server (port 5432); pgAdmin — grafik dastur; psql — konsol.
- `CREATE DATABASE nom;` , `CREATE TABLE nom (ustun TUR cheklovlar, ...);`
- Turlar: `SERIAL` , `INTEGER` , `NUMERIC` , `VARCHAR(n)` , `TEXT` , `BOOLEAN` , `DATE` , `TIMESTAMP` .
- Cheklovlar: `PRIMARY KEY` , `NOT NULL` , `UNIQUE` , `DEFAULT` , `CHECK` , `REFERENCES` .
- `ALTER TABLE` — o'zgartirish; `DROP TABLE` — o'chirish (qaytarib bo'lmaydi!).

Test javoblari

1. Pul summasini saqlash uchun eng to'g'ri tur? — **C) NUMERIC(12, 2)**
2. Avtomatik 1, 2, 3... deb o'suvchi id ustuni uchun tur? — **A) SERIAL**
3. Ustunni majburiy (bo'sh bo'lmaydigan) qiluvchi cheklov? — **D) NOT NULL**
4. Mavjud jadvalga yangi ustun qo'shuvchi buyruq? — **B) ALTER TABLE ... ADD COLUMN**
5. Telefon raqami (+998901234567) uchun eng mos tur? — **C) VARCHAR(20)**
6. SQL'da matn qiymati qanday yoziladi? — **A) 'Ali'**



Ma'lumotlarni qo'shish, ko'rish va saqlash. CRUD operatsiyalari

CRUD konsepsiyasi; INSERT bilan qo'shish, SELECT bilan o'qish (ustunlarni tanlash, WHERE, ORDER BY, LIMIT), UPDATE bilan yangilash, DELETE bilan o'chirish; WHERE'siz UPDATE/DELETE xavfi; o'quvchilar bazasi loyihasi.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- **CRUD** — har qanday dasturning to'rtta asosiy amalini;
- **INSERT** — ma'lumot qo'shish;
- **SELECT** — ma'lumotni o'qish, tanlash va saralash;
- **UPDATE** — o'zgartirish va **DELETE** — o'chirish;
- Eng xavfli xato — **WHERE** siz UPDATE/DELETE — dan qanday saqlanishni;
- To'liq ishlaydigan o'quvchilar bazasini (18-loyiha).

CRUD nima?

Deyarli har qanday dastur ma'lumot bilan to'rtta amal bajaradi. Ularning bosh harflaridan **CRUD** so'zi hosil bo'lgan:

Harf	Amal	SQL	Hayotiy misol (Instagram)	HTTP (39-mavzu)
Create	Yaratish	INSERT	Yangi post joylash	POST
Read	O'qish	SELECT	Lentani ko'rish	GET
Update	Yangilash	UPDATE	Post izohini tahrirlash	PUT/PATCH

[Delete](#)[O'chirish](#)[DELETE](#)[Postni o'chirish](#)[DELETE](#)

Bu darsda 27-mavzuda yaratgan [maktab](#) bazasidagi jadvallar bilan ishlaymiz.

Create — INSERT

SQL

```
-- Bitta qator qo'shish
INSERT INTO sinflar (nomi, sinf_rahbari, xona)
VALUES ('8-A', 'Karimova Nodira', 204);

-- Bir nechta qatorni birdan
INSERT INTO sinflar (nomi, sinf_rahbari, xona) VALUES
    ('8-B', 'Rasulov Akmal', 206),
    ('8-V', 'Yusupova Malika', 210);

-- O'quvchilar (id avtomatik — SERIAL; alochi va yaratilgan — DEFAULT)
INSERT INTO oquvchilar (ism, familiya, tugilgan_sana, email, sinf_id) VALUES
    ('Ali', 'Valiyev', '2012-03-15', 'ali@mail.uz', 1),
    ('Zarina', 'Karimova', '2012-07-02', 'zarina@mail.uz', 1),
    ('Bobur', 'Toshmatov', '2011-11-20', 'bobur@mail.uz', 2),
    ('Madina', 'Ergasheva', '2012-01-09', 'madina@mail.uz', 2),
    ('Sardor', 'Qodirov', '2012-05-30', NULL, 3),
    ('Dilnoza', 'Aminova', '2011-12-12', 'dilnoza@mail.uz', 1);

-- Qo'shilgan qatorni darhol ko'rish (PostgreSQL imkoniyati)
INSERT INTO oquvchilar (ism, familiya, sinf_id)
VALUES ('Jasur', 'Rahimov', 3)
RETURNING id, ism;
```

- Ustunlar ro'yxati va qiymatlar ro'yxati **bir xil tartib va sonda** bo'lishi kerak.
- `id` ni yozmaymiz — `SERIAL` uni o'zi beradi.
- `NULL` — "qiymat yo'q" (bo'sh). Faqat `NOT NULL` bo'lmagan ustunlarga.

Baza cheklovlarni tekshiradi — quyidagilar **xato** beradi:

SQL

```
INSERT INTO oquvchilar (familiya) VALUES ('Aliyev');
-- ❌ null value in column "ism" violates not-null constraint

INSERT INTO oquvchilar (ism, familiya, email) VALUES ('Vali', 'Aliyev',
'ali@mail.uz');
-- ❌ duplicate key value violates unique constraint (email band)

INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES ('Vali', 'Aliyev', 99);
-- ❌ violates foreign key constraint (99-sinf yo'q)

INSERT INTO baholar (oquvchi_id, fan, baho) VALUES (1, 'Matematika', 7);
-- ❌ violates check constraint (baho 2..5 bo'lishi kerak)
```



Bu xatolar — **yaxshi** narsa! Baza noto'g'ri ma'lumot yozilishiga yo'l qo'ymayapti. Xato xabarini diqqat bilan o'qing — u aniq qaysi qoida buzilganini aytadi.

Read — SELECT

SQL

```
-- Barcha ustunlar va barcha qatorlar
SELECT * FROM oquvchilar;

-- Faqat kerakli ustunlar
SELECT ism, familiya, tugilgan_sana FROM oquvchilar;

-- Ustunga boshqa nom berish (AS) va hisoblangan ustun
SELECT ism || ' ' || familiya AS toliq_ism,                -- || - matnlarni
birlashtirish
        EXTRACT(YEAR FROM AGE(tugilgan_sana)) AS yoshi    -- yoshni hisoblash
FROM oquvchilar;
```

Natija

toliq_ism		yoshi
-----+-----		
Ali Valiyev		14
Zarina Karimova		14
Bobur Toshmatov		14
...		

WHERE — shart bo'yicha tanlash

SQL

```
SELECT * FROM oquvchilar WHERE sinf_id = 1;           -- 8-A
o'quvchilari
SELECT * FROM oquvchilar WHERE ism = 'Bobur';
SELECT * FROM oquvchilar WHERE tugilgan_sana < '2012-01-01'; -- 2012-yildan
oldin tug'ilganlar
SELECT * FROM oquvchilar WHERE email IS NULL;         -- emaili
yo'qlar (= NULL emas!)
SELECT * FROM oquvchilar WHERE sinf_id = 1 AND alochi = TRUE; -- VA
SELECT * FROM oquvchilar WHERE sinf_id = 2 OR sinf_id = 3;  -- YOKI
```

Batafsil filtrlash (`LIKE` , `IN` , `BETWEEN`) — 30-mavzuda.

ORDER BY va LIMIT

SQL

```
SELECT ism, familiya FROM oquvchilar ORDER BY familiya;           -- A
dan Z gacha
SELECT ism, tugilgan_sana FROM oquvchilar ORDER BY tugilgan_sana DESC; -- eng
yoshi birinchi
SELECT ism FROM oquvchilar ORDER BY sinf_id, familiya;           --
avval sinf, keyin familiya
SELECT * FROM oquvchilar ORDER BY id LIMIT 3;                     -- faqat
birinchi 3 tasi
SELECT * FROM oquvchilar ORDER BY id LIMIT 3 OFFSET 3;           --
keyingi 3 tasi (2-sahifa)
```



SELECT qismlari tartibi

SELECT ustunlar → FROM jadval → WHERE shart → ORDER BY saralash → LIMIT son. Tartib qat'iy: WHERE ni ORDER BY dan keyin yozib bo'lmaydi.

Update — UPDATE

SQL

```
-- Bitta o'quvchining emailini o'zgartirish
UPDATE oquvchilar
SET email = 'sardor@mail.uz'
WHERE id = 5;

-- Bir nechta ustunni birdan
UPDATE oquvchilar
SET alochi = TRUE, telefon = '+998901234567'
WHERE id = 2;

-- Hisob bilan: 8-A sinfi boshqa xonaga ko'chdi
UPDATE sinflar SET xona = xona + 100 WHERE nomi = '8-A';

-- O'zgargan qatorni ko'rish
UPDATE oquvchilar SET sinf_id = 2 WHERE id = 6 RETURNING *;
```

Delete — DELETE

SQL

```
DELETE FROM oquvchilar WHERE id = 7;

DELETE FROM baholar WHERE sana < '2025-09-01'; -- eski baholarni tozalash
```

🚫 ⚠️ ENG XAVFLI XATO: WHERE'ni unutish

SQL

```
UPDATE oquvchilar SET sinf_id = 1;      -- 😱 BARCHA o'quvchilar 8-A
ga ko'chdi!
DELETE FROM oquvchilar;                 -- 😱 BARCHA o'quvchilar
o'chdi!
```

Baza "Ishonchingiz komilmi?" deb so'ramaydi. **Xavfsizlik qoidasi:** UPDATE yoki DELETE yozishdan oldin, xuddi shu `WHERE` bilan **avval SELECT** qiling va qaysi qatorlar o'zgarishini ko'ring:

SQL

```
SELECT * FROM baholar WHERE sana < '2025-09-01';  -- 1) avval
ko'ramiz: 12 qator
DELETE FROM baholar WHERE sana < '2025-09-01';    -- 2) keyin
o'chiramiz
```

Tranzaksiya — "hammasi yoki hech biri"

Ba'zan bir nechta o'zgarish faqat **birgalikda** ma'noga ega. Masalan, o'quvchini boshqa sinfga ko'chirish va bu haqda yozuv qoldirish:

SQL

```
BEGIN;                                     -- tranzaksiya boshlandi
UPDATE oquvchilar SET sinf_id = 3 WHERE id = 1;
INSERT INTO baholar (oquvchi_id, fan, baho) VALUES (1, 'Matematika', 5);
COMMIT;                                    -- hammasini saqlash
-- Xato bo'lsa: ROLLBACK; - barcha o'zgarishlar bekor qilinadi
```




O'xshatish: pul o'tkazmasi

Do'stingizga 50 000 so'm o'tkazyapsiz: sizning hisobingizdan yechiladi VA do'stingiznikiga tushadi. Agar birinchisi bajarilib, ikkinchisida elektr o'chsa — pul "yo'qolib" qoladi. Tranzaksiya ikkalasi birga bajarilishini yoki ikkalasi ham bekor qilinishini kafolatlaydi.

Amaliy mashg'ulot: o'quvchilar bazasi (18-loyiha)

1 Yuqoridagi `INSERT` lar bilan sinflar va o'quvchilarni qo'shing. O'z sinfdoshlaringizdan yana 5 tasini qo'shing.

2 **Baholar** jadvaliga kamida 20 ta baho qo'shing:

SQL

```
INSERT INTO baholar (oquvchi_id, fan, baho, sana) VALUES
  (1, 'Matematika', 5, '2026-09-10'), (1, 'Informatika', 5, '2026-09-11'),
  (1, 'O'na tili', 4, '2026-09-12'), (2, 'Matematika', 5, '2026-09-10'),
  (2, 'Informatika', 4, '2026-09-11'), (3, 'Matematika', 3, '2026-09-10'),
  (3, 'Informatika', 4, '2026-09-11'), (4, 'Matematika', 5, '2026-09-10'),
  (4, 'O'na tili', 5, '2026-09-12'), (5, 'Informatika', 2, '2026-09-11'),
  (6, 'Matematika', 4, '2026-09-10'), (6, 'Informatika', 5, '2026-09-11');
```

3 Quyidagi so'rovlarni yozing:

1. Barcha o'quvchilarning to'liq ismi, familiya bo'yicha tartiblangan.
2. 2-sinf (sinf_id = 2) o'quvchilari.
3. Eng yoshi katta 3 nafar o'quvchi.
4. Emaili kiritilmagan o'quvchilar.
5. Informatikadan 5 olgan baholar.

4 Sardorning (id=5) emailini qo'shing; Zarinani a'lochi deb belgilang.

5 Bitta o'quvchini o'chiring (avval SELECT bilan tekshirib!). Uning baholari ham o'chdimi? Nima uchun? (`ON DELETE CASCADE`)

▼ 3-topshiriq javoblari

SQL

```
SELECT ism || ' ' || familiya AS toliq_ism FROM oquvchilar ORDER BY  
familiya;  
SELECT * FROM oquvchilar WHERE sinf_id = 2;  
SELECT ism, familiya, tugilgan_sana FROM oquvchilar ORDER BY tugilgan_sana  
LIMIT 3;  
SELECT ism, familiya FROM oquvchilar WHERE email IS NULL;  
SELECT * FROM baholar WHERE fan = 'Informatika' AND baho = 5;
```

Ko'p uchraydigan xatolar

- ❌ `WHERE email = NULL` — hech narsa qaytarmaydi. NULL bilan taqqoslash: `IS NULL` / `IS NOT NULL`.
- ❌ Matn qiymatini qo'shtirnoqsiz yozish: `WHERE ism = Ali` — `column "ali" does not exist`.
- ❌ `INSERT` da ustunlar va qiymatlar soni mos emas: `INSERT has more expressions than target columns`.
- ❌ `WHERE` siz `UPDATE/DELETE`. Avval `SELECT`!

O'zingizni tekshiring

? CRUD'dagi "R" qaysi SQL buyrug'iga mos keladi?

- A. `INSERT`
- B. `SELECT`
- C. `UPDATE`
- D. `REMOVE`

? **DELETE FROM oquvchilar; nima qiladi?**

- A. Faqat birinchi qatorni o'chiradi
- B. Jadvalni o'chiradi
- C. Xato beradi
- D. Jadvaldagi BARCHA qatorlarni o'chiradi

? **Emaili kiritilmagan o'quvchilarni topuvchi to'g'ri shart?**

- A. `WHERE email IS NULL`
- B. `WHERE email = NULL`
- C. `WHERE email = ''`
- D. `WHERE NOT email`

? **Natijani kamayish tartibida saralash uchun?**

- A. `ORDER BY ... ASC`
- B. `SORT BY ... DOWN`
- C. `ORDER BY ... DESC`
- D. `ORDER DESC BY ...`

? **Faqat birinchi 5 ta natijani olish uchun?**

- A. `TOP 5`
- B. `LIMIT 5`
- C. `FIRST 5`
- D. `COUNT 5`

? Bir nechta o'zgarishni "hammasi yoki hech biri" tarzida bajarish nima deyiladi?

- A. Indeks
- B. JOIN
- C. Cheklov
- D. Tranzaksiya



Uyga vazifa

1. 27-mavzudagi `dokon` bazasini to'ldiring: 3 toifa, 15 mahsulot, 5 mijoz.
2. So'rovlar: eng qimmat 5 mahsulot; omborda tugagan (`soni = 0`) mahsulotlar; narxi 50 000 dan arzon mahsulotlar alifbo tartibida.
3. Barcha "Kanselyariya" toifasidagi mahsulotlar narxini 10% oshiring (`SET narx = narx * 1.1`) — avval SELECT bilan tekshiring!



Yodda tuting

- CRUD: Create (`INSERT`), Read (`SELECT`), Update (`UPDATE`), Delete (`DELETE`).
- `INSERT INTO t (a, b) VALUES (1, 'x');`
- `SELECT ustunlar FROM t WHERE shart ORDER BY u DESC LIMIT n;`
- `UPDATE t SET a = 1 WHERE id = 5;` / `DELETE FROM t WHERE id = 5;` — **WHERE'ni unutmang, avval SELECT!**
- `IS NULL` ; tranzaksiya — `BEGIN ... COMMIT / ROLLBACK` .

Test javoblari

1. CRUD'dagi "R" qaysi SQL buyrug'iga mos keladi? — **B) SELECT**
2. DELETE FROM oquvchilar; nima qiladi? — **D) Jadvaldagi BARCHA qatorlarni o'chiradi**
3. Emaili kiritilmagan o'quvchilarni topuvchi to'g'ri shart? — **A) WHERE email IS NULL**
4. Natijani kamayish tartibida saralash uchun? — **C) ORDER BY ... DESC**
5. Faqat birinchi 5 ta natijani olish uchun? — **B) LIMIT 5**
6. Bir nechta o'zgarishni "hammasi yoki hech biri" tarzida bajarish nima deyiladi? — **D) Tranzaksiya**



29-MAVZU · 42 TADAN · III BOB. PYTHON VA MA'LUMOTLAR BAZASI

Jadvallarni bog'lash va JOIN. Index, constraint, data integrity. Backup va restore

INNER, LEFT, RIGHT JOIN va ular farqi; bir nechta jadvalni birlashtirish; indekslar va EXPLAIN; cheklovlar va ma'lumotlar yaxlitligi (ON DELETE); pg_dump va pgAdmin orqali zaxira nusxa va tiklash.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Bir nechta jadvaldagi ma'lumotlarni **JOIN** bilan birlashtirishni;
- `INNER`, `LEFT` va `RIGHT JOIN` farqini;
- Qidiruvni tezlashtiruvchi **indekslarni**;
- Ma'lumotlar **yaxlitligi** va cheklovlar qanday himoya qilishini;
- Bazaning **zaxira nusxasini** olish va tiklashni (backup / restore).

Nima uchun JOIN kerak?

26-mavzuda ma'lumotlarni takrorlamalik uchun ularni turli jadvallarga ajratdik: o'quvchilar alohida, sinflar alohida, baholar alohida. Endi savol: **"Ali Valiyev qaysi sinfda va sinf rahbari kim?"** Javob uchun ikkala jadvalni **birlashtirish** kerak — buni `JOIN` qiladi.



O'xshatish: ikkita ro'yxat

Sizda ikkita qog'oz bor: birinchisida "O'quvchi — sinf raqami", ikkinchisida "Sinf raqami — sinf rahbari". Ali'ning rahbarini bilish uchun birinchi qog'ozdan sinf raqamini topib, ikkinchi qog'ozdan shu raqamni qidirasiz. JOIN aynan shuni avtomatik va millionlab qator uchun bir zumda qiladi.

INNER JOIN

SQL

```
SELECT oquvchilar.ism, oquvchilar.familiya, sinflar.nomi, sinflar.sinf_rahbari
FROM oquvchilar
INNER JOIN sinflar ON oquvchilar.sinf_id = sinflar.id;
```

- `JOIN sinflar` — qaysi jadval bilan birlashtiramiz.
- `ON oquvchilar.sinf_id = sinflar.id` — **qaysi ustunlar bo'yicha** moslashtiramiz (odatda FK = PK).
- `jadval.ustun` — ikki jadvalda bir xil nomli ustun bo'lsa (ikkalasida `id` bor), qaysi biri ekanini aniq ko'rsatish uchun.

Taxalluslar — qisqaroq yozuv

SQL

```
SELECT o.ism, o.familiya, s.nomi AS sinf, s.sinf_rahbari
FROM oquvchilar AS o
JOIN sinflar AS s ON o.sinf_id = s.id           -- JOIN = INNER JOIN
ORDER BY s.nomi, o.familiya;
```

Natija

ism	familiya	sinf	sinf_rahbari
Dilnoza	Aminova	8-A	Karimova Nodira
Zarina	Karimova	8-A	Karimova Nodira
Ali	Valiyev	8-A	Karimova Nodira
Madina	Ergasheva	8-B	Rasulov Akmal
Bobur	Toshmatov	8-B	Rasulov Akmal
...			

JOIN turlari

Agar biror o'quvchining `sinf_id` si bo'sh (NULL) bo'lsa yoki biror sinfda hali o'quvchi bo'lmasa-chi? Shu yerda JOIN turlari farq qiladi:

Tur	Natijada nima bo'ladi	Doira diagrammasi
<code>INNER JOIN</code>	Faqat ikkala jadvalda ham mos keladigan qatorlar	Ikki doiraning kesishmasi
<code>LEFT JOIN</code>	Chap jadvalning HAMMA qatorlari + o'ngdan mos kelganlar (mos kelmasa — NULL)	Butun chap doira
<code>RIGHT JOIN</code>	O'ng jadvalning HAMMA qatorlari + chapdan mos kelganlar	Butun o'ng doira
<code>FULL JOIN</code>	Ikkala jadvalning hamma qatorlari	Ikkala doira

SQL

```
-- Tayyorgarlik: sinfsiz o'quvchi va o'quvchisiz sinf qo'shamiz
INSERT INTO oquvchilar (ism, familiya) VALUES ('Yangi', 'Keldi'); -- sinf_id = NULL
INSERT INTO sinflar (nomi, sinf_rahbari) VALUES ('8-G', 'Azimov B.');
```

-- hali o'quvchisi yo'q

```
-- INNER: "Yangi Keldi" ham, "8-G" ham YO'Q
SELECT o.ism, s.nomi FROM oquvchilar o JOIN sinflar s ON o.sinf_id = s.id;
```

```
-- LEFT: barcha o'quvchilar; "Yangi Keldi" ning sinfi — NULL
SELECT o.ism, s.nomi FROM oquvchilar o LEFT JOIN sinflar s ON o.sinf_id = s.id;
```

```
-- RIGHT: barcha sinflar; "8-G" uchun o'quvchi — NULL
SELECT o.ism, s.nomi FROM oquvchilar o RIGHT JOIN sinflar s ON o.sinf_id = s.id;
```


LEFT JOIN'ning foydali qo'llanilishi: "yo'q"larni topish

SQL

```
-- Hali birorta ham baho olmagan o'quvchilar
SELECT o.ism, o.familiya
FROM oquvchilar o
LEFT JOIN baholar b ON b.oquvchi_id = o.id
WHERE b.id IS NULL;
```



Qaysi JOIN kerakligini bilish uchun so'rang: "Mos kelmagan qatorlar natijada **ko'rinishi kerakmi?**" Yo'q → `INNER`. Chap jadvalning hammasi kerak → `LEFT`. Amalda `RIGHT JOIN` kam ishlatiladi — jadvallar o'rnini almashtirib, `LEFT` bilan yozish mumkin.

Uch jadvalni birlashtirish

SQL

```
-- Kim, qaysi sinfdan, qaysi fandan, qanday baho olgan
SELECT o.ism || ' ' || o.familiya AS oquvchi,
       s.nomi AS sinf,
       b.fan,
       b.baho,
       b.sana
FROM baholar b
JOIN oquvchilar o ON b.oquvchi_id = o.id
JOIN sinflar s    ON o.sinf_id = s.id
WHERE b.fan = 'Matematika'
ORDER BY b.baho DESC, oquvchi;
```

Ko'pga-ko'p: to'garaklar

SQL

```
-- Har bir o'quvchi qaysi to'garaklarda (oraliq jadval orqali)
SELECT o.ism, t.nomi AS togarak
FROM oquvchilar o
JOIN azoliklar a ON a.oquvchi_id = o.id
JOIN togaraklar t ON t.id = a.togarak_id
ORDER BY o.ism;
```

Indekslar — qidiruvni tezlashtirish

Jadvalda 1 000 000 qator bo'lsa, `WHERE familiya = 'Karimova'` uchun baza har bir qatorni birma-bir tekshiradi. **Indeks** — maxsus "ko'rsatkich" bo'lib, kerakli qatorlarni darhol topishga yordam beradi.



O'xshatish: kitob oxiridagi ko'rsatkich

"Fotosintez" so'zini darslikda topish uchun 300 betni varaqlaysizmi? Yo'q — kitob oxiridagi alifboli **ko'rsatkich**ga qaraysiz: "Fotosintez — 47, 112-betlar". Indeks — baza uchun xuddi shunday ko'rsatkich.

SQL

```
-- Familiya bo'yicha tez-tez qidiramiz — indeks yaratamiz
CREATE INDEX idx_oquvchilar_familiya ON oquvchilar (familiya);

-- Tashqi kalitlar JOIN'da ishlatiladi — ularga indeks foydali
CREATE INDEX idx_baholar_oquvchi ON baholar (oquvchi_id);

-- Baza so'rovni qanday bajarishini ko'rish
EXPLAIN SELECT * FROM oquvchilar WHERE familiya = 'Karimova';

DROP INDEX idx_oquvchilar_familiya; -- indeksni o'chirish
```

- `PRIMARY KEY` va `UNIQUE` ustunlarga indeks **avtomatik** yaratiladi.
- Indeks o'qishni tezlashtiradi, lekin **yozishni biroz sekinlashtiradi** va joy egallaydi. Shuning uchun faqat **tez-tez qidiriladigan** ustunlarga qo'yiladi.

- Kichik jadvallarda (yuzlab qator) farq sezilmaydi; minglab va millionlab qatorda — juda katta.

Constraint va ma'lumotlar yaxlitligi (data integrity)

Ma'lumotlar yaxlitligi — bazadagi ma'lumotlar har doim **to'g'ri, izchil va ishonchli** bo'lishi. Buni dastur kodiga emas, **bazaning o'ziga** ishonib topshirish kerak — chunki dasturda xato bo'lishi mumkin, baza esa qoidani doim tekshiradi.

Yaxlitlik turi	Nimani kafolatlaydi	Vosita
Obyekt yaxlitligi	Har bir qator noyob va aniqlanadigan	PRIMARY KEY
Havola yaxlitligi	FK doim mavjud qatorga ishora qiladi	FOREIGN KEY (REFERENCES)
Qiymat yaxlitligi	Qiymatlar to'g'ri turda va chegarada	Turlar, NOT NULL , UNIQUE , CHECK

FK qatori o'chirilganda nima bo'ladi?

Variant	Xatti-harakat	Misol
ON DELETE RESTRICT (standart)	O'chirishga ruxsat bermaydi	O'quvchilari bor sinfni o'chirib bo'lmaydi
ON DELETE CASCADE	Bog'liq qatorlar ham o'chadi	O'quvchi o'chsa — baholari ham
ON DELETE SET NULL	FK ustuni NULL bo'ladi	Sinf yopilsa — o'quvchilar "sinfsiz" qoladi

SQL

```
DELETE FROM sinflar WHERE id = 1;
-- ✗ update or delete on table "sinflar" violates foreign key constraint ...
--   Key (id)=(1) is still referenced from table "oquvchilar".
-- Baza "yetim" o'quvchilar paydo bo'lishiga yo'l qo'ymadi ✓
```

Mavjud jadvalga cheklov qo'shish

SQL

```
ALTER TABLE oquvchilar ADD CONSTRAINT email_formati CHECK (email LIKE '%@%.%');  
ALTER TABLE sinflar ADD CONSTRAINT xona_musbat CHECK (xona > 0);
```

Backup va Restore — zaxira nusxa

Kompyuter buzilishi, tasodifiy `DELETE` yoki viruslar — ma'lumot yo'qolishining sabablari ko'p. **Backup** — bazaning nusxasini saqlab qo'yish, **restore** — nusxadan qayta tiklash. Professional qoida: **"Tekshirilmagan zaxira — zaxira emas"** — tiklashni ham sinab ko'ring.

pgAdmin orqali

- 1 **Backup:** maktab bazasi ustida o'ng tugma → **Backup...** → fayl nomi (maktab_2026-10-06.backup) → Format: **Custom** → **Backup**.
- 2 **Restore:** yangi bo'sh baza yarating (maktab_tiklangan) → o'ng tugma → **Restore...** → faylni tanlang → **Restore**.
- 3 Tiklangan bazadagi jadvallar va ma'lumotlarni tekshiring.

Buyruqlar qatori orqali

Terminal

```
# Butun bazani SQL fayl ko'rinishida saqlash  
pg_dump -U postgres -d maktab -f maktab_backup.sql  
  
# Siqilgan (custom) formatda  
pg_dump -U postgres -d maktab -F c -f maktab.backup  
  
# Tiklash  
createdb -U postgres maktab_yangi  
psql -U postgres -d maktab_yangi -f maktab_backup.sql # .sql fayldan  
pg_restore -U postgres -d maktab_yangi maktab.backup # .backup  
fayldan
```

`maktab_backup.sql` faylini oddiy matn muharririda oching — u bazani qaytadan yaratuvchi `CREATE TABLE` va `INSERT` buyruqlaridan iborat!

Amaliy mashg'ulot: mahsulotlar va buyurtmalar bazasi

(19-loyiha)

dokon.sql

```
CREATE TABLE mijozlar (  
    id        SERIAL PRIMARY KEY,  
    ism       VARCHAR(100) NOT NULL,  
    telefon   VARCHAR(20) UNIQUE  
);  
  
CREATE TABLE mahsulotlar (  
    id        SERIAL PRIMARY KEY,  
    nomi      VARCHAR(100) NOT NULL,  
    narx      NUMERIC(12, 2) NOT NULL CHECK (narx > 0),  
    soni      INTEGER NOT NULL DEFAULT 0 CHECK (soni ≥ 0)  
);  
  
CREATE TABLE buyurtmalar (  
    id        SERIAL PRIMARY KEY,  
    mijoz_id  INTEGER NOT NULL REFERENCES mijozlar(id),  
    sana      TIMESTAMP DEFAULT NOW(),  
    holat     VARCHAR(20) DEFAULT 'yangi' CHECK (holat IN ('yangi',  
'yetkazilmoqda', 'yetkazildi', 'bekor'))  
);  
  
CREATE TABLE buyurtma_tarkibi (  
    buyurtma_id  INTEGER REFERENCES buyurtmalar(id) ON DELETE CASCADE, -- ko'pga-ko'p  
    mahsulot_id  INTEGER REFERENCES mahsulotlar(id),  
    soni         INTEGER NOT NULL CHECK (soni > 0),  
    narx         NUMERIC(12, 2) NOT NULL, -- sotilgan  
    paytdagi_narx NUMERIC(12, 2) NOT NULL,  
    PRIMARY KEY (buyurtma_id, mahsulot_id)  
);  
  
INSERT INTO mijozlar (ism, telefon) VALUES  
    ('Aziz', '+998901111111'), ('Nilufar', '+998902222222'), ('Kamol',  
'+998903333333');  
  
INSERT INTO mahsulotlar (nomi, narx, soni) VALUES  
    ('Daftar', 5000, 200), ('Ruchka', 3000, 500), ('Ryukzak', 180000, 15),  
    ('Kalkulyator', 95000, 0);  
  
INSERT INTO buyurtmalar (mijoz_id, holat) VALUES (1, 'yetkazildi'), (2,  
'yangi'), (1, 'yangi');  
  
INSERT INTO buyurtma_tarkibi VALUES  
    (1, 1, 10, 5000), (1, 2, 5, 3000), (2, 3, 1, 180000), (3, 1, 2, 5000);
```

Quyidagi so'rovlarni yozing:

- 1 Har bir buyurtma: raqami, mijoz ismi, sanasi va holati.
- 2 1-buyurtma tarkibi: mahsulot nomi, soni, narxi va `soni * narx` (summa).
- 3 Hali birorta ham buyurtma bermagan mijozlar (LEFT JOIN + IS NULL).
- 4 Hech qachon sotilmagan mahsulotlar.
- 5 Bazaning zaxira nusxasini oling, `mahsulotlar` jadvalidan bitta qatorni o'chiring, so'ng nusxadan yangi bazaga tiklab, qator o'z joyida ekanini tekshiring.

▼ Javoblar (1–4)

SQL

```
SELECT b.id, m.ism, b.sana, b.holat
FROM buyurtmalar b JOIN mijozlar m ON m.id = b.mijoz_id;

SELECT p.nomi, t.soni, t.narx, t.soni * t.narx AS summa
FROM buyurtma_tarkibi t JOIN mahsulotlar p ON p.id = t.mahsulot_id
WHERE t.buyurtma_id = 1;

SELECT m.ism FROM mijozlar m
LEFT JOIN buyurtmalar b ON b.mijoz_id = m.id
WHERE b.id IS NULL;

SELECT p.nomi FROM mahsulotlar p
LEFT JOIN buyurtma_tarkibi t ON t.mahsulot_id = p.id
WHERE t.mahsulot_id IS NULL;
```

Ko'p uchraydigan xatolar

- ✗ column reference "id" is ambiguous — ikki jadvalda ham `id` bor; `o.id` yoki `s.id` deb aniq yozing.
- ✗ `ON` shartini unutish yoki noto'g'ri ustunlarni bog'lash (`o.id = s.id`) — natija mantiqsiz bo'ladi.
- ✗ LEFT JOIN'dan keyin o'ng jadval bo'yicha `WHERE` qo'yib, uni INNER'ga aylantirib qo'yish.

- ❌ Zaxira nusxani o'sha kompyuterning o'zida saqlash — kompyuter buzilsa, nusxa ham yo'qoladi. Boshqa joyga (fleshka, bulut) ham ko'chiring.

O'zingizni tekshiring

? Faqat ikkala jadvalda ham mos keladigan qatorlarni qaytaruvchi JOIN?

- A. `INNER JOIN`
- B. `LEFT JOIN`
- C. `RIGHT JOIN`
- D. `FULL JOIN`

? Barcha o'quvchilarni (sinfi bo'lmaganlarini ham) sinf nomi bilan chiqarish uchun?

- A. `oquvchilar INNER JOIN sinflar`
- B. `sinflar INNER JOIN oquvchilar`
- C. `oquvchilar LEFT JOIN sinflar`
- D. `oquvchilar RIGHT JOIN sinflar`

? Indeksning asosiy vazifasi?

- A. Ma'lumotlarni shifrlash
- B. Qidiruvni tezlashtirish
- C. Zaxira nusxa olish
- D. Jadvallarni bog'lash

? O'quvchi o'chirilganda uning baholari ham avtomatik o'chishi uchun FK'ga nima qo'shiladi?

- A. `ON DELETE RESTRICT`
- B. `ON DELETE SET NULL`
- C. `NOT NULL`
- D. `ON DELETE CASCADE`

? PostgreSQL bazasining zaxira nusxasini oluvchi buyruq?

- A. `pg_backup`
- B. `pg_dump`
- C. `COPY DATABASE`
- D. `SAVE DATABASE`



Uyga vazifa

- Kutubxona bazangizda (27-mavzu uyga vazifasi): har bir ijara — kitob nomi, muallif, o'quvchi ismi; hali qaytarilmagan kitoblar; hech kim olmagan kitoblar.
- Haftada bir marta avtomatik backup qilish nima uchun muhimligini va qanday tashkil qilish mumkinligini yozing.



Yodda tuting

- `SELECT ... FROM a JOIN b ON a.fk = b.id`; taxalluslar: `FROM oquvchilar o`.
- `INNER` — faqat moslar; `LEFT` — chapning hammasi; `RIGHT` — o'ngning hammasi. "Yo'q"larni topish: `LEFT JOIN + IS NULL`.
- Indeks — tez qidiruv uchun ko'rsatkich (`CREATE INDEX`); PK/UNIQUE'da avtomatik.
- Yaxlitlik: PK, FK, NOT NULL, UNIQUE, CHECK; `ON DELETE CASCADE / RESTRICT / SET NULL`.
- Backup: `pg_dump` / pgAdmin Backup; restore: `psql -f`, `pg_restore`. Tiklashni sinab ko'ring!

✓ Test javoblari

1. Faqat ikkala jadvalda ham mos keladigan qatorlarni qaytaruvchi JOIN? — **A) INNER JOIN**
2. Barcha o'quvchilarni (sinfi bo'lmaganlarini ham) sinf nomi bilan chiqarish uchun? — **C) oquvchilar LEFT JOIN sinflar**
3. Indeksning asosiy vazifasi? — **B) Qidiruvni tezlashtirish**
4. O'quvchi o'chirilganda uning baholari ham avtomatik o'chishi uchun FK'ga nima qo'shiladi? — **D) ON DELETE CASCADE**
5. PostgreSQL bazasining zaxira nusxasini oluvchi buyruq? — **B) pg_dump**



Filtrlash va shartlar. Aggregation va grouping

WHERE'da taqqoslash, AND/OR/NOT, IN, BETWEEN, LIKE/ILIKE, IS NULL; ORDER BY, LIMIT, DISTINCT; agregat funksiyalar COUNT, SUM, AVG, MIN, MAX; GROUP BY va HAVING; statistik hisobotlar tayyorlash.

1 soat · amaliy

Bu darsda nimani o'rganasiz

- Murakkab shartlar bilan aniq ma'lumotlarni tanlashni: `IN`, `BETWEEN`, `LIKE`;
- Takrorlarni olib tashlash (`DISTINCT`) va sahifalash;
- **Agregat funksiyalar:** `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`;
- `GROUP BY` bilan guruhlab hisoblash va `HAVING` bilan guruhlarni filtrlashni;
- Haqiqiy hisobotlar tayyorlashni: "har bir sinf o'rtacha bahosi", "eng ko'p sotilgan mahsulot".

Filtrlash: WHERE chuqurroq

Taqqoslash va mantiq

Operator	Ma'nosi	Misol
<code>=</code> , <code><></code> (yoki <code>≠</code>)	teng, teng emas	<code>fan <> 'Tarix'</code>
<code>></code> , <code><</code> , <code>≥</code> , <code>≤</code>	katta, kichik...	<code>baho ≥ 4</code>
<code>AND</code> , <code>OR</code> , <code>NOT</code>	va, yoki, emas	<code>baho = 5 AND fan = 'Fizika'</code>

IN (...)	ro'yxatdagi qiymatlardan biri	sinf_id IN (1, 3)
BETWEEN a AND b	oraliqda (chegaralar kiradi)	narx BETWEEN 10000 AND 50000
LIKE / ILIKE	naqsh bo'yicha matn qidirish (ILIKE — harf katta-kichikligiga qaramaydi)	ism LIKE 'A%'
IS NULL / IS NOT NULL	bo'sh / bo'sh emas	email IS NOT NULL

SQL

```
-- IN — OR'ning qisqa shakli
SELECT * FROM baholar WHERE fan IN ('Matematika', 'Informatika', 'Fizika');

-- BETWEEN — oraliq (sanalar bilan ham ishlaydi)
SELECT * FROM baholar WHERE sana BETWEEN '2026-09-01' AND '2026-09-30';
SELECT ism, tugilgan_sana FROM oquvchilar WHERE tugilgan_sana BETWEEN '2012-01-01' AND '2012-12-31';

-- NOT bilan
SELECT * FROM oquvchilar WHERE sinf_id NOT IN (1, 2);

-- Qavslar muhim! AND OR'dan oldin bajariladi
SELECT * FROM baholar
WHERE (fan = 'Matematika' OR fan = 'Fizika') AND baho = 5;
```



Qavssiz `WHERE fan = 'Matematika' OR fan = 'Fizika' AND baho = 5` — bu "Matematikadagi **barcha** baholar YOKI Fizikadagi 5 lar" deb tushuniladi. Shubha bo'lsa — qavs qo'ying.

LIKE — naqsh bo'yicha qidirish

Belgi	Ma'nosi
%	istalgan sondagi (0 yoki ko'p) istalgan belgilar
_	aynan bitta istalgan belgi

SQL

```
SELECT ism FROM oquvchilar WHERE ism LIKE 'D%';      -- D bilan boshlanadi:
Dilnoza
SELECT ism FROM oquvchilar WHERE familiya LIKE '%ova'; -- "ova" bilan tugaydi:
Karimova, Aminova
SELECT ism FROM oquvchilar WHERE ism LIKE '%a%';      -- ichida "a" bor
SELECT ism FROM oquvchilar WHERE ism LIKE '_li';      -- 3 harfli, "li" bilan
tugaydi: Ali
SELECT ism FROM oquvchilar WHERE ism ILIKE 'zarina';   -- katta-kichik harfga
qaramaydi
SELECT * FROM oquvchilar WHERE email LIKE '%@mail.uz'; -- mail.uz
pochtasidagilar
```

Saytlardagi **qidiruv maydoni** ko'pincha aynan `ILIKE '%so''z%'` bilan ishlaydi.

DISTINCT — takrorlarsiz

SQL

```
SELECT fan FROM baholar;      -- Matematika, Informatika, Matematika,
... (takrorlar bilan)
SELECT DISTINCT fan FROM baholar; -- har bir fan bir marta
SELECT DISTINCT fan FROM baholar ORDER BY fan;
```

Agregat funksiyalar

Agregat (yig'uvchi) funksiya ko'p qatorni olib, **bitta natija** qaytaradi. Bu — statistika va hisobotlarning asosi.

Funksiya	Vazifasi
<code>COUNT(*)</code>	Qatorlar soni
<code>COUNT(ustun)</code>	NULL bo'lmagan qiymatlar soni
<code>COUNT(DISTINCT ustun)</code>	Noyob qiymatlar soni

SUM(ustun)	Yig'indi
AVG(ustun)	O'rtacha qiymat
MIN(ustun) / MAX(ustun)	Eng kichik / eng katta

SQL

```
SELECT COUNT(*) AS jami_oquvchilar FROM oquvchilar;
SELECT COUNT(email) AS emaili_borlar FROM oquvchilar;
SELECT COUNT(DISTINCT fan) AS fanlar_soni FROM baholar;

SELECT ROUND(AVG(baho), 2) AS ortacha_baho,      -- ROUND — yaxlitlash
       MAX(baho) AS eng_yuqori,
       MIN(baho) AS eng_past
FROM baholar
WHERE fan = 'Matematika';

SELECT MIN(tugilgan_sana) AS eng_kattasi, MAX(tugilgan_sana) AS eng_kichigi
FROM oquvchilar;
```

Natija

```
ortacha_baho | eng_yuqori | eng_past
-----+-----+-----
         4.40 |          5 |          3
```

GROUP BY — guruhlab hisoblash

"Butun maktab o'rtacha bahosi" — bitta son. Lekin ko'pincha **har bir guruh uchun** natija kerak: **har bir fan** bo'yicha o'rtacha, **har bir sinfda** nechta o'quvchi...



O'xshatish: rangli qalamlar

Stol ustida aralash qalamlar. Siz ularni rangiga qarab **uyumlarga ajratasiz** (GROUP BY rang), keyin **har bir uyumni sanaysiz** (COUNT): "qizil — 5, ko'k — 3, yashil — 7".

SQL

```
-- Har bir fan bo'yicha baholar soni va o'rtachasi
SELECT fan,
       COUNT(*) AS baholar_soni,
       ROUND(AVG(baho), 2) AS ortacha
FROM baholar
GROUP BY fan
ORDER BY ortacha DESC;
```

Natija

fan	baholar_soni	ortacha
Ona tili	2	4.50
Informatika	5	4.00
Matematika	5	4.40
...		

SQL

```
-- Har bir sinfda nechta o'quvchi (JOIN + GROUP BY)
SELECT s.nomi AS sinf, COUNT(o.id) AS oquvchilar_soni
FROM sinflar s
LEFT JOIN oquvchilar o ON o.sinf_id = s.id
GROUP BY s.nomi
ORDER BY s.nomi;

-- Har bir o'quvchining o'rtacha bahosi (reyting)
SELECT o.ism || ' ' || o.familiya AS oquvchi,
       COUNT(b.id) AS baholar,
       ROUND(AVG(b.baho), 2) AS ortacha
FROM oquvchilar o
JOIN baholar b ON b.oquvchi_id = o.id
GROUP BY o.id, o.ism, o.familiya
ORDER BY ortacha DESC;
```




GROUP BY'ning oltin qoidasi

`SELECT` dagi har bir ustun yoki **agregat funksiya ichida** bo'lishi, yoki `GROUP BY` da ko'rsatilgan bo'lishi kerak. `SELECT fan, ism, AVG(baho) ... GROUP BY fan` — xato: fan bo'yicha guruhda ko'p ism bor, qaysi birini ko'rsatish kerak?

HAVING — guruhlarni filtrlash

`WHERE` qatorlarni guruhlashdan **oldin** filtrlaydi. `HAVING` esa **guruhlarni** hisob-kitobdan **keyin** filtrlaydi.

SQL

```
-- O'rtacha bahosi 4.5 dan yuqori o'quvchilar
SELECT o.ism, ROUND(AVG(b.baho), 2) AS ortacha
FROM oquvchilar o
JOIN baholar b ON b.oquvchi_id = o.id
GROUP BY o.id, o.ism
HAVING AVG(b.baho) > 4.5;

-- Faqat sentabr baholari asosida (WHERE), kamida 2 ta baho olgan (HAVING) o'quvchilar
SELECT oquvchi_id, COUNT(*) AS soni
FROM baholar
WHERE sana ≥ '2026-09-01'           -- 1) qatorlar filtri
GROUP BY oquvchi_id                -- 2) guruhlash
HAVING COUNT(*) ≥ 2                 -- 3) guruhlar filtri
ORDER BY soni DESC;                 -- 4) saralash
```

SELECT'ning to'liq tuzilishi va bajarilish tartibi

Yozilish tartibi	Bajarilish tartibi	Vazifasi
1. <code>SELECT</code>	5	Qaysi ustunlarni ko'rsatish
2. <code>FROM</code> / <code>JOIN</code>	1	Qaysi jadval(lar)dan
3. <code>WHERE</code>	2	Qatorlarni filtrlash

4.	GROUP BY	3	Guruhlash
5.	HAVING	4	Guruhlarni filtrlash
6.	ORDER BY	6	Saralash
7.	LIMIT	7	Natijalar sonini cheklash

Shuning uchun `WHERE AVG(baho) > 4` yozib bo'lmaydi — `WHERE` bajarilayotganda guruhlar va o'rtacha hali hisoblanmagan!

Amaliy mashg'ulot: hisobotlar

28 va 29-mavzulardagi `maktab` va `dokon` bazalarida quyidagi hisobotlarni tayyorlang:

Maktab bazasi

- 1 Familiyasi "ov" bilan tugaydigan o'quvchilar.
- 2 2012-yilda tug'ilgan o'quvchilar soni.
- 3 Har bir fan bo'yicha: baholar soni, o'rtacha, eng past va eng yuqori baho.
- 4 Har bir sinf bo'yicha o'rtacha baho (3 ta jadval: sinflar, oquvchilar, baholar).
- 5 Kamida bitta "2" olgan o'quvchilar ro'yxati (takrorlarsiz).
- 6 Eng yuqori o'rtacha bahoga ega 3 nafar o'quvchi.

Do'kon bazasi

- 1 Narxi 4 000 dan 100 000 gacha bo'lgan mahsulotlar.
- 2 Har bir buyurtmaning umumiy summasi: `SUM(soni * narx)`.
- 3 Har bir mijoz nechta buyurtma bergani va jami qancha to'lagani.
- 4 Eng ko'p sotilgan mahsulot (dona hisobida).
- 5 Umumiy summasi 50 000 so'mdan oshgan buyurtmalar (HAVING).

▼ Ba'zi javoblar

SQL

```
-- Maktab 4: har bir sinf bo'yicha o'rtacha baho
SELECT s.nomi, ROUND(AVG(b.baho), 2) AS ortacha
FROM sinflar s
JOIN oquvchilar o ON o.sinf_id = s.id
JOIN baholar b ON b.oquvchi_id = o.id
GROUP BY s.nomi
ORDER BY ortacha DESC;

-- Maktab 5: kamida bitta "2" olganlar
SELECT DISTINCT o.ism, o.familiya
FROM oquvchilar o JOIN baholar b ON b.oquvchi_id = o.id
WHERE b.baho = 2;

-- Do'kon 3: mijozlar statistikasi
SELECT m.ism, COUNT(DISTINCT b.id) AS buyurtmalar, COALESCE(SUM(t.soni *
t.narx), 0) AS jami
FROM mijozlar m
LEFT JOIN buyurtmalar b ON b.mijoz_id = m.id
LEFT JOIN buyurtma_tarkibi t ON t.buyurtma_id = b.id
GROUP BY m.id, m.ism
ORDER BY jami DESC;

-- Do'kon 5: 50 000 dan oshgan buyurtmalar
SELECT buyurtma_id, SUM(soni * narx) AS summa
FROM buyurtma_tarkibi
GROUP BY buyurtma_id
HAVING SUM(soni * narx) > 50000;
```

`COALESCE(qiymat, 0)` — qiymat NULL bo'lsa 0 qaytaradi (buyurtmasi yo'q mijoz uchun).

Ko'p uchraydigan xatolar

- ✗ `column "o.ism" must appear in the GROUP BY clause` — SELECT'dagi oddiy ustun GROUP BY'da yo'q.
- ✗ `aggregate functions are not allowed in WHERE` — guruh sharti `HAVING` da yoziladi.
- ✗ `LIKE 'a%'` katta "A" ni topmaydi — `ILIKE` ishlatilg.

- ✗ `COUNT(ustun)` va `COUNT(*)` ni adashtirish — birinchisi NULL'larni sanamaydi.

O'zingizni tekshiring

? "A" harfi bilan boshlanadigan ismlarni topuvchi shart?

- A. `ism = 'A%'`
- B. `ism LIKE 'A%'`
- C. `ism LIKE '%A'`
- D. `ism IN 'A'`

? Jadvaldagi qatorlar sonini qaytaruvchi funksiya?

- A. `SUM(*)`
- B. `LENGTH(*)`
- C. `COUNT(*)`
- D. `TOTAL(*)`

? "Har bir fan bo'yicha o'rtacha baho" uchun nima kerak?

- A. `GROUP BY fan` + `AVG(baho)`
- B. `ORDER BY fan`
- C. `DISTINCT fan`
- D. `WHERE fan = AVG`

? Guruhlarni hisob-kitobdan keyin filtrlash uchun qaysi kalit so'z?

- A. WHERE
- B. FILTER
- C. LIMIT
- D. HAVING

? WHERE narx BETWEEN 100 AND 200 — 200 natijaga kiradimi?

- A. Yo'q
- B. Ha, chegaralar kiradi
- C. Faqat 100 kiradi
- D. Xato beradi

? Takrorlanmas qiymatlarni olish uchun?

- A. UNIQUE
- B. SINGLE
- C. DISTINCT
- D. GROUP



Uyga vazifa

1. Kutubxona bazangiz uchun 5 ta hisobot: eng ko'p o'qilgan kitob, eng faol o'quvchi, har bir muallifning kitoblari soni, qaytarilishi kechikkan ijaralar soni, oylar bo'yicha ijaralar soni (`EXTRACT(MONTH FROM sana)`).
2. SQLBolt saytida 1–12-darslarni bajaring (inglizcha, interaktiv).

✓ Yodda tuting

- Filtrlar: `IN`, `BETWEEN`, `LIKE '%x%'` / `ILIKE`, `IS NULL`, `AND/OR/NOT` (qavslar!).
- `DISTINCT` — takrorsiz; `ORDER BY`, `LIMIT`.
- Agregatlar: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`; `ROUND`, `COALESCE`.
- `GROUP BY` — guruhlab hisoblash; `HAVING` — guruh sharti; `WHERE` — qator sharti.
- Tartib: `SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY` → `LIMIT`.

Test javoblari

1. "A" harfi bilan boshlanadigan ismlarni topuvchi shart? — **B) ism LIKE 'A%'**
2. Jadvaldagi qatorlar sonini qaytaruvchi funksiya? — **C) COUNT(*)**
3. "Har bir fan bo'yicha o'rtacha baho" uchun nima kerak? — **A) GROUP BY fan + AVG(baho)**
4. Guruhlarni hisob-kitobdan keyin filtrlash uchun qaysi kalit so'z? — **D) HAVING**
5. WHERE narx BETWEEN 100 AND 200 — 200 natijaga kiradimi? — **B) Ha, chegaralar kiradi**
6. Takrorlanmas qiymatlarni olish uchun? — **C) DISTINCT**



31-MAVZU · 42 TADAN · III BOB. PYTHON VA MA'LUMOTLAR BAZASI

Yakuniy nazorat ishi — II–III boblar (JavaScript, Python, SQL)

JavaScript, Python va ma'lumotlar bazasi bo'yicha nazorat: 25 ta yopiq test (3 balldan) va 5 ta ochiq savol/amaliy topshiriq (5 balldan) — jami 100 ball.

Takrorlash rejasi, javoblar va baholash mezonlari.

1 soat · nazorat ishi

Nazorat ishi tuzilishi

- **1-qism:** 25 ta yopiq test × 3 ball = **75 ball** (JavaScript — 8, Python — 9, SQL — 8 ta savol).
- **2-qism:** 5 ta ochiq savol va amaliy topshiriq × 5 ball = **25 ball**.
- **Jami: 100 ball.** Vaqt: 80 daqiqa.

Takrorlash rejasi

Mavzu	Bilishingiz kerak
15–16. JS asoslari	let/const, turlar, typeof, operatorlar, ===, if/else, switch, ternary
17–18. Sikllar, funksiyalar, massivlar	for, while, break, function, return, arrow; push/pop, map/filter/reduce, obyektlar
19–20. DOM va hodisalar	querySelector,.textContent,classList,createElement,addEventListener,preventDefault,localStorage
21–22. Python asoslari	print/input, turlar, // va %, f-satr, if/elif/else, for+range, while, try/except, fayllar

23–25. Tuzilmalar,
funksiyalar, OOP

list/tuple/set/dict, slicing, def/return, modullar, json; class, __init__,
self, meros

26–30. Ma'lumotlar
bazasi

jadval/ustun/qator, PK/FK, bog'lanishlar, turlar, CREATE, CRUD,
JOIN, indeks, GROUP BY, HAVING

1-qism: Yopiq testlar (25 × 3 = 75 ball)

JavaScript

? 1. Qiymatini o'zgartirib bo'lmaydigan o'zgaruvchi?

- A. `let`
- B. `var`
- C. `const`
- D. `static`

? 2. `console.log(10 % 4)` natijasi?

- A. 2.5
- B. 2
- C. 40
- D. 6

? 3. `"3" === 3` natijasi?

- A. true
- B. "33"
- C. 6
- D. false

? 4. `for (let i = 1; i ≤ 3; i++)` sikli necha marta bajariladi?

- A. 3
- B. 2
- C. 4
- D. Cheksiz

? 5. `[5, 10, 15].map(x ⇒ x / 5)` natijasi?

- A. 30
- B. [5, 10, 15]
- C. [1, 2, 3]
- D. 6

? 6. `const k = { ism: "Ali" };` — ismni o'qish?

- A. `k(ism)`
- B. `k.ism`
- C. `k→ism`
- D. `k[ism]`

? 7. Tugma bosilishiga javob beruvchi to'g'ri kod?

- A. `btn.addEventListener("click", fn)`
- B. `btn.on("press", fn)`
- C. `btn.click = fn()`
- D. `addEvent(btn, fn)`

? 8. Massivni `localStorage`'ga saqlashdan oldin nima qilinadi?

- A. `Number()`
- B. `toString()`
- C. Hech narsa
- D. `JSON.stringify()`

Python

? 9. `print(17 // 5)` natijasi?

- A. 3.4
- B. 3
- C. 2
- D. 85

? 10. `input()` qaytaradigan qiymat turi?

- A. int
- B. float
- C. str
- D. bool

? 11. JS'dagi `else if` ning Python'dagi ko'rinishi?

- A. `elif`
- B. `else if`
- C. `elsif`
- D. `orif`

? 12. `list(range(1, 5))` natijasi?

- A. [1, 2, 3, 4, 5]
- B. [0, 1, 2, 3, 4]
- C. [1, 5]
- D. [1, 2, 3, 4]

? 13. Qaysi tuzilma o'zgartirilmaydi (immutable)?

- A. list
- B. tuple
- C. dict
- D. set

? 14. `d = {"a": 1}; print(d.get("b", 0))` natijasi?

- A. KeyError
- B. None
- C. 0
- D. 1

? 15. `int("abc")` qanday xato beradi?

- A. ValueError
- B. TypeError
- C. NameError
- D. IndexError

? 16. Obyekt yaratilganda avtomatik chaqiriladigan metod?

- A. `__str__`
- B. `__new__`
- C. `start()`
- D. `__init__`

? 17. `class It(Hayvon):` — bu OOP'ning qaysi tamoyili?

- A. Inkapsulyatsiya
- B. Merosxo'rlik
- C. Polimorfizm
- D. Abstraksiya

Ma'lumotlar bazasi va SQL

? 18. Har bir qatorni noyob aniqlovchi ustun?

- A. Tashqi kalit
- B. Indeks
- C. Birlamchi kalit (PK)
- D. NOT NULL

? 19. Pul summasi uchun eng mos tur?

- A. NUMERIC(12, 2)
- B. REAL
- C. VARCHAR
- D. INTEGER

? 20. CRUD'dagi "C" ga mos SQL buyrug'i?

- A. CREATE
- B. SELECT
- C. UPDATE
- D. INSERT

? 21. `WHERE` siz `DELETE FROM oquvchilar;` nima qiladi?

- A. Xato beradi
- B. Barcha qatorlarni o'chiradi
- C. Birinchi qatorni o'chiradi
- D. Jadvalni o'chiradi

? 22. Chap jadvalning barcha qatorlarini qaytaruvchi JOIN?

- A. INNER JOIN
- B. RIGHT JOIN
- C. LEFT JOIN
- D. CROSS JOIN

? 23. Emaili kiritilmaganlarni topuvchi shart?

- A. `email IS NULL`
- B. `email = NULL`
- C. `email = ''`
- D. `NOT email`

? 24. "Har bir sinfdagi o'quvchilar soni" uchun nima kerak?

- A. `ORDER BY`
- B. `DISTINCT`
- C. `LIMIT`
- D. `GROUP BY` + `COUNT`

? 25. Guruhlangan natijani shart bo'yicha filtrlash?

- A. `WHERE`
- B. `HAVING`
- C. `FILTER`
- D. `IF`

2-qism: Ochiq savollar va amaliy topshiriqlar (5 × 5 = 25 ball)

1-topshiriq (5 ball). JavaScript: DOM

Sahifada `<input id="ism">`, `<button id="btn">` va `<p id="javob">` bor. Tugma bosilganda, agar maydon bo'sh bo'lsa — "Ismingizni kiriting!" qizil rangda, aks holda "Salom, [ism]!" yashil rangda chiqsin.

▼ Namuna javob va baholash

HTML

```
<input id="ism" placeholder="Ism">
<button id="btn">Salomlash</button>
<p id="javob"></p>
<script>
  document.querySelector("#btn").addEventListener("click", () => {
    const ism = document.querySelector("#ism").value.trim();
    const javob = document.querySelector("#javob");
    if (ism === "") {
      javob.textContent = "Ismingizni kiriting!";
      javob.style.color = "red";
    } else {
      javob.textContent = `Salom, ${ism}!`;
      javob.style.color = "green";
    }
  });
</script>
```

Baholash: elementlarni topish — 1; hodisa tinglovchisi — 1; bo'sh tekshiruvi (trim bilan) — 1; textContent bilan chiqarish — 1; ranglar — 1.

2-topshiriq (5 ball). Python: funksiya va ro'yxat

`statistika(ballar)` funksiyasini yozing: u ballar ro'yxatini olib, o'rtacha (2 xona kasr), eng yuqori, eng past ball va 86 dan yuqori ballar sonini dict ko'rinishida qaytarsin. Bo'sh ro'yxat uchun `None` qaytarsin.

▼ Namuna javob

Python

```
def statistika(ballar):  
    if not ballar:  
        return None  
    return {  
        "ortacha": round(sum(ballar) / len(ballar), 2),  
        "eng_yuqori": max(ballar),  
        "eng_past": min(ballar),  
        "alo": len([b for b in ballar if b > 86]),  
    }  
  
print(statistika([90, 75, 88, 60]))  
# {'ortacha': 78.25, 'eng_yuqori': 90, 'eng_past': 60, 'alo': 2}
```

Baholash: def va return — 1; bo'sh holat — 1; o'rtacha va yaxlitlash — 1; max/min — 1; shartli sanash — 1.

3-topshiriq (5 ball). Python OOP

Tortburchak klassini yozing: konstruktor eni va bo'yini oladi; `yuza()`, `perimetr()` metodlari va `__str__` ("Tortburchak 4×5"). Undan meros olgan Kvadrat klassi faqat bitta tomonni oladi.

▼ Namuna javob

Python

```
class Tortburchak:
    def __init__(self, eni, boyi):
        self.eni = eni
        self.boyi = boyi

    def yuza(self):
        return self.eni * self.boyi

    def perimetr(self):
        return 2 * (self.eni + self.boyi)

    def __str__(self):
        return f"Tortburchak {self.eni}x{self.boyi}"

class Kvadrat(Tortburchak):
    def __init__(self, tomon):
        super().__init__(tomon, tomon)

k = Kvadrat(4)
print(k, k.yuza(), k.perimetr())    # Tortburchak 4x4 16 16
```

Baholash: klass va `__init__` — 1; metodlar — 2; `__str__` — 1; meros va `super()` — 1.

4-topshiriq (5 ball). SQL: jadval va CRUD

`kitoblar` jadvalini yarating (id, nomi — majburiy, muallif, yil — 1450 dan katta, narx — pul, mavjud — boolean, standart TRUE). 3 ta kitob qo'shing; 2000-yildan keyin chiqqan kitoblarni narx bo'yicha kamayish tartibida chiqaring; bitta kitob narxini 10% oshiring.

▼ Namuna javob

SQL

```
CREATE TABLE kitoblar (  
    id        SERIAL PRIMARY KEY,  
    nomi      VARCHAR(200) NOT NULL,  
    muallif   VARCHAR(100),  
    yil       INTEGER CHECK (yil > 1450),  
    narx      NUMERIC(10, 2),  
    mavjud    BOOLEAN DEFAULT TRUE  
);  
  
INSERT INTO kitoblar (nomi, muallif, yil, narx) VALUES  
    ('O'tkan kunlar', 'Abdulla Qodiriy', 1926, 65000),  
    ('Ufq', 'Said Ahmad', 1976, 48000),  
    ('Python asoslari', 'Muallif X', 2021, 90000);  
  
SELECT nomi, yil, narx FROM kitoblar WHERE yil > 2000 ORDER BY narx  
DESC;  
  
UPDATE kitoblar SET narx = narx * 1.1 WHERE id = 1;
```

Baholash: jadval va turlar — 1; cheklovlar — 1; INSERT (apostrof to'g'ri) — 1; SELECT + WHERE + ORDER BY — 1; UPDATE + WHERE — 1.

5-topshiriq (5 ball). SQL: JOIN va GROUP BY

`sinflar(id, nomi)`, `oquvchilar(id, ism, sinf_id)`, `baholar(id, oquvchi_id, baho)` jadvallari berilgan. Har bir sinfning nomi, o'quvchilar soni va o'rtacha bahosini chiqaring; faqat o'rtacha bahosi 4 dan yuqori sinflar, o'rtacha baho bo'yicha kamayish tartibida.

▼ Namuna javob

SQL

```
SELECT s.nomi,  
       COUNT(DISTINCT o.id) AS oquvchilar,  
       ROUND(AVG(b.baho), 2) AS ortacha  
FROM sinflar s  
JOIN oquvchilar o ON o.sinf_id = s.id  
JOIN baholar b   ON b.oquvchi_id = o.id  
GROUP BY s.nomi  
HAVING AVG(b.baho) > 4  
ORDER BY ortacha DESC;
```

Baholash: JOIN'lar to'g'ri — 2; GROUP BY va agregatlar — 1 (DISTINCT'ni bilgani — ustiga rag'bat); HAVING — 1; ORDER BY — 1.

Baholash shkalasi

Ball	Natija	Tavsiya
86– 100	A'lo	IV bob — Django'ga tayyorsiz!
71–85	Yaxshi	Xato qilgan mavzular amaliyotini takrorlang.
60– 70	Qoniqarli	Ayniqsa OOP (25) va SQL JOIN/GROUP BY (29–30) ni qayta ishlang — Django'da ular juda kerak bo'ladi.
0–59	Qoniqarsiz	Mentor bilan qo'shimcha mashg'ulot tavsiya etiladi.



III bob yakunlandi!

Siz endi ikki dasturlash tilida (JavaScript va Python) dastur yoza olasiz, OOP tamoyillarini bilasiz va ma'lumotlar bazasini loyihalab, SQL bilan boshqara olasiz. Keyingi bobda bularning hammasini birlashtiramiz: **Python + PostgreSQL + Django = haqiqiy veb-ilova!**

✓ Test javoblari

1. 1. Qiymatini o'zgartirib bo'lmaydigan o'zgaruvchi? — **C) const**
2. 2. console.log(10 % 4) natijasi? — **B) 2**
3. 3. "3" === 3 natijasi? — **D) false**
4. 4. for (let i = 1; i <= 3; i++) sikli necha marta bajariladi? — **A) 3**
5. 5. [5, 10, 15].map(x ⇒ x / 5) natijasi? — **C) [1, 2, 3]**
6. 6. const k = { ism: "Ali" }; — ismni o'qish? — **B) k.ism**
7. 7. Tugma bosilishiga javob beruvchi to'g'ri kod? — **A) btn.addEventListener("click", fn)**
8. 8. Massivni localStorage'ga saqlashdan oldin nima qilinadi? — **D) JSON.stringify()**
9. 9. print(17 // 5) natijasi? — **B) 3**
10. 10. input() qaytaradigan qiymat turi? — **C) str**
11. 11. JS'dagi else if ning Python'dagi ko'rinishi? — **A) elif**
12. 12. list(range(1, 5)) natijasi? — **D) [1, 2, 3, 4]**
13. 13. Qaysi tuzilma o'zgartirilmaydi (immutable)? — **B) tuple**
14. 14. d = {"a": 1}; print(d.get("b", 0)) natijasi? — **C) 0**
15. 15. int("abc") qanday xato beradi? — **A) ValueError**
16. 16. Obyekt yaratilganda avtomatik chaqiriladigan metod? — **D) __init__**
17. 17. class It(Hayvon): — bu OOP'ning qaysi tamoyili? — **B) Merosxo'rlik**
18. 18. Har bir qatorni noyob aniqlovchi ustun? — **C) Birlamchi kalit (PK)**
19. 19. Pul summasi uchun eng mos tur? — **A) NUMERIC(12, 2)**
20. 20. CRUD'dagi "C" ga mos SQL buyrug'i? — **D) INSERT**
21. 21. WHEREsiz DELETE FROM oquvchilar; nima qiladi? — **B) Barcha qatorlarni o'chiradi**
22. 22. Chap jadvalning barcha qatorlarini qaytaruvchi JOIN? — **C) LEFT JOIN**
23. 23. Emaili kiritilmaganlarni topuvchi shart? — **A) email IS NULL**
24. 24. "Har bir sinfdagi o'quvchilar soni" uchun nima kerak? — **D) GROUP BY + COUNT**
25. 25. Guruhlangan natijani shart bo'yicha filtrlash? — **B) HAVING**



32-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Python dasturlash tilini PostgreSQL ma'lumotlar bazasiga bog'lash

Virtual muhit, psycopg kutubxonasi, ulanish, kursor, SQL so'rovlarni Python'dan bajarish, parametrli so'rovlar va SQL injection xavfi, commit va rollback, maxfiy sozlamalar; Python + PostgreSQL CRUD konsol dasturi.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- Python dasturi ma'lumotlar bazasi bilan qanday "gaplashishini";
- Virtual muhit yaratish va `psycopg` kutubxonasini o'rnatishni;
- Bazaga ulanish, so'rov yuborish va natijani olishni;
- Eng xavfli zaiflik — **SQL injection** va undan himoyalaniшни;
- Bazada ishlaydigan to'liq CRUD konsol dasturini (20-loyiha).

Python va baza qanday bog'lanadi?

III bobda SQL'ni pgAdmin orqali qo'lda yozdik. Haqiqiy ilovalarda esa SQL so'rovlarni **dastur** yuboradi: foydalanuvchi formani to'ldiradi → Python ma'lumotni oladi → bazaga `INSERT` yuboradi.





O'xshatish: telefon qo'ng'irog'i

Ulanish (connection) — telefon raqamini terib, suhbat boshlash. **Kursor (cursor)** — suhbat davomida savol berib, javob olish. **Commit** — "kelishdik, yozib qo'yning". **Yopish (close)** — go'shakni qo'yish.

Tayyorgarlik

1. Virtual muhit (venv)

Har bir loyiha o'z kutubxonalarini alohida "quti"da saqlashi kerak — shunda bir loyihaning paketlari boshqasiga xalaqit bermaydi.

Terminal

```
mkdir maktab_app
cd maktab_app
python -m venv venv # "venv" nomli virtual muhit yaratish

# Faollashtirish
venv\Scripts\activate # Windows
source venv/bin/activate # macOS / Linux
# Terminal boshida (venv) yozuvi paydo bo'ladi

pip install "psycopg[binary]" # PostgreSQL uchun Python kutubxonasi
pip freeze > requirements.txt # o'rnatilgan paketlar ro'yxatini saqlash
```



`requirements.txt` — loyihaga kerakli paketlar ro'yxati. Boshqa kompyuterda `pip install -r requirements.txt` bilan hammasini birdan o'rnatish mumkin. `venv` papkasini esa GitHub'ga yuklamang (`.gitignore` ga qo'shing).

2. Baza

28–30-mavzulardagi `maktab` bazasidan foydalanamiz (sinflar, oquvchilar, baholar jadvallari bilan).

Birinchi ulanish

test_ulanish.py

```
import psycopgp

conn = psycopgp.connect(
    host="localhost",
    port=5432,
    dbname="maktab",
    user="postgres",
    password="SIZNING_PAROLINGIZ",
)

cur = conn.cursor()                # kursor — so'rov yuborish
vositasi                           # SQL so'rov
cur.execute("SELECT version();")   # bitta natija qatori
print(cur.fetchone())

cur.close()
conn.close()
```

Natija

```
('PostgreSQL 17.0 on x86_64-windows, compiled by ...',)
```

with bilan — avtomatik yopish (tavsiya etiladi)

Python

```
import psycopgp

with psycopgp.connect("dbname=maktab user=postgres password=SIZNING_PAROLINGIZ
host=localhost") as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT id, ism, familiya FROM oquvchilar ORDER BY
familiya")
        for qator in cur.fetchall():          # barcha natijalar: tuple'lar
ro'yxati
            print(qator)
# with tugaganda: xato bo'lmasa — commit, xato bo'lsa — rollback, so'ng ulanish
yopiladi
```

Natija

```
(6, 'Dilnoza', 'Aminova')
(4, 'Madina', 'Ergasheva')
(2, 'Zarina', 'Karimova')
...
```

Metod	Vazifasi
<code>cur.execute(sql, params)</code>	So'rovni bajarish
<code>cur.fetchone()</code>	Keyingi bitta qator (yoki <code>None</code>)
<code>cur.fetchall()</code>	Barcha qatorlar ro'yxati
<code>cur.fetchmany(5)</code>	Keyingi 5 ta qator
<code>cur.rowcount</code>	O'zgargan qatorlar soni (UPDATE/DELETE dan keyin)
<code>conn.commit()</code>	O'zgarishlarni saqlash
<code>conn.rollback()</code>	O'zgarishlarni bekor qilish



commit'ni unutmang!

`INSERT`, `UPDATE`, `DELETE` dan keyin `conn.commit()` chaqirilmasa, o'zgarishlar **saqlanmaydi** (tranzaksiya, 28-mavzu). `with psycopg.connect(...) as conn:` bloki muvaffaqiyatli tugaganda commit avtomatik bajariladi.



SQL injection — eng xavfli xato

Foydalanuvchi kiritgan qiymatni SQL satriga **to'g'ridan-to'g'ri qo'shish** — veb-saytlardagi eng xavfli va keng tarqalgan zaifliklardan biri.

Python

```
# ❌ HECH QACHON BUNDAY QILMANG
ism = input("Ism: ")
cur.execute(f"SELECT * FROM oquvchilar WHERE ism = '{ism}'")
```

Oddiy foydalanuvchi `Ali` yozadi — hammasi joyida. Lekin buzg'unchi quyidagini yozsa:

Matn

```
' OR '1'='1
```

SQL shunday bo'lib qoladi: `SELECT * FROM oquvchilar WHERE ism = '' OR '1'='1'` — `'1'='1'` doim rost, demak **barcha o'quvchilar** ma'lumoti chiqib ketadi! Yoki undan ham yomoni: `'; DROP TABLE oquvchilar; --` — jadval o'chiriladi.



O'xshatish: buyruq qog'ozi

Siz yordamchingizga "Ombordan ____ ni olib kel" degan qog'oz berasiz va bo'sh joyni mijoz to'ldiradi. Ayyor mijoz "olma. Keyin seyfdagi pullarni ham olib kel" deb yozsa, yordamchingiz ikkalasini ham bajaradi. Parametrli so'rov esa bo'sh joyga yozilgan narsani **faqat mahsulot nomi** deb tushunadi — buyruq sifatida emas.

Python

```
#  TO'G'RI: parametrli so'rov — %s o'rniga qiymatni psycopg o'zi xavfsiz qo'yadi
ism = input("Ism: ")
cur.execute("SELECT * FROM oquvchilar WHERE ism = %s", (ism,)) # (ism,) — bitta elementli tuple!

# Bir nechta parametr
cur.execute(
    "INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES (%s, %s, %s)",
    ("Jasur", "Rahimov", 2),
)
```



Qat'iy qoida

SQL'ga kiradigan har qanday o'zgaruvchan qiymat — **faqat** `%s` **parametr orqali**. f-string, `+` yoki `.format()` bilan SQL yasamang! (Eslatma: `%s` — barcha turlar uchun, son uchun ham; atrofiga qo'shtirnoq qo'yilmaydi.)

CRUD'ni Python'dan bajarish

Python

```
import psycopgp

DSN = "dbname=maktab user=postgres password=SIZNING_PAROLINGIZ host=localhost"

with psycopgp.connect(DSN) as conn:
    with conn.cursor() as cur:
        # CREATE
        cur.execute(
            "INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES (%s, %s, %s) RETURNING id",
            ("Laylo", "Usmonova", 1),
        )
        yangi_id = cur.fetchone()[0]
        print("Yangi o'quvchi id:", yangi_id)

        # READ
        cur.execute("SELECT ism, familiya FROM oquvchilar WHERE id = %s",
            (yangi_id,))
        print(cur.fetchone())

        # UPDATE
        cur.execute("UPDATE oquvchilar SET email = %s WHERE id = %s",
            ("laylo@mail.uz", yangi_id))
        print("Yangilandi:", cur.rowcount, "qator")

        # DELETE
        cur.execute("DELETE FROM oquvchilar WHERE id = %s", (yangi_id,))
        print("O'chirildi:", cur.rowcount, "qator")
    # bu yerda avtomatik conn.commit()
```

Natijani dict ko'rinishida olish

Python

```
from psycopg.rows import dict_row

with psycopg.connect(DSN, row_factory=dict_row) as conn:
    rows = conn.execute("SELECT id, ism, familiya FROM oquvchilar LIMIT 2").fetchall()
    for r in rows:
        print(r["ism"], r["familiya"])    # indeks o'rniga nom bilan – o'qilishi oson
# [{'id': 1, 'ism': 'Ali', 'familiya': 'Valiyev'}, ...]
```

Maxfiy sozlamalarni koddan ajratish

Parolni kod ichida yozib, GitHub'ga yuklasangiz — uni butun dunyo ko'radi! Sozlamalarni **muhit o'zgaruvchilari** (environment variables) yoki alohida faylda saqlang:

Terminal

```
pip install python-dotenv
```

.env (GitHub'ga YUKLANMAYDI!)

```
DB_NAME=maktab
DB_USER=postgres
DB_PASSWORD=SIZNING_PAROLINGIZ
DB_HOST=localhost
```

.gitignore

```
venv/
.env
__pycache__/
```

db.py

```
import os
import psycopg
from psycopg.rows import dict_row
from dotenv import load_dotenv

load_dotenv()          # .env faylidagi qiymatlarni o'qiydi

def ulanish():
    return psycopg.connect(
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        host=os.getenv("DB_HOST", "localhost"),
        row_factory=dict_row,
    )
```

Amaliy mashg'ulot: Python + PostgreSQL CRUD dasturi (20-loyiha)

23-mavzudagi o'quvchilar dasturini endi **haqiqiy bazada** ishlaydigan qilamiz. Ma'lumotlar dastur yopilganda ham saqlanib qoladi.

```

import psycpg
from db import ulanish

def royxat():
    with ulanish() as conn:
        rows = conn.execute("""
            SELECT o.id, o.ism, o.familiya, s.nomi AS sinf,
                   ROUND(AVG(b.baho), 2) AS ortacha
            FROM oquvchilar o
            LEFT JOIN sinflar s ON s.id = o.sinf_id
            LEFT JOIN baholar b ON b.oquvchi_id = o.id
            GROUP BY o.id, s.nomi
            ORDER BY o.familiya
        """).fetchall()
    print(f"\n{'ID':<4}{'Ism':<12}{'Familiya':<14}{'Sinf':<6}{'O'rtacha':>8}")
    print("-" * 44)
    for r in rows:
        print(f"{r['id']:<4}{r['ism']:<12}{r['familiya']:<14}{r['sinf'] or '-':<6}{r['ortacha'] or '-':>8}")

def qoshish():
    ism = input("Ism: ").strip()
    familiya = input("Familiya: ").strip()
    if not ism or not familiya:
        print("❌ Ism va familiya majburiy")
        return
    with ulanish() as conn:
        sinflar = conn.execute("SELECT id, nomi FROM sinflar ORDER BY
nomi").fetchall()
        print("Sinflar:", ", ".join(f"{s['id']}={s['nomi']}" for s in sinflar))
        try:
            sinf_id = int(input("Sinf id: "))
            r = conn.execute(
                "INSERT INTO oquvchilar (ism, familiya, sinf_id) VALUES (%s,
%s, %s) RETURNING id",
                (ism, familiya, sinf_id),
            ).fetchone()
            print(f"✅ Qo'shildi (id = {r['id']})")
        except ValueError:
            print("❌ Sinf id son bo'lishi kerak")

```



```

except psycopg.errors.ForeignKeyViolation:
    print("❌ Bunday sinf yo'q")

def baho_qoyish():
    try:
        oquvchi_id = int(input("O'quvchi id: "))
        fan = input("Fan: ").strip()
        baho = int(input("Baho (2-5): "))
    except ValueError:
        print("❌ Son kiriting")
        return
    try:
        with ulanish() as conn:
            conn.execute(
                "INSERT INTO baholar (oquvchi_id, fan, baho) VALUES (%s, %s, %s)",
                (oquvchi_id, fan, baho),
            )
            print("✅ Baho qo'yildi")
    except psycopg.errors.CheckViolation:
        print("❌ Baho 2 dan 5 gacha bo'lishi kerak")
    except psycopg.errors.ForeignKeyViolation:
        print("❌ Bunday o'quvchi yo'q")

def qidirish():
    soz = input("Ism yoki familiya qismi: ").strip()
    with ulanish() as conn:
        rows = conn.execute(
            "SELECT id, ism, familiya FROM oquvchilar WHERE ism ILIKE %s OR familiya ILIKE %s",
            (f"%{soz}%", f"%{soz}%"),
        ).fetchall()
        # % belgilarini QIYMATga qo'shamiz, SQL'ga emas
    for r in rows:
        print(f" {r['id']}. {r['ism']} {r['familiya']}")
    if not rows:
        print(" Hech kim topilmadi")

def ochirish():
    try:
        oquvchi_id = int(input("O'chiriladigan o'quvchi id: "))

```

```

except ValueError:
    print("✗ Son kiriting")
    return
with ulanish() as conn:
    r = conn.execute("SELECT ism, familiya FROM oquvchilar WHERE id = %s",
(oquvchi_id,)).fetchone()
    if not r:
        print("✗ Topilmadi")
        return
    if input(f"{r['ism']} {r['familiya']} o'chirilsinmi? (ha/yo'q):
").lower() == "ha":
        conn.execute("DELETE FROM oquvchilar WHERE id = %s", (oquvchi_id,))
        print("✅ O'chirildi")

MENYU = {
    "1": ("Ro'yxat", royxat),
    "2": ("O'quvchi qo'shish", qoshish),
    "3": ("Baho qo'yish", baho_qoyish),
    "4": ("Qidirish", qidirish),
    "5": ("O'chirish", ochirish),
}

if __name__ == "__main__":
    while True:
        print("\n===== MAKTAB BAZASI =====")
        for k, (nom, _) in MENYU.items():
            print(f"{k}. {nom}")
        print("0. Chiqish")
        t = input("> ").strip()
        if t == "0":
            break
        if t in MENYU:
            MENYU[t][1]()

```

Topshiriqlar:

- 1 Dasturni ishga tushiring. pgAdmin'da bir vaqtning o'zida jadvalni ochib, dastur qo'shgan ma'lumotlar bazada paydo bo'layotganini kuzating.
- 2 "Qidirish" ga `' OR '1'='1` yozib ko'ring — parametrli so'rov tufayli hech qanday "teshik" yo'qligiga ishonch hosil qiling.
- 3 "Sinf statistikasi" menyu bandini qo'shing (30-mavzudagi GROUP BY so'rovi).

Ko'p uchraydigan xatolar

- ❌ `ModuleNotFoundError: No module named 'psycopg'` — virtual muhit faollashtirilmagan yoki paket o'rnatilmagan.
- ❌ `connection refused` — PostgreSQL serveri ishlamayapti yoki port/host noto'g'ri.
- ❌ `(ism)` yozish — bu tuple emas, oddiy qavs! Bitta parametr uchun: `(ism,)` — vergul bilan.
- ❌ `'%s'` — qo'shtirnoq ichida yozish; to'g'risi shunchaki `%s`.
- ❌ O'zgartirishdan keyin commit qilinmagani uchun ma'lumot "yo'qolib" qolishi.

O'zingizni tekshiring

? Python'dan PostgreSQL bilan ishlash uchun ishlatiladigan kutubxona?

- A. requests
- B. psycopg
- C. pygame
- D. flask

? INSERT dan keyin o'zgarishni bazada saqlash uchun nima chaqiriladi?

- A. `cur.save()`
- B. `conn.close()`
- C. `cur.fetchone()`
- D. `conn.commit()`

? SQL injection'dan himoyalangan to'g'ri kod?

- A. `cur.execute("... WHERE ism = %s", (ism,))`
- B. `cur.execute(f"... WHERE ism = '{ism}'")`
- C. `cur.execute("... WHERE ism = '" + ism + "'")`
- D. `cur.execute("... WHERE ism = {}".format(ism))`

? So'rovning barcha natija qatorlarini oluvchi metod?

- A. `fetchone()`
- B. `getall()`
- C. `fetchall()`
- D. `readall()`

? Parol kabi maxfiy sozlamalarni qayerda saqlash to'g'ri?

- A. Kod ichida, izoh bilan
- B. `.env` faylida (GitHub'ga yuklanmaydi)
- C. README faylida
- D. Ma'lumotlar bazasi nomida



Uyga vazifa

- 29-mavzudagi `dokon` bazasi uchun Python dasturi: mahsulotlar ro'yxati, mahsulot qo'shish, narxni o'zgartirish, omborda tugaganlarni ko'rsatish.
- "Buyurtma berish" funksiyasi: mijoz va mahsulotlarni tanlaydi, `buyurtmalar` va `buyurtma_tarkibi` ga yozadi, ombordagi `soni` ni kamaytiradi — hammasi **bitta tranzaksiyada**.

✓ Yodda tuting

- Virtual muhit: `python -m venv venv` → faollashtirish → `pip install "psycopg[binary]"`.
- `with psycopg.connect(...) as conn:` → `conn.execute(sql, params)` → `fetchone/fetchall`; blok tugaganda avtomatik commit.
- **Faqat parametrli so'rovlar:** `%s` + tuple — SQL injection'dan himoya.
- Maxfiy sozlamalar — `.env` da, `.gitignore` ga qo'shilgan.

✓ Test javoblari

1. Python'dan PostgreSQL bilan ishlash uchun ishlatiladigan kutubxona? — **B) psycopg**
2. INSERT dan keyin o'zgarishni bazada saqlash uchun nima chaqiriladi? — **D) conn.commit()**
3. SQL injection'dan himoyalangan to'g'ri kod? — **A) cur.execute("... WHERE ism = %s", (ism,))**
4. So'rovning barcha natija qatorlarini oluvchi metod? — **C) fetchall()**
5. Parol kabi maxfiy sozlamalarni qayerda saqlash to'g'ri? — **B) .env faylida (GitHub'ga yuklanmaydi)**



33-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django frameworkiga kirish. Arxitektura, loyiha va ilova (App) yaratish

Framework nima va nima uchun Django, MVT arxitekturasini, so'rovning Django ichidagi yo'li, o'rnatish, startproject va startapp, loyiha fayllari, settings.py, runserver, PostgreSQL'ga ulash.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- **Framework** nima va Django nima uchun shunchalik mashhurligini;
- Django'ning **MVT** (Model–View–Template) arxitekturasini;
- Foydalanuvchi so'rovi Django ichida qanday yo'l bosib o'tishini;
- Django'ni o'rnatish, **loyiha** va **ilova (app)** yaratishni;
- Loyihani PostgreSQL bazasiga ulashni.

Shu darsdan boshlab 38-mavzugacha bitta haqiqiy loyiha — "**Maktab blogi**"ni bosqichma-bosqich quramiz.

Framework nima?

32-mavzuda Python va baza bilan konsol dastur yozdik. Endi uni **veb-sayt**ga aylantirish uchun ko'p narsa kerak: brauzer so'rovlarini qabul qilish, manzillarni (URL) tahlil qilish, HTML sahifalar yasash, formalarni tekshirish, foydalanuvchilarni ro'yxatdan o'tkazish, parollarni xavfsiz saqlash, admin panel... Bularning hammasini noldan yozish — oylar.

Framework — tayyor "karkas": dasturchilar tomonidan ko'p yillar davomida sinalgan, keng tarqalgan vazifalarni allaqachon hal qilgan kod to'plami. Siz faqat **o'z loyihangizga xos** qismlarni yozasiz.



O'xshatish: uy qurish

Noldan dastur yozish — g'ishtni o'zingiz pishirib, uy qurish. **Framework** bilan ishlash — tayyor karkas (poydevor, devorlar, elektr va suv quvurlari) ustida xonalarni o'zingizga moslab bezash. Tezroq, xavfsizroq va sifatliroq.

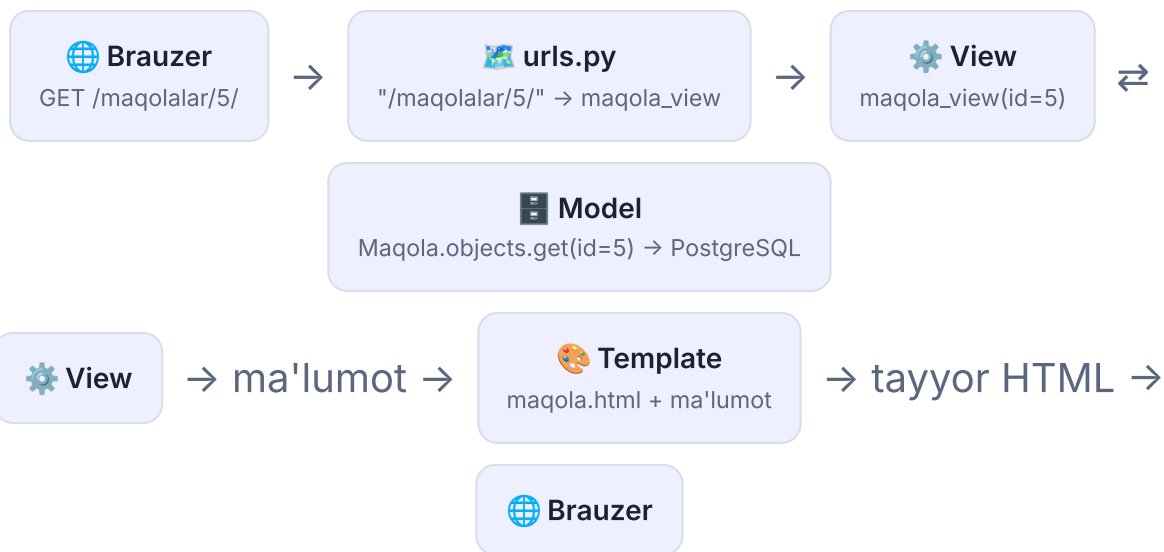
Nima uchun Django?

- 🔋 **"Batareyalar bilan birga"** — admin panel, foydalanuvchilar tizimi, formalar, ORM, xavfsizlik — hammasi tayyor.
- 🔒 **Xavfsiz** — SQL injection, XSS, CSRF kabi hujumlardan avtomatik himoya.
- 📈 **Masshtablanadi** — Instagram, Pinterest, Spotify, Mozilla saytlari Django'da ishlaydi (yoki ishlagan).
- 📚 **Ajoyib hujjatlar** va katta jamiyat. Shiori: "Muddati bor perfeksionistlar uchun veb-framework".

MVT arxitekturası

Django loyihasi kodni uch qismga ajratadi — har biri o'z ishi bilan shug'ullanadi:

Qism	Fayl	Vazifasi	Restoran o'xshatishi
Model	<code>models.py</code>	Ma'lumotlar tuzilmasi va baza bilan ishlash	Ombor va mahsulotlar ro'yxati
View	<code>views.py</code>	Mantiq: so'rovni qabul qilish, ma'lumot olish, javob tayyorlash	Oshpaz: buyurtmani tayyorlaydi
Template	<code>templates/*.html</code>	Ko'rinish: HTML shablon	Likopcha va bezak — taom qanday ko'rinishi
URL	<code>urls.py</code>	Qaysi manzil qaysi View'ga borishi	Ofitsiant: buyurtmani kerakli oshpazga yetkazadi



So'rovning Django ichidagi to'liq yo'li

i Boshqa frameworklarda **MVC** (Model–View–Controller) atamasi ishlatiladi. Django'da nomlar boshqacha: Django'ning **Viewi** — MVC'dagi Controller, **Templatei** — MVC'dagi View. G'oya bir xil.

Loyiha (project) va ilova (app)

- **Loyiha (project)** — butun veb-sayt: umumiy sozlamalar, asosiy URL'lar. Bitta.
- **Ilova (app)** — saytning mustaqil qismi, bitta vazifaga mas'ul: `blog`, `accounts` (foydalanuvchilar), `shop`. Loyihada bir nechta bo'ladi va ularni boshqa loyihalarda qayta ishlatish mumkin.



O'xshatish: maktab binosi

Loyiha — maktab binosi (umumiy qoidalar, kirish eshigi, qo'ng'iroq). **Ilovalar** — binodagi xonalar: kutubxona, sport zali, oshxona. Har biri o'z ishini qiladi, lekin hammasi bitta binoda va umumiy qoidalariga bo'ysunadi.

O'rnatish va loyiha yaratish

1

Loyiha papkasi va virtual muhit:

Terminal

```
mkdir maktab_blog
cd maktab_blog
python -m venv venv
venv\Scripts\activate          # Windows  (macOS/Linux: source
venv/bin/activate)

pip install django "psycopg[binary]" python-dotenv
python -m django --version      # 5.x
```

- 2 Loyiha yaratish (oxiridagi **nuqta**ga e'tibor bering — joriy papkada yaratadi):

Terminal

```
django-admin startproject config .
```

- 3 Serverni ishga tushirish:

Terminal

```
python manage.py runserver
```

Natija

```
Watching for file changes with StatReloader
System check identified no issues (0 silenced).
October 06, 2026 - 15:00:00
Django version 5.x, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Brauzerda <http://127.0.0.1:8000> ni oching — raketa 🚀 va "The install worked successfully!" yozuvi. Tabriklaymiz!

- 4 Ilova yaratish:

Terminal

```
python manage.py startapp blog
```

`config/settings.py`

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "blog",                                # ← bizning ilovamiz  
]
```



`runserver` — faqat **ishlab chiqish** (development) uchun server. Kod o'zgarganda avtomatik qayta ishga tushadi. Internetga chiqarish uchun boshqa server ishlatiladi (41-mavzu). Serverni to'xtatish: `Ctrl + C`.

Loyiha tuzilmasi

Matn

```
maktab_blog/  
├─ manage.py          ← loyihani boshqarish buyruqlari (runserver, migrate,  
...)                  ← ...)  
├─ venv/              ← virtual muhit (GitHub'ga yuklanmaydi)  
├─ config/            ← LOYIHA sozlamalari  
│   ├── __init__.py  
│   └── settings.py    ← barcha sozlamalar: ilovalar, baza, til, vaqt  
mintaqasi  
├─ urls.py            ← asosiy URL'lar ("bosh ofitsiant")  
├─ asgi.py / wsgi.py ← serverga joylash uchun (41-mavzu)  
└─ blog/  
    ├── migrations/    ← bazadagi o'zgarishlar tarixi (36-mavzu)  
    ├── __init__.py  
    ├── admin.py        ← admin panel sozlamalari (37-mavzu)  
    ├── apps.py          ← ilova sozlamalari  
    ├── models.py        ← MODELLAR (36-mavzu)  
    ├── tests.py         ← testlar  
    └── views.py         ← VIEW'lar (34-mavzu)
```

Buyruq	Vazifasi
<code>python manage.py runserver</code>	Serverni ishga tushirish
<code>python manage.py startapp nom</code>	Yangi ilova yaratish
<code>python manage.py makemigrations</code>	Model o'zgarishlaridan migratsiya yaratish
<code>python manage.py migrate</code>	Migratsiyalarni bazaga qo'llash
<code>python manage.py createsuperuser</code>	Admin foydalanuvchi yaratish
<code>python manage.py shell</code>	Loyiha ichida Python konsoli

settings.py — asosiy sozlamalar

config/settings.py (muhim qismlari)

```
from pathlib import Path
import os
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent # loyiha papkasi
load_dotenv(BASE_DIR / ".env")

SECRET_KEY = os.getenv("SECRET_KEY", "dev-uchun-vaqtinchalik-kalit") #
MAXFIY!
DEBUG = os.getenv("DEBUG", "True") = "True" # ishlab chiqishda True,
serverda False
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")

LANGUAGE_CODE = "uz" # admin panel o'zbek
tilida
TIME_ZONE = "Asia/Tashkent"
USE_I18N = True
USE_TZ = True
```



SECRET_KEY va DEBUG

`SECRET_KEY` — loyihaning maxfiy kaliti (parollar, sessiyalar xavfsizligi uchun). Uni GitHub'ga yuklamang — `.env` da saqlang. `DEBUG = True` xato bo'lganda batafsil ma'lumot ko'rsatadi — o'rganish uchun ajoyib, lekin internetdagi saytda **har doim False** bo'lishi shart.

PostgreSQL'ga ulash

Django standart holatda **SQLite** (bitta fayldagi oddiy baza) ishlatadi. Biz PostgreSQL'ga ulaymiz:

- 1 pgAdmin'da yangi baza yarating: `maktab_blog`.
- 2 Loyiha papkasida `.env` fayl:

`.env`

```
SECRET_KEY=bu-yerga-uzun-tasodifiy-satr-yozing
DEBUG=True
DB_NAME=maktab_blog
DB_USER=postgres
DB_PASSWORD=SIZNING_PAROLINGIZ
DB_HOST=localhost
DB_PORT=5432
```

3

`settings.py` dagi `DATABASES` ni almashtiring:

`config/settings.py`

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.getenv("DB_NAME"),
        "USER": os.getenv("DB_USER"),
        "PASSWORD": os.getenv("DB_PASSWORD"),
        "HOST": os.getenv("DB_HOST", "localhost"),
        "PORT": os.getenv("DB_PORT", "5432"),
    }
}
```

4

Django'ning o'z jadvallarini (foydalanuvchilar, sessiyalar...) yarating:

Terminal

```
python manage.py migrate
```

Natija

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  ...
```

pgAdmin'da `maktab_blog` bazasini yangilang — `auth_user`, `django_session` kabi jadvallar paydo bo'ldi! Ularni Django **o'zi** yaratdi.

5

`.gitignore` fayl:

```
.gitignore

venv/
.env
__pycache__/
*.pyc
db.sqlite3
```

Birinchi sahifa (oldindan ko'rish)

Keyingi mavzuda batafsil o'rganamiz, lekin hoziroq natija ko'raylik:

`blog/views.py`

```
from django.http import HttpResponse

def bosh_sahifa(request):
    return HttpResponse("<h1>Maktab blogiga xush kelibsiz! 🚀</h1>")
```

`config/urls.py`

```
from django.contrib import admin
from django.urls import path
from blog import views

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", views.bosh_sahifa),          # "" — bosh sahifa
    (http://127.0.0.1:8000/)
]
```

Serverni ishga tushiring va bosh sahifani yangilang — raketa o'rnida sizning xabaringiz!

Amaliy mashg'ulot

- 1 Yuqoridagi barcha qadamlarni bajaring: `venv` → Django → `config` loyihasi → `blog` ilovasi → PostgreSQL → `migrate` → birinchi sahifa.
- 2 `pip freeze > requirements.txt` bilan paketlar ro'yxatini saqlang.
- 3 Loyihani Git repozitoriyasiga aylantiring va GitHub'ga push qiling (`.env` va `venv` yuklanmaganiga ishonch hosil qiling!).
- 4 **Tajriba:** `views.py` da ataylab xato qiling (masalan, `HttpRespons`) va sahifani yangilang. Django'ning sariq xato sahifasini o'qing — u qaysi faylning qaysi qatorida xato borligini ko'rsatadi (bu `DEBUG=True` tufayli). Xatoni tuzating.
- 5 `python manage.py` ni argumentsiz ishga tushiring va mavjud buyruqlar ro'yxatini ko'ring.

Ko'p uchraydigan xatolar

- ✗ `ModuleNotFoundError: No module named 'django'` — virtual muhit faollashtirilmagan (terminal boshida `(venv)` yo'q).
- ✗ `startproject config` (nuqtasiz) — ortiqcha ichma-ich papka yaratiladi: `config/config/`.
- ✗ Ilovani `INSTALLED_APPS` ga qo'shishni unutish — modellar va shablonlar topilmaydi.
- ✗ `Error: That port is already in use` — server allaqachon boshqa terminalda ishlayapti. Uni to'xtating yoki `runserver 8001`.
- ✗ `password authentication failed` — `.env` dagi parol noto'g'ri yoki `load_dotenv` chaqirilmagan.

O'zingizni tekshiring

? Django'ning arxitekturasi qanday ataladi?

- A. MVC
- B. REST
- C. MVT (Model–View–Template)
- D. CRUD

? Ma'lumotlar tuzilmasi va baza bilan ishlash Django'ning qaysi qismida?

- A. View
- B. Model
- C. Template
- D. URL

? Yangi ilova yaratish buyrug'i?

- A. `python manage.py startapp blog`
- B. `django-admin newapp blog`
- C. `python manage.py createapp blog`
- D. `pip install blog`

? Yaratilgan ilovani loyihaga qayerda ro'yxatdan o'tkazamiz?

- A. `urls.py`
- B. `views.py`
- C. `manage.py`
- D. `settings.py` → `INSTALLED_APPS`

? Internetdagi (production) saytda `DEBUG` qiymati qanday bo'lishi kerak?

- A. True
- B. False
- C. Ahamiyati yo'q
- D. None

? Django'ning ishlab chiqish serverini ishga tushiruvchi buyruq?

- A. `python manage.py start`
- B. `django run`
- C. `python manage.py runserver`
- D. `python server.py`



Uyga vazifa

1. Ikkinchi ilova — `pages` yarating va unga "Biz haqimizda" sahifasini qo'shing (`/haqida/` manzilida).
2. MVT arxitekturasini o'zingiz tanlagan hayotiy o'xshatish bilan (restoran emas!) chizib tushuntiring.

3. Django rasmiy qo'llanmasining 1-qismini o'qing (inglizcha; brauzer tarjimonidan foydalanishingiz mumkin).

✓ Yodda tuting

- Framework — tayyor karkas; Django — "batareyalar bilan birga", xavfsiz Python framework'i.
- MVT: Model (ma'lumot) — View (mantiq) — Template (ko'rinish) + URL (yo'naltirish).
- `django-admin startproject config .` → `python manage.py startapp blog` → `INSTALLED_APPS` ga qo'shish.
- `runserver` — ishlab chiqish serveri; `migrate` — bazaga jadvallar yaratish.
- PostgreSQL: `DATABASES` sozlamasi; maxfiy qiymatlar — `.env` da.

✓ Test javoblari

1. Django'ning arxitekturasini qanday ataladi? — **C) MVT (Model–View–Template)**
2. Ma'lumotlar tuzilmasi va baza bilan ishlash Django'ning qaysi qismida? — **B) Model**
3. Yangi ilova yaratish buyrug'i? — **A) python manage.py startapp blog**
4. Yaratilgan ilovani loyihaga qayerda ro'yxatdan o'tkazamiz? — **D) settings.py → INSTALLED_APPS**
5. Internetdagi (production) saytda DEBUG qiymati qanday bo'lishi kerak? — **B) False**
6. Django'ning ishlab chiqish serverini ishga tushiruvchi buyruq? — **C) python manage.py runserver**



34-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django URL va View komponentlari. URL va View yordamida sahifalar yaratish

So'rov va javob obyektlari, funksiya asosidagi view'lar, URL marshrutlash, path converterlar (int, slug), ilova urls.py va include, nomlangan URL'lar, GET parametrlari, 404 va redirect, Class Based View'lar bilan tanishuv.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- **View** — so'rovni qabul qilib, javob qaytaruvchi funksiya ekanini;
- URL'larni view'larga bog'lash (**marshrutlash**) va ilovaga alohida `urls.py` yaratishni;
- Manzildan qiymat olish: `/maqola/5/` → `id = 5`;
- Qidiruv parametrlari (`?q=...`), 404 xatosi va qayta yo'naltirishni;
- Funksiya va Class asosidagi view'lar farqini.

So'rov va javob

Har bir view — bu **funksiya**: u brauzerdan kelgan **so'rov (request)** obyektni oladi va **javob (response)** obyektni qaytaradi.

```

from django.http import HttpResponse

def salom(request):
    # request – so'rov haqida hamma narsa: metod, manzil, foydalanuvchi,
    # parametrlar...
    print(request.method)          # "GET" yoki "POST"
    print(request.path)            # "/salom/"
    return HttpResponse("Salom!") # javob – brauzerga qaytadigan matn/HTML

```



O'xshatish: pochta oynasi

Pochtaga xat (so'rov) berasiz: unda kimdan, qayerga, nima so'ralgani yozilgan. Xodim (view) xatni o'qiydi, kerakli ishni qiladi va sizga javob xatini (response) qaytaradi. Har bir oyna (URL) o'z xizmatiga mas'ul.

request xossasi	Nima beradi
request.method	So'rov turi: "GET" (sahifani ko'rish) yoki "POST" (forma yuborish)
request.GET	Manzildagi parametrlar: /qidiruv/?q=python → request.GET.get("q")
request.POST	Formadan yuborilgan ma'lumotlar (37-mavzu)
request.user	Tizimga kirgan foydalanuvchi
request.path	So'ralgan manzil

URL marshrutlash

Django har bir so'rov uchun `urlpatterns` ro'yxatini **yuqoridan pastga** ko'rib chiqadi va birinchi mos kelgan yo'lning view'ini chaqiradi.

Ilova uchun alohida urls.py

Barcha URL'larni bitta `config/urls.py` ga yozish katta loyihada chalkashadi. To'g'ri usul: har bir ilova o'z `urls.py` siga ega bo'ladi, asosiy fayl esa ularni `include` bilan ularadi.

blog/urls.py (yangi fayl yarating)

```
from django.urls import path
from . import views

app_name = "blog" # nomlar maydoni

urlpatterns = [
    path("", views.bosh_sahifa, name="bosh"), # /
    path("haqida/", views.haqida, name="haqida"), #
    /haqida/
    path("maqola/<int:id>/", views.maqola, name="maqola"), #
    /maqola/5/
    path("kategoriya/<slug:nomi>/", views.kategoriya, name="kategoriya"), #
    /kategoriya/sport/
    path("qidiruv/", views.qidiruv, name="qidiruv"), #
    /qidiruv/?q=...
]
```

config/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("blog.urls")), # blog ilovasining barcha URL'lari
]
```

/maqola/5/



config/urls.py

"" → include("blog.urls")



blog/urls.py

"maqola/<int:id>/"



views.maqola(request, id=5)

Path converterlar — manzildan qiymat olish

Converter

Nimaga mos keladi

Misol URL

View oladi

<code><int:id></code>	Musbat butun son	<code>/maqola/42/</code>	<code>id=42</code> (int)
<code><str:nom></code>	<code>/</code> dan boshqa har qanday matn	<code>/user/ali/</code>	<code>nom="ali"</code>
<code><slug:nomi></code>	Lotin harf, raqam, <code>-</code> , <code>_</code>	<code>/kategoriya/maktab-hayoti/</code>	<code>nomi="maktab-hayoti"</code>



Slug — sarlavhaning URL uchun qulay ko'rinishi: "Maktab hayoti" → `maktab-hayoti`. Chiroyli manzillar SEO uchun foydali (11-mavzu).

View'lar yozish

Hozircha ma'lumotlar bazasi o'rniga oddiy Python ro'yxatidan foydalanamiz (36-mavzuda haqiqiy modelga almashtiramiz):


```

from django.http import HttpResponseRedirect, Http404, JsonResponse
from django.shortcuts import redirect

# Vaqtinchalik "baza"
MAQOLALAR = [
    {"id": 1, "sarlavha": "Bilimlar bellashuvi", "kategoriya": "tadbirlar",
    "matn": "8-sinflar o'rtasida..."},
    {"id": 2, "sarlavha": "Yangi kompyuter xonasi", "kategoriya": "maktab-
    hayoti", "matn": "20 ta kompyuter..."},
    {"id": 3, "sarlavha": "Futbol bo'yicha g'alaba", "kategoriya": "sport",
    "matn": "Shahar chempionati..."},
]

def bosh_sahifa(request):
    html = "<h1>Maktab blogi</h1><ul>"
    for m in MAQOLALAR:
        html += f'<li><a href="/maqola/{m["id"]}/">{m["sarlavha"]}</a></li>'
    html += "</ul>"
    return HttpResponseRedirect(html)

def haqida(request):
    return HttpResponseRedirect("<h1>Biz haqimizda</h1><p>5-maktab o'quvchilarining
    blogi.</p>")

def maqola(request, id):
    # id — URL'dagi <int:id> dan
    keladi
    for m in MAQOLALAR:
        if m["id"] == id:
            return HttpResponseRedirect(f"<h1>{m['sarlavha']}</h1><p>{m['matn']}</p><a
            href='/'><← Orqaga</a>")
    raise Http404("Bunday maqola topilmadi") # 404 sahifa

def kategoriya(request, nomi):
    topilganlar = [m for m in MAQOLALAR if m["kategoriya"] == nomi]
    if not topilganlar:
        return redirect("blog:bosh") # bosh sahifaga qayta
    yo'naltirish
    royxat = "".join(f"<li>{m['sarlavha']}</li>" for m in topilganlar)

```

```

return HttpResponse(f"<h1>Kategoriya: {nomi}</h1><ul>{royxat}</ul>")

def qidiruv(request):
    q = request.GET.get("q", "").strip() # /qidiruv/?q=futbol
    natija = [m for m in MAQOLALAR if q.lower() in m["sarlavha"].lower()] if q
    else []
    return HttpResponse(f"<h1>'{q}' bo'yicha {len(natija)} ta natija</h1>")

def api_maqolalar(request):
    return JsonResponse({"maqolalar": MAQOLALAR}) # JSON javob (39-mavzuga muvaddima)

```

Sinab ko'ring:

- `http://127.0.0.1:8000/` — ro'yxat
- `/maqola/2/` — maqola; `/maqola/99/` — 404
- `/kategoriya/sport/` ; `/kategoriya/yoq/` — bosh sahifaga qaytaradi
- `/qidiruv/?q=futbol`



Diqqat

View ichida HTML'ni satr sifatida yozish — faqat o'rganish uchun. U noqulay va xavfli (foydalanuvchi matni tekshirilmay HTML'ga qo'shilyapti). Keyingi mavzuda HTML'ni **shablonlarga** (templates) ko'chiramiz — Django ularda matnni avtomatik xavfsiz qiladi.

Nomlangan URL'lar

Har bir yo'lga `name=` berdik. Endi manzilni qo'lda yozish o'rniga nomi bo'yicha yasash mumkin:

Python

```

from django.urls import reverse

reverse("blog:maqola", args=[5]) # "/maqola/5/"
redirect("blog:haqida")          # /haqida/ ga yo'naltirish
# Shablonlarda: {% url 'blog:maqola' 5 %} (35-mavzu)

```

Afzalligi: kelajakda `"maqola/<int:id>/"` ni `"post/<int:id>/"` ga o'zgartirsangiz, faqat `urls.py` da bitta qator o'zgaradi — butun saytdagi havolalar o'zi yangilanadi.

HTTP status kodlari Django'da

Holat	Django'da	Kod
Oddiy javob	<code>HttpResponse(...)</code>	200
Topilmadi	<code>raise Http404(...)</code> yoki <code>get_object_or_404</code> (36-mavzu)	404
Qayta yo'naltirish	<code>redirect("nom")</code>	302
JSON	<code>JsonResponse({...})</code>	200

Class Based View (CBV) — klass asosidagi view'lar

Hozirgacha **funksiya asosidagi view** (Function Based View, FBV) yozdik. Django yana bir usulni taklif qiladi — view'ni **klass** sifatida yozish (25-mavzudagi OOP!). Ko'p uchraydigan vazifalar uchun Django tayyor klasslar beradi va siz ulardan **meros** olasiz.

blog/views.py

```
from django.views import View
from django.views.generic import TemplateView

class SalomView(View):
    def get(self, request):                # GET so'rov uchun metod
        return HttpResponse("Salom, GET!")

    def post(self, request):                # POST so'rov uchun metod
        return HttpResponse("Salom, POST!")

class AloqaView(TemplateView):             # tayyor klass: shablonni
    ko'rsatadi                               # 35-mavzuda yaratamiz
    template_name = "blog/aloka.html"
```

```
path("salom/", views.SalomView.as_view(), name="salom"),      # CBV uchun
.as_view()
```

	FBV (funksiya)	CBV (klass)
Yozilishi	Oddiy, aniq, boshlovchilar uchun tushunarli	Qisqa, lekin "sehr" ko'p
GET/POST ajratish	<code>if request.method == "POST":</code>	Alohida <code>get()</code> , <code>post()</code> metodlar
Qayta ishlatish	Qiyinroq	Meros orqali oson; tayyor <code>ListView</code> , <code>DetailView</code> , <code>CreateView</code> ...

Bu kursda asosan FBV ishlatamiz (tushunish oson), 38-mavzuda esa tayyor CBV'lar bilan CRUD qanchalik qisqa bo'lishini ko'rasiz.

Amaliy mashg'ulot

- 1 `blog/urls.py` yarating va `config/urls.py` da `include` qiling.
- 2 Yuqoridagi view'larni yozing va har bir manzilni brauzerda sinab ko'ring. `/maqola/abc/` ni ham oching — nima uchun 404? (`int` converter harflarni qabul qilmaydi.)
- 3 Bosh sahifadagi havolalarni `reverse("blog:maqola", args=[m["id"]])` bilan yasang.
- 4 Yangi yo'llar qo'shing:
 - `/kalkulyator/<int:a>/<int:b>/` — ikki sonning yig'indisi, ayirmasi, ko'paytmasini ko'rsatsin;
 - `/salom/<str:ism>/` — "Salom, Ali!" deb javob bersin;
 - `/api/maqolalar/` — JSON qaytarsin (`JsonResponse`).
- 5 `/tasodifiy/` — tasodifiy maqolaga `redirect` qilsin (`random.choice`).

Ko'p uchraydigan xatolar

- ❌ `Page not found (404)` — URL'da yakuniy `/` yo'q yoki yo'l `urlpatterns` da ro'yxatdan o'tmagan. Django xato sahifasida barcha mavjud yo'llar ro'yxatini ko'rsatadi — tekshiring.
- ❌ `maqola() got an unexpected keyword argument 'pk'` — URL'dagi nom (`<int:pk>`) va funksiya parametri (`id`) mos emas.
- ❌ View `return` qilmaydi — `The view didn't return an HttpResponse object`.
- ❌ `NoReverseMatch` — `reverse` / `redirect` dagi URL nomi noto'g'ri yoki `app_name:` unutilgan.
- ❌ CBV'da `.as_view()` unutilgan.

O'zingizni tekshiring

? Django view'i nimani qabul qilib, nimani qaytaradi?

- A. HTML qabul qilib, SQL qaytaradi
- B. request (so'rov) qabul qilib, response (javob) qaytaradi
- C. URL qabul qilib, model qaytaradi
- D. Hech narsa qabul qilmaydi

? `path("maqola/<int:id>/", views.maqola)` — `/maqola/7/` so'ralganda view nima oladi?

- A. `id="7"` (matn)
- B. Hech narsa
- C. `id=7` (son)
- D. `maqola=7`

? Ilova URL'larini asosiy `urls.py` ga ulash uchun?

- A. `include("blog.urls")`
- B. `import blog`
- C. `connect("blog")`
- D. `urls.add(blog)`

? `/qidiruv/?q=futbol` dagi "futbol" ni view'da qanday olasiz?

- A. `request.POST["q"]`
- B. `request.q`
- C. `request.path`
- D. `request.GET.get("q")`

? Boshqa sahifaga qayta yo'naltiruvchi funksiya?

- A. `HttpResponse()`
- B. `redirect()`
- C. `Http404()`
- D. `goto()`

? Class Based View'ni URL'ga ulashda nima yoziladi?

- A. `views.SalomView`
- B. `views.SalomView()`
- C. `views.SalomView.as_view()`
- D. `views.SalomView.get`

Uyga vazifa

1. `pages` ilovasi (33-mavzu uyga vazifasi) uchun alohida `urls.py` yarating: `/haqida/`, `/aloqa/`, `/qoidalar/`.
2. `/jadval/<int:son>/` — berilgan sonning 1–10 gacha ko'paytirish jadvalini HTML jadval sifatida qaytaruvchi view.
3. `/yosh/<int:yil>/` — tug'ilgan yil bo'yicha yoshni hisoblovchi view; yil kelajakda bo'lsa — 404.

Yodda tuting

- View: `def view(request): return HttpResponse(...)`.
- Ilovada `urls.py` + `app_name`; asosiy faylda `path("", include("blog.urls"))`.
- Converterlar: `<int:id>`, `<str:nom>`, `<slug:nomi>` — view parametriga keladi.
- `request.GET.get("q")`, `raise Http404`, `redirect("app:nom")`, `JsonResponse`, `name=` + `reverse`.
- CBV: `class V(View): def get(self, request) → V.as_view()`.

✓ Test javoblari

1. Django view'i nimani qabul qilib, nimani qaytaradi? — **B) request (so'rov) qabul qilib, response (javob) qaytaradi**
2. `path("maqola/<int:id>/", views.maqola)` — `/maqola/7/` so'ralganda view nima oladi? — **C) id=7 (son)**
3. Ilova URL'larini asosiy `urls.py` ga ulash uchun? — **A) `include("blog.urls")`**
4. `/qidiruv/?q=futbol` dagi "futbol" ni view'da qanday olasiz? — **D) `request.GET.get("q")`**
5. Boshqa sahifaga qayta yo'naltiruvchi funksiya? — **B) `redirect()`**
6. Class Based View'ni URL'ga ulashda nima yoziladi? — **C) `views.SalomView.as_view()`**



35-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Template tizimi va dinamik sahifalar.

Template va statik fayllar

render() va kontekst, Django Template Language: o'zgaruvchilar, {% if %}, {% for %}, filtrlar, {% url %}; shablon merosi (extends, block, include); CSS, JavaScript va rasmlar — static fayllar.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- HTML'ni view'dan ajratib, **shablon** (template) fayllarda saqlashni;
- View'dan shablonga ma'lumot uzatishni (`render` va kontekst);
- Django shablon tili: o'zgaruvchilar, shartlar, sikllar, filtrlar;
- **Shablon merosi** — umumiy sahifa "skeleti"ni bir marta yozishni;
- CSS, JavaScript va rasmlarni (**static fayllar**) ulashni.

Bu dars I-II boblardagi HTML/CSS/JS bilimlaringizni Django bilan birlashtiradi!

Template nima?

34-mavzuda HTML'ni Python satri ichida yozdik — noqulay va xavfli. **Template (shablon)** — oddiy HTML fayl, lekin unda "**bo'sh joylar**" bor, ularni Django ma'lumot bilan to'ldiradi.



O'xshatish: tabriknoma shabloni

Do'konda tayyor tabriknoma: "Hurmatli _____, sizni _____ bilan tabriklaymiz!" Siz faqat bo'sh joylarni to'ldirasiz. Shablon bir xil, ism va sabab har safar boshqa. Django template ham shunday: `<h1>{{ sarlavha }}</h1>` — har bir maqola uchun o'z sarlavhasi qo'yiladi.

Shablonlar papkasi va render()

- 1 Papkalar tuzilmasini yarating (ilova nomini **ikki marta** takrorlash — nomlar to'qnashmasligi uchun Django an'anasi):

Matn

```
blog/  
└─ templates/  
    └─ blog/  
        ├── bosh_sahifa.html  
        └─ maqola.html
```

- 2 View'da `render` ishlatish:

blog/views.py

```
from django.shortcuts import render  
from django.http import Http404  
  
MAQOLALAR = [ ... ] # 34-mavzudagi ro'yxat  
  
def bosh_sahifa(request):  
    kontekst = { # shablonga  
        yuboriladigan ma'lumotlar (dict)  
        "sarlavha": "Maktab blogi",  
        "maqolalar": MAQOLALAR,  
    }  
    return render(request, "blog/bosh_sahifa.html", kontekst)  
  
def maqola(request, id):  
    for m in MAQOLALAR:  
        if m["id"] == id:  
            return render(request, "blog/maqola.html", {"maqola": m})  
    raise Http404("Maqola topilmadi")
```

`render(request, shablon_yo'li, kontekst)` — shablonni topadi, kontekstdagi ma'lumotlar bilan to'ldiradi va tayyor HTML javob qaytaradi.

Django Template Language (DTL)

Sintaksis	Vazifasi	Misol
<code>{{ ... }}</code>	O'zgaruvchini chiqarish	<code>{{ maqola.sarlavha }}</code>
<code>{% ... %}</code>	Teg — mantiq (shart, sikl, havola)	<code>{% for m in maqolalar %}</code>
<code>{{ x filtr }}</code>	Filtr — qiymatni o'zgartirish	<code>{{ ism upper }}</code>
<code>{# ... #}</code>	Izoh	<code>{# bu ko'rinmaydi #}</code>

O'zgaruvchilar

```
blog/templates/blog/maqola.html

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>{{ maqola.sarlavha }}</title>
</head>
<body>
  <h1>{{ maqola.sarlavha }}</h1>           {# dict kaliti yoki obyekt atributi – nuqta bilan #}
  <p>Kategoriya: {{ maqola.kategoriya }}</p>
  <p>{{ maqola.matn }}</p>
  <a href="/">← Bosh sahifa</a>
</body>
</html>
```



Avtomatik xavfsizlik

Agar matnda `<script>` bo'lsa, Django uni **avtomatik ekranlaydi** (zararsiz matnga aylantiradi) — XSS hujumlaridan himoya (19-mavzuni eslang). 34-mavzudagi satr bilan yasalgan HTML'dan farqi aynan shu.

Shartlar: {% if %}

Django shablon

```
{% if maqolalar %}
    <p>Jami {{ maqolalar|length }} ta maqola</p>
{% else %}
    <p>Hozircha maqola yo'q 😞</p>
{% endif %}

{% if user.is_authenticated %}
    <p>Salom, {{ user.username }}!</p>
{% elif yosh ≥ 14 and sinf = 8 %}
    ...
{% endif %}
```

Sikllar: {% for %}

blog/templates/blog/bosh_sahifa.html

```
<h1>{{ sarlavha }}</h1>
<ul>
    {% for m in maqolalar %}
        <li>
            {{ forloop.counter }}.                {# 1, 2, 3 –
tartib raqami #}
            <a href="{% url 'blog:maqola' m.id %}">{{ m.sarlavha }}</a>
            <small>({{ m.kategoriya }})</small>
        </li>
    {% empty %}
        <li>Maqolalar yo'q</li>                    {# ro'yxat bo'sh
bo'lsa #}
    {% endfor %}
</ul>
```

- `{% url 'blog:maqola' m.id %}` — 34-mavzudagi nomlangan URL'dan havola yasaydi. Manzillarni qo'lda yozmang!
- `forloop.counter` (1 dan), `forloop.first`, `forloop.last` — sikl haqida ma'lumot.



Shablonda faqat **ko'rsatish mantiqi** bo'lsin. Murakkab hisob-kitob, bazaga so'rov — view'ning ishi. DTL ataylab cheklangan: unda Python funksiyalarini to'g'ridan-to'g'ri chaqirib bo'lmaydi.

Filtrlar

Django shablon

<code>{{ ism upper }}</code>	→ ALI
<code>{{ ism lower }}</code>	→ ali
<code>{{ ism title }}</code>	→ Ali Valiyev
<code>{{ matn truncatewords:20 }}</code>	→ birinchi 20 so'z...
<code>{{ matn truncatechars:100 }}</code>	→ birinchi 100 belgi...
<code>{{ maqolalar length }}</code>	→ 3
<code>{{ sana date:"d.m.Y" }}</code>	→ 06.10.2026
<code>{{ sana date:"d E Y, H:i" }}</code>	→ 06 oktabr 2026, 15:30
<code>{{ narx floatformat:2 }}</code>	→ 12500.00
<code>{{ izoh default:"Izoh yo'q" }}</code>	→ qiymat bo'sh bo'lsa
<code>{{ matn linebreaks }}</code>	→ qator o'tishlarini <code><p></code> va <code>
</code> ga aylantiradi

Shablon merosi — eng muhim imkoniyat

Saytning har bir sahifasida bir xil header, menyu, footer va CSS bor. Ularni har bir shablonga nusxalash — xato. **Shablon merosi**: umumiy skelet bitta `base.html` da yoziladi, boshqa sahifalar faqat **o'zgaruvchi qismlarini** to'ldiradi.



O'xshatish: daftar varag'i

Maktab daftarining har bir varag'ida bir xil chiziqlar va hoshiya bor (`base.html`). Har bir varaqqa faqat o'sha kunning darsi yoziladi (block content). Hoshiyani har safar qayta chizmaysiz.

```

{% load static %}
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Maktab blogi{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'blog/css/style.css' %}">
</head>
<body>
  <header class="header">
    <a class="logo" href="{% url 'blog:bosh' %}">🏠 Maktab blogi</a>
    <nav>
      <a href="{% url 'blog:bosh' %}">Bosh sahifa</a>
      <a href="{% url 'blog:haqida' %}">Biz haqimizda</a>
    </nav>
  </header>

  <main class="konteyner">
    {% block content %}{% endblock %}          {# - har bir sahifa shu joyni
to'ldiradi #}
  </main>

  <footer class="footer">© 2026 5-maktab o'quvchilari</footer>
  <script src="{% static 'blog/js/main.js' %}"></script>
</body>
</html>

```

blog/templates/blog/bosh_sahifa.html

```
{% extends "blog/base.html" %}

{% block title %}Bosh sahifa – Maktab blogi{% endblock %}

{% block content %}
    <h1>So'nggi maqolalar</h1>
    <div class="kartalar">
        {% for m in maqolalar %}
            {% include "blog/_karta.html" %}
        {% empty %}
            <p>Hozircha maqola yo'q.</p>
        {% endfor %}
    </div>
{% endblock %}
```

blog/templates/blog/_karta.html

```
{# Qayta ishlatiladigan bo'lak: bitta maqola kartasi #}
<article class="karta">
    <span class="belgi">{{ m.kategoriya }}</span>
    <h3><a href="{% url 'blog:maqola' m.id %}">{{ m.sarlavha }}</a></h3>
    <p>{{ m.matn|truncatewords:15 }}</p>
</article>
```

blog/templates/blog/maqola.html

```
{% extends "blog/base.html" %}

{% block title %}{{ maqola.sarlavha }}{% endblock %}

{% block content %}
    <article>
        <h1>{{ maqola.sarlavha }}</h1>
        <p class="belgi">{{ maqola.kategoriya }}</p>
        {{ maqola.matn|linebreaks }}
        <a href="{% url 'blog:bosh' %}">← Barcha maqolalar</a>
    </article>
{% endblock %}
```

Teg	Vazifasi
<code>{% extends "base.html" %}</code>	Ota shablondan meros olish (faylning birinchi qatori bo'lishi shart)
<code>{% block nom %}...{% endblock %}</code>	O'zgaradigan joy (ota shablonda e'lon qilinadi, bola uni to'ldiradi)
<code>{% include "fayl.html" %}</code>	Boshqa shablon bo'lagini qo'shish (karta, menyu)

Static fayllar: CSS, JavaScript, rasmlar

CSS, JS va logotip kabi o'zgarmaydigan fayllar Django'da **static fayllar** deyiladi.

Matn

```
blog/
├── static/
│   └── blog/
│       ├── css/style.css
│       ├── js/main.js
│       └── img/logo.png
```

Django shablon

```
{% load static %}                                {# shablon boshida
bir marta #}
<link rel="stylesheet" href="{% static 'blog/css/style.css' %}">

<script src="{% static 'blog/js/main.js' %}"></script>
```


config/settings.py

```
STATIC_URL = "static/" # brauzerdagi manzil boshlanishi
STATICFILES_DIRS = [BASE_DIR / "static"] # (ixtiyoriy) loyiha darajasidagi
umumiy static papka
STATIC_ROOT = BASE_DIR / "staticfiles" # serverga joylashda yig'iladigan
joy (41-mavzu)
```

blog/static/blog/css/style.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: "Segoe UI", Arial, sans-serif; background:
#f8fafc; color: #1f2937; }
.header { display: flex; justify-content: space-between; align-items: center;
padding: 16px 32px;
background: #4f46e5; }
.header a { color: white; text-decoration: none; margin-left: 18px; }
.logo { font-weight: 800; font-size: 1.3rem; margin-left: 0 !important; }
.konteyner { max-width: 1000px; margin: 30px auto; padding: 0 16px; min-height:
60vh; }
.kartalar { display: grid; grid-template-columns: repeat(auto-fill,
minmax(260px, 1fr)); gap: 20px; }
.karta { background: white; padding: 20px; border-radius: 14px; box-shadow: 0
6px 18px rgba(0,0,0,.06);
transition: transform .2s; }
.karta:hover { transform: translateY(-4px); }
.karta h3 a { color: #1e1b4b; text-decoration: none; }
.belgi { display: inline-block; background: #eef2ff; color: #4f46e5; padding:
2px 10px;
border-radius: 999px; font-size: 13px; font-weight: 600; }
.footer { text-align: center; padding: 24px; color: #6b7280; }
```



Media fayllar — foydalanuvchilar yuklaydigan fayllar (maqola rasmlari, avatarlar). Ular static'dan farq qiladi va `MEDIA_URL` / `MEDIA_ROOT` bilan sozlanadi — 36-mavzuda `ImageField` bilan birga ko'ramiz.



Static fayl o'zgarishi ko'rinmasa — brauzer eski nusxani keshlagan. `Ctrl` + `F5` bilan sahifani to'liq yangilang. Yangi `static` papka qo'shganda serverni qayta ishga tushiring.

Amaliy mashg'ulot

- 1 `base.html`, `bosh_sahifa.html`, `_karta.html`, `maqola.html` shablonlarini yarating va view'larni `render` ga o'tkazing.
- 2 Static papkada `style.css` yarating va ulang. Sahifa 8-mavzudagi portfolio kabi chiroyli ko'rinishiga erishing.
- 3 `haqida.html` sahifasini `base.html` dan meros olib yarating.
- 4 `main.js` da kichik interaktivlik qo'shing: "🌙" tugmasi bosilganda tungi rejim (19-mavzu).
- 5 Kategoriya sahifasini (`kategoriya.html`) yarating: sarlavha `{{ nomi|title }}` va shu kategoriyadagi maqolalar (`_karta.html` ni `include` qilib).
- 6 Bosh sahifada maqolalar sonini ko'rsating va oxirgi kartadan keyin chiziq chizilmasin (`forloop.last`).

Ko'p uchraydigan xatolar

- ❌ `TemplateDoesNotExist` — shablon yo'li noto'g'ri yoki papka nomi `template` (s harfisiz). To'g'risi: `blog/templates/blog/fayl.html`.
- ❌ Invalid block tag 'static' — shablon boshida `{% load static %}` yo'q.
- ❌ `{% endfor %}`, `{% endif %}`, `{% endblock %}` ni unutish — `TemplateSyntaxError`.
- ❌ `{% extends %}` birinchi qatorda emas.
- ❌ `{{ maqola[sarlavha] }}` — DTL'da kvadrat qavs yo'q, faqat nuqta: `{{ maqola.sarlavha }}`.

O'zingizni tekshiring

? Shablonda o'zgaruvchi qiymatini chiqarish sintaksisi?

- A. `{% ism %}`
- B. `{{ ism }}`
- C. `${ism}`
- D. `<?= ism ?>`

? View'dan shablonga ma'lumot qanday uzatiladi?

- A. Global o'zgaruvchi orqali
- B. URL orqali
- C. `render(request, "shablon.html", kontekst_dict)`
- D. localStorage orqali

? Umumiy skeletdan meros olish uchun qaysi teg?

- A. `{% extends "base.html" %}`
- B. `{% include "base.html" %}`
- C. `{% import base %}`
- D. `{% inherit base %}`

? **Shablonda CSS faylni to'g'ri ulash?**

- A. `href="style.css"`
- B. `href="/blog/style.css"`
- C. `href="{{ static.css }}"`
- D. `href="{% static 'blog/css/style.css' %}"`

? `{{ matn|truncatewords:10 }}` nima qiladi?

- A. Matnni 10 marta takrorlaydi
- B. Matnning birinchi 10 so'zini ko'rsatadi
- C. 10 ta belgini o'chiradi
- D. Matnni 10 ga bo'ladi

? **Sikl ichida ro'yxat bo'sh bo'lgandagi holat uchun qaysi teg?**

- A. `{% else %}`
- B. `{% none %}`
- C. `{% empty %}`
- D. `{% null %}`



Uyga vazifa

1. Blog dizaynini to'liq responsiv qiling (10-mavzu): telefonda bitta ustun, burger menyu.
2. Maxsus 404 sahifa: `templates/404.html` yarating (u `DEBUG=False` bo'lganda ko'rinadi — 41-mavzuda tekshiramiz).

3. `pages` ilovasidagi sahifalarni ham umumiy `base.html` dan meros oladigan qiling (Maslahat: `config/settings.py` → `TEMPLATES[0]["DIRS"] = [BASE_DIR / "templates"]` va loyiha darajasidagi `templates/base.html`).

✓ Yodda tuting

- Shablonlar: `ilova/templates/ilova/*.html`; view'da `render(request, "ilova/fayl.html", {...})`.
- DTL: `{{ o'zgaruvchi }}`, `{% teg %}`, `{{ x|filtr }}`; `{% if %}`, `{% for %}`...`{% empty %}`, `{% url 'app:nom' arg %}`.
- Meros: `base.html` da `{% block content %}`; bolada `{% extends %}` + `{% block %}`; bo'laklar — `{% include %}`.
- Static: `ilova/static/ilova/...`, shablonda `{% load static %}` + `{% static 'yo'l' %}`.
- Django matnni avtomatik ekranlaydi — XSS'dan himoya.

✓ Test javoblari

1. Shablonda o'zgaruvchi qiymatini chiqarish sintaksisi? — **B) {{ ism }}**
2. View'dan shablonga ma'lumot qanday uzatiladi? — **C) render(request, "shablon.html", kontekst_dict)**
3. Umumiy skeletdan meros olish uchun qaysi teg? — **A) {% extends "base.html" %}**
4. Shablonda CSS faylni to'g'ri ulash? — **D) href="{% static 'blog/css/style.css' %}"**
5. {{ matn|truncatewords:10 }} nima qiladi? — **B) Matnning birinchi 10 so'zini ko'rsatadi**
6. Sikl ichida ro'yxat bo'sh bo'lgandagi holat uchun qaysi teg? — **C) {% empty %}**



36-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Models va ORM texnologiyasi. Model yaratish va migratsiyalar

Model — jadvalning Python klassi, maydon turlari, ForeignKey bog'lanishlari, __str__ va Meta; makemigrations va migrate; ORM bilan CRUD: create, all, filter, get, order_by, update, delete; get_object_or_404; view'larni bazaga ulash.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- **Model** — ma'lumotlar bazasi jadvalini Python klassi sifatida yozishni;
- Maydon turlari va jadvallar o'rtasidagi bog'lanishlarni (`ForeignKey`);
- **Migratsiya** — modelni haqiqiy jadvalga aylantirish jarayonini;
- **ORM** yordamida SQL yozmasdan ma'lumot qo'shish, o'qish, filtrlash, o'zgartirish va o'chirishni;
- Blog sahifalarini haqiqiy PostgreSQL bazasiga ulashni.

ORM nima?

III bobda SQL yozdik, 32-mavzuda esa uni Python ichida satr sifatida yubordik. Django'da yanada qulay yo'l bor — **ORM** (Object-Relational Mapping): jadvallar — **klasslar**, qatorlar — **obyektlar**, SQL o'rniga — **Python kodi**.

SQL	Django ORM
<pre>SELECT * FROM blog_maqola;</pre>	<pre>Maqola.objects.all()</pre>
<pre>SELECT * FROM blog_maqola WHERE id = 5;</pre>	<pre>Maqola.objects.get(id=5)</pre>

```
... WHERE sarlavha  
ILIKE '%futbol%'  
ORDER BY yaratilgan  
DESC
```

```
Maqola.objects.filter(sarlavha__icontains="futbol").order_by("-  
yaratilgan")
```

```
INSERT INTO  
blog_maqola (...)  
VALUES (...);
```

```
Maqola.objects.create(sarlavha="...", ...)
```



O'xshatish: tarjimon

Siz o'zbekcha (Python) gapirasiz, baza esa faqat inglizcha (SQL) tushunadi. ORM — tarjimon: sizning `Maqola.objects.filter(...)` gapingizni to'g'ri va xavfsiz SQL'ga o'giradi. Bonus: SQL injection'dan avtomatik himoya (32-mavzu) va bazani almashtirsangiz (SQLite → PostgreSQL) kod o'zgarmaydi.

Model yaratish

blog/models.py

```
from django.db import models
from django.contrib.auth.models import User
from django.urls import reverse


class Kategoriya(models.Model):
    nomi = models.CharField("Nomi", max_length=100, unique=True)
    slug = models.SlugField(unique=True)

    class Meta:
        verbose_name = "Kategoriya"
        verbose_name_plural = "Kategoriyalar"
        ordering = ["nomi"]

    def __str__(self):
        return self.nomi


class Maqola(models.Model):
    sarlavha = models.CharField("Sarlavha", max_length=200)
    matn = models.TextField("Matn")
    kategoriya = models.ForeignKey(
        Kategoriya,
        on_delete=models.PROTECT,          # kategoriyada maqola bo'lsa –
o'chirib bo'lmaydi
        related_name="maqolalar",
    )
    muallif = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="maqolalar")
    chop_etilgan = models.BooleanField("Chop etilgan", default=True)
    korishlar = models.PositiveIntegerField(default=0)
    yaratilgan = models.DateTimeField(auto_now_add=True)    # yaratilganda
avtomatik
    yangilangan = models.DateTimeField(auto_now=True)      # har saqlanganda
avtomatik

    class Meta:
        verbose_name = "Maqola"
        verbose_name_plural = "Maqolalar"
```

```

        ordering = ["-yaratilgan"] # yangilari
birinchi

    def __str__(self):
        return self.sarlavha

    def get_absolute_url(self):
        return reverse("blog:maqola", args=[self.id])

```

Har bir model — `models.Model` dan **meros** olgan klass (25-mavzu!). Har bir atribut — jadval **ustuni**. `id` ustunini yozmaymiz — Django uni avtomatik qo'shadi.

Asosiy maydon turlari

Django maydoni	PostgreSQL turi	Qachon
<code>CharField(max_length=...)</code>	<code>VARCHAR(n)</code>	Qisqa matn: ism, sarlavha
<code>TextField()</code>	<code>TEXT</code>	Uzun matn: maqola
<code>IntegerField()</code> , <code>PositiveIntegerField()</code>	<code>INTEGER</code>	Butun son
<code>DecimalField(max_digits,</code> <code>decimal_places)</code>	<code>NUMERIC</code>	Pul
<code>BooleanField()</code>	<code>BOOLEAN</code>	Ha/yo'q
<code>DateTimeField()</code> , <code>DateField()</code>	<code>DATE</code> , <code>TIMESTAMP</code>	Sana, vaqt
<code>EmailField()</code> , <code>URLField()</code> , <code>SlugField()</code>	<code>VARCHAR</code> + tekshiruv	Email, havola, slug
<code>ImageField(upload_to=...)</code>	<code>VARCHAR</code> (fayl yo'li)	Rasm (<code>pip install Pillow</code> kerak)
<code>ForeignKey(Model, on_delete=...)</code>	FK ustun <code>..._id</code>	Birga-ko'p bog'lanish
<code>ManyToManyField(Model)</code>	Oraliq jadval (avtomatik)	Ko'pga-ko'p (teglar)

Maydon parametrlari

- `null=True` — bazada NULL bo'lishi mumkin; `blank=True` — formada bo'sh qoldirish mumkin.
- `default=...` — standart qiymat; `unique=True` — takrorlanmas; `choices=[...]` — tanlov ro'yxati.
- Birinchi argument (`"Sarlavha"`) — admin panel va formalarda ko'rinadigan chiroyli nom.

on_delete — 29-mavzudagi yaxlitlik qoidalari

Django	SQL ma'nosi	Bizning misolda
<code>CASCADE</code>	<code>ON DELETE CASCADE</code>	Foydalanuvchi o'chsa — maqolalari ham
<code>PROTECT</code>	<code>RESTRICT</code>	Maqolasi bor kategoriyanı o'chirib bo'lmaydi
<code>SET_NULL</code>	<code>SET NULL</code> (<code>null=True</code> kerak)	—

Migratsiyalar — modeldan jadvalga

Model — faqat Python kodi. Bazada jadval paydo bo'lishi uchun ikki qadam kerak:

Terminal

```
python manage.py makemigrations # 1) model o'zgarishlaridan "reja"
(migratsiya fayli) yaratish
python manage.py migrate # 2) rejani bazaga qo'llash (CREATE TABLE
...)
```

Natija

```
Migrations for 'blog':
  blog/migrations/0001_initial.py
    - Create model Kategoriya
    - Create model Maqola
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying blog.0001_initial... OK
```



O'xshatish: qurilish chizmasi

`models.py` — uyning yangi loyihasi. `makemigrations` — quruvchilar uchun "nimani o'zgartirish kerak" degan ish rejasini yozish ("2-qavatga xona qo'shilsin"). `migrate` — rejani haqiqatda bajarish. Migratsiya fayllari — barcha o'zgarishlar tarixi (Git kabi!).

pgAdmin'da `blog_kategoriya` va `blog_maqola` jadvallarini toping (nom: `ilova_model`). `blog_maqola` da `kategoriya_id` va `muallif_id` FK ustunlariga e'tibor bering. Django qanday SQL yozganini ko'rish: `python manage.py sqlmigrate blog 0001`.



Qoida

`models.py` ni har o'zgartirganingizda (maydon qo'shish, nomini o'zgartirish) — **`makemigrations`** va **`migrate`**. Migratsiya fayllarini qo'lda tahrirlamang va Git'ga commit qiling.

ORM bilan ishlash: Django shell

Terminal

```
python manage.py createsuperuser      # admin foydalanuvchi (maqola muallifi uchun ham kerak)
python manage.py shell                 # loyiha ichidagi Python konsoli
```

Create — yaratish

Python

```
from blog.models import Kategoriya, Maqola
from django.contrib.auth.models import User

admin = User.objects.first()

sport = Kategoriya.objects.create(nomi="Sport", slug="sport")
tadbir = Kategoriya.objects.create(nomi="Tadbirlar", slug="tadbirlar")

m = Maqola.objects.create(
    sarlavha="Futbol bo'yicha g'alaba",
    matn="Maktabimiz jamoasi shahar chempionatida birinchi o'rinni egalladi!",
    kategoriya=sport,
    muallif=admin,
)
print(m.id, m.yaratilgan)

# Ikkinchi usul: obyekt yaratib, keyin saqlash
m2 = Maqola(sarlavha="Bilimlar bellashuvi", matn="...", kategoriya=tadbir,
muallif=admin)
m2.save()
```

Read — o'qish

Python

```
Maqola.objects.all()                # barchasi (QuerySet)
Maqola.objects.count()               # soni
Maqola.objects.get(id=1)             # aynan bittasi (topilmasa –
DoesNotExist xatosi!)
Maqola.objects.first(), Maqola.objects.last()

Maqola.objects.filter(chop_etilgan=True)      # WHERE chop_etilgan
= TRUE
Maqola.objects.filter(kategoriya__slug="sport") # bog'langan jadval
bo'yicha (JOIN!)
Maqola.objects.filter(sarlavha__icontains="futbol") # ILIKE '%futbol%'
Maqola.objects.filter(korishlar__gte=100)        # ≥ 100
Maqola.objects.filter(yaratilgan__year=2026)
Maqola.objects.exclude(chop_etilgan=False)      # NOT
Maqola.objects.order_by("-korishlar")[:5]        # ORDER BY ... DESC
LIMIT 5

m = Maqola.objects.get(id=1)
print(m.sarlavha, m.kategoriya.nomi, m.muallif.username) # FK orqali
bog'langan obyektga o'tish
sport.maqolalar.all()                        # teskari yo'nalish
(related_name)
```

Lookup	SQL
maydon=x	= x
maydon__gt / __gte / __lt / __lte	> / ≥ / < / ≤
maydon__contains / __icontains	LIKE '%x%' / ILIKE
maydon__startswith	LIKE 'x%'
maydon__in=[1, 2]	IN (1, 2)
maydon__isnull=True	IS NULL
fk__maydon	JOIN orqali bog'langan jadval ustuni

Update va Delete

Python

```
m = Maqola.objects.get(id=1)
m.sarlavha = "Futbol: shahar chempionlari!"
m.korishlar += 1
m.save() # UPDATE

Maqola.objects.filter(kategoriya__slug="sport").update(chop_etilgan=False) #
ko'pini birdan

Maqola.objects.get(id=2).delete() # DELETE
```

Agregatsiya (30-mavzu Django'da)

Python

```
from django.db.models import Count, Sum, Avg

Maqola.objects.aggregate(jami=Sum("korishlar"), ortacha=Avg("korishlar"))
# {'jami': 340, 'ortacha': 85.0}

Kategoriya.objects.annotate(soni=Count("maqolalar")) # GROUP BY: har bir
kategoriyadagi maqolalar soni
for k in Kategoriya.objects.annotate(soni=Count("maqolalar")):
    print(k.nomi, k.soni)
```



QuerySet "dangasa": `Maqola.objects.filter(...)` yozilganda baza hali so'ralmaydi — faqat natija haqiqatan kerak bo'lganda (sikl, `print`, shablon) bitta SQL yuboriladi. Shu tufayli `filter().exclude().order_by()` zanjiri samarali ishlaydi.

View'larni bazaga ulash

Endi 34–35-mavzulardagi **MAQOLALAR** ro'yxatini o'chirib, haqiqiy modeldan foydalanamiz:

```

from django.shortcuts import render, get_object_or_404
from django.db.models import Q, F
from .models import Maqola, Kategoriya

def bosh_sahifa(request):
    maqolalar =
    Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
    "muallif")
    return render(request, "blog/bosh_sahifa.html", {"maqolalar": maqolalar})

def maqola(request, id):
    m = get_object_or_404(Maqola, id=id, chop_etilgan=True)    # topilmasa –
    avtomatik 404
    Maqola.objects.filter(id=m.id).update(korishlar=F("korishlar") + 1)    #
    ko'rishlar +1
    return render(request, "blog/maqola.html", {"maqola": m})

def kategoriya(request, nomi):
    kat = get_object_or_404(Kategoriya, slug=nomi)
    return render(request, "blog/kategoriya.html", {
        "kategoriya": kat,
        "maqolalar": kat.maqolalar.filter(chop_etilgan=True),
    })

def qidiruv(request):
    q = request.GET.get("q", "").strip()
    natija = Maqola.objects.none()
    if q:
        natija = Maqola.objects.filter(
            Q(sarlavha__icontains=q) | Q(matn__icontains=q),    # Q – YOKI
            chop_etilgan=True,
        )
    return render(request, "blog/qidiruv.html", {"q": q, "maqolalar": natija})

```

Shablonlar deyarli o'zgarmaydi — dict kalitlari o'rniga model atributlari: `{{ m.sarlavha }}`, `{{ m.kategoriya.nomi }}`, `{{ m.yaratilgan|date:"d.m.Y" }}`, havola uchun `{{ m.get_absolute_url }}`.


```
{# m — Maqola obyekti #}
<article class="karta">
  <a class="belgi" href="{% url 'blog:kategoriya' m.kategoriya.slug %}">{{
m.kategoriya.nomi }}</a>
  <h3><a href="{{ m.get_absolute_url }}">{{ m.sarlavha }}</a></h3>
  <p>{{ m.matn|truncatewords:20 }}</p>
  <small>👉 {{ m.muallif.username }} · 📅 {{ m.yaratilgan|date:"d.m.Y" }} · 👁
{{ m.korishlar }}</small>
</article>
```



`select_related("kategoriya", "muallif")` — kategoriya va muallifni **bitta JOIN so'rov** bilan oladi. Usiz har bir karta uchun alohida so'rov ketardi (10 ta maqola = 21 ta so'rov). Bu "N+1 muammosi" deyiladi.

Amaliy mashg'ulot

- 1 `Kategoriya` va `Maqola` modellarini yozing, `makemigrations` + `migrate`. pgAdmin'da jadvallarni ko'ring.
- 2 `createsuperuser` bilan admin yarating.
- 3 `shell` orqali 3 ta kategoriya va 6 ta maqola yarating. ORM so'rovlarini mashq qiling: sport maqolalari, sarlavhasida "maktab" bor maqolalar, eng ko'p ko'rilgan 3 tasi.
- 4 View'larni bazaga ulang, `MAQOLALAR` ro'yxatini o'chiring. `kategoriya.html` va `qidiruv.html` shablonlarini (`base.html` dan meros, kartalar uchun `_karta.html`) yarating. `base.html` dagi menyuga qidiruv formasini qo'shing: `<form action="{% url 'blog:qidiruv' %}"><input name="q"></form>`. Sayt endi bazadagi ma'lumotni ko'rsatishini tekshiring.
- 5 Bosh sahifaga kategoriyalar ro'yxatini maqolalar soni bilan qo'shing (`annotate(Count(...))`).
- 6 **Model o'zgartirish:** `Maqola` ga `rasm = models.ImageField(upload_to="maqolalar/", blank=True)` qo'shing (`pip install Pillow`), migratsiya qiling. `settings.py` ga `MEDIA_URL = "media/"`, `MEDIA_ROOT = BASE_DIR / "media"`; `config/urls.py` oxiriga:

Python

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT) # faqat DEBUG rejimida
```

Shablonda: `{% if m.rasm %}{% endif %}`.

Ko'p uchraydigan xatolar

- ✗ `no such table / relation "blog_maqola" does not exist` — `migrate` bajarilmagan.
- ✗ `No changes detected` — ilova `INSTALLED_APPS` da yo'q yoki fayl saqlanmagan.
- ✗ `Maqola matching query does not exist` — `get()` hech narsa topmadi; view'larda `get_object_or_404` ishlatilg.
- ✗ `get() returned more than one` — `get` faqat bitta natija uchun; ko'p bo'lsa `filter`.
- ✗ Yangi majburiy maydon qo'shganda migratsiya standart qiymat so'raydi — `default=...` yoki `null=True` bering.

O'zingizni tekshiring

? ORM nima?

- A. Ma'lumotlar bazasi turi
- B. Jadvallar bilan Python klass va obyektlari orqali ishlash texnologiyasi
- C. HTML shablon tili
- D. Server dasturi

? Modelni o'zgartirgandan keyin bajariladigan ikki buyruq?

- A. `runserver` va `shell`
- B. `startapp` va `migrate`
- C. `makemigrations` va `migrate`
- D. `createsuperuser` va `collectstatic`

? Uzun maqola matni uchun eng mos maydon?

- A. `TextField`
- B. `CharField(max_length=50)`
- C. `SlugField`
- D. `IntegerField`

? Sarlavhasida "olimpiada" so'zi bor maqolalar (katta-kichik harfga qaramay)?

- A. `Maqola.objects.get(sarlavha="olimpiada")`
- B. `Maqola.objects.all("olimpiada")`
- C. `Maqola.objects.filter(sarlavha="%olimpiada%")`
- D. `Maqola.objects.filter(sarlavha__icontains="olimpiada")`

? Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya?

- A. `Maqola.objects.get()`
- B. `get_object_or_404()`
- C. `find_or_404()`
- D. `Http404()`

? "Har bir maqolaning bitta kategoriyasi, kategoriyada ko'p maqola" — qaysi maydon?

- A. `ManyToManyField`
- B. `OneToOneField`
- C. `ForeignKey` (Maqola modelida)
- D. `CharField`



Uyga vazifa

1. **Kutubxona tizimi (22-loyiha) — modellar:** `Muallif`, `Kitob` (FK muallif, janr — `choices`, yil, sahifalar, mavjud), `Oquvchi`. Migratsiya va shell orqali ma'lumotlar.
2. **Teglar:** `Teg` modeli va `Maqola.teglar = models.ManyToManyField(Teg, blank=True)`.
Shell'da: `m.teglar.add(t1, t2)`, `Maqola.objects.filter(teglar__nomi="olimpiada")`.



Yodda tuting

- Model — `class X(models.Model)`; maydonlar — ustunlar; `__str__`, `Meta`, `get_absolute_url`.
- `ForeignKey(..., on_delete=CASCADE/PROTECT/SET_NULL, related_name=...)`, `ManyToManyField`.
- `makemigrations` → `migrate` — har model o'zgarishidan keyin.
- ORM: `create`, `all`, `get`, `filter(maydon__lookup=...)`, `exclude`, `order_by`, `save`, `update`, `delete`, `annotate/aggregate`.
- View'larda `get_object_or_404`, `select_related`.

✓ Test javoblari

1. ORM nima? — **B) Jadvallar bilan Python klass va obyektlari orqali ishlash texnologiyasi**
2. Modelni o'zgartirgandan keyin bajariladigan ikki buyruq? — **C) makemigrations va migrate**
3. Uzun maqola matni uchun eng mos maydon? — **A) TextField**
4. Sarlavhasida "olimpiada" so'zi bor maqolalar (katta-kichik harfga qaramay)? — **D) `Maqola.objects.filter(sarlavha__icontains="olimpiada")`**
5. Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya? — **B) `get_object_or_404()`**
6. "Har bir maqolaning bitta kategoriyasi, kategoriyada ko'p maqola" — qaysi maydon? — **C) ForeignKey (Maqola modelida)**



37-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Admin panel bilan ishlash. Django Forms yordamida ma'lumotlarni qabul qilish

Superuser, modellarni admin'da ro'yxatdan o'tkazish va sozlash (list_display, search_fields, list_filter, prepopulated_fields); Form va ModelForm, CSRF himoyasi, validatsiya, clean metodlari, xabarlar; ro'yxatdan o'tish va kirish tizimi.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Django'ning tayyor **admin paneli** orqali ma'lumotlarni boshqarishni va uni sozlashni;
- **Django Forms** — foydalanuvchidan ma'lumotni qabul qilish, tekshirish va bazaga saqlashni;
- **CSRF** hujumi va `{% csrf_token %}` himoyasini;
- O'z validatsiya qoidalarini yozishni;
- **Ro'yxatdan o'tish va kirish** (Login/Register) tizimini (24–25-loyihalar).

Django Admin — tayyor boshqaruv paneli

Har bir saytga "orqa eshik" kerak: maqolalarni qo'shish, tahrirlash, foydalanuvchilarni boshqarish. Boshqa frameworklarda buni haftalab yozish kerak — Django'da u **tayyor**. Bu Django'ning eng mashhur imkoniyatlaridan biri.

1. Superuser va kirish

Terminal

```
python manage.py createsuperuser
# Username: admin
# Email: admin@maktab.uz
# Password: ***** (yozilayotganda ko'rinmaydi — bu normal)
python manage.py runserver
```

<http://127.0.0.1:8000/admin/> ni oching va kiring. Hozircha faqat "Foydalanuvchilar" va "Guruhlar" bor.

2. Modellarni ro'yxatdan o'tkazish

blog/admin.py

```
from django.contrib import admin
from .models import Kategoriya, Maqola

admin.site.register(Kategoriya)
admin.site.register(Maqola)
```

Sahifani yangilang — "Blog" bo'limida Kategoriyalar va Maqolalar paydo bo'ldi. Endi ularni qo'shish, tahrirlash va o'chirish mumkin — bir qator kod yozmasdan (tayyor formalar, validatsiya, tasdiqlash oynalari)! Ro'yxatda obyektlar `__str__` qaytargan matn bilan ko'rinadi (36-mavzu).

3. Admin'ni sozlash

blog/admin.py

```
from django.contrib import admin
from .models import Kategoriya, Maqola

@admin.register(Kategoriya)
class KategoriyaAdmin(admin.ModelAdmin):
    list_display = ["nomi", "slug"]
    prepopulated_fields = {"slug": ("nomi",)} # nom yozilganda slug
    avtomatik to'ladi

@admin.register(Maqola)
class MaqolaAdmin(admin.ModelAdmin):
    list_display = ["sarlavha", "kategoriya", "muallif", "chop_etilgan",
                    "korishlar", "yaratilgan"]
    list_filter = ["chop_etilgan", "kategoriya", "yaratilgan"] # o'ng
    tomonda filtrlar
    search_fields = ["sarlavha", "matn"] # qidiruv
    maydoni
    list_editable = ["chop_etilgan"] #
    ro'yxatning o'zida tahrirlash
    list_per_page = 20
    date_hierarchy = "yaratilgan" # sana
    bo'yicha navigatsiya
    readonly_fields = ["korishlar", "yaratilgan", "yangilangan"]
    actions = ["chop_etish"]

@admin.action(description="Tanlanganlarni chop etish")
def chop_etish(self, request, queryset):
    soni = queryset.update(chop_etilgan=True)
    self.message_user(request, f"{soni} ta maqola chop etildi")

admin.site.site_header = "Maktab blogi – boshqaruv paneli"
admin.site.site_title = "Maktab blogi"
```

Sozlama

Vazifasi

<code>list_display</code>	Ro'yxatda qaysi ustunlar ko'rinadi
<code>list_filter</code>	O'ng paneldagi filtrlar
<code>search_fields</code>	Qidiruv qaysi maydonlar bo'yicha
<code>list_editable</code>	Ro'yxatning o'zida o'zgartiriladigan maydonlar
<code>prepopulated_fields</code>	Bir maydondan boshqasini avtomatik to'ldirish (slug)
<code>readonly_fields</code>	Faqat o'qish uchun
<code>actions</code>	Tanlangan obyektlar ustida ommaviy amallar



Foydalanuvchilar va ruxsatlar

Admin'da yangi foydalanuvchi yaratib, unga **"Staff status"** bering va faqat "Maqola" uchun "qo'shish/o'zgartirish" ruxsatlarini belgilang — u admin'ga kira oladi, lekin kategoriyalar va foydalanuvchilarni ko'rmaydi. Masalan, sinf muxbirlari uchun.

Django Forms — nima uchun?

Admin panel — faqat ma'murlar uchun. Oddiy foydalanuvchilar (izoh qoldirish, ro'yxatdan o'tish, maqola yuborish) uchun **sayt ichida** formalar kerak. 3-mavzuda HTML formalarni yozdik va 20-mavzuda JavaScript bilan tekshirdik. Lekin brauzerdagi tekshiruvni chetlab o'tish oson — **server ham tekshirishi shart**.

Django Forms uchta ishni bajaradi:

1. **HTML yaratadi** — input'lar, label'lar avtomatik;
2. **Tekshiradi (validatsiya)** — majburiy maydonlar, email formati, uzunlik, o'z qoidalariningiz;
3. **Tozalaydi** — matnni kerakli turga (son, sana) aylantiradi va xavfsiz qiladi.

Oddiy Form: "Aloqa" formasi

blog/forms.py (yangi fayl)

```
from django import forms

class AloqaForm(forms.Form):
    ism = forms.CharField(label="Ismingiz", max_length=100)
    email = forms.EmailField(label="Email")
    mavzu = forms.ChoiceField(label="Mavzu", choices=[
        ("taklif", "Taklif"),
        ("xato", "Xato haqida xabar"),
        ("boshqa", "Boshqa"),
    ])
    xabar = forms.CharField(label="Xabar", widget=forms.Textarea(attrs={"rows":
5}), min_length=10)

    def clean_ism(self):                                # bitta maydon uchun o'z
tekshiruvimiz
        ism = self.cleaned_data["ism"].strip()
        if any(belgi.isdigit() for belgi in ism):
            raise forms.ValidationError("Ismda raqam bo'lmashligi kerak")
        return ism.title()
```

```

from django.contrib import messages
from django.shortcuts import render, redirect
from .forms import AloqaForm

def aloqa(request):
    if request.method == "POST":
        form = AloqaForm(request.POST)
        to'ldiramiz
        if form.is_valid():
            o'tdi
            data = form.cleaned_data
            (dict)
            print("Yangi xabar:", data["ism"], data["email"], data["xabar"])
            messages.success(request, "Rahmat! Xabaringiz yuborildi.")
            return redirect("blog:aloqa")
            uchun redirect
            # ✗ xato bo'lsa – forma xatolar bilan qayta ko'rsatiladi
        else:
            marta ochildi
            form = AloqaForm()
            return render(request, "blog/aloqa.html", {"form": form})
            # bo'sh forma
            # forma yuborildi
            # kelgan ma'lumot bilan
            # ✔ hamma tekshiruvdan
            # tozalangan ma'lumot
            # qayta yuborilmasligi
            # GET – sahifa birinchi

```

blog/templates/blog/aloqa.html

```
{% extends "blog/base.html" %}

{% block content %}
    <h1>Biz bilan bog'laning</h1>

    {% if messages %}
        {% for msg in messages %}<p class="xabar">{{ msg }}</p>{% endfor %}
    {% endif %}

    <form method="post" novalidate>
        {% csrf_token %}                                {# ← MAJBURIY: CSRF himoyasi #}
        {{ form.as_p }}                                  {# barcha maydonlar <p> ichida, xatolar
bilan #}
        <button type="submit">Yuborish</button>
    </form>
{% endblock %}
```

blog/urls.py

```
path("aloqa/", views.aloqa, name="aloqa"),
```



Django formasini qayta ishlashning standart sxemasi



CSRF nima?

CSRF (Cross-Site Request Forgery) — yovuz sayt sizning brauzeringiz nomidan boshqa saytga yashirin forma yuboradigan hujum (masalan, siz tizimga kirgan blogga "barcha maqolalarni o'chir" so'rovi). `{% csrf_token %}` formaga yashirin maxfiy kod qo'shadi; Django POST so'rovni faqat shu kod to'g'ri bo'lsa qabul qiladi. Usiz: **403 Forbidden — CSRF verification failed**.

Formani qo'lda chiqarish (dizayn uchun)

Django shablon

```
{% for field in form %}
  <div class="maydon">
    {{ field.label_tag }}
    {{ field }}
    {% if field.errors %}<small class="xato">{{ field.errors.0 }}</small>{%
endif %}
  </div>
{% endfor %}
```

ModelForm — modeldan avtomatik forma

Agar forma **model**ga ma'lumot saqlasa, maydonlarni qayta yozish shart emas — `ModelForm` ularni modeldan o'zi oladi.

```
from .models import Maqola

class MaqolaForm(forms.ModelForm):
    class Meta:
        model = Maqola
        fields = ["sarlavha", "kategoriya", "matn"] # foydalanuvchi
        to'ldiradigan maydonlar
        widgets = {
            "sarlavha": forms.TextInput(attrs={"placeholder": "Qiziqarli
sarlavha..."}),
            "matn": forms.Textarea(attrs={"rows": 8}),
        }

    def clean_sarlavha(self):
        s = self.cleaned_data["sarlavha"]
        if len(s) < 5:
            raise forms.ValidationError("Sarlavha kamida 5 belgidan iborat
bo'lsin")
        return s
```

```

from django.contrib.auth.decorators import login_required
from .forms import MaqolaForm

@login_required                                # faqat tizimga kirganlar
uchun
def maqola_yozish(request):
    if request.method == "POST":
        form = MaqolaForm(request.POST)
        if form.is_valid():
            maqola = form.save(commit=False)      # hali bazaga yozmay
            maqola.muallif = request.user         # muallifni o'zimiz
            maqola.chop_etilgan = False          # admin tasdiqlaguncha
            maqola.save()                        # endi saqlaymiz
            messages.success(request, "Maqola yuborildi! Admin tasdiqlagach
            chop etiladi.")
            return redirect("blog:bosh")
        else:
            form = MaqolaForm()
            return render(request, "blog/maqola_form.html", {"form": form})

```

`commit=False` — formadan obyekt yaratadi, lekin bazaga saqlamaydi: shunda formada bo'lmagan maydonlarni (`muallif`) qo'shib, keyin saqlash mumkin.

Ro'yxatdan o'tish va kirish (Login / Register)

Django'da foydalanuvchilar tizimi tayyor: `User` modeli, parollarni **xeshlab** (shifrlab bo'lmaydigan ko'rinishda) saqlash, kirish/chiqish view'lari.

Kirish va chiqish — tayyor view'lar

config/urls.py

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("django.contrib.auth.urls")), # login/,
    path("logout/", password_change/ ...
    path("", include("blog.urls")),
]
```

config/settings.py

```
LOGIN_REDIRECT_URL = "blog:bosh" # kirgandan keyin
LOGOUT_REDIRECT_URL = "blog:bosh" # chiqqandan keyin
LOGIN_URL = "login" # @login_required kirmaganlarni shu
yerga yuboradi
```

templates/registration/login.html (loyiha darajasida yoki blog/templates/registration/)

```
{% extends "blog/base.html" %}
{% block content %}
    <h1>Kirish</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button>Kirish</button>
    </form>
    <p>Akkauntingiz yo'qmi? <a href="{% url 'blog:royxat' %}">Ro'yxatdan
o'ting</a></p>
{% endblock %}
```


Ro'yxatdan o'tish

blog/views.py

```
from django.contrib.auth import login
from django.contrib.auth.forms import UserCreationForm

def royxatdan_otish(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)      # tayyor forma: username +
        parol + tasdiqlash
        if form.is_valid():
            user = form.save()                      # parol avtomatik xeshlanadi
            login(request, user)                   # darhol tizimga kiritamiz
            messages.success(request, f"Xush kelibsiz, {user.username}!")
            return redirect("blog:bosh")
        else:
            form = UserCreationForm()
    return render(request, "blog/royxat.html", {"form": form})
```

blog/urls.py (qo'shimcha)

```
path("royxat/", views.royxatdan_otish, name="royxat"),
path("yozish/", views.maqola_yozish, name="yozish"),
```

base.html menyusiga

```
{% if user.is_authenticated %}
<span>👤 {{ user.username }}</span>
<a href="{% url 'blog:yozish' %}">✍️ Maqola yozish</a>
<form method="post" action="{% url 'logout' %}" style="display:inline">
    {% csrf_token %}<button>Chiqish</button>           {# Django 5'da logout
- POST so'rov #}
</form>
{% else %}
<a href="{% url 'login' %}">Kirish</a>
<a href="{% url 'blog:royxat' %}">Ro'yxatdan o'tish</a>
{% endif %}
```



Parollar bazada **ochiq holda saqlanmaydi** — pgAdmin'da `auth_user` jadvalining `password` ustuniga qarang: `pbkdf2_sha256$...` kabi xesh. Hatto admin ham parolingizni bilmaydi. Hech qachon parolni o'zingiz oddiy `CharField` da saqlamang!

Amaliy mashg'ulot

- 1 Admin'ni sozlang (yuqoridagi `KategoriyaAdmin`, `MaqolaAdmin`) va admin orqali 5 ta maqola qo'shing. Filtr, qidiruv va ommaviy amalni sinang.
- 2 "Aloqa" formasini yarating. Bo'sh, noto'g'ri email va raqamli ism bilan yuborib, xato xabarlarini ko'ring. `{% csrf_token %}` ni olib tashlab, 403 xatoni ko'ring — keyin qaytaring.
- 3 Aloqa xabarlarini bazada saqlang: `Xabar` modeli (ism, email, mavzu, matn, sana) → `ModelForm` → admin'da ko'rinsin.
- 4 Ro'yxatdan o'tish (24-loyiha) va kirish/chiqish (25-loyiha) tizimini qo'shing; menyuda foydalanuvchi holatiga qarab havolalar o'zgarsin.
- 5 "Maqola yozish" sahifasini (`@login_required`) yarating. Tizimga kirmasdan `/yozish/` ga kirib ko'ring — login sahifasiga yo'naltiradi.

Ko'p uchraydigan xatolar

- ✗ `CSRF verification failed` — formada `{% csrf_token %}` yo'q yoki `method="post"` yozilmagan.
- ✗ Forma yuborilgach sahifa yangilansa, ma'lumot **qayta yuboriladi** — muvaffaqiyatli POST'dan keyin doim `redirect`.
- ✗ `form.cleaned_data` ni `is_valid()` dan **oldin** ishlatish — `AttributeError`.
- ✗ Admin'da model ko'rinmayapti — `admin.py` da ro'yxatdan o'tkazilmagan yoki server qayta ishga tushirilmagan.
- ✗ `ModelForm`'da `fields` ko'rsatilmagan — Django xato beradi (xavfsizlik uchun barcha maydonlarni avtomatik ochmaydi).

O'zingizni tekshiring

? Modelni admin panelda ko'rsatish uchun nima qilinadi?

- A. `settings.py` ga qo'shiladi
- B. `admin.py` da `admin.site.register(Model)`
- C. Migratsiya qilinadi
- D. Shablon yaratiladi

? Admin ro'yxatida qaysi ustunlar ko'rinishini belgilovchi sozlama?

- A. `fields`
- B. `list_filter`
- C. `search_fields`
- D. `list_display`

? POST formada majburiy bo'lgan xavfsizlik tegi?

- A. `{% csrf_token %}`
- B. `{% secure %}`
- C. `{% load form %}`
- D. `{% token %}`

? **Forma ma'lumotlari to'g'riligini tekshiruvchi metod?**

- A. `form.save()`
- B. `form.check()`
- C. `form.is_valid()`
- D. `form.validate()`

? **Model asosida avtomatik forma yaratuvchi klass?**

- A. `forms.Form`
- B. `forms.ModelForm`
- C. `admin.ModelAdmin`
- D. `models.Form`

? **View'ni faqat tizimga kirgan foydalanuvchilar uchun ochiq qiluvchi dekorator?**

- A. `@admin_only`
- B. `@csrf_exempt`
- C. `@staff`
- D. `@login_required`



Uyga vazifa

1. **Izohlar:** `Izoh` modeli (maqola FK, muallif FK, matn, sana). Maqola sahifasida izohlar ro'yxati va faqat tizimga kirganlar uchun izoh formasi.
2. Kutubxona loyihangiz (22-loyiha) uchun admin'ni to'liq sozlang: kitoblar ro'yxatida muallif, janr, yil; filtr — janr va mavjudlik; qidiruv — nom va muallif.

✓ Yodda tuting

- Admin: `createsuperuser`; `@admin.register(Model)` + `ModelAdmin` (`list_display`, `list_filter`, `search_fields`...).
- Forma sxemasi: GET → bo'sh forma; POST → `Form(request.POST)` → `is_valid()` → `cleaned_data` / `save()` → `redirect`.
- `{% csrf_token %}` — har bir POST formada; `clean_maydon()` — o'z tekshiruvlaringiz.
- `ModelForm` + `save(commit=False)`; `messages` — foydalanuvchiga xabar.
- Auth: `include("django.contrib.auth.urls")`, `UserCreationForm`, `login()`, `@login_required`, `user.is_authenticated`.

✓ Test javoblari

1. Modelni admin panelda ko'rsatish uchun nima qilinadi? — **B) admin.py da admin.site.register(Model)**
2. Admin ro'yxatida qaysi ustunlar ko'rinishini belgilovchi sozlama? — **D) list_display**
3. POST formada majburiy bo'lgan xavfsizlik tegi? — **A) {% csrf_token %}**
4. Forma ma'lumotlari to'g'riligini tekshiruvchi metod? — **C) form.is_valid()**
5. Model asosida avtomatik forma yaratuvchi klass? — **B) forms.ModelForm**
6. View'ni faqat tizimga kirgan foydalanuvchilar uchun ochiq qiluvchi dekorator? — **D) @login_required**



CRUD amallarini yaratish. CRUD asosida mini loyiha

Maktab blogini to'liq CRUD ilovaga aylantirish: ro'yxat (sahifalash bilan), batafsil ko'rish, yaratish, tahrirlash, o'chirish (tasdiqlash bilan), faqat muallifga ruxsat; xuddi shu ishni Class Based View'lar (ListView, DetailView, CreateView, UpdateView, DeleteView) bilan qilish; sinovdan o'tkazish.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Model, View, Template va Form'ni bitta to'liq ishlaydigan ilovaga **birlashtirishni**;
- To'rtta CRUD amalini veb-interfeys orqali amalga oshirishni;
- **Ruxsatlarni**: maqolani faqat uning muallifi tahrirlay va o'chira olishini;
- Uzun ro'yxatni sahifalarga bo'lishni (pagination);
- Xuddi shu ilovani Django'ning tayyor **Class Based View**'lari bilan bir necha qatorda yozishni;
- Loyihani sinovdan o'tkazishni (21-loyiha: Blog veb-ilovasi).

Reja: CRUD = 5 ta sahifa

Amal	URL	View	Shablon
R — ro'yxat	/	maqola_royxat	maqola_list.html
R — batafsil	/maqola/<id>/	maqola_batafsil	maqola_detail.html
C — yaratish	/maqola/yangi/	maqola_yaratish	maqola_form.html

U — tahrirlash	/maqola/<id>/tahrirlash/	maqola_tahrirlash	maqola_form.html (o'sha!)
D — o'chirish	/maqola/<id>/ochirish/	maqola_ochirish	maqola_confirm_delete.html

Bu jadval — har qanday CRUD ilovaning "retsepti". Kutubxona, talabalar tizimi, mahsulotlar — hammasi xuddi shu 5 ta sahifadan iborat.

1. URL'lar

blog/urls.py

```
from django.urls import path
from . import views

app_name = "blog"

urlpatterns = [
    path("", views.maqola_royxat, name="bosh"),
    path("maqola/yangi/", views.maqola_yaratish, name="yaratish"),          #
    <int:id> dan OLDIN!
    path("maqola/<int:id>/", views.maqola_batafsil, name="maqola"),
    path("maqola/<int:id>/tahrirlash/", views.maqola_tahrirlash,
name="tahrirlash"),
    path("maqola/<int:id>/ochirish/", views.maqola_ochirish, name="ochirish"),
    path("kategoriya/<slug:nomi>/", views.kategoriya, name="kategoriya"),
    path("qidiruv/", views.qidiruv, name="qidiruv"),
    path("royxat/", views.royxatdan_otish, name="royxat"),
]
```


2. Forma

blog/forms.py

```
from django import forms
from .models import Maqola

class MaqolaForm(forms.ModelForm):
    class Meta:
        model = Maqola
        fields = ["sarlavha", "kategoriya", "matn", "chop_etilgan"]
        widgets = {"matn": forms.Textarea(attrs={"rows": 10})}

    def clean_sarlavha(self):
        s = self.cleaned_data["sarlavha"].strip()
        if len(s) < 5:
            raise forms.ValidationError("Sarlavha kamida 5 belgi bo'lsin")
        return s
```

3. View'lar (funksiya asosida)

blog/views.py

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.core.paginator import Paginator
from django.db.models import F
from django.http import HttpResponseForbidden
from django.shortcuts import get_object_or_404, redirect, render

from .forms import MaqolaForm
from .models import Maqola

# ----- READ: ro'yxat -----
def maqola_royxat(request):
    maqolalar = (Maqola.objects.filter(chop_etilgan=True)
                  .select_related("kategoriya", "muallif"))
    sahifalovchi = Paginator(maqolalar, 6) # har sahifada 6 ta
    sahifa = sahifalovchi.get_page(request.GET.get("page")) # ?page=2
    return render(request, "blog/maqola_list.html", {"sahifa": sahifa})

# ----- READ: batafsil -----
def maqola_batafsil(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    # Chop etilmagan maqolani faqat muallifi ko'ra oladi
    if not maqola.chop_etilgan and maqola.muallif != request.user:
        return HttpResponseForbidden("Bu maqola hali chop etilmagan")
    Maqola.objects.filter(id=id).update(korishlar=F("korishlar") + 1)
    return render(request, "blog/maqola_detail.html", {"maqola": maqola})

# ----- CREATE -----
@login_required
def maqola_yaratish(request):
    if request.method == "POST":
        form = MaqolaForm(request.POST)
        if form.is_valid():
            maqola = form.save(commit=False)
            maqola.muallif = request.user
            maqola.save()
```

```

        messages.success(request, "✅ Maqola yaratildi")
        return redirect(maqola) #
get_absolute_url() ga
    else:
        form = MaqolaForm()
        return render(request, "blog/maqola_form.html", {"form": form, "sarlavha":
"Yangi maqola"})

# ----- UPDATE -----
@login_required
def maqola_tahrirlash(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    if maqola.muallif != request.user: # 🔒 faqat muallif
        return HttpResponseForbidden("Siz bu maqolani tahrirlay olmaysiz")
    if request.method == "POST":
        form = MaqolaForm(request.POST, instance=maqola) # instance – mavjud
obyektni yangilash
        if form.is_valid():
            form.save()
            messages.success(request, "✅ O'zgarishlar saqlandi")
            return redirect(maqola)
        else:
            form = MaqolaForm(instance=maqola) # forma eski
            qiyamatlar bilan to'lgan
            return render(request, "blog/maqola_form.html", {"form": form, "sarlavha":
"Tahrirlash"})

# ----- DELETE -----
@login_required
def maqola_ochirish(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    if maqola.muallif != request.user:
        return HttpResponseForbidden("Siz bu maqolani o'chira olmaysiz")
    if request.method == "POST": # faqat
tasdiqlangandan keyin
        maqola.delete()
        messages.success(request, "🗑️ Maqola o'chirildi")
        return redirect("blog:bosh")
    return render(request, "blog/maqola_confirm_delete.html", {"maqola":
maqola})

```



Xavfsizlik: ikki qoida

1. **Ruxsatni serverda tekshiring.** Shablonda "Tahrirlash" tugmasini yashirish yetarli emas — ayyor foydalanuvchi `/maqola/5/tahrirlash/` manzilini qo'lda yozishi mumkin. Shuning uchun view'da `if maqola.muallif != request.user`.
2. **O'chirish — faqat POST.** Agar GET so'rov bilan o'chirsak, oddiy havola (yoki boshqa saytdagi rasm teg!) maqolani o'chirib yuborishi mumkin. Avval tasdiqlash sahifasi, keyin POST forma + CSRF.

`MaqolaForm(request.POST, instance=maqola)` — **yaratish va tahrirlash** uchun bitta forma va bitta shablon: `instance` berilsa — mavjud obyekt yangilanadi, berilmasa — yangisi yaratiladi.

4. Shablonlar

blog/templates/blog/maqola_list.html

```
{% extends "blog/base.html" %}
{% block title %}Maktab blogi{% endblock %}

{% block content %}
  <div class="sarlavha-qator">
    <h1>So'nggi maqolalar</h1>
    {% if user.is_authenticated %}
      <a class="tugma" href="{% url 'blog:yaratish' %}">👉 Yangi maqola</a>
    {% endif %}
  </div>

  <div class="kartalar">
    {% for m in sahifa %}
      {% include "blog/_karta.html" %}
    {% empty %}
      <p>Hozircha maqola yo'q. Birinchisini siz yozing!</p>
    {% endfor %}
  </div>

  {% if sahifa.has_other_pages %}
    <nav class="sahifalash">
      {% if sahifa.has_previous %}
        <a href="?page={{ sahifa.previous_page_number }}">← Oldingi</a>
      {% endif %}
      <span>{{ sahifa.number }} / {{ sahifa.paginator.num_pages }}</span>
      {% if sahifa.has_next %}
        <a href="?page={{ sahifa.next_page_number }}">Keyingi →</a>
      {% endif %}
    </nav>
  {% endif %}
{% endblock %}
```

```

{% extends "blog/base.html" %}
{% block title %}{{ maqola.sarlavha }}{% endblock %}

{% block content %}
    {% for msg in messages %}<p class="xabar">{{ msg }}</p>{% endfor %}

    <article class="maqola">
        <a class="belgi" href="{% url 'blog:kategoriya' maqola.kategoriya.slug %}">
            {{ maqola.kategoriya }}</a>
        <h1>{{ maqola.sarlavha }}</h1>
        <p class="meta">👤 {{ maqola.muallif.username }} · 📅 {{
maqola.yaratilgan|date:"d.m.Y H:i" }} · 👁 {{ maqola.korishlar }}</p>
        {% if not maqola.chop_etilgan %}<p class="ogohlantirish">✍️ Qoralama –
faqat siz ko'ryapsiz</p>{% endif %}
        {{ maqola.matn|linebreaks }}

        {% if user == maqola.muallif %}                                {# tugmalar faqat
muallifga ko'rinadi #}
            <div class="amallar">
                <a class="tugma" href="{% url 'blog:tahrirlash' maqola.id %}">✍️
Tahrirlash</a>
                <a class="tugma qizil" href="{% url 'blog:ochirish' maqola.id %}">🗑️
O'chirish</a>
            </div>
        {% endif %}
    </article>
    <a href="{% url 'blog:bosh' %}">← Barcha maqolalar</a>
{% endblock %}

```

```
{% extends "blog/base.html" %}
{% block title %}{{ sarlavha }}{% endblock %}

{% block content %}
    <h1>{{ sarlavha }}</h1>
    <form method="post" class="forma">
        {% csrf_token %}
        {% for field in form %}
            <div class="maydon">
                {{ field.label_tag }}
                {{ field }}
                {% for xato in field.errors %}<small class="xato">{{ xato }}</small>{%
endfor %}
            </div>
        {% endfor %}
        <button type="submit" class="tugma">💾 Saqlash</button>
        <a href="{% url 'blog:bosh' %}">Bekor qilish</a>
    </form>
{% endblock %}
```

```
{% extends "blog/base.html" %}
{% block content %}
    <h1>O'chirishni tasdiqlang</h1>
    <p><<<b>{{ maqola.sarlavha }}</b>>> maqolasini o'chirmoqchimisiz? Bu amalni
qaytarib bo'lmaydi.</p>
    <form method="post">
        {% csrf_token %}
        <button type="submit" class="tugma qizil">Ha, o'chirish</button>
        <a href="{{ maqola.get_absolute_url }}">Yo'q, ortga</a>
    </form>
{% endblock %}
```

```
.sarlavha-qator { display: flex; justify-content: space-between; align-items: center; }
.tugma { display: inline-block; background: #4f46e5; color: #fff; padding: 10px 18px; border-radius: 10px;
        text-decoration: none; border: none; font: inherit; cursor: pointer; }
.tugma.qizil { background: #dc2626; }
.forma .maydon { display: grid; gap: 6px; margin-bottom: 16px; }
.forma input, .forma select, .forma textarea { padding: 10px; border: 2px solid #e5e7eb; border-radius: 10px; font: inherit; }
.xato { color: #dc2626; }
.xabar { background: #dcfce7; padding: 12px; border-radius: 10px; }
.sahifalash { display: flex; gap: 16px; justify-content: center; margin: 30px 0; }
.amallar { display: flex; gap: 10px; margin: 20px 0; }
```

5. Xuddi shu narsa — Class Based View'lar bilan

CRUD shunchalik keng tarqalganki, Django uning uchun **tayyor klasslar** beradi. Yuqoridagi ~80 qator view kodi ~40 qatorga qisqaradi:


```

from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.contrib.messages.views import SuccessMessageMixin
from django.urls import reverse_lazy
from django.views.generic import ListView, DetailView, CreateView, UpdateView,
DeleteView

from .forms import MaqolaForm
from .models import Maqola

class MaqolaList(ListView):
    queryset =
    Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
    "muallif")
    paginate_by = 6 # shablonda: page_obj, object_list
    # shablon (standart): blog/maqola_list.html

class MaqolaDetail(DetailView):
    model = Maqola
    pk_url_kwarg = "id" # URL'da <int:id> ishlatganimiz
    uchun
    # shablon: blog/maqola_detail.html, obyekt: {{ object }} yoki {{ maqola }}

class MaqolaCreate(LoginRequiredMixin, SuccessMessageMixin, CreateView):
    model = Maqola
    form_class = MaqolaForm
    success_message = "✅ Maqola yaratildi"
    extra_context = {"sarlavha": "Yangi maqola"} # shablondagi {{ sarlavha
    }} uchun

    def form_valid(self, form):
        form.instance.muallif = self.request.user
        return super().form_valid(form) # saqlash va get_absolute_url() ga
    o'tish

class FaqatMuallif(UserPassesTestMixin):
    """Faqat muallifga ruxsat beruvchi 'aralashma' (mixin)."""
    def test_func(self):
        return self.get_object().muallif == self.request.user

```

```
class MaqolaUpdate(LoginRequiredMixin, FaqatMuallif, SuccessMessageMixin,
UpdateView):
    model = Maqola
    form_class = MaqolaForm
    pk_url_kwarg = "id"
    success_message = "✅ O'zgarishlar saqlandi"
    extra_context = {"sarlavha": "Tahrirlash"}

class MaqolaDelete(LoginRequiredMixin, FaqatMuallif, DeleteView):
    model = Maqola
    pk_url_kwarg = "id"
    success_url = reverse_lazy("blog:bosh")
```

blog/urls.py (CBV varianti)

```
from . import views_cbv as v

urlpatterns = [
    path("", v.MaqolaList.as_view(), name="bosh"),
    path("maqola/yangi/", v.MaqolaCreate.as_view(), name="yaratish"),
    path("maqola/<int:id>/", v.MaqolaDetail.as_view(), name="maqola"),
    path("maqola/<int:id>/tahrirlash/", v.MaqolaUpdate.as_view(),
name="tahrirlash"),
    path("maqola/<int:id>/ochirish/", v.MaqolaDelete.as_view(),
name="ochirish"),
]
```



CBV shablon nomlari

Tayyor klasslar shablonni **avtomatik** nomi bo'yicha qidiradi: `ilova/model_list.html`, `model_detail.html`, `model_form.html`, `model_confirm_delete.html`. Shuning uchun biz shablonlarni oldindan aynan shunday nomladik! `ListView` da ro'yxat — `page_obj` (yoki `maqola_list`), shuning uchun `maqola_list.html` dagi sahifa ni `page_obj` ga almashtiring.

Klass

Nima qiladi

`ListView`

Ro'yxat + sahifalash

<code>DetailView</code>	Bitta obyekt (topilmasa 404)
<code>CreateView</code>	GET — bo'sh forma, POST — tekshirish va yaratish
<code>UpdateView</code>	Eski qiymatlar bilan forma, saqlash
<code>DeleteView</code>	Tasdiqlash sahifasi va POST bilan o'chirish
<code>LoginRequiredMixin</code>	<code>@login_required</code> ning CBV varianti
<code>UserPassesTestMixin</code>	<code>test_func()</code> True qaytarmasa — 403

6. Sinovdan o'tkazish

Loyihani topshirishdan oldin har bir ssenariyni tekshiring. Ikki xil brauzerda (yoki biri "Inkognito" rejimida) ikki foydalanuvchi bilan kiring:

#	Ssenariy	Kutilgan natija	✓
1	Mehmon bosh sahifani ochadi	Chop etilgan maqolalar, "Yangi maqola" tugmasi yo'q	☐
2	Mehmon <code>/maqola/yangi/</code> ni qo'lda ochadi	Login sahifasiga yo'naltiriladi	☐
3	Ali maqola yaratadi (sarlavha 3 belgi)	"Kamida 5 belgi" xatosi, forma ma'lumotlari yo'qolmaydi	☐
4	Ali to'g'ri maqola yaratadi	Maqola sahifasi + yashil xabar; admin'da muallif — Ali	☐
5	Vali Ali'ning maqolasini ochadi	Tahrirlash/o'chirish tugmalari ko'rinmaydi	☐
6	Vali <code>/maqola/ID/tahrirlash/</code> ni qo'lda ochadi	403 Forbidden	☐
7	Ali maqolani o'chiradi	Tasdiqlash sahifasi → "Ha" → ro'yxatda yo'q	☐

8	15 ta maqola bo'lsa	Sahifalash: 3 ta sahifa	☐
9	/maqola/9999/	404	☐

Avtomatik test (qisqacha tanishuv)

blog/tests.py

```
from django.contrib.auth.models import User
from django.test import TestCase
from django.urls import reverse
from .models import Kategoriya, Maqola

class MaqolaTestlari(TestCase):
    def setUp(self):
        self.ali = User.objects.create_user("ali", password="parol12345")
        self.vali = User.objects.create_user("vali", password="parol12345")
        kat = Kategoriya.objects.create(nomi="Sport", slug="sport")
        self.m = Maqola.objects.create(sarlavha="Test maqola", matn="...",
            kategoriya=kat, muallif=self.ali)

    def test_bosh_sahifa_ochiladi(self):
        javob = self.client.get(reverse("blog:bosh"))
        self.assertEqual(javob.status_code, 200)
        self.assertContains(javob, "Test maqola")

    def test_begona_tahrirlay_olmaydi(self):
        self.client.login(username="vali", password="parol12345")
        javob = self.client.get(reverse("blog:tahrirlash", args=[self.m.id]))
        self.assertEqual(javob.status_code, 403)
```

Terminal

```
python manage.py test blog
```

Natija

```
Creating test database for alias 'default'...
```

```
..
```

```
-----  
Ran 2 tests in 0.412s
```

```
OK
```

Testlar alohida, vaqtinchalik bazada ishlaydi — haqiqiy ma'lumotlaringizga tegmaydi.

Amaliy mashg'ulot (21-loyiha: Blog veb-ilovasi)

- 1 Yuqoridagi URL, forma, view va shablonlarni loyihangizga qo'shing (funksiya variantida).
- 2 Sinov jadvalidagi 9 ta ssenariyni bajaring va belgilang.
- 3 View'larni CBV variantiga o'tkazing va ssenariylarni qayta tekshiring.
- 4 "Mening maqolalarim" sahifasi: `/mening/` — joriy foydalanuvchining barcha maqolalari (qoralamalar bilan).
- 5 Loyihani GitHub'ga push qiling; README'da: loyiha tavsifi, texnologiyalar, ishga tushirish yo'riqnomasi, skrinshotlar.

▼ 4-topshiriq yechimi

Python

```
@login_required  
def mening_maqolalarim(request):  
    maqolalar =  
    Maqola.objects.filter(muallif=request.user).select_related("kategoriya")  
    return render(request, "blog/mening.html", {"maqolalar": maqolalar})  
  
# urls.py: path("mening/", views.mening_maqolalarim, name="mening"),
```

Ko'p uchraydigan xatolar

- ❌ `maqola/yangi/` yo'li `maqola/<int:id>/` dan **keyin** yozilsa — muammo yo'q (chunki "yangi" son emas), lekin `<str:...>` ishlatilsa — "yangi" so'zi id deb olinadi. Aniq yo'llarni doim oldin yozing.
- ❌ Tahrirlashda `instance=` unutilgan — yangi nusxa yaratiladi, eskisi o'zgarmaydi.
- ❌ Ruxsat faqat shablonda tekshirilgan — URL'ni qo'lda yozib, boshqa bironing maqolasini o'chirish mumkin bo'lib qoladi.
- ❌ GET so'rov bilan o'chirish.
- ❌ `TemplateDoesNotExist: blog/maqola_form.html` — CBV standart nomli shablonni qidirayпти; faylni shunday nomlang yoki `template_name` bering.

O'zingizni tekshiring

? **ModelForm bilan mavjud obyektни tahrirlash uchun formaga nima beriladi?**

- A. `id=`
- B. `object=`
- C. `instance=`
- D. `update=True`

? **Nima uchun o'chirish faqat POST so'rov bilan bo'lishi kerak?**

- A. POST tezroq ishlaydi
- B. Oddiy havola yoki begona sayt tasodifan/ataylab o'chirib yubormasligi uchun (CSRF himoyasi bilan)
- C. GET so'rovda id bo'lmaydi
- D. Django GET'ni qo'llab-quvvatlamaydi

? "Faqat muallif tahrirlay oladi" tekshiruvi qayerda bo'lishi SHART?

- A. View'da (serverda)
- B. Faqat shablonda
- C. CSS'da
- D. JavaScript'da

? Ro'yxatni sahifalarga bo'luvchi Django klassi?

- A. `Splitter`
- B. `PageView`
- C. `Limiter`
- D. `Paginator`

? Yangi obyekt yaratuvchi tayyor CBV?

- A. `ListView`
- B. `CreateView`
- C. `NewView`
- D. `FormView`

? CBV'da tizimga kirishni talab qiluvchi aralashma (mixin)?

- A. `AuthMixin`
- B. `UserPassesTestMixin`
- C. `LoginRequiredMixin`
- D. `SessionMixin`



Uyga vazifa (IV bob yakunlovchi loyiha)

Quyidagilardan birini to'liq CRUD ilova sifatida yarating (model + admin + forma + 5 sahifa + ruxsatlar + sahifalash + kamida 2 ta test):

1. **Kutubxona boshqaruv tizimi** (22-loyiha): kitoblar CRUD'i, kitobni "olish/qaytarish".
2. **Talabalar boshqaruv tizimi** (23-loyiha): talabalar, guruhlar, baholar; guruh bo'yicha o'rtacha baho.
3. **E'lonlar taxtasi**: foydalanuvchilar e'lon joylaydi (rasm bilan), toifa va qidiruv bo'yicha filtr.



IV bob yakunlandi!

- CRUD = ro'yxat, batafsil, yaratish, tahrirlash, o'chirish — 5 ta URL/view/shablon.
- `ModelForm(request.POST, instance=obj)` — yaratish va tahrirlash uchun bitta forma.
- Ruxsatlar serverda; o'chirish — tasdiqlash + POST.
- `Paginator`; tayyor CBV'lar: `ListView`, `DetailView`, `CreateView`, `UpdateView`, `DeleteView` + mixinlar.
- Siz endi to'liq ishlaydigan, xavfsiz veb-ilova yarata olasiz! Keyingi bobda unga **API** qo'shamiz va **internetga chiqaramiz**.

✓ Test javoblari

1. ModelForm bilan mavjud obyektни tahrirlash uchun formaga nima beriladi? — **C) instance=**
2. Nima uchun o'chirish faqat POST so'rov bilan bo'lishi kerak? — **B) Oddiy havola yoki begona sayt tasodifan/ataylab o'chirib yubormasligi uchun (CSRF himoyasi bilan)**
3. "Faqat muallif tahrirlay oladi" tekshiruvi qayerda bo'lishi SHART? — **A) View'da (serverda)**
4. Ro'yxatni sahifalarga bo'luvchi Django klassi? — **D) Paginator**
5. Yangi obyekt yaratuvchi tayyor CBV? — **B) CreateView**
6. CBV'da tizimga kirishni talab qiluvchi aralashma (mixin)? — **C) LoginRequiredMixin**



39-MAVZU · 42 TADAN · V BOB. DRF VA DEPLOY

Django REST Framework (DRF) ga kirish.

Serializerlar va API View'lar

API nima, REST arxitekturası, JSON formati, HTTP metodlari va status kodlari; DRF'ni o'rnatish, Serializer va ModelSerializer, validatsiya, @api_view va APIView, Response, brauzerda ko'rinadigan API.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- **API** nima va zamonaviy ilovalar nima uchun unga muhtojligini;
- **REST** arxitekturası qoidalari va **JSON** formatini;
- Django REST Framework'ni o'rnatish va sozlashni;
- **Serializer** — Python obyektlarini JSON'ga va aksincha aylantirishni;
- API View'lar yozib, ularni brauzerda sinashni.

API nima?

IV bobda Django **HTML sahifalar** qaytardi — ularni faqat brauzer ko'rsata oladi. Lekin bugun bitta xizmatdan ko'plab "mijozlar" foydalanadi:

-  Mobil ilova (Android, iOS) — unga HTML kerak emas, faqat **ma'lumot** kerak;
-  React/Vue kabi frontend dasturlar — sahifani o'zi chizadi, serverdan faqat ma'lumot oladi;
-  Telegram bot, boshqa saytlar, aqlli soatlar...

API (Application Programming Interface — dasturlash interfeysi) — dasturlar bir-biri bilan "gaplashishi" uchun kelishilgan qoidalar. Veb-API — ma'lumotni **JSON** ko'rinishida beradigan URL'lar to'plami.



O'xshatish: restoran (yana!) va yetkazib berish

IV bobdagi sayt — restoran zali: taom chiroyli likopchada (HTML) beriladi. **API** — yetkazib berish xizmati: o'sha oshxona, o'sha taom, lekin konteynerda (JSON). Mijoz uni o'z uyida (mobil ilova, React sayt) o'zi xohlagan likopchaga soladi. Oshxona (back-end) bitta — mijozlar ko'p.



Mobil ilova



React sayt



Telegram bot

⇔ JSON ⇔



Django REST API

/api/maqolalar/



PostgreSQL

JSON — ma'lumot almashish tili

24-mavzuda tanishgan **JSON** (JavaScript Object Notation) — API'larning asosiy formati. U Python'dagi dict/list'ga va JavaScript'dagi obyekt/massivga juda o'xshaydi:

JSON

```
{
  "id": 5,
  "sarlavha": "Bilimlar bellashuvi",
  "chop_etilgan": true,
  "korishlar": 128,
  "kategoriya": { "id": 2, "nomi": "Tadbirlar" },
  "teglar": ["olimpiada", "8-sinf"],
  "rasm": null
}
```

Qoidalar: kalitlar **qo'sh qo'shtirnoqda**; qiymatlar — matn, son, `true` / `false`, `null`, obyekt `{}` yoki massiv `[]`.

REST arxitekturası

REST — API'ni tartibli va tushunarli loyihalash uslubi. Asosiy g'oya: hamma narsa — **resurs** (maqola, foydalanuvchi, mahsulot), har bir resursning o'z **URLi** bor, amallar esa **HTTP metodlari** bilan bildiriladi.

Amal (CRUD)	HTTP metod	URL	Javob kodi
Barcha maqolalar ro'yxati	GET	/api/maqolalar/	200 OK
Bitta maqola	GET	/api/maqolalar/5/	200 / 404
Yangi maqola yaratish	POST	/api/maqolalar/	201 Created
To'liq yangilash	PUT	/api/maqolalar/5/	200
Qisman yangilash	PATCH	/api/maqolalar/5/	200
O'chirish	DELETE	/api/maqolalar/5/	204 No Content

- URL'da **ot** (resurs), ko'plikda: /api/maqolalar/ , /api/getMaqola/ , /api/maqola/ochir/5/  — amal metod orqali bildiriladi.
- Xatolar ham status kod bilan: 400 — noto'g'ri ma'lumot, 401 — tizimga kirilmagan, 403 — ruxsat yo'q, 404 — topilmadi.

DRF'ni o'rnatish

Django REST Framework (DRF) — Django'da API yaratish uchun eng mashhur kutubxona.

Terminal

```
pip install djangorestframework
pip freeze > requirements.txt
```

```

INSTALLED_APPS = [
    # ...
    "rest_framework",
    "blog",
]

REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",  # o'qish –
        hamмага, yozish – kirganlarga
    ],
}

```

Serializer — tarjimon

Bazadan olingan **Maqola** obyektini to'g'ridan-to'g'ri JSON qilib bo'lmaydi. **Serializer** ikki yo'nalishda ishlaydi:

- **Serializatsiya:** model obykti → Python dict → JSON (javob yuborish uchun);
- **Deserializatsiya:** JSON → tekshirish (validatsiya) → model obykti (kelgan ma'lumotni saqlash uchun).



Django Forms bilan o'xshashlik

Serializer — API uchun **Forma** (37-mavzu): u ham maydonlarni belgilaydi, ma'lumotni tekshiradi va saqlaydi. Farqi: forma HTML bilan, serializer JSON bilan ishlaydi. **Form** ↔

Serializer, **ModelForm** ↔ **ModelSerializer**.

```
from rest_framework import serializers
from .models import Kategoriya, Maqola

class KategoriyaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Kategoriya
        fields = ["id", "nomi", "slug"]

class MaqolaSerializer(serializers.ModelSerializer):
    muallif = serializers.ReadOnlyField(source="muallif.username") #
    faqat o'qish uchun
    kategoriya_nomi = serializers.ReadOnlyField(source="kategoriya.nomi")

    class Meta:
        model = Maqola
        fields = ["id", "sarlavha", "matn", "kategoriya", "kategoriya_nomi",
                  "muallif", "chop_etilgan", "korishlar", "yaratilgan"]
        read_only_fields = ["korishlar", "yaratilgan"]

    def validate_sarlavha(self, qiymat): # Forms'dagi
    clean_sarlavha kabi
        if len(qiymat.strip()) < 5:
            raise serializers.ValidationError("Sarlavha kamida 5 belgi
            bo'lsin")
        return qiymat.strip()
```

Shell'da sinab ko'ramiz

Python

```
from blog.models import Maqola
from blog.serializers import MaqolaSerializer

m = Maqola.objects.first()
s = MaqolaSerializer(m)
print(s.data)                                # obyekt → dict

s = MaqolaSerializer(Maqola.objects.all()[:2], many=True)  # ro'yxat uchun
many=True
print(s.data)

# Kelgan ma'lumotni tekshirish
s = MaqolaSerializer(data={"sarlavha": "Hi", "matn": "...", "kategoriya": 1})
print(s.is_valid())                          # False
print(s.errors)                             # {'sarlavha': ["Sarlavha
kamida 5 belgi bo'lsin"]}
```

Natija

```
{'id': 1, 'sarlavha': "Futbol bo'yicha g'alaba", 'matn': '...', 'kategoriya':
1, 'kategoriya_nomi': 'Sport',
  'muallif': 'admin', 'chop_etilgan': True, 'korishlar': 12, 'yaratilgan':
'2026-10-06T15:30:00+05:00'}
False
{'sarlavha': [ErrorDetail(string="Sarlavha kamida 5 belgi bo'lsin",
code='invalid')]}
```

API View'lar

1-usul: `@api_view` — funksiya asosida

blog/api_views.py (yangi fayl)

```
from django.shortcuts import get_object_or_404
from rest_framework import status
from rest_framework.decorators import api_view
from rest_framework.response import Response

from .models import Maqola
from .serializers import MaqolaSerializer

@api_view(["GET", "POST"])
def maqolalar_api(request):
    if request.method == "GET":
        maqolalar = Maqola.objects.filter(chop_etilgan=True)
        serializer = MaqolaSerializer(maqolalar, many=True)
        return Response(serializer.data) # 200 +

JSON

    # POST — yangi maqola
    serializer = MaqolaSerializer(data=request.data) #
    request.data — kelgan JSON
    if serializer.is_valid():
        serializer.save(muallif=request.user) #
    muallifni o'zimiz beramiz
    return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(["GET"])
def maqola_api(request, id):
    maqola = get_object_or_404(Maqola, id=id, chop_etilgan=True)
    return Response(MaqolaSerializer(maqola).data)
```


2-usul: `APIView` — klass asosida

Python

```
from rest_framework.views import APIView

class MaqolalarAPI(APIView):
    def get(self, request):
        serializer = MaqolaSerializer(Maqola.objects.filter(chop_etilgan=True),
many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = MaqolaSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)      # xato bo'lsa –
avtomatik 400
        serializer.save(muallif=request.user)
        return Response(serializer.data, status=status.HTTP_201_CREATED)
```

blog/urls.py

```
from . import api_views

urlpatterns += [
    path("api/maqolalar/", api_views.maqolalar_api, name="api_maqolalar"),
    path("api/maqolalar/<int:id>/", api_views.maqola_api, name="api_maqola"),
]
```

Brauzerda ko'rinadigan API

Serverni ishga tushirib, <http://127.0.0.1:8000/api/maqolalar/> ni oching. DRF chiroyli, **interaktiv** sahifa ko'rsatadi: JSON javob, status kod va (tizimga kirgan bo'lsangiz) pastda POST qilish uchun forma. Bu API'ni sinashning eng oson usuli.

Boshqa usullar:

Terminal

```
# Terminal orqali (curl)
curl http://127.0.0.1:8000/api/maqolalar/
curl http://127.0.0.1:8000/api/maqolalar/1/
```

JavaScript'dan (brauzer konsoli yoki frontend)

```
fetch("http://127.0.0.1:8000/api/maqolalar/")
  .then(javob => javob.json())
  .then(maqolalar => {
    maqolalar.forEach(m => console.log(m.sarlavha, "-", m.muallif));
  });
```



Professional dasturchilar API'ni sinash uchun **Postman** yoki VS Code'dagi **Thunder Client** kengaytmasidan foydalanadi: istalgan metod, sarlavha va JSON tana bilan so'rov yuborish mumkin.

Amaliy mashg'ulot

- 1 DRF'ni o'rnatish va sozlash.
- 2 `serializers.py` yaratish, shell'da serializatsiya va validatsiyani sinab ko'ring.
- 3 `@api_view` bilan ikkita endpoint yaratish va brauzerda oching.
- 4 Admin orqali tizimga kiring, so'ng brauzerdagi API sahifasida POST forma orqali yangi maqola yaratish (JSON):

JSON

```
{
  "sarlavha": "API orqali yaratilgan maqola",
  "matn": "Bu maqola DRF yordamida qo'shildi!",
  "kategoriya": 1
}
```

Javob kodi `201` ekanini tekshiring. Sarlavhani 3 belgili qilib yuboring — `400` va xato xabari.

- 5 `/api/kategoriyalar/` endpointini yarating (`KategoriyaSerializer`), har bir kategoriyada maqolalar soni ham bo'lsin (Maslahat: `serializers.IntegerField(source="maqolalar.count", read_only=True)`).

Ko'p uchraydigan xatolar

- ❌ `Object of type Maqola is not JSON serializable` — obyekt serializer'dan o'tkazilmagan.
- ❌ Ro'yxat uchun `many=True` unutilgan — `AttributeError`.
- ❌ `serializer.data` ni `is_valid()` dan oldin o'qish (`data=` bilan yaratilganda).
- ❌ `"rest_framework"` `INSTALLED_APPS` ga qo'shilmagan — brauzerdagi API sahifasi shablon xatosi beradi.
- ❌ POST so'rov `403` qaytaradi — tizimga kirilmagan (`IsAuthenticatedOrReadOnly`).

O'zingizni tekshiring

? REST API'da yangi resurs yaratish uchun qaysi HTTP metod ishlatiladi?

- A. GET
- B. POST
- C. PUT
- D. DELETE

? Muvaffaqiyatli yaratilgan resurs uchun qaytariladigan status kod?

- A. 200 OK
- B. 204 No Content
- C. 201 Created
- D. 302 Found

? Serializer'ning vazifasi?

- A. Model obyektlarini JSON'ga va JSON'ni tekshirilgan obyektlarga aylantirish
- B. Bazaga ulanish
- C. HTML shablon yaratish
- D. URL'larni sozlash

? Qaysi URL REST qoidalariga mos?

- A. `/api/getAllMaqolalar/`
- B. `/api/maqola/ochirish/5/`
- C. `/api/maqolaYarat/`
- D. `/api/maqolalar/5/`

? Bir nechta obyektни serializatsiya qilish uchun qaysi parametr kerak?

- A. `all=True`
- B. `many=True`
- C. `list=True`
- D. `multiple=True`

? DRF view'ida kelgan JSON ma'lumot qayerda?

- A. `request.POST`
- B. `request.GET`
- C. `request.data`
- D. `request.json`

Uyga vazifa

1. Kutubxona loyihangiz (38-mavzu uyga vazifasi) uchun `KitobSerializer` va `/api/kitoblar/` (GET, POST) endpointini yarating.
2. Ochiq API'larni o'rganing: brauzerda `https://api.github.com/users/torvalds` ni oching. JSON tuzilmasini tahlil qilib, qaysi maydonlar borligini yozing.

Yodda tuting

- API — dasturlar orasidagi interfeys; veb-API JSON qaytaradi va ko'p mijozlarga xizmat qiladi.
- REST: resurs — URL (ot, ko'plik), amal — HTTP metod (GET/POST/PUT/PATCH/DELETE), natija — status kod.
- DRF: `pip install djangorestframework` + `INSTALLED_APPS`.
- `ModelSerializer` — model ↔ JSON; `validate_maydon()`; `many=True`; `is_valid()`, `save()`, `errors`.
- `@api_view` / `APIView` + `Response(data, status=...)`.

✓ Test javoblari

1. REST API'da yangi resurs yaratish uchun qaysi HTTP metod ishlatiladi? — **B) POST**
2. Muvaffaqiyatli yaratilgan resurs uchun qaytariladigan status kod? — **C) 201 Created**
3. Serializer'ning vazifasi? — **A) Model obyektlarini JSON'ga va JSON'ni tekshirilgan obyektlarga aylantirish**
4. Qaysi URL REST qoidalariga mos? — **D) /api/maqolalar/5/**
5. Bir nechta obyektни serializatsiya qilish uchun qaysi parametr kerak? — **B) many=True**
6. DRF view'ida kelgan JSON ma'lumot qayerda? — **C) request.data**



DRF yordamida REST API yaratish. CRUD API ishlab chiqish

Generic view'lar (ListCreateAPIView, RetrieveUpdateDestroyAPIView), ViewSet va Router, ruxsatlar (faqat muallif), sahifalash, qidiruv va saralash, token autentifikatsiya, CORS; Front-end (JavaScript fetch) va Back-end integratsiyasi; mahsulotlar API'si.

2 soat · amaliy

 **Bu darsda nimani o'rganasiz**

- To'liq CRUD API'ni (GET, POST, PUT, PATCH, DELETE) bir necha qatorda yozishni;
- **ViewSet** va **Router** — URL'larni avtomatik yaratishni;
- API ruxsatlari: "faqat muallif o'zgartira oladi";
- Sahifalash, qidiruv va saralashni;
- **Front-end va Back-end integratsiyasi**: II bobdagi JavaScript bilan Django API'dan foydalanishni (29-loyiha);
- Mahsulotlar boshqaruv API'sini (27–28-loyihalar).

Abstraksiya zinapoyasi

39-mavzuda `@api_view` bilan har bir metodni qo'lda yozdik. DRF bir xil ishni tobora qisqaroq yozish imkonini beradi:

Daraja	Vosita	Kod hajmi	Qachon
1	<code>@api_view</code> / <code>APIView</code>	Ko'p, hammasi qo'lda	Nostandart mantiq

2	Generic view'lar (<code>ListCreateAPIView</code> ...)	Kam	Standart CRUD, URL'larni o'zingiz boshqarasiz
3	<code>ModelViewSet</code> + <code>Router</code>	Eng kam	To'liq standart CRUD



O'xshatish: ovqat tayyorlash

1-daraja — hamma narsani noldan pishirish. 2-daraja — yarim tayyor mahsulotlardan. 3-daraja — tayyor taomni isitib berish. Har birining o'z o'rni bor: maxsus taom uchun noldan pishirasiz, oddiy tushlik uchun — tayyoridan foydalanasiz.

2-daraja: Generic view'lar

blog/api_views.py

```
from rest_framework import generics
from .models import Maqola
from .serializers import MaqolaSerializer

class MaqolaListCreate(generics.ListCreateAPIView):          # GET (ro'yxat) +
    queryset =                                               POST (yaratish)
    queryset =
    Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
    "muallif")
    serializer_class = MaqolaSerializer

    def perform_create(self, serializer):                    # saqlashdan oldin
    muallifni qo'shamiz
        serializer.save(muallif=self.request.user)

class MaqolaDetail(generics.RetrieveUpdateDestroyAPIView):  # GET (bitta) +
    queryset = Maqola.objects.all()                          PUT + PATCH + DELETE
    serializer_class = MaqolaSerializer
```


blog/urls.py

```
path("api/maqolalar/", api_views.MaqolaListCreate.as_view()),
path("api/maqolalar/<int:pk>/", api_views.MaqolaDetail.as_view()),    #
generic'lar "pk" kutadi
```

Atigi ~15 qatorda to'liq CRUD API! Ammo hozir **istalgan** tizimga kirgan foydalanuvchi boshqa birovning maqolasini o'zgartira oladi — buni tuzatamiz.

Ruxsatlar (permissions)

blog/permissions.py (yangi fayl)

```
from rest_framework import permissions

class MuallifYokiFaqatOqish(permissions.BasePermission):
    """O'qish – hammaga; o'zgartirish va o'chirish – faqat muallifga."""

    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:    # GET, HEAD,
OPTIONS – "xavfsiz" metodlar
            return True
        return obj.muallif == request.user
```

blog/api_views.py

```
from rest_framework import permissions
from .permissions import MuallifYokiFaqatOqish

class MaqolaDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = Maqola.objects.all()
    serializer_class = MaqolaSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
MuallifYokiFaqatOqish]
```

Tayyor ruxsat	Ma'nosi
AllowAny	Hammaga ochiq
IsAuthenticated	Faqat tizimga kirganlar
IsAuthenticatedOrReadOnly	O'qish — hammaga, yozish — kirganlarga
IsAdminUser	Faqat admin (staff)

Bu 38-mavzudagi "faqat muallif tahrirlay oladi" qoidasining API'dagi ko'rinishi. Begona foydalanuvchi PUT/DELETE yuborsa — 403 Forbidden .

3-daraja: ViewSet va Router

blog/api_views.py

```
from rest_framework import viewsets, permissions, filters
from .models import Maqola, Kategoriya
from .serializers import MaqolaSerializer, KategoriyaSerializer
from .permissions import MuallifYokiFaqatOqish

class MaqolaViewSet(viewsets.ModelViewSet):          # list, create, retrieve,
update, partial_update, destroy
    serializer_class = MaqolaSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
MuallifYokiFaqatOqish]
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ["sarlavha", "matn"]              # ?search=futbol
    ordering_fields = ["yaratilgan", "korishlar"]      # ?ordering=-
korishlar

    def get_queryset(self):
        qs =
Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
"muallif")
        kategoriya = self.request.query_params.get("kategoriya")    # ?
kategoriya=sport
        if kategoriya:
            qs = qs.filter(kategoriya__slug=kategoriya)
        return qs

    def perform_create(self, serializer):
        serializer.save(muallif=self.request.user)

class KategoriyaViewSet(viewsets.ReadOnlyModelViewSet):    # faqat o'qish: list
+ retrieve
    queryset = Kategoriya.objects.all()
    serializer_class = KategoriyaSerializer
```

config/urls.py

```
from rest_framework.routers import DefaultRouter
from blog.api_views import MaqolaViewSet, KategoriyaViewSet

router = DefaultRouter()
router.register("maqolalar", MaqolaViewSet, basename="maqola")
router.register("kategoriyalar", KategoriyaViewSet)

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include(router.urls)),          # ← barcha API URL'lari
    # avtomatik
    path("api-auth/", include("rest_framework.urls")), # brauzerdagi API'da
    # "Log in" tugmasi
    path("", include("blog.urls")),
]
```



39-mavzu va yuqoridagi generic misol uchun `blog/urls.py` ga qo'shgan `api/maqolalar/...` yo'llarini endi **o'chiring** — aks holda bir xil manzil ikki joyda e'lon qilinadi.

Router avtomatik yaratgan URL'lar:

URL	Metodlar	ViewSet metodi
<code>/api/</code>	GET	Barcha endpointlar ro'yxati (DRF bosh sahifasi)
<code>/api/maqolalar/</code>	GET, POST	<code>list</code> , <code>create</code>
<code>/api/maqolalar/5/</code>	GET, PUT, PATCH, DELETE	<code>retrieve</code> , <code>update</code> , <code>partial_update</code> , <code>destroy</code>
<code>/api/kategoriyalar/</code> , <code>/api/kategoriyalar/2/</code>	GET	<code>list</code> , <code>retrieve</code>

Sahifalash

config/settings.py

```
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES":
    ["rest_framework.permissions.IsAuthenticatedOrReadOnly"],
    "DEFAULT_PAGINATION_CLASS":
    "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 10,
}
```

GET /api/maqolalar/?page=2&search=maktab&ordering=-korishlar

```
{
  "count": 23,
  "next": "http://127.0.0.1:8000/api/maqolalar/?page=3&search=maktab&ordering=-korishlar",
  "previous": "http://127.0.0.1:8000/api/maqolalar/?page=1&search=maktab&ordering=-korishlar",
  "results": [
    { "id": 12, "sarlavha": "Maktab kutubxonasi yangilandi", "korishlar": 340,
    "...": "..." }
  ]
}
```

Mobil ilova va JavaScript uchun: Token autentifikatsiya

Brauzer sessiya (cookie) orqali tizimda qoladi. Mobil ilova va boshqa dasturlar esa har bir so'rovda **token** — maxfiy kalit yuboradi.

config/settings.py

```
INSTALLED_APPS += ["rest_framework.authtoken"]

REST_FRAMEWORK["DEFAULT_AUTHENTICATION_CLASSES"] = [
    "rest_framework.authentication.SessionAuthentication",
    "rest_framework.authentication.TokenAuthentication",
]
```

config/urls.py

```
from rest_framework.authtoken.views import obtain_auth_token
urlpatterns += [path("api/token/", obtain_auth_token)] # POST
username+password → token
```

Terminal

```
python manage.py migrate
curl -X POST http://127.0.0.1:8000/api/token/ -d
"username=ali&password=parol12345"
# {"token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b"}

curl -X POST http://127.0.0.1:8000/api/maqolalar/ \
-H "Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b" \
-H "Content-Type: application/json" \
-d '{"sarlavha": "Token bilan yozildi", "matn": "Salom!", "kategoriya":
1}'
```



Token — parol kabi maxfiy! Uni kodga yozib GitHub'ga yuklamang. Haqiqiy loyihalarda tokenlar HTTPS orqali uzatiladi (41-mavzu).

Front-end va Back-end integratsiyasi (29-loyiha)

Endi kursning eng muhim lahzasi: **II bobdagi JavaScript** (front-end) va **Django API** (back-end) birlashadi. Front-end alohida sayt bo'lishi mumkin (masalan, Netlify'da), shuning uchun avval **CORS** ni sozlaymiz.

CORS — boshqa saytdan so'rovlarga ruxsat

Brauzer xavfsizlik uchun bir saytdagi JavaScript'ga **boshqa domendagi** API'ga so'rov yuborishni taqiqlaydi (**CORS** — Cross-Origin Resource Sharing). Ruxsat **server** tomonidan beriladi:

Terminal

```
pip install django-cors-headers
```

config/settings.py

```
INSTALLED_APPS += ["corsheaders"]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",          # ← ro'yxatning eng
    boshiga
    "django.middleware.security.SecurityMiddleware",
    # ... qolganlari
]

CORS_ALLOWED_ORIGINS = [
    "http://127.0.0.1:5500",                          # VS Code Live Server
    "https://maktab-blog.netlify.app",                 # internetdagi front-end
]
```

Front-end: maqolalar o'quvchi sahifa

frontend/index.html (Live Server bilan oching)

```
<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Maktab blogi – JS mijoz</title>
<style>
  body { font-family: "Segoe UI", sans-serif; max-width: 760px; margin: 30px
auto; padding: 0 16px; background: #f8fafc; }
  .qidiruv { display: flex; gap: 8px; margin-bottom: 16px; }
  .qidiruv input { flex: 1; padding: 10px; border: 2px solid #e5e7eb; border-
radius: 10px; }
  .karta { background: #fff; padding: 16px 20px; border-radius: 14px; margin-
bottom: 12px; box-shadow: 0 4px 14px rgba(0,0,0,.06); }
  .belgi { background: #eef2ff; color: #4f46e5; padding: 2px 10px; border-
radius: 999px; font-size: 13px; }
  .xato { color: #dc2626; }
  button { padding: 10px 16px; border: none; border-radius: 10px; background:
#4f46e5; color: #fff; cursor: pointer; }
</style>
</head>
<body>
  <h1>📖 Maktab blogi</h1>
  <form class="qidiruv" id="qidiruv">
    <input id="soz" placeholder="Qidirish...">
    <button>Izlash</button>
  </form>
  <p id="holat">Yuklanmoqda... </p>
  <div id="royxat"></div>
  <button id="yana" hidden>Yana yuklash</button>

<script>
  const API = "http://127.0.0.1:8000/api/maqolalar/";
  const royxat = document.querySelector("#royxat");
  const holat = document.querySelector("#holat");
  const yanaBtn = document.querySelector("#yana");
  let keyingi = null;

  async function yukla(url, tozala = true) {
```



```

    holat.textContent = "Yuklanmoqda...";
    holat.className = "";
    try {
        const javob = await fetch(url);
        if (!javob.ok) throw new Error(`Server xatosi: ${javob.status}`);
        const data = await javob.json(); // { count, next, previous,
results }

        if (tozala) royxat.innerHTML = "";
        data.results.forEach(m => {
            const karta = document.createElement("article");
            karta.className = "karta";
            const sarlavha = document.createElement("h3");
            sarlavha.textContent = m.sarlavha; // textContent
- XSS'dan himoya
            const meta = document.createElement("p");
            meta.innerHTML = `<span class="belgi"></span> 🖐️ <b></b> · 👁️
${m.korishlar}`;
            meta.querySelector(".belgi").textContent = m.kategoriya_nomi;
            meta.querySelector("b").textContent = m.muallif;
            const matn = document.createElement("p");
            matn.textContent = m.matn.slice(0, 150) + (m.matn.length > 150 ? "...": "");
            karta.append(sarlavha, meta, matn);
            royxat.append(karta);
        });

        holat.textContent = `Jami: ${data.count} ta maqola`;
        keyingi = data.next;
        yanaBtn.hidden = !keyingi;
    } catch (err) {
        holat.textContent = `❌ ${err.message}. Django server ishlayaptimi?`;
        holat.className = "xato";
    }
}

document.querySelector("#qidiruv").addEventListener("submit", e => {
    e.preventDefault();
    const soz = document.querySelector("#soz").value.trim();
    yukla(`${API}?search=${encodeURIComponent(soz)}`);
});

yanaBtn.addEventListener("click", () => yukla(keyingi, false)); // keyingi
sahifa

```

```
yukla(API);  
</script>  
</body>  
</html>
```

Nima bo'lyapti? JavaScript (front-end, port 5500) → `fetch` → Django API (back-end, port 8000) → PostgreSQL → JSON → JavaScript sahifani chizadi. Bu — **full-stack** ilovaning to'liq zanjiri, va siz uning barcha qismlarini o'zingiz yozdingiz! 🎉

 **index.html + JS**
Live Server :5500

— `fetch(GET /api/maqolalar/)` →

 **Django + DRF**
:8000

⇔ SQL ⇔  **PostgreSQL**

Amaliy mashg'ulot: mahsulotlar boshqaruv API'si (27–28-loyihalar)

1 Yangi ilova: `python manage.py startapp shop`. Model:

shop/models.py

```
from django.db import models  
  
class Mahsulot(models.Model):  
    nomi = models.CharField(max_length=200)  
    tavsif = models.TextField(blank=True)  
    narx = models.DecimalField(max_digits=12, decimal_places=2)  
    soni = models.PositiveIntegerField(default=0)  
    faol = models.BooleanField(default=True)  
    yaratilgan = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.nomi
```

- 2 `MahsulotSerializer` (narx musbat bo'lsin — `validate_narx`), `MahsulotViewSet` (qidiruv — nom; saralash — narx, yaratilgan), router'ga qo'shing.
- 3 Ruxsat: o'qish — hammaga, qo'shish/o'zgartirish/o'chirish — faqat admin (`IsAdminUser`; maslahat: `get_permissions` metodida `self.action` ga qarab).
- 4 `?narx_min=100000&narx_max=500000` filtrini `get_queryset` da qo'shing.
- 5 Barcha amallarni brauzerdagi API sahifasida yoki Thunder Client'da sinang: GET (ro'yxat, bitta), POST, PATCH (faqat narx), DELETE.
- 6 **Front-end:** 19-mavzudagi kartalar dizayni bilan mahsulotlar katalogi sahifasini yarating — ma'lumotlar API'dan kelsin.

▼ 3-qadam uchun maslahat

Python

```
class MahsulotViewSet(viewsets.ModelViewSet):
    queryset = Mahsulot.objects.filter(faol=True)
    serializer_class = MahsulotSerializer

    def get_permissions(self):
        if self.action in ("list", "retrieve"):
            return [permissions.AllowAny()]
        return [permissions.IsAdminUser()]
```

Ko'p uchraydigan xatolar

- ✗ Brauzer konsolida `blocked by CORS policy` — `corsheaders` o'rnatilmagan, middleware ro'yxat boshida emas yoki front-end manzili `CORS_ALLOWED_ORIGINS` da yo'q (port ham mos bo'lishi kerak!).
- ✗ `ViewSet`'da `get_queryset` ishlatilganda router'da `basename` unutilgan.
- ✗ Generic view URL'ida `<int:id>` — generic'lar standart holatda `pk` kutadi.
- ✗ `fetch` natijasini `.json()` siz ishlatish yoki `await` ni unutish.

O'zingizni tekshiring

? Bitta klassda list, create, retrieve, update, destroy amallarini beruvchi DRF klassi?

- A. `APIView`
- B. `ListAPIView`
- C. `ModelViewSet`
- D. `Serializer`

? `ViewSet` uchun URL'larni avtomatik yaratuvchi vosita?

- A. Router (`DefaultRouter`)
- B. `path()`
- C. Paginator
- D. Middleware

? `SAFE_METHODS` qaysi metodlarni o'z ichiga oladi?

- A. POST, PUT
- B. DELETE
- C. Hammasi
- D. GET, HEAD, OPTIONS

? Netlify'dagi JS sayt Django API'ga so'rov yuborganda "CORS" xatosi chiqdi. Nima qilish kerak?

- A. JavaScript kodini o'chirish
- B. Django'da `django-cors-headers` ni sozlab, front-end manzilini ruxsat etilganlarga qo'shish
- C. Brauzerni almashtirish
- D. API'ni HTML'ga aylantirish

? API'da qidiruv uchun DRF'ning qaysi vositasi ishlatiladi?

- A. `Paginator`
- B. `OrderingFilter`
- C. `SearchFilter` + `search_fields`
- D. `Router`

Uyga vazifa

1. JS front-end'ga "Maqola qo'shish" formasini qo'shing: token bilan `POST` so'rov yuboring (`headers: { "Authorization": "Token ...", "Content-Type": "application/json" }`).
2. Kutubxona API'ngizni `ViewSet` + `Router`'ga o'tkazing, qidiruv va sahifalash qo'shing.

Yodda tuting

- Generic: `ListCreateAPIView`, `RetrieveUpdateDestroyAPIView`; `perform_create` — muallifni qo'shish.
- `ModelViewSet` + `DefaultRouter` — to'liq CRUD API bir necha qatorda.
- Ruxsatlar: `permission_classes`, o'z `BasePermission` klassingiz (`has_object_permission`).
- `SearchFilter`, `OrderingFilter`, `PageNumberPagination`; token autentifikatsiya.
- Integratsiya: front-end `fetch` ⇌ DRF API; `django-cors-headers` bilan CORS.

✓ Test javoblari

1. Bitta klassda list, create, retrieve, update, destroy amallarini beruvchi DRF klassi? — **C) ModelViewSet**
2. ViewSet uchun URL'larni avtomatik yaratuvchi vosita? — **A) Router (DefaultRouter)**
3. SAFE_METHODS qaysi metodlarni o'z ichiga oladi? — **D) GET, HEAD, OPTIONS**
4. Netlify'dagi JS sayt Django API'ga so'rov yuborganda "CORS" xatosi chiqdi. Nima qilish kerak? — **B) Django'da django-cors-headers ni sozlab, front-end manzilini ruxsat etilganlarga qo'shish**
5. API'da qidiruv uchun DRF'ning qaysi vositasi ishlatiladi? — **C) SearchFilter + search_fields**



41-MAVZU · 42 TADAN · V BOB. DRF VA DEPLOY

Django loyihasini serverga joylash (Deploy)

Deploy nima, development va production farqi, hosting turlari; loyihani tayyorlash: DEBUG, SECRET_KEY, ALLOWED_HOSTS, gunicorn, whitenoise, dj-database-url, requirements.txt, collectstatic; Render'ga bosqichma-bosqich joylash, PostgreSQL, superuser; PythonAnywhere muqobili; deploy'dan keyingi tekshiruv.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- **Deploy** nima va "mening kompyuterimda ishlaydi" dan "internetda ishlaydi" ga qanday o'tishni;
- Development va production muhitlari farqini;
- Django loyihasini serverga tayyorlashni (xavfsizlik sozlamalari, statik fayllar, server dasturi);
- Loyihani **Render** bulutli platformasiga PostgreSQL bilan bosqichma-bosqich joylashni (30-loyiha);
- Joylangan saytni tekshirish va yangilashni.

Deploy nima?

Hozirgacha blogimiz faqat sizning kompyuteringizda, `127.0.0.1:8000` da ishladi — uni hech kim ko'ra olmaydi. **Deploy** (joylash) — dasturni foydalanuvchilar ishlatishi uchun internetga ulangan **serverga** o'rnatish va ishga tushirish.



O'xshatish: oshxona va restoran

Uyda ovqat pishirish (development) — o'zingiz uchun, xato qilsangiz ham hech kim bilmaydi. Restoran ochish (production) — boshqa narsa: sanitariya qoidalar (xavfsizlik), katta oshxona (server), doimiy ish vaqti (24/7), ko'p mijozlarga bir vaqtda xizmat. Retsept bir xil, lekin talablar butunlay boshqa.

	Development (ishlab chiqish)	Production (ishchi muhit)
Qayerda	Sizning kompyuteringiz	Internetdagi server
<code>DEBUG</code>	<code>True</code> — batafsil xatolar	<code>False</code> — xatolar yashirin (xavfsizlik!)
Server dasturi	<code>runserver</code> (sinov uchun)	<code>gunicorn</code> (ishonchli, tez)
Statik fayllar	Django o'zi beradi	<code>collectstatic</code> + <code>WhiteNoise</code>
Maxfiy kalitlar	<code>.env</code> fayl	Hosting'dagi muhit o'zgaruvchilari
Manzil	<code>127.0.0.1:8000</code>	<code>https://maktab-blog.onrender.com</code>

Hosting turlari (Django uchun)

Platforma	Turi	Afzalligi	E'tibor bering
Render	PaaS (bulut platformasi)	GitHub'dan avtomatik deploy, PostgreSQL, HTTPS bepul	Bepul tarifda sayt "uxlab qoladi" (birinchi ochilishi sekin); bepul baza muddati cheklangan
Railway	PaaS	Juda oson, PostgreSQL bir tugma bilan	Bepul limit cheklangan
PythonAnywhere	Python uchun maxsus hosting	O'quvchilar uchun eng oddiy, brauzerdan boshqarish	Bepul tarifda faqat SQLite/MySQL
VPS (DigitalOcean, mahalliy provayderlar)	Virtual server	To'liq nazorat	Hammasini o'zingiz sozlaysiz (Linux, Nginx)

PaaS (Platform as a Service) — server, operatsion tizim va tarmoqni platforma boshqaradi, siz faqat kodingizni berasiz. Bu darsda **Render** bilan ishlaymiz. Tariflar va bepul limitlar vaqt o'tishi bilan o'zgaradi

1-bosqich: loyihani production uchun tayyorlash

Kerakli paketlar

Terminal

```
pip install gunicorn whitenoise dj-database-url  
pip freeze > requirements.txt
```

Paket	Vazifasi
<code>gunicorn</code>	Production server dasturi (<code>runserver</code> o'rniga)
<code>whitenoise</code>	Statik fayllarni (CSS, JS) tez va xavfsiz berish
<code>dj-database-url</code>	Bazani bitta <code>DATABASE_URL</code> satridan sozlash



`gunicorn` Windows'da ishlamaydi — bu normal: u faqat serverda (Linux) ishga tushadi. Kompyuteringizda `runserver` dan foydalanishda davom eting.

settings.py — muhitga qarab sozlash

config/settings.py

```
import os
from pathlib import Path
import dj_database_url
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")          # lokal kompyuterda .env,
serverda — muhit o'zgaruvchilari

# 🛡️ Xavfsizlik
SECRET_KEY = os.environ["SECRET_KEY"]    # o'rnatilmagan bo'lsa — xato
(ataylab!)
DEBUG = os.getenv("DEBUG", "False") == "True" # standart — False
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
CSRF_TRUSTED_ORIGINS = [f"https://{h}" for h in ALLOWED_HOSTS if h not in
("127.0.0.1", "localhost")]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",      # ←
    SecurityMiddleware'dan keyin
    "django.contrib.sessions.middleware.SessionMiddleware",
    # ... qolganlari o'zgarishsiz
]

# 🐘 Baza: serverda DATABASE_URL, lokal kompyuterda .env'dagi qiymatlar
DATABASES = {
    "default": dj_database_url.config(
        default=f"postgres://{os.getenv('DB_USER')}:
{os.getenv('DB_PASSWORD')}@"
        f"{os.getenv('DB_HOST', 'localhost')}:{os.getenv('DB_PORT',
'5432')}/{os.getenv('DB_NAME')}",
        conn_max_age=600,
    )
}

# 📁 Statik fayllar
STATIC_URL = "static/"
```

```

STATIC_ROOT = BASE_DIR / "staticfiles" # collectstatic shu
yerga yig'adi
STORAGES = {
    "default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
    "staticfiles": {"BACKEND":
"whitenoise.storage.CompressedManifestStaticFilesStorage"},
}

# 🛡️ Production'da qo'shimcha himoya
if not DEBUG:
    SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
    SESSION_COOKIE_SECURE = True
    CSRF_COOKIE_SECURE = True

```

❌ Production'ning oltin qoidolari

1. `DEBUG = False` — aks holda xato sahifasi buzg'unchiga kodingiz, sozlamalar va hatto parollar haqida ma'lumot beradi.
2. `SECRET_KEY` **maxfiy va yangi** — GitHub'dagi yoki darsdagi kalitni ishlatmang. Yangisini yarating: `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`
3. `ALLOWED_HOSTS` — faqat sizning domeningiz.

Lokal kompyuteringizdagi `.env` ga `DEBUG=True` qo'shib qo'ying — shunda kompyuteringizda ishlash qulay qoladi.

Build skripti

Server har deploy'da bajaradigan buyruqlar:

build.sh (loyiha ildizida)

```

#!/usr/bin/env bash
set -o errexit # biror buyruq xato bersa - to'xtash

pip install -r requirements.txt
python manage.py collectstatic --no-input
python manage.py migrate

```

Terminal

```
git update-index --chmod=+x build.sh    # Windows'da faylni "bajariladigan"  
qilish (Git orqali)  
# macOS/Linux: chmod +x build.sh
```

Lokal tekshiruv

Terminal

```
python manage.py check --deploy          # Django production uchun maslahatlar  
beradi  
python manage.py collectstatic          # staticfiles/ papkasi paydo bo'ladi  
(.gitignore ga qo'shing)
```

.gitignore

```
venv/  
.env  
__pycache__/  
staticfiles/  
media/
```

GitHub'ga push

Terminal

```
git add .  
git commit -m "Production uchun tayyorlandi: gunicorn, whitenoise, dj-database-  
url"  
git push
```

2-bosqich: Render'ga joylash

1 render.com da **GitHub** akkauntingiz bilan ro'yxatdan o'ting.

2 **PostgreSQL bazasi yarating:** Dashboard → **New +** → **Postgres** → nom (`maktab-blog-db`), region (Frankfurt — O'zbekistonga yaqinroq), bepul tarif → **Create Database**. Tayyor bo'lgach, **Internal Database URL** ni nusxalang (`postgresql:// ...`).

3 **Web Service yarating:** **New +** → **Web Service** → GitHub repozitoriyangizni tanlang.

Maydon	Qiymat
Name	<code>maktab-blog</code> (manzil: <code>maktab-blog.onrender.com</code>)
Language / Runtime	Python 3
Build Command	<code>./build.sh</code>
Start Command	<code>gunicorn config.wsgi:application</code>
Instance Type	Free

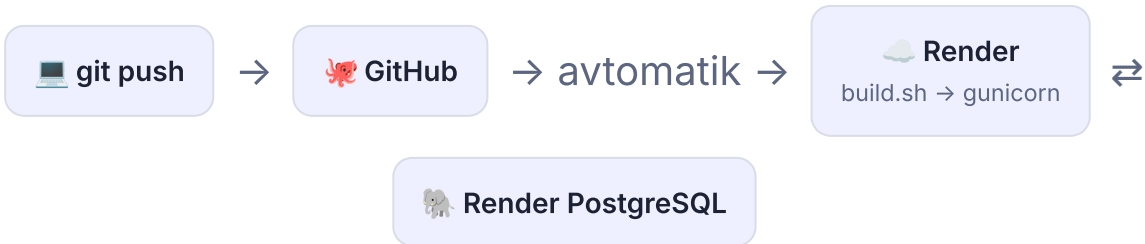
4 **Environment Variables** (muhit o'zgaruvchilari) bo'limida qo'shing:

Key	Value
<code>SECRET_KEY</code>	yangi yaratilgan uzun tasodifiy kalit
<code>DEBUG</code>	<code>False</code>
<code>ALLOWED_HOSTS</code>	<code>maktab-blog.onrender.com</code>
<code>DATABASE_URL</code>	2-qadamdagi Internal Database URL
<code>PYTHON_VERSION</code>	kompyuteringizdagi versiya, masalan <code>3.12.7</code>

5 **Create Web Service** bosing. **Logs** oynasida jarayonni kuzating: paketlar o'rnatiladi → collectstatic → migrate → gunicorn ishga tushadi. "**Your service is live** 🎉" yozuvini kuting (bir necha daqiqa).

6 **Admin yaratish:** servis sahifasida **Shell** yorlig'i (agar tarifingizda mavjud bo'lsa) → `python manage.py createsuperuser` . Shell mavjud bo'lmasa, lokal kompyuterdan **External Database URL** bilan bajarish mumkin: `.env` dagi baza sozlamalari o'rninga vaqtincha `DATABASE_URL= ... external ...` qo'yib, `python manage.py createsuperuser` (keyin qaytaring!).

7 Saytingizni oching: `https://maktab-blog.onrender.com` 🎉 Havolani do'stlaringiz va ota-onangizga yuboring!



Har bir `git push` dan keyin Render saytni avtomatik qayta joylaydi (CI/CD)

3-bosqich: deploy'dan keyin

Tekshiruv ro'yxati

Tekshiruv	✓
Bosh sahifa ochiladi, CSS ishlayapti (dizayn buzilmagan)	☐
<code>/admin/</code> ga kirish mumkin, maqola qo'shish ishlaydi	☐
Ro'yxatdan o'tish, kirish, maqola yaratish/tahrirlash/o'chirish ishlaydi	☐
<code>/maqola/9999/</code> — chiroyli 404 sahifa (sariq Django xatosi EMAS!)	☐
<code>/api/maqolalar/</code> JSON qaytaradi	☐
Manzilda  (HTTPS)	☐
Telefonda to'g'ri ko'rinadi	☐
40-mavzudagi JS front-end'ni Netlify'ga joylab, <code>API</code> manzilini Render'dagisiga almashtiring va <code>CORS_ALLOWED_ORIGINS</code> ga Netlify manzilini qo'shing	☐

Yangilash

Terminal

```
# Kodni o'zgartirdingiz → lokal sinadingiz → yubordingiz
git add .
git commit -m "Izohlar funksiyasi qo'shildi"
git push          # Render avtomatik qayta joylaydi; yangi migratsiyalar
build.sh'da bajariladi
```

Xatolarni topish

- **Logs** — barcha xatolar shu yerda. "Build failed" bo'lsa — build log'ini, sayt 500 qaytarsa — runtime log'ini o'qing.
- `DisallowedHost` → `ALLOWED_HOSTS` da domen yo'q yoki xato yozilgan.
- CSS yo'q (oddiy matn ko'rinishidagi sayt) → WhiteNoise middleware yoki `collectstatic` muammosi.
- `CSRF verification failed` (forma yuborishda) → `CSRF_TRUSTED_ORIGINS` da `https://` bilan domen yo'q.
- `ModuleNotFoundError` → paket `requirements.txt` ga yozilmagan (`pip freeze > requirements.txt` ni unutgansiz).
- Hech qachon muammoni "tuzatish" uchun serverda `DEBUG=True` qo'ymang!



Media fayllar haqida

Bulutli platformalarning disk xotirasi ko'pincha **vaqtinchalik**: har deploy'da foydalanuvchilar yuklagan rasmlar o'chib ketishi mumkin. Haqiqiy loyihalarda media fayllar alohida xizmatda (Cloudinary, Amazon S3) saqlanadi. O'quv loyihasi uchun bu muammo emas, lekin buni bilib qo'ying.

Muqobil: PythonAnywhere (qisqacha)

- 1 pythonanywhere.com da bepul "Beginner" akkaunt oching.
- 2 **Consoles → Bash:** `git clone https://github.com/USERNAME/maktab_blog.git`, virtual muhit yarating va `pip install -r requirements.txt`.
- 3 **Web → Add a new web app → Manual configuration** → Python versiyasi. WSGI fayliga loyiha yo'li va `DJANGO_SETTINGS_MODULE = "config.settings"` ni yozing.

- 4 Bepul tarifda PostgreSQL yo'q — `DATABASE_URL` o'rniga SQLite (`sqlite:///db.sqlite3`) ishlatiladi. Static fayllar uchun **Web** sahifasida `/static/` → `staticfiles` papkasini ko'rsating.
- 5 **Reload** — sayt `USERNAME.pythonanywhere.com` da.

Amaliy mashg'ulot (30-loyiha)

- 1 Blog loyihangizni production uchun tayyorlang va `python manage.py check --deploy` ogohlantirishlarini o'qing.
- 2 Render'da PostgreSQL va Web Service yarating, loyihani joylang.
- 3 Tekshiruv ro'yxatidagi barcha bandlarni bajaring.
- 4 Kichik o'zgarish kiriting (masalan, footer matni), push qiling va avtomatik qayta joylashni kuzating.
- 5 README.md ga sayt havolasini, skrinshotlarni va "Qanday ishga tushirish" bo'limini qo'shing.

O'zingizni tekshiring

? **Production serverda** `DEBUG` qanday bo'lishi kerak va nima uchun?

- A. True — xatolarni ko'rish uchun
- B. False — xato sahifalari maxfiy ma'lumotlarni oshkor qilmasligi uchun
- C. Ahamiyati yo'q
- D. None

? **Production'da `runserver` o'rniga ishlatiladigan server dasturi?**

- A. whitenoise
- B. pip
- C. psqI
- D. gunicorn

? **Barcha statik fayllarni bitta papkaga yig'uvchi buyruq?**

- A. `python manage.py collectstatic`
- B. `python manage.py migrate`
- C. `python manage.py static`
- D. `pip install static`

? **Sayingiz `DisallowedHost` xatosini berdi. Sababi?**

- A. Baza ulanmagan
- B. CSS topilmadi
- C. Domen `ALLOWED_HOSTS` da yo'q
- D. Parol noto'g'ri

? **Maxfiy kalitlar (`SECRET_KEY`, baza paroli) production'da qayerda saqlanadi?**

- A. `settings.py` ichida
- B. Hosting'ning muhit o'zgaruvchilarida (environment variables)
- C. GitHub README'da
- D. HTML shablonda

? Render'ga ulangan loyihani yangilash uchun nima qilinadi?

- A. Serverga qayta ro'yxatdan o'tish
- B. Loyihani o'chirib, qayta yaratish
- C. O'zgarishlarni GitHub'ga `git push` qilish
- D. Hech narsa — yangilab bo'lmaydi



Uyga vazifa

1. IV bob uyga vazifasidagi loyihangizni (kutubxona / talabalar / e'lonlar) ham internetga joylang.
2. Barcha joylangan loyihalaringiz havolalarini GitHub profilingizdagi README'ga va portfolio saytingizga qo'shing — bu sizning birinchi "rezyume"ingiz!



Yodda tuting

- Deploy — loyihani internetdagi serverga joylash; production ≠ development.
- Tayyorlash: `DEBUG=False`, maxfiy `SECRET_KEY`, `ALLOWED_HOSTS`, `CSRF_TRUSTED_ORIGINS`; `gunicorn`, `whitenoise`, `django-database-url`; `requirements.txt`.
- `build.sh`: `install` → `collectstatic` → `migrate`; start: `gunicorn config.wsgi:application`.
- Render: Postgres + Web Service + Environment Variables → har `git push` da avtomatik deploy.
- Muammolarni Logs orqali toping; serverda hech qachon `DEBUG=True` qo'ymang.

✓ Test javoblari

1. Production serverda DEBUG qanday bo'lishi kerak va nima uchun? — **B) False — xato sahifalari maxfiy ma'lumotlarni oshkor qilmasligi uchun**
2. Production'da runserver o'rniga ishlatiladigan server dasturi? — **D) gunicorn**
3. Barcha statik fayllarni bitta papkaga yig'uvchi buyruq? — **A) python manage.py collectstatic**
4. Saytingiz DisallowedHost xatosini berdi. Sababi? — **C) Domen ALLOWED_HOSTS da yo'q**
5. Maxfiy kalitlar (SECRET_KEY, baza paroli) production'da qayerda saqlanadi? — **B) Hosting'ning muhit o'zgaruvchilarida (environment variables)**
6. Render'ga ulangan loyihani yangilash uchun nima qilinadi? — **C) O'zgarishlarni GitHub'ga git push qilish**



42-MAVZU · 42 TADAN · V BOB. DRF VA DEPLOY

Yakuniy nazorat ishi — IV–V boblar (Django, DRF, Deploy) va yakuniy loyiha

Django, Django REST Framework va deploy bo'yicha nazorat: 25 ta yopiq test va 5 ta ochiq topshiriq (jami 100 ball); yakuniy loyiha talablari, g'oyalar, baholash mezonlari va taqdimot rejasi.

1 soat · nazorat ishi

Nazorat ishi tuzilishi

- **1-qism:** 25 ta yopiq test × 3 ball = **75 ball** (Python+PostgreSQL — 3, Django — 14, DRF — 5, Deploy — 3).
- **2-qism:** 5 ta ochiq savol va amaliy topshiriq × 5 ball = **25 ball**.
- **Jami: 100 ball.** Vaqt: 80 daqiqa.
- Sahifa oxirida — kursning **yakuniy loyihasi** talablari (31-loyiha).

Takrorlash rejasi

Mavzu	Bilishingiz kerak
32. Python + PostgreSQL	venv, psycopg, connect/execute/fetch, commit, parametrli so'rovlar va SQL injection
33. Django'ga kirish	MVT, project/app, startproject/startapp, INSTALLED_APPS, runserver, settings, DATABASES
34. URL va View	request/response, path, include, converterlar, name/reverse, Http404, redirect, CBV

35. Template va static	render, {{ }}, {% %}, filtrlar, extends/block/include, {% url %}, {% static %}
36. Model va ORM	maydon turlari, ForeignKey, makemigrations/migrate, objects.all/filter/get/create, lookups
37. Admin va Forms	admin.register, ModelAdmin, Form/ModelForm, is_valid, csrf_token, login_required, UserCreationForm
38. CRUD	5 sahifa, instance=, ruxsatlar, POST bilan o'chirish, Paginator, generic CBV'lar
39–40. DRF	REST, JSON, metodlar va status kodlar, Serializer, @api_view, generics, ViewSet, Router, permissions, CORS
41. Deploy	DEBUG=False, SECRET_KEY, ALLOWED_HOSTS, gunicorn, whitenoise, collectstatic, muhit o'zgaruvchilari

1-qism: Yopiq testlar (25 × 3 = 75 ball)

? 1. SQL injection'dan himoyalangan so'rov qaysi?

- A. `cur.execute(f"SELECT * FROM t WHERE ism='{ism}'")`
- B. `cur.execute("SELECT * FROM t WHERE ism=%s", (ism,))`
- C. `cur.execute("SELECT * FROM t WHERE ism='" + ism + "'")`
- D. Hammasi xavfsiz

? 2. psycopg'da INSERT dan keyin o'zgarishni saqlash?

- A. `cur.save()`
- B. `cur.fetchall()`
- C. `conn.close()`
- D. `conn.commit()` (yoki `with` blokining muvaffaqiyatli tugashi)

? 3. Loyiha kutubxonalarini alohida saqlash uchun nima yaratiladi?

- A. Virtual muhit (venv)
- B. Yangi baza
- C. Git branch
- D. Docker

? 4. Django arxitekturasidagi "T" nima?

- A. Table
- B. Test
- C. Template
- D. Token

? 5. Yangi ilova yaratilgandan keyin uni qayerga qo'shish kerak?

- A. `urls.py`
- B. `settings.py` → `INSTALLED_APPS`
- C. `models.py`
- D. `manage.py`

? 6. `path("kitob/<int:pk>/", views.kitob)` — view qanday e'lon qilinadi?

- A. `def kitob(request):`
- B. `def kitob(pk):`
- C. `def kitob(request, id):`
- D. `def kitob(request, pk):`

? 7. Ilova URL'larini asosiy faylga ulash?

- A. `path("", include("blog.urls"))`
- B. `import blog.urls`
- C. `urls.append(blog)`
- D. `path("", blog)`

? 8. Shablonda o'zgaruvchi chiqarish?

- A. `{% ism %}`
- B. `[[ ism ]]`
- C. `{{ ism }}`
- D. `${ ism }`

? 9. Umumiy skeletdan meros olish tegi?

- A. `{% include %}`
- B. `{% extends %}`
- C. `{% block %}`
- D. `{% load %}`

? 10. Shablonda CSS ulash uchun avval nima yoziladi?

- A. `{% css %}`
- B. `{% import static %}`
- C. `{% url 'static' %}`
- D. `{% load static %}`

? 11. Model o'zgargandan keyin bajariladigan buyruqlar ketma-ketligi?

- A. `makemigrations` → `migrate`
- B. `migrate` → `makemigrations`
- C. `runserver` → `migrate`
- D. `collectstatic`

? 12. "Har bir kitobning bitta muallifi, muallifning ko'p kitobi" — Kitob modelida qaysi maydon?

- A. `ManyToManyField(Muallif)`
- B. `CharField`
- C. `ForeignKey(Muallif, on_delete=...)`
- D. `OneToOneField`

? 13. Narxi 50 000 dan kichik kitoblar?

- A. `Kitob.objects.get(narx<50000)`
- B. `Kitob.objects.filter(narx__lt=50000)`
- C. `Kitob.objects.all(narx=50000)`
- D. `Kitob.filter("narx < 50000")`

? 14. Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya?

- A. `Model.objects.get()`
- B. `Http404()`
- C. `find()`
- D. `get_object_or_404()`

? 15. Admin ro'yxatidagi ustunlarni belgilovchi sozlama?

- A. `list_display`
- B. `fields`
- C. `columns`
- D. `search_fields`

? 16. POST formada majburiy xavfsizlik tegi?

- A. `{% secure %}`
- B. `{% load csrf %}`
- C. `{% csrf_token %}`
- D. `{% token %}`

? 17. Mavjud obyektни ModelForm bilan tahrirlash?

- A. `Form(request.POST, id=5)`
- B. `Form(request.POST, instance=obj)`
- C. `Form.update(obj)`
- D. `obj.form()`

? 18. Faqat tizimga kirganlar uchun ochiq view (funksiya)?

- A. `@admin_required`
- B. `@csrf_exempt`
- C. `@api_view`
- D. `@login_required`

? 19. REST API'da resursni o'chirish metodi va javob kodi?

- A. GET, 200
- B. POST, 201
- C. DELETE, 204
- D. PUT, 404

? 20. Model ↔ JSON aylantiruvchi DRF klassi?

- A. `ModelSerializer`
- B. `ModelForm`
- C. `ModelAdmin`
- D. `JsonResponse`

? 21. Bir nechta obyektни serializatsiya qilish?

- A. `Serializer(qs, all=True)`
- B. `Serializer(qs, many=True)`
- C. `Serializer.list(qs)`
- D. `Serializer(qs)`

? 22. To'liq CRUD API'ni bitta klassda beruvchi DRF klassi va URL'larni avtomatik yaratuvchi vosita?

- A. `APIView` + `path`
- B. `Serializer` + `include`
- C. `ListView` + `Paginator`
- D. `ModelViewSet` + `DefaultRouter`

? 23. Boshqa domendagi JS sayt API'ga murojaat qila olishi uchun Django'da nima sozlanadi?

- A. CSRF
- B. ALLOWED_HOSTS
- C. CORS (`django-cors-headers`)
- D. WhiteNoise

? 24. Production'da Django ilovasini ishga tushiruvchi server dasturi?

- A. `gunicorn`
- B. `runserver`
- C. `pip`
- D. `psql`

? 25. Internetdagi saytda qaysi sozlama noto'g'ri?

- A. `DEBUG = False`
- B. `DEBUG = True`
- C. `ALLOWED_HOSTS = ["sayt.onrender.com"]`
- D. `SECRET_KEY` muhit o'zgaruvchisida

2-qism: Ochiq savollar va amaliy topshiriqlar (5 × 5 = 25 ball)

1-topshiriq (5 ball). Nazariy savol

Foydalanuvchi brauzerda `/maqola/5/` ni ochganda Django ichida nima sodir bo'lishini MVT tushunchalari bilan bosqichma-bosqich tushuntiring.

▼ Namuna javob va baholash

(1) So'rov Django'ga keladi; (2) `urls.py` yo'llarni tekshirib, `maqola/<int:id>/` ga mosini topadi va `id=5` bilan view'ni chaqiradi; (3) View Model orqali (ORM) bazadan 5-maqolani so'raydi — `get_object_or_404(Maqola, id=5)`; topilmasa 404; (4) View maqolani kontekst sifatida Template'ga beradi — `render(...)`; (5) Shablon ma'lumot bilan to'ldirilib, tayyor HTML javob brauzerga qaytariladi.

Baholash: URL — 1; View — 1; Model/ORM/baza — 1; Template/render — 1; to'g'ri ketma-ketlik va 404 holati — 1.

2-topshiriq (5 ball). Model va admin

`Kitob` modelini yozing: nomi (200 belgi), muallif (100 belgi), janr (tanlov: roman, she'riyat, ilmiy), yil, narx (pul), mavjud (standart — ha), qo'shilgan sana (avtomatik). `__str__` nomini qaytarsin.

Admin'da ro'yxatda nomi, muallif, janr, mavjud ko'rinsin; janr va mavjudlik bo'yicha filtr, nom va muallif bo'yicha qidiruv.

▼ Namuna javob

Python

```
from django.db import models
from django.contrib import admin

class Kitob(models.Model):
    JANRLAR = [("roman", "Roman"), ("sheriyat", "She'riyat"),
               ("ilmiy", "Ilmiy")]
    nomi = models.CharField(max_length=200)
    muallif = models.CharField(max_length=100)
    janr = models.CharField(max_length=20, choices=JANRLAR)
    yil = models.PositiveIntegerField()
    narx = models.DecimalField(max_digits=10, decimal_places=2)
    mavjud = models.BooleanField(default=True)
    qoshilgan = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.nomi

@admin.register(Kitob)
class KitobAdmin(admin.ModelAdmin):
    list_display = ["nomi", "muallif", "janr", "mavjud"]
    list_filter = ["janr", "mavjud"]
    search_fields = ["nomi", "muallif"]
```

Baholash: maydon turlari to'g'ri — 2; choices va default/auto_now_add — 1; `__str__` — 1; admin sozlamalari — 1.

3-topshiriq (5 ball). View va shablon

Mavjud kitoblar ro'yxatini ko'rsatuvchi view va shablon yozing: `?q=` parametri bo'lsa, nomi bo'yicha qidirsin; ro'yxat bo'sh bo'lsa "Kitob topilmadi" chiqsin; har bir kitob nomi batafsil

sahifaga havola bo'lsin.

▼ Namuna javob

Python

```
def kitoblar(request):
    q = request.GET.get("q", "").strip()
    qs = Kitob.objects.filter(mavjud=True)
    if q:
        qs = qs.filter(nomi__icontains=q)
    return render(request, "kutubxona/kitoblar.html", {"kitoblar": qs,
"q": q})
```

Django shablon

```
{% extends "kutubxona/base.html" %}
{% block content %}
    <form><input name="q" value="{{ q }}" placeholder="Qidirish">
<button>Izlash</button></form>
    <ul>
        {% for k in kitoblar %}
            <li><a href="{% url 'kutubxona:kitob' k.id %}">{{ k.nomi }}</a>
– {{ k.muallif }}</li>
        {% empty %}
            <li>Kitob topilmadi</li>
        {% endfor %}
    </ul>
{% endblock %}
```

Baholash: filter va icontains — 1; GET parametr — 1; render va kontekst — 1; for/empty — 1;

{% url %} bilan havola — 1.

4-topshiriq (5 ball). DRF API

`Kitob` uchun to'liq CRUD API yarating (`/api/kitoblar/`): o'qish — hammaga, o'zgartirish — faqat tizimga kirganlarga; narx musbat bo'lishi tekshirilsin; nom bo'yicha qidiruv.

▼ Namuna javob

Python

```
from rest_framework import serializers, viewsets, permissions, filters
from rest_framework.routers import DefaultRouter

class KitobSerializer(serializers.ModelSerializer):
    class Meta:
        model = Kitob
        fields = "__all__"

    def validate_narx(self, v):
        if v <= 0:
            raise serializers.ValidationError("Narx musbat bo'lishi kerak")
        return v

class KitobViewSet(viewsets.ModelViewSet):
    queryset = Kitob.objects.all()
    serializer_class = KitobSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    filter_backends = [filters.SearchFilter]
    search_fields = ["nomi"]

router = DefaultRouter()
router.register("kitoblar", KitobViewSet)
# urls.py: path("api/", include(router.urls))
```

Baholash: serializer — 1; validatsiya — 1; ViewSet — 1; ruxsat — 1; router va qidiruv — 1.

5-topshiriq (5 ball). Deploy

Django loyihasini Render'ga joylash uchun bajariladigan asosiy qadamlarni va production sozlamalarini sanab, har birini qisqa tushuntiring.

▼ Namuna javob

1. `gunicorn`, `whitenoise`, `django-database-url` o'rnatish va `requirements.txt` ni yangilash.
2. `settings.py`: `DEBUG=False`, `SECRET_KEY` va baza — muhit o'zgaruvchilaridan, `ALLOWED_HOSTS` va `CSRF_TRUSTED_ORIGINS` — domen, `WhiteNoise` middleware, `STATIC_ROOT`.
3. `build.sh`: `pip install` → `collectstatic` → `migrate`. Kod GitHub'ga push.
4. Render'da PostgreSQL va Web Service; Start: `gunicorn config.wsgi:application`; Environment Variables.
5. Deploy'dan keyin: `superuser` yaratish, tekshiruv ro'yxati, Logs orqali xatolarni topish.

Baholash: har bir to'g'ri tushuntirilgan bosqich — 1 ball.

Baholash shkalasi

Ball	Natija
86–100	A'lo — kurs dasturini to'liq o'zlashtirgansiz
71–85	Yaxshi
60–70	Qoniqarli
0–59	Qoniqarsiz — mentor bilan takrorlash tavsiya etiladi

Yakuniy loyiha (31-loyiha)

Kurs davomida o'rganilgan **barcha** bilimlarni bitta loyihada namoyish eting. Loyiha individual yoki 2–3 kishilik jamoada (13-mavzu) bajariladi.

Majburiy talablar

Qism	Talab	Mavzular
Front-end	Semantik HTML, o'z CSS dizayni, responsiv (telefon + kompyuter), kamida bitta JS interaktivlik	2–12, 15–20
Ma'lumotlar bazasi	PostgreSQL; kamida 3 ta model, kamida bitta <code>ForeignKey</code>	26–30, 36
Back-end	Django: CRUD (5 sahifa), shablon merosi, formalar va validatsiya, admin panel	33–38
Foydalanuvchilar	Ro'yxatdan o'tish / kirish; kamida bitta "faqat muallif" ruxsati	37–38
API	Kamida bitta resurs uchun DRF API (GET + POST)	39–40
Algoritmik qism	Shart va sikllarga asoslangan mantiq: statistika, reyting, hisob-kitob, filtrlash	16–18, 21–23, 30
Git va deploy	GitHub'da kamida 15 ta mazmunli commit, README; internetga joylangan	11, 13, 41

Loyiha g'oyalari

- 📖 **Maktab kutubxonasi:** kitoblar katalogi, ijaraga olish/qaytarish, kechikish jarimasi, eng ko'p o'qilgan kitoblar reytingi.
- 📝 **Onlayn test tizimi:** o'qituvchi test yaratadi, o'quvchilar yechadi, natijalar va reyting jadvali.
- 🏠 **Sinf jurnali:** o'quvchilar, fanlar, baholar, davomat; o'rtacha baholar va hisobotlar.
- 🛒 **Maktab do'koni / yarmarka:** mahsulotlar, savat, buyurtmalar, ombor hisobi.
- 🎯 **To'garaklar portali:** to'garaklar, joylar soni, ro'yxatdan o'tish, jadval.
- 💡 **O'z g'oyangiz** — mentor bilan kelishgan holda.

Bajarish rejasi

- 1 G'oya va rejalashtirish** (1-hafta): funksiyalar ro'yxati, ER-diagramma (26-mavzu), Figma'da 2–3 sahifa maketi (10-mavzu), GitHub repozitoriya va Kanban doskasi.
- 2 Back-end** (2-hafta): modellar, migratsiyalar, admin, CRUD view'lar va formalar.

- 3 **Front-end** (3-hafta): shablonlar, dizayn, responsivlik, JS interaktivlik.
- 4 **API, sinov va deploy** (4-hafta): DRF endpoint, sinov ssenariylari va testlar, Render'ga joylash, README.
- 5 **Taqdimot.**

Baholash mezonlari (100 ball)

Mezon	Ball
Funksionallik: barcha majburiy talablar ishlaydi, xatolarsiz	30
Kod sifati: tushunarli nomlar, tartibli tuzilma, takrorlanmaslik, xavfsizlik (CSRF, ruxsatlar, parametrli so'rovlar)	20
Dizayn va foydalanish qulayligi: chiroyli, responsiv, tushunarli	15
Ma'lumotlar bazasi tuzilmasi: to'g'ri modellar va bog'lanishlar	10
Git, README va deploy: commit tarixi, hujjat, ishlaydigan havola	10
Taqdimot: maqsadni, ishlash prinsipini va texnologiyalarni tushuntira olish, savollarga javob	15

Taqdimot rejasini (5–7 daqiqa)

1. **Muammo va g'oya** — loyiha kimga va nima uchun kerak? (30 soniya)
2. **Jonli namoyish** — internetdagi sayt: asosiy ssenariylar (3 daqiqa).
3. **Texnik tuzilma** — modellar diagrammasi, qaysi texnologiyalar qayerda ishlatilgan (1–2 daqiqa).
4. **Qiyinchiliklar va xulosalar** — eng qiyin muammo va uni qanday hal qildingiz; nimani yaxshilagan bo'lardingiz (1 daqiqa).
5. **Savollarga javob.**

Loyiha havolasi o'quv dasturiga ko'ra GitHub repozitoriyasi yoki mentor tasdiqlagan boshqa platforma (Google Drive, Telegram kanal/guruhdagi video-taqdimot) orqali topshiriladi.

Tabriklaymiz — kurs yakunlandi!

Siz 66 soat davomida veb-sahifaning birinchi tegidan internetda ishlaydigan, ma'lumotlar bazasi va API'ga ega full-stack ilovagacha bo'lgan yo'lni bosib o'tdingiz: **HTML → CSS → JavaScript → Python → PostgreSQL → Django → DRF → Deploy**.

Keyingi qadamlar:

- Ushbu saytdagi "Full-stack qo'llanma" kursida JavaScript backend (Node.js, Express) va React bilan tanishing.
- **Telegram botlar** (Python + Aiogram) — Python bilimlaringizni yangi yo'nalishda qo'llang.
- Loyihalar qilishda davom eting: har bir yangi loyiha — portfoliingizga yangi qadam.
- Olimpiadalar, xakatonlar va IT-tanlovlarda qatnashing.

Omad! 

✓ Test javoblari

1. 1. SQL injection'dan himoyalangan so'rov qaysi? — **B) `cur.execute("SELECT * FROM t WHERE ism=%s", (ism,))`**
2. 2. psycopg2'da INSERT dan keyin o'zgarishni saqlash? — **D) `conn.commit()` (yoki with blokining muvaffaqiyatli tugashi)**
3. 3. Loyiha kutubxonalarini alohida saqlash uchun nima yaratiladi? — **A) Virtual muhit (venv)**
4. 4. Django arxitekturasidagi "T" nima? — **C) Template**
5. 5. Yangi ilova yaratilgandan keyin uni qayerga qo'shish kerak? — **B) `settings.py` → `INSTALLED_APPS`**
6. 6. `path("kitob/<int:pk>/", views.kitob)` — view qanday e'lon qilinadi? — **D) `def kitob(request, pk):`**
7. 7. Ilova URL'larini asosiy faylga ulash? — **A) `path("", include("blog.urls"))`**
8. 8. Shablonda o'zgaruvchi chiqarish? — **C) `{{ ism }}`**
9. 9. Umumiy skeletdan meros olish tegi? — **B) `{% extends %}`**
10. 10. Shablonda CSS ulash uchun avval nima yoziladi? — **D) `{% load static %}`**
11. 11. Model o'zgargandan keyin bajariladigan buyruqlar ketma-ketligi? — **A) `makemigrations` → `migrate`**
12. 12. "Har bir kitobning bitta muallifi, muallifning ko'p kitobi" — Kitob modelida qaysi maydon? — **C) `ForeignKey(Muallif, on_delete=...)`**
13. 13. Narxi 50 000 dan kichik kitoblar? — **B) `Kitob.objects.filter(narx__lt=50000)`**
14. 14. Obyekt topilmasa avtomatik 404 qaytaruvchi funksiya? — **D) `get_object_or_404()`**
15. 15. Admin ro'yxatidagi ustunlarni belgilovchi sozlama? — **A) `list_display`**
16. 16. POST formada majburiy xavfsizlik tegi? — **C) `{% csrf_token %}`**
17. 17. Mavjud obyektni ModelForm bilan tahrirlash? — **B) `Form(request.POST, instance=obj)`**
18. 18. Faqat tizimga kirganlar uchun ochiq view (funksiya)? — **D) `@login_required`**
19. 19. REST API'da resursni o'chirish metodi va javob kodi? — **C) `DELETE, 204`**
20. 20. Model ↔ JSON aylantiruvchi DRF klassi? — **A) `ModelSerializer`**
21. 21. Bir nechta obyektni serializatsiya qilish? — **B) `Serializer(qs, many=True)`**
22. 22. To'liq CRUD API'ni bitta klassda beruvchi DRF klassi va URL'larni avtomatik yaratuvchi vosita? — **D) `ModelViewSet + DefaultRouter`**
23. 23. Boshqa domendagi JS sayt API'ga murojaat qila olishi uchun Django'da nima sozlanadi? — **C) `CORS (django-cors-headers)`**
24. 24. Production'da Django ilovasini ishga tushiruvchi server dasturi? — **A) `gunicorn`**
25. 25. Internetdagi saytda qaysi sozlama noto'g'ri? — **B) `DEBUG = True`**